

Лекции о сложности алгоритмов

$$\begin{aligned}\bar{T}_A(s) &= \sum_{x \in X_s} C_A^T(x) P_s(x) \leq \\ &\leq \sum_{x \in X_s} \left(\max_{x \in X_s} C_A^T(x) \right) P_s(x) = \\ &= \sum_{x \in X_s} T_A(s) P_s(x) = T_A(s) \sum_{x \in X_s} P_s(x) = \\ &= T_A(s).\end{aligned}$$

С. А. Абрамов

Лекции о сложности алгоритмов

*Допущено учебно-методическим советом по прикладной математике
и информатике УМО по классическому университетскому
образованию в качестве учебного пособия для студентов высших
учебных заведений, обучающихся по специальности и направлению
«Прикладная математика и информатика» и по направлению
«Информационные технологии»*

Издание четвертое, исправленное и дополненное, электронное

Москва
Издательство МЦНМО
2024

УДК 510.52
ББК 22.12
А16

Абрамов С. А.

Лекции о сложности алгоритмов

4-е издание, исправленное и дополненное, электронное

М.: МЦНМО, 2024

255 с.

ISBN 978-5-4439-3420-4

В книге излагаются основные (начальные) разделы теории сложности алгоритмов. Сложности рассматриваются в худшем случае и в среднем. Ряд основных понятий теории сложности, как-то: оценки снизу и сверху, нижняя граница сложности алгоритмов некоторого класса, оптимальный алгоритм и т. д., рассматривается не только в обычном функциональном, но и в асимптотическом смысле: асимптотические оценки, асимптотическая нижняя граница, оптимальность по порядку сложности и т. д. Показывается, что при исследовании существования алгоритма решения задачи, имеющего «не очень высокую» сложность, важную роль может играть сводимость одной задачи к другой.

Изложение сопровождается анализом сложности ряда алгоритмов арифметики, сортировки и поиска, вычислительной геометрии, теории графов и др.

Для студентов, специализирующихся в области математики и информатики.

<https://t.me/portalToIT>

Издательство Московского центра
непрерывного математического образования
119002, Москва, Большой Власьевский пер., 11. Тел. (499) 241-08-04
<http://www.mccme.ru>

ISBN 978-5-4439-3420-4

© Абрамов С. А., 2024.

© МЦНМО, 2024.

Предисловие

https://t.me/it_boooks/2

Предлагаемая книга основана на курсе лекций, читаемых автором на факультете вычислительной математики и кибернетики (ВМК) МГУ им. М. В. Ломоносова. Ее цель — обсуждение основных понятий теории сложности и некоторых методов анализа сложности алгоритмов. Это обсуждение сопровождается подробным рассмотрением со сложностной точки зрения ряда алгоритмов арифметики, сортировки и поиска, вычислительной геометрии, теории графов и др. Материал группируется по главам и параграфам в соответствии с разделами самой теории сложности — сложность в худшем случае, сложность в среднем, нижние границы сложности и т. д. Для того, чтобы сосредоточиться именно на понятиях и подходах теории сложности, при этом не затрачивая слишком много времени на объяснение деталей трудных для понимания алгоритмов, в примерах исследуются либо алгоритмы достаточно известные (сложностные характеристики которых менее известны), либо алгоритмы, суть которых может быть изложена коротко. Сложностные вопросы рассматриваются в книге довольно подробно, но книга значительно уступает по широте охвата алгоритмического материала книгам Д. Кнута [25, 26, 27], А. Ахо, Дж. Хопкрофта и Дж. Ульмана [8], Т. Кормена, Ч. Лейзерсона, Р. Ривеста [28], С. Бааса и А. ван Гелдера [61], Ж. Брассара и П. Берли [64], отражающим обширный спектр вопросов построения алгоритмов, и книгам по специальным алгоритмическим разделам математики — вычислительной геометрии (Ф. Препарата, М. Шеймос [44]), алгоритмической теории чисел (Э. Бах, Дж. Шаллит [62]), комбинаторике (Э. Рейнгольд, Ю. Нивергельт, Н. Део [46], С. Джукна [75]) и др. Здесь надо сказать, что названная литература, как и некоторые другие издания, послужила источником ряда примеров и задач, предлагаемых в этой книге.

В последних трех параграфах, касающихся классов P и NP , понятия NP -полноты и т. д., ряд фактов дается без доказательства. Это объясняется тем, что в книге используются менее формальные модели вычислений, чем, скажем, машина Тьюринга, а доказательства упомянутых фактов опираются на полностью формализованные модели. Недостающие доказательства могут быть найдены, например, в [5], [15], [20], [31].

В приложениях даются дополнительные сведения по затрагиваемым в основном тексте вопросам. Исключение составляют приложения А, F: в первом из них содержатся необходимые сведения о ряде алгоритмов сортировки и поиска, во втором — о методе решения линейных рекуррентных уравнений с постоянными коэффициентами; эти два приложения носят характер напоминания.

Библиографические комментарии даются в сносках.

Каждая из глав снабжена задачами для самостоятельного решения, среди которых помимо легких имеются и такие, которые углубляют материал главы, поэтому полезно по крайней мере прочитывать условия всех задач. Задача содержит указание к решению в тех, например, случаях (довольно редких), когда в основном тексте имеется отсылка к этой задаче. По всем главам для задач используется сквозная нумерация. Многие из предлагаемых задач имеют вид утверждений, подразумевается, что каждое такое утверждение надо доказать.

Сжатое изложение курса этих лекций представлено в виде расширяемого эссе [3] на веб-странице

<http://www.ccas.ru/sabramov/essay/>

Автор благодарен своим коллегам по ВМК МГУ В.Б.Алексееву и В.П.Иванникову, а также Е.В.Зиме (университет Уилфрида Лорье, Ватерлоо, Канада) и М.Петковшеку (университет Любляны, Словения), беседы и дискуссии с которыми существенно помогли в отборе материала и выработке общей схемы курса лекций и этой книги, при этом надо особо отметить, что Е.В.Зима принял участие в написании главы 6. Большая благодарность и А.В.Бернштейну (ИСА РАН), А.А.Васину (ВМК МГУ), В.А.Серебрякову (ВЦ РАН), сделавшим полезные замечания по ряду разделов книги. Автор признателен рецензентам М.Н.Вялому (ВЦ РАН) и С.Б.Гашкову (мехмат МГУ) — их пометки на полях рукописи оказали значительную помощь на заключительном этапе работы над книгой. Особая благодарность за советы и многочисленные конструктивные замечания Е.А.Бордаченковой (ВМК МГУ) — научному редактору книги.

* * *

Первое издание книги вышло в 2009 г., второе — в 2012 г., третье — в 2020 г. Настоящее четвертое издание книги отличается от предыдущего исправлением замеченных погрешностей и опечаток. Добавлено некоторое количество задач. Автор благодарит М.Кауэрса (университет И.Кеплера в Линце, Австрия), А.А.Разборова (Чикагский университет) за полезные советы.

Введение

Исследование *сложности* алгоритма помогает понять степень его практической приемлемости. Сравнительный анализ сложности нескольких алгоритмов решения одной и той же задачи позволяет делать обоснованный выбор лучшего из них. Например, если для решения задачи предлагается новый алгоритм, то необходимы доводы, говорящие о его преимуществах в сравнении с известными ранее алгоритмами, и анализ сложности может предоставить такие доводы.

Слово «сложность» в этом контексте является математическим термином, а не общим обозначением препятствия к выполнению замысла. С понятием сложности связываются затраты времени или памяти, соответствующие худшему случаю, либо затраты в среднем; при этом, чтобы обсуждать худший или «средний» случай, нужно, прежде всего, договориться, как определяется *размер входа* (размер входных данных) алгоритма и в чем измеряются *затраты* при работе алгоритма над фиксированным входом.

Часто размер входа определяют как общее число символов в представлении входа, но возможны и другие пути. В задачах сортировки и поиска размер входа — это, как правило, количество элементов n входного массива, в задачах на графах — число вершин $|V|$ или число ребер $|E|$ входного графа $G = (V, E)$, но, с другой стороны, $|V|$ и $|E|$ могут рассматриваться и совместно как две *компоненты размера* входа. В арифметических задачах размером входа может быть, например, максимум абсолютных величин входных целых чисел, или количество цифр в двоичной записи этого максимума, или же, как уже говорилось, суммарное количество двоичных цифр всех входных чисел и т. д. — выбор делается в зависимости от характера задачи.

Для алгоритмов сортировки соответствующие затраты времени достаточно полно характеризуются количеством сравнений и перемещений элементов массива. Для алгоритмов на графах учитываемые затраты могут складываться, например, из операций над матрицей смежности или над массивом списков смежных вершин данного графа и из некоторых дополнительных вычислительных операций. Для арифметических алгоритмов в качестве объема затрат может быть взято количество всех выполняемых арифметических операций, или,

как альтернатива, лишь наиболее дорогих операций (например, мультипликативных — умножений и делений); более тщательный анализ требует рассмотрения количества *битовых* операций. Если алгоритм является *рандомизированным* (содержит обращения к генератору случайных чисел с известным распределением), то затраты, вообще говоря, не определяются однозначно входом алгоритма, но зависят от полученных случайных чисел. Можно рассматривать усредненные затраты для каждого конкретного входа; такие затраты уже будут функцией входа.

При фиксированном значении размера входа сами входы алгоритма могут варьироваться, при этом меняются и затраты; алгоритм может быть охарактеризован наибольшими или средними (в соответствии с распределением вероятностей на входах данного размера) затратами. В обоих случаях сложность алгоритма — это функция размера входа или, соответственно, нескольких компонент размера входа. Для этого понятия иногда используется термин *вычислительная сложность*, чтобы избежать путаницы с описательной (в другой терминологии — изобразительной) сложностью, которая тем или иным способом определяется исходя из самой записи (текста) алгоритма; одним из видов описательной сложности является длина записи алгоритма, и именно так понятие сложности трактуется в некоторых теориях¹. Мы будем рассматривать только вычислительную сложность, называя ее просто сложностью.

Понятие сложности является математическим уточнением довольно расплывчатого термина «трудоемкость», с помощью которого, наряду с не более ясными «быстродействием» и «эффективностью», иногда пытаются характеризовать алгоритмы. Принятие в качестве сложности именно затрат в среднем или затрат, соответствующих худшему (а не, скажем, лучшему) случаю, помогает оценить достаточность имеющихся вычислительных ресурсов для выполнения алгоритма.

Итак, сложность является функцией числового, чаще всего целого, аргумента, иногда — нескольких таких аргументов. Теория сложности изучает эти функции, сопоставленные как отдельным алгоритмам, так и классам алгоритмов решения некоторой задачи. При рассмотрении класса алгоритмов возникает семейство функций, для которого могут обсуждаться вопросы о нахождении *нижней границы*, о существовании минимальной функции этого семейства (если такая функция существует, то соответствующий ей алгоритм — *оптимальный* в рассматриваемом классе) и т. д. Важным является также

¹ См., например, статью «Алгоритма сложность» в [58].

вопрос о существовании в данном классе алгоритмов, нацеленных на решение конкретной задачи, алгоритма такого, сложность которого растет с увеличением размера входа не слишком быстро, например, остается ограниченной некоторым полиномом, вид которого не оговаривается (такой алгоритм принято называть *полиномиальным*).

При исследовании существования алгоритма решения задачи, имеющего «не очень высокую» сложность, важную роль играет *сводимость* какой-то задачи к другой — из возможности быстро решить некоторую задачу может следовать такая же возможность для ряда других, и наоборот, невозможность быстрого решения какой-то одной задачи может автоматически повлечь такую же невозможность для ряда других задач.

Сложности многих алгоритмов трудно или вообще нельзя представить в простом «замкнутом» виде; помимо этого, точное значение сложности алгоритма для каждого конкретного значения размера входа часто не представляет особого интереса, актуальным же является исследование роста сложности при возрастании размера входа (особый интерес к исходным данным большого размера оправдан тем, что на них «захлебываются» тривиальные алгоритмы). Поэтому в теории сложности широко используется *асимптотическое* оценивание. Однако сравнение сложностей различных алгоритмов на основе асимптотических оценок этих сложностей возможно не для всех типов таких оценок.

Этот круг вопросов, наряду с рассмотрением общих достаточно элементарных подходов и методов, которые могут оказаться полезными при сложностном анализе алгоритмов, составляет содержание предлагаемого курса теории сложности алгоритмов.

Часто используемые обозначения

$C_A^T(x)$, $C_A^S(x)$ — временные и пространственные затраты алгоритма A для входа (входных данных) x ;

$\|x\|$ — размер входа x ;

$T_A(s)$, $S_A(s)$ — значения временной и пространственной сложности алгоритма A для значения s размера входа;

$\lambda(n)$ — количество цифр в двоичной записи неотрицательного целого n (битовая длина n);

$\lambda^*(n)$ — количество единиц в двоичной записи неотрицательного целого n ;

$\lfloor a \rfloor$ — наибольшее целое, меньшее или равное вещественному a ;

$\lceil a \rceil$ — наименьшее целое, большее или равное вещественному a ;

$\mathbb{N}$, $\mathbb{N}^+$ — множества неотрицательных и положительных целых чисел;

$\mathbb{Z}$ — множество (кольцо) целых чисел;

$\mathbb{Z}_k$ — кольцо вычетов по модулю k ;

$\mathbb{Q}$, $\mathbb{R}$, $\mathbb{C}$ — множества (поля) рациональных, вещественных и комплексных чисел;

$m \mid n$ — делимость целого n на целое m ;

Π_n — множество всех перестановок чисел $1, 2, \dots, n$;

$\square$ — конец доказательства.

Сложности алгоритмов как функции числовых аргументов. Сложность в худшем случае

§ 1. Затраты алгоритма для данного входа. Сложность по числу операций

Пусть по входным данным (входу) x алгоритм A вычисляет результат (выход) y . Такое вычисление связано с затратами времени и памяти. Допустим, что достигнуто соглашение о том, как измеряются эти затраты, тогда можно рассмотреть функции затрат

$$C_A^T(x), \quad C_A^S(x),$$

где верхний индекс T указывает на временные затраты¹, S — на пространственные затраты (затраты памяти и пространственные затраты — это синонимы; пространственные затраты на хранение x не включаются в $C_A^S(x)$), соответствующие вычислениям, связанным с применением A к входу x ; буквы T и S возникают из английских слов *time* — время и *space* — пространство. Вид этих функций может быть очень непростым; их трудно исследовать методами математического анализа, потому что аргумент x не является, вообще говоря, числом и не принадлежит какому-либо метрическому пространству. Можно ввести некоторую неотрицательную числовую характеристику $\|x\|$ возможных входов алгоритма и оценивать функции затрат с помощью функций, зависящих не от x , а от $\|x\|$:

$$C_A^T(x) \leq \varphi(\|x\|), \quad C_A^S(x) \leq \psi(\|x\|). \quad (1.1)$$

Если φ и ψ не слишком быстро растут при возрастании (числового) аргумента, то эти оценки могут обнадеживать автора или потребителя алгоритма A .

¹ Здесь и далее имеется в виду временные (связанные с расходом времени), а не временные (непостоянные, краткосрочные) затраты. Аналогично — временная сложность и т. д.

Избираемую нами величину $\|\cdot\|$ принято называть *размером входа*. В соответствии с нашим замыслом размер входа должен характеризовать «громоздкость» исходных данных; то, что $\|x\|$ является числом, позволяет говорить о росте φ, ψ и исследовать этот рост. Выбор размера входа — существенный этап оценивания затрат; нужна, во всяком случае, уверенность, что оценки вида (1.1) будут нести полезную информацию.

В дальнейшем, однако, предметом нашего изучения будут не функции затрат, а некоторые другие функции, о которых мы скажем после ряда примеров. Предварительно условимся о некоторых обычных для литературы по сложности алгоритмов обозначениях. Пусть $a \in \mathbb{R}$, т. е. a — вещественное число. Значение $\lfloor a \rfloor$ есть результат округления a до целого в меньшую сторону: $\lfloor 3,14 \rfloor = 3$, $\lfloor -3,14 \rfloor = -4$ (эту величину — целую часть a — записывают и как $[a]$). Соответственно, $\lceil a \rceil$ есть результат округления a до целого в большую сторону: $\lceil 3,14 \rceil = 4$, $\lceil -3,14 \rceil = -3$. Если a — целое, то $\lfloor a \rfloor = \lceil a \rceil = a$.

Рассмотрим три простых примера.

Пример 1.1. Исходя из определения произведения матриц, мы получаем алгоритм умножения двух квадратных матриц порядка n , требующий n^3 операций умножения. Этот алгоритм далее будет обозначаться буквами ММ, от английских слов *matrix multiplication* — матричное умножение.

Пример 1.2. Один из очевидных алгоритмов проверки простоты данного целого $n \geq 2$, который мы будем называть алгоритмом пробных делений, состоит в выяснении того, имеется ли среди чисел $2, 3, \dots, \lfloor \sqrt{n} \rfloor$ хотя бы одно, делящее n ; легко заметить, что $d \leq \lfloor \sqrt{n} \rfloor$ для целого $d \geq 2$, если и только если $\frac{n}{d} \geq d$. Количество проверок делимости n на те или иные целые числа (для краткости можно говорить о «числе делений») колеблется от 0 до $\lfloor \sqrt{n} \rfloor - 1$. Этот алгоритм далее будет обозначаться буквами ТД, от английских слов *trial divisions* — пробные деления.

Пример 1.3. При сортировке простыми вставками число сравнений¹ колеблется от $n - 1$ до $\frac{n(n-1)}{2}$, в зависимости от входного

¹ Основные алгоритмы сортировки и поиска приводятся в приложении А. Обсуждая конкретные алгоритмы сортировки, мы для краткости иногда будем опускать само слово «алгоритм», говоря, например, не об алгоритме сортировки простыми вставками, а просто о сортировке простыми вставками и т. д. Все сортировки, которые мы рассматриваем, являются сортировками с помощью сравнений, т. е. никаких других операций, кроме сравнений и перемещений (присваива-

массива $x_1, x_2, \dots, x_n$. Если интересоваться числом перемещений элементов (операций обмена $\leftrightarrow$), то оно колеблется от 0 до $\frac{n(n-1)}{2}$. Эта сортировка далее будет обозначаться буквой I, от английского insert — вставка.

В этих примерах уже фактически сказано, что считается размером входа и затратами. В примере 1.1 размер входа — целое неотрицательное n — это порядок матриц. В примере 1.2 размер входа — само входное число (сам вход) n . В примерах 1.1, 1.2 количество исследуемых операций однозначно определяется по n . Столь жесткая зависимость затрат от размера входа может и не иметь места, как это видно из примера 1.3, где размер входа — это длина n массива. В связи с этим примером остановимся пока на операциях сравнения. Количество сравнений $\frac{n(n-1)}{2}$, требуемое в худшем случае, характеризует рассматриваемый алгоритм среди всех алгоритмов сортировки как «имеющий квадратичную сложность». Для придания этим словам точного смысла должно быть, прежде всего, определено само понятие сложности.

Определение 1.1. Пусть на возможных входах x алгоритма A определена неотрицательная числовая функция $\|x\|$ (размер входа). Пусть также определены целочисленные неотрицательные функции $C_A^T(x)$, $C_A^S(x)$ временных и пространственных затрат алгоритма A . Тогда *временной и пространственной сложностями* алгоритма A называются функции числового аргумента

$$T_A(n) = \max_{\|x\|=n} C_A^T(x), \quad S_A(n) = \max_{\|x\|=n} C_A^S(x) \quad (1.2)$$

(областью изменения n как аргумента функций $T_A(n)$ и $S_A(n)$ является множество значений размера $\|\cdot\|$). Более полно каждая такая сложность именуется сложностью *в худшем случае*.

Два примечания к определению 1.1.

1) Значение $C_A^T(x)$ есть, как правило, число каких-то операций, а $C_A^S(x)$ — число хранимых величин какого-то типа (затраты на хранение исходных данных не учитываются), поэтому предположение о целочисленности значений этих функций вполне естественно. Без этого

ний или обменов), над элементами массива производиться не может, — исключение составляет пример 29.1. Элементы сортируемых массивов считаются попарно различными. Если не оговорено противное, то речь всегда идет о сортировке по возрастанию. Размером входа каждой из сортировок мы без специального упоминания считаем число n элементов массива (это число мы называем также *длиной* массива).

предположения в (1.2) пришлось бы использовать $\sup$ вместо $\max$, что сделало бы многие рассуждения о сложности более тяжеловесными.

2) Видимо, вместо «сложность в худшем случае» правильнее было бы сказать «сложность как затраты в худшем случае», но такова принятая здесь терминология. Мы пока (до § 5) не рассматриваем сложность в среднем, и поэтому, не опасаясь путаницы, будем называть сложность в худшем случае просто сложностью.

Вернемся к алгоритмам ММ, ТД, I из примеров 1.1, 1.2, 1.3. Рассмотрим вначале временные сложности $T_{\text{ММ}}(n)$, $T_{\text{ТД}}(n)$, $T_I(n)$. Получаем $T_{\text{ММ}}(n) = n^3$ и $T_I(n) = \frac{n(n-1)}{2}$. Для $T_{\text{ТД}}(n)$ у нас нет столь простой формулы. Заметим, например, что для значений n от 111 до 120 мы имеем

n :	111	112	113	114	115	116	117	118	119	120
$T_{\text{ТД}}(n)$:	2	1	9	1	4	1	2	1	6	1

Изменение этой функции при $n \rightarrow \infty$ можно охарактеризовать так: для любых n значение этой функции не превосходит $\lfloor \sqrt{n} \rfloor - 1$, и найдутся сколь угодно большие n , для которых ее значение равно $\lfloor \sqrt{n} \rfloor - 1$.

В примерах 1.2 и 1.3 не возникает необходимости в хранении больших объемов величин сверх объема входа (сортировка простыми вставками не требует дополнительного массива). Дополнительная память, если и нужна, — это несколько дополнительных переменных (ячеек), используемых алгоритмом.

Поэтому можно считать, что $S_I(n)$, $S_{\text{ТД}}(n)$ ограничены константами. В примере 1.1 нам требуется дополнительная матрица, в которой будет формироваться результат; имеем $S_{\text{ММ}}(n) = n^2$.

Во всех этих примерах мы, как легко заметить, принимаем во внимание лишь часть затрат и в соответствии с этим определяем сложность алгоритма. В примере 1.1 учитывались только умножения как более дорогие (доминирующие по затратности) операции. В примере 1.3 мы рассматриваем либо сравнения, либо обмены. И во всех примерах мы игнорировали затраты по выполнению управляющих операторов, хотя, скажем, при выполнении простейшего оператора цикла

for $i = 1$ to n do ... od

тратится время на изменение параметра цикла, проверку условия продолжения цикла и т. д.

Обычно в таких ситуациях считается, что мы учитываем основную часть затрат для худшего случая. Если пытаться точно подсчитывать

все, не упуская ничего, то получение и обоснование оценки сложности заурядного алгоритма может превратиться в очень тяжелую работу. Принимая решение о том, в чем измеряются временные затраты, мы разделяем важное и неважное. Разумеется, при этом необходима осторожность, без которой к разряду неучитываемых «дешевых» операций могут быть отнесены такие, на выполнение которых в реальных вычислениях уйдет существенное время из-за их огромного количества.

Отметим еще, что, вообще говоря, время выполнения операции зависит от размеров ее операндов, это относится и к арифметическим операциям, и к операции сравнения (например, длинные числа сравнивать труднее, чем короткие и т.д.). Аналогично дело обстоит с пространственными затратами и соответственно с пространственной сложностью; здесь, прежде чем находить сложность, надо решить, что является «единицей хранения»: скажем, при умножении двух матриц мы можем получить матрицу с элементами, которые записываются более длинно, чем элементы исходных матриц. Однако без оговорок считается, что результат операции всегда уместается в одной ячейке.

Иногда приходится отвлекаться от того, что десятичная или двоичная запись иррационального числа не может быть размещена в ячейке какого-либо устройства. Более того, в теории сложности как разделе алгебры¹ рассматривают, например, алгоритмы, применимые к элементам произвольного абстрактного кольца или поля, и вопрос о представлении этих элементов в реальном компьютере или в абстрактной машине не обсуждается вообще.

Определение 1.2. Пусть функция $C_A^T(x)$ в определении 1.1 отражает затраты на выполнение лишь какого-то одного типа операций в предположении, что все эти операции требуют одинакового времени выполнения, не зависящего от вида и размера операндов, и это время равно 1. Тогда соответствующая функция $T_A(n)$ — это сложность *по числу операций* рассматриваемого типа. Привлечение в качестве $C_A^S(x)$ функции, отражающей только количество хранимых дополнительных величин того или иного фиксированного типа, приводит к пространственной сложности $S_A(n)$ *по числу величин* рассматриваемого типа.

Такую сложность называют сложностью по числу операций или числу величин рассматриваемого типа, в предположении о равноза-

¹ См., например, [35].

тратности всех рассматриваемых операций и равнообъемности всех учитываемых величин. Сложность арифметических алгоритмов по числу умножений и делений называют *мультипликативной* сложностью. Из основных арифметических операций умножение и деление являются наиболее дорогими, и при рассмотрении арифметических алгоритмов часто интересуются именно мультипликативной сложностью.

Несколько позднее мы будем говорить о *битовой* сложности, которая позволяет принимать в расчет различие в размерах операндов и рассматривать при нахождении сложности непохожие операции, сопоставляя их затратность на битовом уровне.

Формулы (1.2) вновь заставляют вспомнить о важности понятия размера входа. В силу разных причин не всегда этот размер легко и естественно может быть введен так, что сложность алгоритма возрастает при увеличении размера входа. Но, по крайней мере, функция $\|\cdot\|$ должна обеспечивать определенность значений $T_A(n)$, $S_A(n)$ для всех достаточно больших n — иначе было бы невозможно говорить о поведении $T_A(n)$, $S_A(n)$ при $n \rightarrow \infty$.

Заведомо неудачный размер входа — это, скажем, число строк n данной матрицы, если речь идет об алгоритмах вычисления произведения всех элементов и если матрица не обязательно является квадратной: при выбранном таким образом размере входа мы имеем тождественное равенство $T_A(n) = \infty$ для сложности по числу умножений любого такого алгоритма A , так как при фиксированном числе строк число столбцов и число элементов могут быть сколь угодно большими.

Другие возможные затруднения, вызванные неудачным выбором размера входа, будут обсуждаться в § 13.

Достаточно очевидно, что если алгоритм A состоит в последовательном (друг за другом) выполнении алгоритмов U и V , то мы, вообще говоря, не можем утверждать, что $T_A(n) = T_U(n) + T_V(n)$, так как, например, размер входа алгоритма U может существенно отличаться, как в большую, так и в меньшую сторону, от размера его выхода.

Пусть A и B — некоторые алгоритмы. Как можно заметить, из выполнения для всех n неравенства $T_A(n) < T_B(n)$ не следует, что $C_A^T(x) < C_B^T(x)$ для всех возможных входов x (достаточно вспомнить сортировку слияниями и пузырьковую сортировку; последняя выполняется быстрее первой, коль скоро массив задан уже в упорядоченном виде). Аналогично дело обстоит с пространственными сложностью и затратами.

В некоторых случаях алгоритму сопоставляют несколько сложностей. Итоговые временные затраты сортировки массива с помощью сравнений складываются из затрат на сравнение и на перемещение элементов. Без учета типа элементов массива нельзя сказать, какая из операций является более дорогой. Поэтому для алгоритмов сортировки используются две сложности: с одной стороны — по числу сравнений, а с другой — по числу перемещений элементов, т. е. присваиваний ($:=$) или обменов ($\leftrightarrow$). Если же рассматривается некоторый арифметический алгоритм, то, исследовав его мультипликативную сложность, можно дополнительно интересоваться, в качестве уточнения, *аддитивной* сложностью (числом сложений и вычитаний в худшем случае) и т. д.

Пусть некоторому алгоритму A сопоставлены две, скажем, временные сложности $T'_A(n)$ и $T''_A(n)$. Например, $T'_A(n)$ и $T''_A(n)$ могут быть сложностями по разным операциям. Если операции первого и второго вида предполагаются имеющими одинаковую затратность, то это еще не дает оснований считать, что сложность $\tilde{T}_A(n)$, которую мы определим, рассматривая совместно операции обоих видов, будет равна $T'_A(n) + T''_A(n)$, потому что максимумы этих двух видов затрат могут достигаться на разных входах одного и того же размера. Можно только утверждать, что

$$\tilde{T}_A(n) \leq T'_A(n) + T''_A(n).$$

Пример 1.4. Чтобы проиллюстрировать указанное обстоятельство, мы вновь обратимся к сортировке простыми вставками, которая может рассматриваться в двух вариантах I_1 и I_2 : для вставки элемента x_{i+1} в уже упорядоченную часть $x_1, x_2, \dots, x_i$ элементы этой упорядоченной части просматриваются либо в порядке $x_i, x_{i-1}, \dots$, либо в порядке $x_1, x_2, \dots$. В первом варианте максимум числа сравнений достигается тогда, когда входной массив имеет обратную к требуемой упорядоченности: $x_1 > x_2 > \dots > x_n$, и на этом же входе достигается максимум числа обменов; следовательно, имеем $\tilde{T}_{I_1}(n) = T'_{I_1}(n) + T''_{I_1}(n) = n(n-1)$, где $T'_{I_1}(n), T''_{I_1}(n)$ суть сложности по числу сравнений и обменов.

Во втором варианте максимум числа сравнений достигается, когда массив уже упорядочен так, как требуется: $x_1 < x_2 < \dots < x_n$, а максимум числа обменов — когда $x_1 > x_2 > \dots > x_n$. Если $i > 1$, то при вставке элемента x_{i+1} в уже упорядоченную часть $x_1, x_2, \dots, x_i$ потребуется тем меньше обменов, чем больше потребуется сравнений. Если элемент займет место с номером k , то это потребует $i - k + 1$ обменов. Число

же сравнений равно k , если $k \leq i$, и равно $k - 1$, если $k = i + 1$. Суммарное число сравнений и обменов равно $i + 1$, если $k \leq i$, и равно i , если $k = i + 1$. Поэтому максимум общего числа сравнений и обменов достигается, например, на входном массиве, имеющем обратную к требуемой упорядоченность: $x_1 > x_2 > \dots > x_n$. Легко устанавливается, что $\tilde{T}_{I_2}(n) = \sum_{i=2}^n (i + 1) = \frac{(n + 3)(n - 1)}{2}$.

Рассматривая сложности $\tilde{T}_{I_1}(n)$, $\tilde{T}_{I_2}(n)$ первого и второго вариантов сортировки простыми вставками по общему числу сравнений и обменов, мы имеем

$$\tilde{T}_{I_1}(n) = n(n - 1), \quad \tilde{T}_{I_2} = \frac{(n + 3)(n - 1)}{2}, \quad (1.3)$$

и, таким образом,

$$\tilde{T}_{I_1}(n) / \tilde{T}_{I_2}(n) \rightarrow 2 \quad (1.4)$$

при возрастании n .

Обладающий большей общей сложностью $\tilde{T}_{I_1}$ первый вариант алгоритма рассматривается в литературе значительно чаще второго, не в последнюю очередь из-за того, что его запись как программы несколько проще.

Заметим попутно, что среди сортировок с квадратичной сложностью по числу сравнений чрезвычайно низкой сложностью по числу перемещений, а именно сложностью, равной $n - 1$, обладает сортировка выбором.

В некоторых случаях к учитываемым операциям относят операции весьма разнородные, и время выполнения каждой из них считают равным 1 — обсуждаемый далее пример 3.1, как и некоторые другие, даст соответствующую иллюстрацию. При нахождении каждой такой «смешанной» сложности затраты для каждого конкретного входа учитываются разом все вместе.

Этот параграф завершим обсуждением тех предположений о выполнении рассматриваемых алгоритмов, которые неявно уже использовались нами. Мы предполагали, что вычисления проводятся некоторой машиной, память которой состоит из ячеек. При этом как объем каждой ячейки, так и общее число ячеек считается бесконечным, что, разумеется, является чистой абстракцией. Поместив в ячейку результат каких-то вычислений или считав содержимое некоторой ячейки, машина обращается еще к какой-то ячейке и т. д. Относительно этого процесса предполагается, что сам переход от ячейки к ячейке не связан с ка-

кими-либо затратами. Этот принцип лежит в основе модели вычислений, называемой РАМ («равнодоступная адресная машина» — довольно неуклюжий, но принятый термин; имеется в виду машина с равнодоступными ячейками памяти)¹. Модель РАМ существенно отличается в этом смысле от другой известной модели вычислений — машины Тьюринга (МТ), где время перехода от одной ячейки ленты к другой зависит от расстояния между ячейками; при этом одна ячейка ленты МТ содержит один символ фиксированного конечного алфавита².

Употребление слова «модель» в этом контексте подчеркивает то обстоятельство, что реальные вычислительные устройства (компьютеры) работают на тех или иных специфических принципах, но при исследовании вычислительной сложности алгоритмов этой спецификой можно до известного предела пренебрегать и исходить из упрощающей системы допущений.

Далее мы будем еще обращаться к понятию модели вычислений; всюду до главы 5 без оговорок подразумевается модель РАМ.

§ 2. Асимптотические оценки: формализм

Для характеристики роста сложности по числу сравнений сортировки простыми вставками первого и второго типа мы можем использовать известный из математического анализа символ O и написать $T(n) = O(n^2)$ (здесь и далее $n \rightarrow \infty$). Мы можем также выделить главные по росту слагаемые в (1.3):

$$\tilde{T}_1(n) = n^2 + O(n), \quad \tilde{T}_2(n) = \frac{1}{2}n^2 + O(n), \quad (2.1)$$

хотя в этом и нет ощутимого практического смысла в силу простоты функций $\tilde{T}_1(n)$, $\tilde{T}_2(n)$. В то же время, как это хорошо известно, асимптотическая формула $f(n) = O(g(n))$ является удобным средством оценивания нетривиально устроенной функции $f(n)$ с помощью более простой функции $g(n)$; столь же полезными оказываются и оценки вида $f(n) = o(g(n))$. Но когда мы говорим, что сортировка выбором, пузырьковая сортировка и сортировка простыми вставками имеют

¹ В литературе на английском языке — RAM (random-access machine).

² Раньше в литературе по вычислительной математике и программированию слово «машина» употреблялось как обозначение некоторого реального вычислительного устройства; устойчивым словосочетанием было, например, «электронная вычислительная машина» (ЭВМ). Сейчас в этом значении в основном употребляется слово «компьютер», а слово «машина» часто используется при обсуждении абстрактных вычислительных устройств: машин Тьюринга, равнодоступных адресных машин и т. д.

квадратичные сложности, мы имеем в виду не только то, что соответствующие сложности допускают оценку $O(n^2)$, но что эти сложности являются величинами порядка n^2 ; в математическом анализе это иногда записывается как $T(n) \asymp n^2$, где $T(n)$ — рассматриваемая функция, в данном случае — сложность. В последние годы в теории сложности алгоритмов вместо $f(n) \asymp g(n)$ стали писать $f(n) = \Theta(g(n))$.

Определение 2.1. Функции $f(n)$ и $g(n)$ имеют одинаковый порядок (пишут $f(n) = \Theta(g(n))$), если найдутся положительные c_1, c_2, N такие, что неравенства

$$c_1 |g(n)| \leq |f(n)| \leq c_2 |g(n)| \quad (2.2)$$

выполнены для всех $n > N$.

Без труда проверяется, что отношение «иметь одинаковый порядок» является отношением эквивалентности на множестве функций, определенных для всех достаточно больших значений n (в нашем случае эти значения целые). Несимметричность записи $f(n) = \Theta(g(n))$ в сравнении с записью $f(n) \asymp g(n)$ ($f(n)$ и $g(n)$ как бы не равноправны в первой записи, хотя имеем дело с отношением эквивалентности) объясняется тем, что обычно эту запись используют, когда $g(n)$ проще, чем $f(n)$.

Итак, для сложности $T(n)$ по числу сравнений для любой из упомянутых сортировок (кроме упомянутой вскользь сортировки слияниями) мы имеем $T(n) = \Theta(n^2)$. Это более сильное утверждение, чем $T(n) = O(n^2)$, так как $T(n) = O(n^2)$ является лишь асимптотической верхней оценкой: в соответствии с известным из математического анализа определением

$$f(n) = O(g(n)) \iff \exists_{c, N > 0} \forall_{n > N} |f(n)| \leq c |g(n)|,$$

например, $n = O(n^2)$, но неверно, что $n = \Theta(n^2)$. Здесь и далее, пользуясь кванторами $\exists, \forall$, мы записываем связываемые ими переменные, равно как и условия, определяющие множества значений этих переменных, в виде индексных выражений при кванторах. Это часто позволяет обходиться без дополнительных скобок и облегчает чтение формул.

Иногда бывают полезными нижние асимптотические оценки.

Определение 2.2. Мы пишем $f(n) = \Omega(g(n))$, если найдутся положительные c, N такие, что для всех $n > N$ выполнено $|f(n)| \geq c |g(n)|$.

Следующее предложение выводится из определений символов O, Ω и Θ .

Предложение 2.1. Соотношение $f(n) = \Theta(g(n))$ имеет место тогда и только тогда, когда одновременно $f(n) = O(g(n))$ и $f(n) = \Omega(g(n))$; помимо этого, $f(n) = \Omega(g(n))$ тогда и только тогда, когда $g(n) = O(f(n))$.

Если размер входа является целым положительным числом, то возникающие функции являются последовательностями. Для единообразия мы, как правило, будем говорить о функциях, подразумевая, но не упоминая специально, что каждая такая функция определена лишь для целых положительных значений аргумента (возможно даже, только для достаточно больших целых положительных значений аргумента). Итак, при $n \rightarrow \infty$ оценки вида $f(n) = \Lambda(g(n))$, где Λ — один из символов Ω , O , Θ , предполагают, что функции $f(n)$, $g(n)$ определены для всех достаточно больших n . Соответствующее неравенство из числа

$$|f(n)| \geq c_1 |g(n)|, \quad |f(n)| \leq c_2 |g(n)|, \quad c_1 |g(n)| \leq |f(n)| \leq c_2 |g(n)| \quad (2.3)$$

тоже, в соответствии с определением, должно выполняться лишь для n , больших некоторого N . Заметим, однако, что если $f(n)$ и $g(n)$ определены для всех $n \in \mathbb{N}^+$ и принимают при $1 \leq n \leq N$ ненулевые значения, то можно считать, что соответствующее неравенство из перечисленных в (2.3) выполняется для всех n , так как, положив

$$m = \min_{1 \leq n \leq N} \frac{|f(n)|}{|g(n)|}, \quad M = \max_{1 \leq n \leq N} \frac{|f(n)|}{|g(n)|},$$

мы можем заменить c_1, c_2 в (2.3) на $c'_1 = \min\{c_1, m\}$, $c'_2 = \max\{c_2, M\}$. Это замечание в некоторых случаях будет для нас полезным.

Вернемся к примеру 1.2. Для сложности алгоритма пробных делений было бы ошибкой утверждать, что его сложность по числу делений есть $\Theta(\sqrt{n})$. Но оценка $O(\sqrt{n})$, разумеется, верна и, более того, является точной в смысле следующего определения.

Определение 2.3. Если имеет место оценка $f(n) = O(g(n))$, то она называется *точной*, коль скоро существует неограниченно возрастающая последовательность неотрицательных целых чисел $\{n_k\}$ такая, что для $\varphi(k) = f(n_k)$, $\psi(k) = g(n_k)$ имеет место $\varphi(k) = \Omega(\psi(k))$.

Для упомянутых $\varphi(k)$ и $\psi(k)$ в силу этого определения и семантики символа Θ выполнено $\varphi(k) = \Theta(\psi(k))$.

При рассмотрении алгоритма пробных делений для доказательства точности оценки $O(\sqrt{n})$ можно взять n_k равным k -му простому числу, $k = 1, 2, \dots$

Понятие точности оценки вида $f(n) = O(g(n))$ можно определить также с помощью знакомого из математического анализа символа o ; напомним, что $u(n) = o(v(n))$ при $n \rightarrow \infty$, коль скоро $u(n) = \alpha(n)v(n)$ и $\lim_{n \rightarrow \infty} \alpha(n) = 0$.

Предложение 2.2. Пусть $f(n) = O(g(n))$. Эта оценка является точной, если и только если неверно, что $f(n) = o(g(n))$.

Доказательство. Пусть оценка является точной, и $\{n_k\}$ — возрастающая последовательность, о которой говорится в определении 2.3. Тогда существует положительная константа c такая, что $|f(n_k)| \geq c|g(n_k)|$, $k = 1, 2, \dots$, и соотношение $f(n) = o(g(n))$ места не имеет. Обратно, если неверно, что $f(n) = o(g(n))$, то по определению символа o существуют $\varepsilon > 0$ и возрастающая последовательность $\{n_k\}$ натуральных чисел такие, что $|f(n_k)| \geq \varepsilon|g(n_k)|$, $k = 1, 2, \dots$. Если при этом выполнена оценка $f(n) = O(g(n))$, то эта оценка точна в соответствии с определением 2.3. $\square$

Для рассматриваемой сложности алгоритма пробных делений не верна, скажем, оценка $O(\log n)$, потому что для этой сложности оценка $O(\sqrt{n})$ является точной и в то же время $\log n = o(\sqrt{n})$.

Нелишним будет заметить, что сложность алгоритма пробных делений допускает оценки $O(n)$, $O(n^5)$, $O(n \log n)$ и т.д., хотя, разумеется, эти оценки являются более грубыми в сравнении с $O(\sqrt{n})$. Еще раз подчеркнем, что оценка $f(n) = O(g(n))$ есть асимптотическая *верхняя* оценка, равно как оценка $f(n) = \Omega(g(n))$ — асимптотическая *нижняя*¹. Как, например, из $l < 5$ и $m < 100$ нельзя вывести, что $l < m$, так и из $f(n) = O(n^2)$, $g(n) = O(n^3)$ нельзя вывести, что хотя бы для достаточно больших n выполняется $f(n) < g(n)$. Оценка вида $T_A(n) = O(g(n))$ (или $S_A(n) = O(g(n))$) подходит для того, чтобы «похвалить» алгоритм A , т.е. охарактеризовать его сложность как достаточно низкую (речь идет лишь об оценках вида $O(g(n))$, а не о более тонких оценках, включающих символ O и имеющих вид, подобный (2.1)), но не для того, чтобы «раскритиковать» его — для таких целей скорее подойдет оценка вида $T_A(n) = \Omega(h(n))$. Зная, например, что сложность по числу обменов для сортировки выбором есть $O(n)$, а для сорти-

¹ В книге [10] отмечено, что положение с символом O схоже с тем, которое возникнет, если кто-нибудь «вместо слов „меньше чем“ начнет писать $= M$, например, так: $3 = M(5)$. На вопрос: „Что значит $M(5)$?“ — он должен ответить: „Нечто, что меньше, чем 5“. Таким образом, он быстро привыкает читать M как „нечто, что меньше, чем“, приближаясь к тем самым словам, которые употребляем мы, вводя соотношение $f(s) = O(\varphi(s))$ ».

ровки простыми вставками — $\Omega(n^2)$, мы обоснованно заключаем, что для достаточно больших n первая сложность меньше второй.

Оценки вида $T_A(n) = \Theta(g(n))$, соединяющие в себе оценки $T_A(n) = O(g(n))$ и $T_A(n) = \Omega(g(n))$, в соответствующих ситуациях подходят и для характеристики сложности как сравнительно низкой, и, наоборот, как сравнительно высокой¹.

При всем этом, иногда можно услышать сообщения о новых алгоритмах, сопровождаемые рассуждениями в духе следующего (подразумевается, что n — размер входа): «Лучший из известных ранее алгоритмов решения этой задачи требует $O(n^3)$ операций, а предлагаемый нами алгоритм — лишь $O(n)$. Таким образом, достигнуто улучшение на два порядка по числу операций». Но информация, содержащаяся в первой из этих двух фраз, не дает достаточных оснований для сделанного заключения. Более того, на основе этой информации вообще нельзя сказать, какой из двух алгоритмов — известный ранее или новый — требует меньше операций при больших n , ведь речь идет лишь об оценках сверху, и возможно, что первая из них может быть улучшена.

Если про оценку $O(g(n))$ известно, что она точная, то это расширяет возможности ее использования. Допустим, что нам известен алгоритм распознавания простоты числа n , имеющий мультипликативную сложность $O(\log^d n)$ при некотором $d > 0$. Тогда мультипликативная сложность этого алгоритма для бесконечного множества значений n (но, может быть, не для всех n) будет меньше, чем мультипликативная сложность алгоритма пробных делений, и для этого вывода достаточно того, что сложность алгоритма пробных делений допускает точную оценку $O(\sqrt{n})$.

Утверждение, что $f(n)$ и $g(n)$ асимптотически эквивалентны, записываемое как $f(n) \sim g(n)$, означает, как известно, что $f(n) = g(n) + o(g(n)) = g(n)(1 + o(1))$. Утверждение, что $f(n) \sim g(n)$, является, очевидно, более сильным, чем утверждение, что $f(n) = \Theta(g(n))$. Заметим кстати, что из формул (2.1) следует

$$\tilde{T}_{1_1}(n) \sim n^2, \quad \tilde{T}_{1_2}(n) \sim \frac{1}{2}n^2 \quad (2.4)$$

¹ Θ - и Ω -нотации вошли в литературу по сложности алгоритмов с появлением статьи Д. Кнута [77], в которой автор, в частности, пишет о бессмысленности нижних оценок вида $O(f(n))$ и о невозможности использования оценок такого рода как оценок сложности при сравнении алгоритмов. В [77] отмечается также, что Ω -нотация использовалась ранее в работах Э. Ч. Титчмарша, известного математика первой половины XX века.

(например, $\tilde{T}_1(n) = n^2 + O(n) = n^2 \left(1 + O\left(\frac{1}{n}\right)\right) = n^2(1 + o(1))$), но не наоборот. Из (2.4) следует только, что

$$\tilde{T}_1(n) = n^2 + o(n^2), \quad \tilde{T}_2(n) = \frac{1}{2}n^2 + o(n^2),$$

при этом из $v(n) = o(n^2)$ не следует, что $v(n) = O(n)$, что доказывается примером $v(n) = n^{3/2}$.

Слова « $f(n)$ имеет асимптотику $g(n)$ » означают, что $f(n) \sim g(n)$; например, $\tilde{T}_1(n)$ имеет асимптотику n^2 , а $\tilde{T}_2(n)$ имеет асимптотику $\frac{1}{2}n^2$.

Иногда используются *комбинированные* размеры входа, имеющие несколько компонент, т. е. пары чисел, тройки, четверки и т. д. Функция затрат и сложность в этом случае имеют несколько аргументов. Здесь по-прежнему могут использоваться, например, асимптотические оценки вида $\Theta(g(n_1, n_2))$, где под знаком Θ расположена функция двух переменных n_1, n_2 , причем $n_1, n_2 \rightarrow \infty$; определение Θ легко модифицируется на случай двух и большего числа переменных:

$$f(n_1, n_2) = \Theta(g(n_1, n_2)) \iff \iff \exists_{c_1, c_2, N > 0} \forall_{n_1, n_2 > N} c_1 |g(n_1, n_2)| \leq |f(n_1, n_2)| \leq c_2 |g(n_1, n_2)|. \quad (2.5)$$

То же самое с Ω , O и o . При этом, если имеет место оценка $f(n_1, n_2) = O(g(n_1, n_2))$, то мы назовем ее точной, коль скоро неверно, что $f(n_1, n_2) = o(g(n_1, n_2))$.

Сложности многих алгоритмов трудно или невозможно представить элементарного вида функциями от размера входа. Помимо этого, точное значение сложности алгоритма для каждого конкретного значения размера входа часто не представляет особого интереса, актуальным же является исследование роста сложности при возрастании размера входа. Поэтому асимптотическое оценивание широко используется в исследовании сложности алгоритмов.

§ 3. Асимптотические оценки: два примера

Если мы изначально имеем эскизное описание алгоритма, не содержащее мелких деталей, но полностью отражающее его идею, то уже этого эскиза может быть достаточно для получения некоторой информативной асимптотической оценки сложности; проработка деталей алгоритма будет влиять на скрытые за символами O , Ω , Θ значения констант.

Пример 3.1. Займемся задачей построения *выпуклой оболочки* конечного множества M точек координатной плоскости, т. е. наименьшего выпуклого многоугольника H , содержащего все множество M (рис. 1). Множество M задается массивом координат принадлежащих ему точек; требуется построить массив координат вершин многоугольника H при обходе этого многоугольника, начиная с какой-нибудь его вершины, например, против часовой стрелки¹.

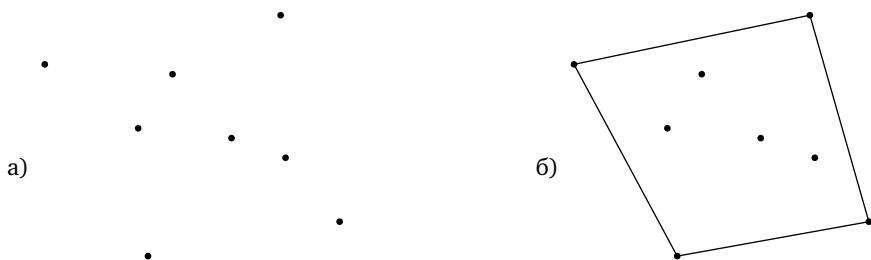


Рис. 1. а) Конечное множество M точек плоскости; б) выпуклая оболочка множества M .

Пусть n — число элементов множества M , будем считать это число размером входа. Алгоритм, основанный на переборе всех подмножеств множества M с проверкой для каждого из них, является ли оно множеством вершин искомого многоугольника H , имеет очень высокую сложность $\Omega(2^n)$. Обсудим идею значительно более экономного алгоритма Р.Л.Грэхема (этот алгоритм мы обозначим буквой G).

Можно довольно быстро найти среди точек множества M такую, которая обязательно будет одной из вершин многоугольника H : достаточно выбрать в M точку P с наименьшей ординатой, а если таких точек несколько, то из этих нескольких взять ту, которая имеет наименьшую абсциссу. Дополнительно найдем точку O , которая принадлежит многоугольнику H , но не совпадает ни с одной из его вершин: возьмем для этого какие-нибудь две точки из M и найдем середину соединяющего их отрезка (если впоследствии вдруг окажется, что эта точка принадлежит M , то можно будет удалить ее из M , так как она заведомо не является вершиной H).

¹ «Наглядно можно представлять себе дело так: в точках M вбиты гвозди, на которые натянута резинка, охватывающая их все, — эта резинка и будет выпуклой оболочкой множества гвоздей» [28]. Но в нашем понимании построение выпуклой оболочки предполагает еще перечисление вершин в порядке их обхода.

Используя какую-нибудь сортировку с помощью сравнений, все точки множества M можно упорядочить по возрастанию углов между отрезком OP и отрезками, соединяющими O с точками множества M , при этом мы считаем, что величина каждого угла принадлежит полуинтервалу $[0, 2\pi)$. Если вдруг обнаружится, что два каких-то угла равны, то упорядочим соответствующие точки по удаленности от O , но для краткости будем говорить просто о сортировке по величине угла. Соединив точки в этом порядке (будем обозначать их $P_1, P_2, \dots, P_n$, при этом $P_1 = P$), и соединив дополнительно P_n с P_1 , мы получим замкнутую несамопересекающуюся ломаную, но ограниченный этой ломаной многоугольник может содержать вмятины и не быть выпуклым (см. рис. 2а). Тогда среди вершин $P_2, P_3, \dots, P_n$ найдется хотя бы одна, скажем P_k , *вмятая*, которая принадлежит треугольнику $P_{k-1}OP_{k+1}$ при $k < n$ и треугольнику $P_{n-1}OP_1$ при $k = n$ (рис. 2б). Вмятую вершину можно исключить из дальнейшего рассмотрения, соединив напрямую P_{k-1} с P_{k+1} , или, соответственно, P_1 с P_{n-1} . Удалив все вмятые вершины, мы получим требуемый многоугольник. Такова общая идея алгоритма. Задержимся на удалении вмятых вершин.

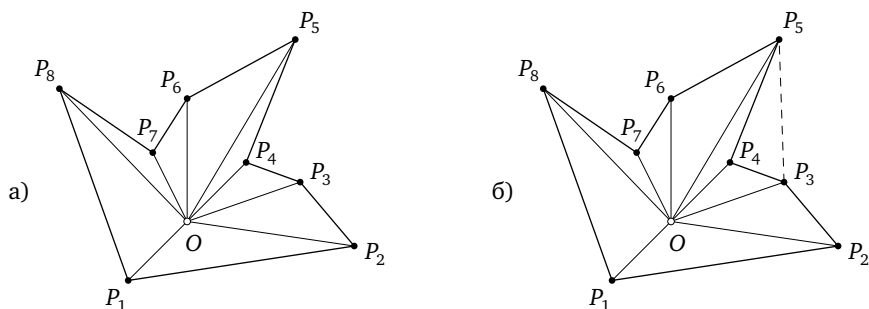


Рис. 2. а) Точки, упорядоченные по величине угла $\angle P_1OP_i$, $i = 1, 2, \dots, n$; б) вершина P_4 — первая вмятая вершина в последовательности $P_2, P_3, \dots, P_8$.

Вмятые вершины можно обнаружить просмотром точек $P_2, P_3, \dots, P_n, P_1$: переходя от вершины P_i к вершине P_{i+1} , $i = 2, 3, \dots, n-1$, можно сразу проверять, принадлежит ли P_i треугольнику $P_{i-1}OP_{i+1}$, а при переходе от P_n к P_1 проверять, принадлежит ли P_n треугольнику $P_{n-1}OP_1$. Если да, то P_i или соответственно P_n удаляется, но после этого надо проверить, не окажется ли теперь вмятой предыдущая из неудаленных вершин, — на рис. 2б видно, что после удаления P_4 вершина P_3 становится вмятой. Возможно, что удаление одной

вмятой вершины повлечет удаление нескольких уже рассмотренных вершин, но вершина P_1 никогда не будет удалена — она изначально выбиралась такой. При $i < n - 1$ после P_{i+1} мы рассматриваем P_{i+2} и вновь пытаемся освободиться от вмятых вершин с меньшими номерами и т. д. Последний шаг — переход от P_n к P_1 и завершающая попытка освободиться от вмятых вершин.

Затраты этапа построения точек P и O ограничены значением $c_1 n$, где c_1 — некоторая константа.

Если используется сортировка, сложность которой по числу сравнений есть $r(n)$, то в алгоритме Грэхема может потребоваться не более $c_2 r(n)$ операций для сортировки точек по величине угла, константа c_2 отражает затраты на сравнение двух углов и сравнение расстояний от O до двух данных точек.

Покажем, что описанный процесс удаления вмятых вершин потребует затрат, не превосходящих по величине $c_3 n$, где c_3 — некоторая константа (в частности, учитывающая затраты на проверку принадлежности точки треугольнику). В самом деле, если переход от P_i к P_{i+1} сопровождается проверкой вмятости некоторого числа v_i вершин, то число удаленных при этом вершин равно $v_i - 1$. Но общее число удаленных вершин меньше n . Поэтому $\sum_{i=2}^n (v_i - 1) < n$, и, как следствие,

$$\sum_{i=2}^n v_i < 2n. \quad (3.1)$$

Это означает, что сложность $T_G(n)$ алгоритма Грэхема по общему числу арифметических операций и сравнений не превосходит $c' r(n) + c'' n$, где c' и c'' суть некоторые положительные константы. Сложность любой сортировки массивов длины n по числу сравнений не может быть меньше $n/2$, так как каждый элемент должен пройти хотя бы одно сравнение и в каждом сравнении участвуют два элемента. Имеем

$$T_G(n) \leq c' r(n) + c'' n = r(n) \left(c' + c'' \frac{n}{r(n)} \right) \leq r(n) (c' + 2c''),$$

откуда $T_G(n) = O(r(n))$. С другой стороны, точки изначально заданного множества могут иметь любое расположение, и сортировка может потребовать $r(n)$ сравнений. Поэтому $T_G(n) = \Theta(r(n))$.

После того как точки упорядочены по величине угла, информацию об их координатах можно представить в виде двунаправленного списка¹, и тогда удаление вмятых вершин не будет связано с ка-

¹ См. [14, с. 251].

кими-либо перемещениями координат точек, и, подходя формально, можно было бы затраты на перемещения в процессе удаления вмятых вершин считать равными нулю. При менее формальном подходе эти затраты можно считать ограниченными сверху значением sp , где s — некоторая константа: переход от массива к двунаправленному списку и, в силу (3.1), работа со ссылками во время удаления вмятых вершин потребуют затрат, ограниченных величинами такого вида. В свою очередь, сложность любой сортировки по числу перемещений элементов не может быть меньше чем $n/2$ (так как не исключено, например, что изначальный порядок элементов является обратным к требуемому). Отсюда сложность алгоритма Грэхема по числу перемещений не превосходит произведения некоторой константы и сложности используемой сортировки по числу перемещений. Аналогично, для сложности по общему числу арифметических операций, сравнений и перемещений мы имеем оценку $\Theta(s(n))$, где $s(n)$ — соответствующая сложность используемой сортировки.

Основываясь на эскизном описании алгоритма Грэхема, мы получили следующее.

Для любой сортировки массивов длины n , имеющей некоторую сложность $s(n)$ по общему числу сравнений и перемещений элементов, существует алгоритм построения выпуклой оболочки n точек, заданных массивом своих координат на плоскости, сложность которого по общему числу арифметических операций, сравнений и перемещений есть $\Theta(s(n))$.

Если предполагать, что пространственная сложность используемой сортировки допускает оценку $O(n)$, то пространственная сложность алгоритма Грэхема допускает оценку $\Theta(n)$ (напомним, что от исходного массива мы переходим к двунаправленному списку).

Завершающее этот пример замечание. Алгоритм Грэхема использует сравнения и арифметические операции. Может ли какая-то сортировка числовых массивов с помощью сравнений и арифметических операций иметь меньшую сложность по общему числу операций, чем любая из сортировок, использующих только сравнения? Отрицательный ответ на этот вопрос обоснован в § 29.

Пример 3.2. Пусть $G = (V, E)$ — ориентированный граф без кратных ребер и $v \in V$. Вояжем по G , выходящим из вершины v , будем называть любой путь, который

- начинается в вершине v ,
- не проходит ни по одному из ребер дважды,

- завершается в вершине, из которой не выходит ни одного непройденного ребра

(вояж не обязательно охватывает все ребра G). Примером выходящего из вершины 1 вояжа в изображенном на рис. 3 графе служит $(1, 2, 2, 3, 1, 4)$.

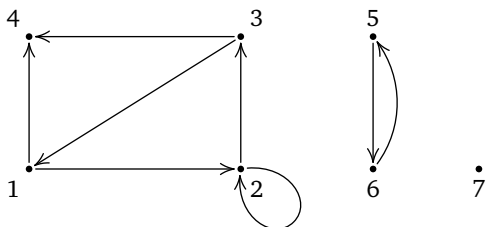


Рис. 3. Ориентированный граф.

Построение какого-то одного выходящего из данной вершины v вояжа может быть выполнено произвольным блужданием по непройденным ребрам графа, завершающимся в вершине, из которой не выходит непройденных ребер. Систематизация блужданий может состоять в том, что, попав в некоторую вершину, мы выбираем для продолжения пути ребро, ведущее в вершину с наименьшим доступным номером.

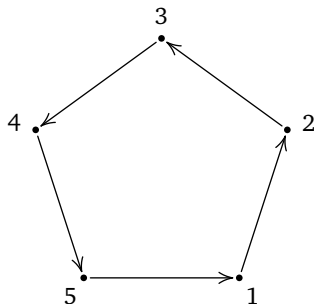


Рис. 4. Граф в виде кольца (любой вояж охватывает все ребра).

Определяемый этим алгоритмом вояж может захватить и все ребра — например, когда граф имеет вид кольца (на рис. 4 изображен такой граф, для которого $|E| = |V| = 5$). Входом алгоритма является граф G и вершина v . Если мы рассматриваем $|E|$ как размер входа, то для сложности любого алгоритма построения вояжа обязательно

имеет место нижняя оценка $\Omega(|E|)$ — ввиду, например, того, что для любого фиксированного $|E|$ существует граф, имеющий вид кольца. Возникает вопрос, можно ли обосновать верхнюю оценку $O(|E|)$? Если да, то в итоге это дало бы оценку $\Theta(|E|)$.

Ответ на вопрос о $O(|E|)$ зависит от объема затрат, связанных с решениями о включении ребер в вояж. Как будет показано, эти затраты существенно зависят от выбранного способа представления исходного графа.

Для представления графов, не имеющих кратных ребер, часто используются *матрицы смежности*. Матрица смежности — квадратная матрица порядка $|V|$, состоящая из нулей и единиц (фактически, булева матрица), в которой элемент с индексами i, j равен единице, если и только если из i -й вершины выходит ребро в j -ю вершину. Такой способ представления действительно удобен во многих задачах. Но когда речь идет о блужданиях по графу, при которых однажды пройденное ребро изымается из дальнейшего рассмотрения, более удобным оказывается представление графа в виде массива $a_1, a_2, \dots, a_{|V|}$ списков смежных вершин. В списке a_i , $i = 1, 2, \dots, |V|$, в порядке возрастания содержатся номера вершин, в которые входят ребра, выходящие из вершины с номером i . Для графа, приведенного на рис. 3, имеем

$$a_1 = (2, 4), \quad a_2 = (2, 3), \quad a_3 = (1, 4), \quad a_4 = (), \quad a_5 = (6), \quad a_6 = (5), \quad a_7 = ();$$

для графа в виде кольца —

$$a_1 = (2), \quad a_2 = (3), \quad \dots, \quad a_{|V|-1} = (|V|), \quad a_{|V|} = (1)$$

(в этом графе $|V| = |E|$). Представление в виде массива списков смежных вершин возможно и для графов, имеющих кратные ребра; в этом случае номера вершин в каждом списке располагаются в порядке неубывания.

Начав вояж из вершины с номером v , мы смотрим, не пуст ли список a_v : если он пуст, то вояж закончен. Если же список a_v не пуст, переносим первый его элемент (пусть это будет, например, i) в список вершин, через которые шаг за шагом проходит вояж; затем рассматриваем список a_i , если он пуст, то вояж закончен, иначе повторяем все то же самое, что проделывали в предыдущей вершине, и т. д.:

```

l := cons(v, nil); i := v;
while not null(ai) do i := car(ai); l := cons(i, l); ai := cdr(ai)

```

od

Мы здесь использовали операции над списками, применяемые в языке Лисп: *car* — первый элемент списка, *cdr* — список после удаления первого его элемента, *cons* — вставка элемента в начало списка, *null* — предикат проверки пустоты списка, *nil* — обозначение пустого списка.

Затраты, связанные с удлинением пути на один шаг и с проверкой, не завершен ли вояж, ограничены сверху константой (эти затраты суть четыре операции над списками), длина всего пути не превосходит $|E|$, и затраты во всех случаях не превосходят произведения некоторой константы на $|E|$.

Список l содержит в обратном порядке все вершины, через которые последовательно проходит вояж. Если желателен прямой порядок, то надо перевернуть l :

```
l' := nil;
while not null(l) do l' := cons(car(l), l'); l := cdr(l) od
```

Построение списка l' потребует некоторых дополнительных операций над списками, число этих операций не превзойдет произведения некоторой константы c на длину списка l , но эта длина не превосходит $|E|$, следовательно, число операций на этом этапе не превзойдет $c|E|$.

Естественно считать пространственные затраты на хранение списка с числовыми элементами пропорциональными длине списка. В случае, например, графа в виде кольца каждый из списков l и l' имеет длину $|E| + 1$.

Обозначим буквами VL описанный алгоритм построения вояжа, предполагая, что исходный ориентированный граф задается массивом списков смежных вершин. Из сказанного следует, что для временных затрат алгоритма VL мы имеем

$$C_{VL}^T(G, v) \leq c|E|, \quad (3.2)$$

где c — некоторая константа. Это неравенство выполнено для любого имеющего $|E|$ ребер графа. Поэтому если значение $|E|$ принято за размер входа (напомним, что сам вход — это граф и выделенная в нем вершина), то из (3.2) будет следовать $T_{VL}(|E|) \leq c|E|$, откуда

$$T_{VL}(|E|) = O(|E|). \quad (3.3)$$

Вместе с указанной ранее оценкой $\Omega(|E|)$ оценка (3.3) дает

$$T_{VL}(|E|) = \Theta(|E|).$$

Нетрудно вывести аналогичную оценку и для пространственной сложности.

Если массивы списков смежных вершин используются в качестве представления ориентированных графов, то построение начинающегося с заданной вершины v вояжа в графе $G = (V, E)$ возможно с помощью алгоритма, который при выборе числа $|E|$ ребер в качестве размера входа имеет сложность $\Theta(|E|)$ по числу операций над списками. Эта же оценка имеет место для пространственной сложности, если считать, что пространственные затраты на хранение списка с числовыми элементами пропорциональны длине этого списка.

Если бы граф представлялся матрицей смежности, то мы для используемого алгоритма не смогли бы получить оценку сложности $\Theta(|E|)$, так как при блуждании по графу вход в каждую вершину сопровождается проверкой, есть ли непройденное до сих пор ребро, выходящее из этой вершины. Например, для графа в виде кольца при $|E| = |V| = n$ мы имеем матрицу смежности порядка n :

$$\begin{pmatrix} 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & \dots & 1 \\ 1 & 0 & 0 & \dots & 0 \end{pmatrix}.$$

Начиная вояж из первой вершины, мы потратим $2 + 3 + \dots + n + 1 = \Omega(n^2)$ сравнений элементов матрицы с единицей при поиске первых подходящих вершин.

Разумеется, базовые операции над списками и операции сравнения элементов матрицы смежности с единицей различаются по затратности (сравнение с единицей — это очень дешевая операция). Но какими бы ни были положительные c' и c'' , отражающие затраты на операцию сравнения с 1 и, соответственно, на операцию над списком, неравенство $c'|E|^2 > c''|E|$ будет иметь место для всех достаточно больших $|E|$. Константы, скрытые в оценках $\Omega(|E|^2)$ и $O(|E|)$, найденных для сложностей алгоритма при использовании двух разных способов представления графа, различаются между собой. Но тем не менее мы можем сравнивать рост первой и второй сложностей на основе этих оценок. Этот пример показателен и в том отношении, что «дешевых» операций может быть слишком много (их число может слишком быстро расти при увеличении размера входа) и их игнорирование может дать неправильное представление о реальной временной сложности алгоритма.

Если рассматривать две компоненты $|V|, |E|$ размера входа, то сложностью алгоритма VL будет функция $T'_{VL}(|V|, |E|)$ двух перемен-

ных. Для того чтобы устанавливать для нее асимптотические оценки, эта функция должна быть определена для всех (или, по крайней мере, всех достаточно больших) натуральных значений $|V|, |E|$. Это значит, что для всех таких $|V|, |E|$ должен существовать граф, имеющий $|V|$ вершин и $|E|$ ребер. Проблема будет снята, если, например, допустить к рассмотрению и такие ориентированные графы, которые имеют кратные ребра. Оценка $T'_{VL}(|V|, |E|) = O(|E|)$ будет, очевидно, выполняться. Можно обосновать и оценку $T'_{VL}(|V|, |E|) = \Omega(|E|)$: достаточно рассмотреть имеющий $|E|$ ребер граф в виде цветка (рис. 5),

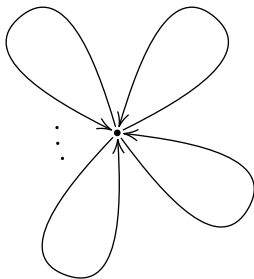


Рис. 5. Граф с одной вершиной и произвольным заданным числом ребер.

добавив к нему $|V| - 1$ изолированных вершин (подобных вершине 7 на рис. 3) и взяв в качестве v вершину в центре «цветка». В итоге $T'_{VL}(|V|, |E|) = \Theta(|E|)$.

§ 4. Длина числа как возможный размер входа

Часто, хотя и не всегда, для алгоритмов целочисленной арифметики, входом которых является целое неотрицательное число n , размером входа выбирают не само n , а его *битовую длину*, или, иными словами, количество $\lambda(n)$ цифр в его двоичной записи:

$$\lambda(n) = \begin{cases} 1, & \text{если } n = 0, \\ \lfloor \log_2 n \rfloor + 1, & \text{если } n > 0. \end{cases}$$

Во второй строчке этого определения можно заменить $\lfloor \log_2 n \rfloor + 1$ на $\lceil \log_2(n+1) \rceil$ — см. задачу 3.

Пример 4.1. Вернемся к алгоритму пробных делений (пример 1.2). Наряду с его уже рассмотренной сложностью $T_{TD}(n)$ введем еще одну сложность (также по числу делений) $T_{TD}^*(m)$, $m = \lambda(n)$. Легко обнаружить, что эта новая функция ведет себя не так, как функция $T_{TD}(n)$,

для которой характерны резкие скачки. Для начальных значений m имеем

m :	2	3	4	5	6	7
n :	2—3	4—7	8—15	16—31	32—63	64—127
$\hat{n}$:	3	7	13	31	61	127
$T_{\text{TD}}^*(m)$:	1	1	2	4	6	10

(в строке m указывается битовая длина входа; в строке n указывается диапазон, в котором располагаются значения n , битовая длина m которых равна числу в предыдущей строке; в строке $\hat{n}$ указывается одно из тех чисел этого диапазона, на которых достигается максимум числа делений; в последней строке указано значение сложности $T_{\text{TD}}^*(m)$). В поведении $T_{\text{TD}}^*(m)$ не видно резких скачков. Переход от размера $\|n\| = n$ к размеру $\|n\|^* = m = \lambda(n)$ влечет более основательное преобразование функции $T(n)$, чем простая замена переменной. Теперь одним и тем же размером m обладают несколько входов n , затраты для которых, вообще говоря, различны, и, в соответствии с определением сложности, мы должны брать максимум этих затрат. Данным размером m обладают все целые n такие, что

$$2^{m-1} \leq n < 2^m. \quad (4.1)$$

Согласно постулату Бертрана, при $m > 1$ в таком диапазоне обязательно встретится простое число.

Постулат Бертрана.¹ При любом целом $N > 1$ имеется простое число, принадлежащее интервалу $(N, 2N)$.

С помощью постулата Бертрана мы можем доказать, что $T_{\text{TD}}^*(m) = \Theta(2^{m/2})$. В самом деле, обозначим через p_m наибольшее простое число битовой длины m . Количество делений, требующееся алгоритму для работы с p_m , будет равно $\lfloor \sqrt{p_m} \rfloor - 1$. В силу (4.1) имеем $\lfloor 2^{(m-1)/2} \rfloor - 1 \leq \lfloor \sqrt{p_m} \rfloor - 1$. Таким образом,

$$T_{\text{TD}}^*(m) \geq \lfloor 2^{(m-1)/2} \rfloor - 1 > \frac{1}{\sqrt{2}} 2^{m/2} - 2 = \left(\frac{1}{\sqrt{2}} - \frac{1}{2^{m/2-1}} \right) 2^{m/2},$$

и при $m \geq 4$ имеем

$$T_{\text{TD}}^*(m) > \left(\frac{1}{\sqrt{2}} - \frac{1}{2} \right) 2^{m/2}.$$

¹ Сформулирован Ж. Бертраном в 1845 г. и доказан П. Л. Чебышевым в 1852 г. Обсуждение доказательства выходит за рамки этого курса, доказательство Чебышева см. в [55] (статья «О простых числах»). Доказательства, использующие лишь элементарные средства, можно найти в [50] (приложение «Доказательство постулата Бертрана (теоремы Чебышева)») и в [16, § 5].

В то же время, в силу (4.1) очевидно, что для всех m

$$T_{\text{TD}}^*(m) < 2^{m/2} - 1 < 2^{m/2}.$$

Отсюда следует требуемое.

Исследуем общий вопрос о возможности перехода от оценок $T_A^*(m)$ к оценкам $T_A(n)$ и наоборот для какого-либо алгоритма A , считая, что эти две сложности соответствуют одному и тому же способу учета затрат, но разным размерам входа — $\|\cdot\|$ и $\|\cdot\|^*$, принимающим значения в $\mathbb{N}^+$, причем если $\|x\| = n$ для некоторого входа x , то $\|x\|^* = m = \lambda(n) = \lfloor \log_2 n \rfloor + 1$. В таком случае, очевидно, выполнено

$$T_A^*(m) = \max_{2^{m-1} \leq n < 2^m} T_A(n). \quad (4.2)$$

Ясно, что $T_A(n)$ несет более полную информацию о рассматриваемом алгоритме A , чем $T_A^*(m)$: значения $T_A^*(m)$, $m = 1, 2, \dots$, образуют подпоследовательность последовательности $T_A(n)$, $n = 1, 2, \dots$. Поэтому естественно, что переход от оценок для $T_A^*(m)$ к оценкам для $T_A(n)$ приводит к более грубому результату, чем тот, который может быть получен при изначальном рассмотрении размера $\|x\| = n$.

Довольно типична ситуация, когда от оценки через логарифм делается переход к оценке через $m = \lambda(n)$. Полезно простое следствие соотношений (4.1):

$$m - 1 \leq \log_2 n < m. \quad (4.3)$$

Пусть для алгоритма A , вход которого — целое неотрицательное n , размером входа первоначально считается само n , и пусть, например, $T_A(n) = O(\log^3 n)$, т. е. для некоторой фиксированной константы c и всех достаточно больших n выполнено $T_A(n) < c \log_2^3 n$. Приняв теперь за размер входа $m = \lambda(n)$, получаем, что для всех достаточно больших m и каждого такого неотрицательного целого n_0 , что $\lambda(n_0) = m$, в силу правого неравенства из (4.3) выполняется $C_A^T(n_0) < c \log_2^3 n_0 < cm^3$. Но это значит, что и для максимума $T_A^*(m)$ затрат алгоритма A на входах размера m выполнено это неравенство. Отсюда $T_A^*(m) = O(m^3)$. Подобные переходы от $\log n$ к m возможны и для использующих Ω асимптотических оценок. Наряду с левым неравенством из (4.3) используется то, что при $c < 1$ для всех достаточно больших m выполнено $m - 1 > cm$ (например, $m - 1 > \frac{1}{2}m$ для всех $m > 2$). В содержащих Θ асимптотических оценках переход от логарифма к m обосновывается тем, что Θ соединяет в себе O и Ω . Сказанное легко применяется, например, к оценкам, в которых под символом O, Ω, Θ располагается $\log^k n$. Более общая ситуация рассматривается в задаче 33.

Пример 4.2. Идея бинарного алгоритма возведения a в целую неотрицательную степень n , называемого также алгоритмом повторного возведения в квадрат (мы будем обозначать его буквами RS, от английского названия алгоритма repeated squaring — повторное возведение в квадрат), состоит в том, что если двоичная запись n есть $\beta_k \dots \beta_1 \beta_0$, то вычисление a^n может быть сведено к вычислению последовательности значений $q_i = a^{(2^i)}$, $i = 0, 1, \dots, k$ (каждый следующий элемент последовательности получаем возведением в квадрат предыдущего), и подсчету произведения u тех q_i , для которых $\beta_i = 1$. Например, $a^{11} = a^8 \cdot a^2 \cdot a$ (5 умножений), так как $(11)_{10} = (1011)_2$.

Займемся сложностью по числу умножений этого алгоритма. Используя $\lambda(n)$ для обозначения битовой длины n и обозначение $\lambda^*(n)$ для числа единиц в двоичной записи n , мы легко получаем следующее.

Если рассматривать n как размер входа бинарного алгоритма вычисления a^n , $n \in \mathbb{N}^+$, то сложность $T_{RS}(n)$ по числу умножений для этого алгоритма равна $\lambda(n) + \lambda^*(n) - 2$. Если в качестве размера входа рассматривать $m = \lambda(n)$, то сложность $T_{RS}^*(m)$ по числу умножений равна $2m - 2$.

Для $T_{RS}^*(m)$ и $T_{RS}(n)$ имеем очевидные асимптотические оценки

$$T_{RS}^*(m) = \Theta(m), \quad T_{RS}(n) = \Theta(\log n).$$

Заметим, что, не принимая в расчет детали алгоритма RS, мы не можем вывести из $T_{RS}^*(m) = 2m - 2$ равенство $T_{RS}(n) = \lambda(n) + \lambda^*(n) - 2$, хотя обратный вывод очевиден.

Вполне допустимо не указывать основание логарифма в асимптотических равенствах, в которых переход к другому основанию не нарушает этого равенства. В равенстве $T_{RS}(n) = \Theta(\log n)$ мы не указываем основание логарифма, так как переход от одного основания к другому — это умножение логарифма на ненулевую константу, при этом очевидно, что если $f(n) = \Theta(g(n))$ и $c \neq 0$, то $f(n) = \Theta(c \cdot g(n))$, аналогично дело обстоит с символами O, Ω, o . При желании можно считать, что $\log n$ в подобных формулах — это, например, $\log_2 n$. Не следует, конечно, забывать, что во многих случаях основание логарифма существенно, и, например, неверно, что $3^{\log_2 n} = \Theta(3^{\log_4 n})$.

В рассмотренном алгоритме вычисления a^n предполагается, что показатель степени n задан как массив двоичных цифр. Если показатель n задан как число, то его двоичные цифры $\beta_0, \beta_1, \dots$ могут быть получены одна за другой нахождением соответствующих частных и остатков от деления на 2. Это не изменит оценок $\Theta(\log n)$, $\Theta(m)$ как для общего числа операций, так и для числа мультипликации

тивных операций. Но при этом бинарный алгоритм возведения в степень может быть использован, например, для вычисления A^n , где A — матрица большого порядка и т.д. В этом смысле обсуждаемые операции умножения естественно подсчитывать отдельно и именно их считать составляющими главные затраты алгоритма.

Задачу вычисления a^n в некотором множестве с ассоциативным умножением (т. е. в полугруппе) часто формулируют как задачу об *аддитивных цепочках* для n , т. е. о наборах целых чисел $n_1 < n_2 < \dots < n_k$, в которых

- $n_1 = 1$, $n_k = n$;
- для каждого i , $1 < i \leq k$, найдутся s, t такие, что $1 \leq t \leq s < i$ и $n_t + n_s = n_i$.

Каждая аддитивная цепочка для n задает способ вычисления a^n . Например, аддитивная цепочка 1, 2, 3, 4, 8, 11 для 11 задает тот способ, который соответствует бинарному алгоритму возведения в степень. Задача быстрого возведения в степень n с помощью умножений — это фактически задача построения короткой аддитивной цепочки для n .

Обозначение $l(n)$ закреплено за длиной самой короткой аддитивной цепочки для n . Бинарный алгоритм возведения в степень использует свою специфическую аддитивную цепочку (назовем ее бинарной). Бинарная цепочка не для всех n имеет длину $l(n)$ (см. задачу 28). Но, как мы выясним несколько позже, длина бинарной цепочки и $l(n)$ имеют одинаковый порядок при $n \rightarrow \infty$. Помимо этого бинарные цепочки легко и быстро строятся, чего в общем случае нельзя сказать об аддитивных цепочках длины $l(n)$.

Некоторые предположения относительно функции $l(n)$ до настоящего времени не доказаны, хотя и подтверждены проверкой для многих n . К числу таковых относится, например, гипотеза Шольца—Брауэра: $l(2^n - 1) \leq n - 1 + l(n)$ для всех $n > 0$.¹

Задачи

1. Для любого $n \in \mathbb{Z}$ выполнено $n = \left\lfloor \frac{n}{2} \right\rfloor + \left\lceil \frac{n}{2} \right\rceil$, для любого $a \in \mathbb{R}$ выполнено $\lceil a \rceil = -\lfloor -a \rfloor$.

2. Пусть $x \in \mathbb{R}$. Положим $\{x\} = x - \lfloor x \rfloor$ (дробная доля или дробная часть числа x ; например, $\{3,14\} = 0,14$, $\{-3,14\} = 0,86$). Тогда зна-

¹ Подробнее об аддитивных цепочках см. в [26, разд. 4.6.3].

чение $\frac{1}{2} - \left| \{x\} - \frac{1}{2} \right|$ равно расстоянию от x до ближайшего к нему целого числа на числовой прямой.

3. Для любых $k, n \in \mathbb{N}^+, k > 1$, выполнено $\lfloor \log_k n \rfloor + 1 = \lceil \log_k(n+1) \rceil$.

Указание. Пусть $m \in \mathbb{N}^+$ таково, что $k^{m-1} \leq n < k^m$, тогда $k^{m-1} < n+1 \leq k^m$. Прологарифмировать по основанию k обе системы неравенств.

4. Предположим, что в нашем распоряжении нет операции обмена $a \leftrightarrow b$ и мы заменяем ее тремя присваиваниями:

$$c := a; \quad a := b; \quad b := c;$$

чему в этом случае равны сложности первого и второго варианта сортировки простыми вставками по суммарному числу сравнений и присваиваний? Сохраняется ли соотношение (1.4)?

5. Пусть полином $f(n) = a_m n^m + \dots + a_1 n + a_0$ имеет ненулевой старший коэффициент a_m . Тогда

а) $f(n) = O(n^k) \Leftrightarrow k \geq m$,

б) $f(n) = \Omega(n^k) \Leftrightarrow k \leq m$,

в) $f(n) = \Theta(n^k) \Leftrightarrow k = m$, при $k = m$ также выполнено $f(n) \sim a_m n^m$.

6. Следует ли из $f(n) = \Omega(g(n))$, что $g(n) = o(f(n))$?

7. Пусть $P(n)$ — количество простых множителей целого числа $n > 1$ с учетом кратности. Имеет место точная оценка $P(n) = O(\log n)$.

8. (Продолжение предыдущей задачи.) Пусть

$$P^*(m) = \max_{2^{m-1} \leq n < 2^m} P(n),$$

$m = 2, 3, \dots$ Верно ли, что $P^*(m) = \Theta(m)$?

9. Указать все вещественные значения δ , при которых справедлива оценка

а) $T_{\text{TD}}(n) = O(n^\delta)$,

б) $T_{\text{TD}}(n) = \Omega(n^\delta)$,

в) $T_{\text{TD}}(n) = \Theta(n^\delta)$,

где $T_{\text{TD}}(n)$ — сложность алгоритма пробных делений (пример 1.2).

10. Пусть $T_{\text{TD}}(n)$ — сложность алгоритма пробных делений для проверки простоты числа n . Верно ли, что $T_{\text{TD}}(n) = \Omega(n^{1/3})$?

11. Пусть $T_{\text{TD}}^*(m)$ — сложность алгоритма пробных делений для проверки простоты числа n , $m = \lambda(n)$. Верно ли, что $T_{\text{TD}}^*(m) = \Omega(2^{m/3})$?

12. Железнодорожный сортировочный узел устроен так, как показано на рис. 6. На правой стороне собрано некоторое число вагонов двух типов (на рис. 6 — черные и белые), по n штук каждого типа. Тупик может вместить все $2n$ вагонов. Требуется, пользуясь тремя сортировочными операциями В, ИЗ, МИМО, собрать вагоны на левой стороне

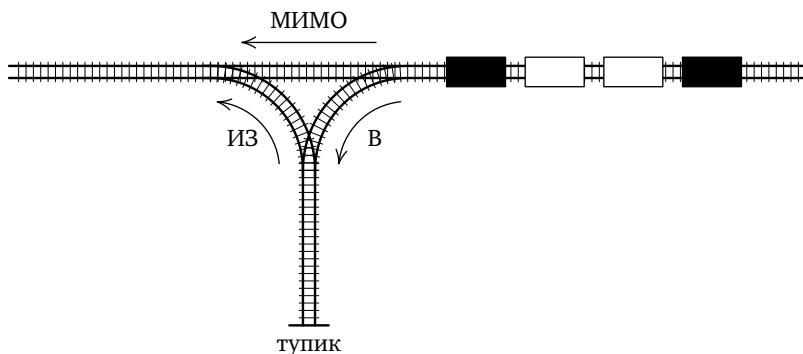


Рис. 6. Сортировочный узел.

так, чтобы типы вагонов чередовались. Указать такой алгоритм решения этой задачи, сложность которого по числу сортировочных операций при рассмотрении n в качестве размера входа равнялась бы $3n - 1$.

13. Хорошо известно, что мультипликативная сложность метода Гаусса решения системы n линейных уравнений с n неизвестными допускает оценки

$$1) O(n^3), \quad 2) \frac{1}{3}n^3 + O(n^2), \quad 3) \Theta(n^3).$$

а) Из какой оценки (указать номер) следуют две остальные?

б) Можно ли из приведенных оценок выбрать такую, которая является следствием любой из остальных?

в) Является ли оценка $\Omega(n^3)$ следствием какой-либо из оценок 1, 2, 3?

14. Верно ли, что $f(n) = \Theta(g(n))$, если и только если одновременно $f(n) = O(g(n))$ и $g(n) = O(f(n))$?

15. В приложении В выведена асимптотическая оценка (В.3) суммы $\sum_{k=1}^n \sqrt{k}$. Из (В.3) для этой же суммы, очевидно, следует оценка $\Theta(n^{3/2})$.

а) Для каких $\alpha \in \mathbb{R}$ выполнено $\sum_{k=1}^n \lfloor \sqrt{k} \rfloor = \Theta(n^\alpha)$?

б) Найти точную формулу для суммы $\sum_{k=1}^n \lfloor \sqrt{k} \rfloor$.

Указание. б) Пусть k — целое положительное число. Тогда

$$\lfloor \sqrt{k^2} \rfloor = \lfloor \sqrt{k^2 + 1} \rfloor = \dots = \lfloor \sqrt{(k+1)^2 - 1} \rfloor = k, \quad (k+1)^2 - k^2 = 2k + 1.$$

16. Пусть $T_A(n)$ и $T'_A(n)$ — сложности алгоритма A по двум видам используемых операций, $T''_A(n)$ — сложность по суммарному числу операций обоих видов. Какие из следующих соотношений обязательно выполняются:

а) $T_A(n) + T'_A(n) = T''_A(n)$,

б) $T_A(n) + T'_A(n) \leq T''_A(n)$,

в) $T_A(n) + T'_A(n) \geq T''_A(n)$,

г) $T_A(n) + T'_A(n) \neq T''_A(n)$?

17. Пусть A — некоторый алгоритм, входом которого является целое неотрицательное число x . Пусть $T_A(x)$ и $T_A^*(m)$ — сложности этого алгоритма по одному и тому же виду операций, но для T_A в качестве размера входа берется само x , а для T_A^* — битовая длина x (т.е. $m = \lambda(x)$). Может ли для какого-нибудь A быть так, что для всех x

а) $T_A(x) > T_A^*(\lambda(x))$,

б) $T_A(x) < T_A^*(\lambda(x))$,

в) $T_A(x) = T_A^*(\lambda(x))$,

г) $T_A(x) \neq T_A^*(\lambda(x))$?

18. Для сложности $T_{QS}(n)$ быстрой сортировки выполняется оценка $T_{QS}(n) = \Theta(n^2)$. (Обозначение QS происходит от английского названия быстрой сортировки — quick sort.)

Указание. Оценка $T_{QS}(n) = \Omega(n^2)$ устанавливается предъявлением примера. Неравенство $T_{QS}(n) < n^2$ можно доказать индукцией по n , используя то, что при фиксированном $n \in \mathbb{N}^+$ квадратичная функция $(m-1)^2 + (n-m)^2$ от m принимает на отрезке $[1, n]$ свое максимальное значение в одном из концов этого отрезка.

19. Алгоритм, использующий только аддитивные операции и сравнения целых чисел для проверки того, является ли данное целое положительное n точным квадратом, может быть основан на вычислении последовательности значений $1, 1+3, 1+3+5, \dots$ до появления в ней первого большего или равного n элемента. Сложность по числу аддитивных операций и операций сравнения этого алгоритма (назовем его sq_1) допускает оценку $\Theta(\sqrt{n})$. Сохранится ли эта оценка, если дополнительно использовать операцию нахождения целой части половины числа (считается, что затратность этой операции такая же, как у аддитивных операций) для того, чтобы предварительно выяснять, на какую максимальную степень двойки — четную или нечетную — делится n (алгоритм sq_2)?

20. (Продолжение предыдущей задачи.) Пусть $T_{\text{sq}_1}(n)$, $T_{\text{sq}_2}(n)$ — сложности, соответственно, первого и второго вариантов рассмотрен-

ного алгоритма и

$$T_{sq_1}^*(m) = \max_{2^{m-1} \leq n < 2^m} T_{sq_1}(n), \quad T_{sq_2}^*(m) = \max_{2^{m-1} \leq n < 2^m} T_{sq_2}(n),$$

$m = 2, 3, \dots$

а) Как связаны значения функций $T_{sq_1}^*(m)$ и $T_{sq_2}^*(m)$?

б) Имеются ли среди оценок $\Theta(m)$, $O(\log m)$, $\Theta(2^{m/2})$, $O(2^m)$ такие, которые верны для $T_{sq_2}^*(m)$, и если да, то какие именно?

21. Изменим в алгоритме Грэхема (пример 3.1) этап удаления вмятых вершин: будем обходить — возможно, многократно — против часовой стрелки вершины построенной несамопересекающейся ломаной и удалять вмятые вершины до тех пор, пока при очередном обходе не окажется безуспешной попытка найти хотя бы одну вмятую вершину. Убедиться в том, что сложность этого варианта рассматриваемого этапа алгоритма Грэхема есть $\Omega(n^2)$, где n — начальное число вершин ломаной (в то время как сложность этого этапа в алгоритме Грэхема есть $O(n)$).

22. Рассмотрим в главных чертах алгоритм Шеймоса—Хоя¹ построения пересечения $P \cap Q$ двух выпуклых многоугольников P и Q содержащих соответственно l и m вершин. Считаем, что многоугольники задаются массивами $P_1, P_2, \dots, P_l$ и $Q_1, Q_2, \dots, Q_m$ координат своих последовательных вершин.

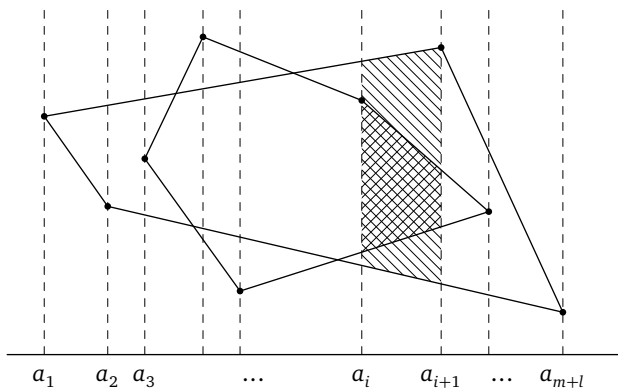


Рис. 7. Алгоритм Шеймоса—Хоя ($l = 5$, $m = 4$).

Упорядочим множество всех абсцисс обоих многоугольников по возрастанию (для простоты считаем, что абсциссы попарно различны, хотя это и несущественно), в результате получим абсциссы $a_1 < a_2 < \dots < a_{l+m}$ (рис. 7). Теперь для каждого $i = 1, 2, \dots, l + m - 1$

¹ См. [44, гл. 7].

строим пересечение трапеций, вырезаемых прямыми $x = a_i$, $x = a_{i+1}$ в заданных многоугольниках. (В вырожденном случае такая трапеция превращается в треугольник.) Из получившихся пересечений трапеций можно собрать $P \cap Q$, удаляя лишние вершины.

Привлекая дополнительно те или иные структуры данных (массивы, списки, ...) и уточняя по мере надобности какие-то этапы алгоритма, добиться того, чтобы в итоге алгоритм (обозначим его буквами SH) имел следующие сложностные характеристики. Если размер входа — это $r = l + m$, то $T_{SH}(r) = \Theta(r)$; если размер входа — это $s = \max\{l, m\}$, то $T'_{SH}(s) = \Theta(s)$; если размер входа имеет две компоненты l и m , то $T''_{SH}(l, m) = \Theta(l + m)$ — мы рассматриваем операции сравнения и перемещения элементов, арифметические операции, как мультипликативные, так и аддитивные, все вместе. Для пространственной сложности — $S_{SH}(r)$, $S'_{SH}(s)$ или $S''_{SH}(l, m)$ в зависимости от выбора размера входа — получаем те же оценки $\Theta(r)$, $\Theta(s)$ или $\Theta(l + m)$.

Указание. Если исходные многоугольники представить, например, двуправленными списками координат последовательных вершин, то список абсцисс $(a_1, a_2, \dots, a_{l+m})$, $a_1 < a_2 < \dots < a_{l+m}$, можно получить с временными затратами, не превосходящими $c_0(l + m)$.

При определении пространственной сложности можно заметить, что общее число вершин искомого пересечения не превосходит $3(l + m)$ — это следует непосредственно из самого алгоритма Шеймоса—Хоя: при построении пересечения двух трапеций может возникнуть не более двух дополнительных вершин.

23. Множество M , состоящее из n точек, задано массивом координат этих точек. Точка p задана своими координатами, по крайней мере одна из которых отлична от нуля. Требуется найти такую точку q плоскости, что все точки множества M попадают в одну из полуплоскостей, на которые разбивает плоскость прямая, проходящая через q в направлении вектора из точки $(0, 0)$ в p . Алгоритм поиска должен иметь сложность $O(n)$ по числу арифметических операций и операций сравнения.

24. Расширить алгоритм построения вояжа в ориентированном графе $G = (V, E)$ с выделенной начальной вершиной v (пример 3.2) так, чтобы помимо вояжа алгоритм выдавал также информацию о том, охватил ли вояж все ребра графа. Размер входа имеет две компоненты $|V|$, $|E|$. Для графов, не содержащих изолированных вершин, т. е. вершин, подобных вершине 7 на рис. 3, сложность расширенного алгоритма должна быть $O(|E|)$.

25. Задача проверки существования и фактического построения эйлера цикла в данном графе для ориентированного случая выглядит так: выяснить, имеется ли в данном ориентированном графе цикл, проходящий по всем ребрам графа по одному разу, и если существует, то построить этот цикл. Воспользовавшись рассмотренным алгоритмом построения вояжа (пример 3.2), дать алгоритм решения этой задачи. Размер входа имеет две компоненты $|V|$, $|E|$. Для графов, не содержащих изолированных вершин, сложность предлагаемого алгоритма должна быть $O(|E|)$.

26. Путник столкнулся со стеной, простирающейся бесконечно в обе стороны. Имеется дверь в этой стене, но путник не знает ни расстояния до двери, ни направления к ней. Предложить позволяющий добраться до двери алгоритм, сложность которого допускает оценку $O(n)$, где n — число шагов (неизвестное путнику), изначально отделяющее путника от двери.

Указание. Сумма $s_k = 1 + q + \dots + q^k$, $q > 1$, есть величина порядка q^k , т. е. $s_k = \Theta(q^k)$.

27. Будем считать затратностью любого построения на плоскости с помощью циркуля и линейки количество линий (окружностей или прямых), проводимых в ходе построения. Рассмотрим задачу построения отрезка, длина которого равна $1/n$ от длины исходного отрезка, считая n размером входа. Предложить для этой задачи такой алгоритм решения, сложность которого допускает оценку $O(\log n)$.¹

28. При $n = 15$ возможно вычисление a^n с меньшим числом умножений, чем требуется бинарному алгоритму возведения в степень.

29. Пусть двоичная запись числа $n \in \mathbb{N}$ есть $\beta_k \dots \beta_1 \beta_0$. Алгоритм

```

 $q := a; u := 1;$ 
for  $i = 0$  to  $k - 1$  do
    if  $\beta_i = 1$  then  $u := u \cdot q$  fi;
     $q := q^2$ 
od;
 $u := u \cdot q$ 

```

вычисляет a^n . Сравнить сложность этого алгоритма по числу умножений с аналогичной сложностью бинарного алгоритма возведения в степень, рассмотренного в примере 4.2, при использовании n или соответственно $m = \lambda(n)$ в качестве размера входа. Как изменить

¹ Разбор задач на построение с точки зрения их сложности содержится, например, в [6], [16, § 15].

предложенный здесь алгоритм, чтобы его сложность совпала со сложностью бинарного алгоритма, рассмотренного в примере 4.2? (Написать псевдокод.)

30. Пусть A — некоторый алгоритм, входом которого является произвольное натуральное число n . Пусть $T_A(n)$, $T_A^*(m)$ — сложности этого алгоритма по числу арифметических операций при использовании n и соответственно $m = \lambda(n)$ в качестве размера входа. Пусть $f(n)$ — некоторая функция, определенная для всех натуральных значений аргумента, такая, что $T_A(n) \geq f(n)$. Верно ли, что неравенство $T_A^*(m) \geq f(2^{m-1})$ обязательно выполнено для всех натуральных значений m ?

31. Пусть A — некоторый алгоритм, входом которого является произвольное натуральное число n . Пусть $T_A(n)$, $T_A^*(m)$ — сложности этого алгоритма по числу арифметических операций при использовании n и соответственно $m = \lambda(n)$ в качестве размера входа. Пусть $f(m)$ — некоторая функция, определенная для всех целых неотрицательных значений аргумента, такая, что $T_A^*(m) \geq f(m)$. Верно ли, что неравенство $T_A(n) \geq f(\lfloor \log_2 n \rfloor)$ обязательно выполнено для всех натуральных значений n ?

Указание. Рассмотреть алгоритм RS для обоснования отрицательного ответа.

32. Пусть A — некоторый алгоритм, входом которого является произвольное натуральное число n . Пусть $T_A(n)$, $T_A^*(m)$ — сложности этого алгоритма по числу арифметических операций при использовании n и соответственно $m = \lambda(n)$ в качестве размера входа. Определяется ли однозначно

- а) функция $T_A^*(m)$ по функции $T_A(n)$;
- б) функция $T_A(n)$ по функции $T_A^*(m)$?

33. Предположим, что две сложности $T_A(n)$, $T_A^*(m)$ некоторого алгоритма A получаются из одной и той же функции затрат $C_A^T(x)$, но с использованием разных размеров входа — $\|\cdot\|$ и $\|\cdot\|^*$, принимающих значения в $\mathbb{N}^+$, причем если $\|x\| = n$ для некоторого входа x , то $\|x\|^* = m = \lambda(n) = \lfloor \log_2 n \rfloor + 1$. В таком случае справедливы следующие утверждения:

а) Пусть $f(x)$ — неубывающая функция вещественной переменной. Тогда если $T_A^*(m) = O(f(m))$, то $T_A(n) = O(f(\log_2 n + 1))$; как следствие, при $f(x+1) = O(f(x))$ имеем $T_A(n) = O(f(\log_2 n))$.

б) Пусть $g(x)$ — неубывающая функция вещественной переменной. Тогда

- (i) если $T_A(n) = O(g(n))$, то $T_A^*(m) = O(g(2^m))$,
- (ii) если $T_A(n) = \Omega(g(n))$, то $T_A^*(m) = \Omega(g(2^{m-1}))$; как следствие, при $g(x/2) = \Omega(g(x))$ имеем $T_A^*(m) = \Omega(g(2^m))$.

Указание. Предварительно доказать вспомогательные утверждения:

а) Пусть $f(x)$ — неубывающая функция вещественной переменной. Тогда если $T_A^*(m) \leq f(m)$, то $T_A(n) \leq f(\log_2 n + 1)$.

б) Пусть $g(x)$ — неубывающая функция вещественной переменной. Тогда

(i) если $T_A(n) \leq g(n)$, то $T_A^*(m) \leq g(2^m)$,

(ii) если $T_A(n) \geq g(n)$, то $T_A^*(m) \geq g(2^{m-1})$.

34. Сложность $T(n)$ по количеству операций над числами некоторого алгоритма умножения квадратных числовых матриц порядка n есть $n^{\omega+o(1)}$, где число ω таково, что $2 < \omega \leq 3$. Следует ли из этого, что

а) $T(n) = O(n^\omega)$;

б) $T(n) = \Omega(n^\omega)$?

35. Для функции $l(n)$, определенной в конце § 4, выполнено $l(ab) \leq l(a) + l(b) - 1$ для всех $a, b \in \mathbb{N}^+$.

36. (Продолжение предыдущей задачи.) Можно ли утверждать, что $l(ab) = l(a) + l(b) - 1$ для всех $a, b \in \mathbb{N}^+$?

Глава 2

Сложность в среднем

§ 5. Понятие сложности в среднем

До сих пор мы исследовали сложность в худшем случае, теперь обратимся к сложности в среднем. Мы ограничимся ситуацией, когда при каждом фиксированном значении s размера входа сами соответствующие входы образуют конечное множество $X_s = \{x: \|x\| = s\}$. Будем предполагать, что каждому $x \in X_s$ приписана некоторая вероятность $P_s(x)$:

$$0 \leq P_s(x) \leq 1,$$

и при этом для каждого s

$$\sum_{x \in X_s} P_s(x) = 1.$$

Таким образом, при каждом допустимом значении s размера входа множество X_s превращается в вероятностное пространство; по умолчанию считается, что вероятность распределена равномерно:

$$P_s(x) = \frac{1}{N_s}, \quad (5.1)$$

где N_s — количество элементов множества X_s . Функции от s

$$\sum_{x \in X_s} C_A^T(x) P_s(x) \quad \text{и} \quad \sum_{x \in X_s} C_A^S(x) P_s(x) \quad (5.2)$$

называются временной и, соответственно, пространственной сложностью в среднем алгоритма A . Более подробно:

Определение 5.1. Пусть при любом допустимом значении s множество X_s всех входов размера s является вероятностным пространством, в силу чего временные и пространственные затраты алгоритма A для входов x (т.е. $C_A^T(x)$ и $C_A^S(x)$) размера s являются случайными величинами на X_s . Сложностью в среднем называется математическое ожидание (первая или вторая сумма из (5.2)) соответствующей случайной величины.

Сложность в среднем может принимать и нецелые значения, даже если функция затрат целочисленна.

Для временной и пространственной сложности в среднем алгоритма A мы будем использовать обозначения $\bar{T}_A(\cdot)$, $\bar{S}_A(\cdot)$.

Теорема 5.1. Для любого алгоритма A при любом распределении вероятностей на множестве X_s , где s — некоторое допустимое значение размера $\|\cdot\|$, сложность в среднем не превосходит сложности в худшем случае:

$$\bar{T}_A(s) \leq T_A(s), \quad \bar{S}_A(s) \leq S_A(s).$$

Доказательство. Докажем, например, первое неравенство:

$$\begin{aligned} \bar{T}_A(s) &= \sum_{x \in X_s} C_A^T(x) P_s(x) \leq \sum_{x \in X_s} (\max_{x \in X_s} C_A^T(x)) P_s(x) = \\ &= \sum_{x \in X_s} T_A(s) P_s(x) = T_A(s) \sum_{x \in X_s} P_s(x) = T_A(s). \end{aligned}$$

Второе неравенство доказывается аналогично. $\square$

Ниже в этом параграфе мы рассмотрим временную сложность в среднем ряда алгоритмов.

Пример 5.1. Как было уже установлено (пример 4.2), бинарный алгоритм возведения в степень n требует $\lambda(n) + \lambda^*(n) - 2$ умножений; если в качестве размера взять $m = \lambda(n)$, то сложность в худшем случае будет равна $2m - 2$. Подсчитаем сложность в среднем, привлекая m как размер входа. Всего имеется 2^{m-1} неотрицательных целых битовой длины m ; если считать все эти числа равновероятными, то искомая сложность есть

$$\frac{1}{2^{m-1}} \sum_{n=2^{m-1}}^{2^m-1} (\lambda(n) + \lambda^*(n) - 2) = m - 2 + \frac{1}{2^{m-1}} \sum_{n=2^{m-1}}^{2^m-1} \lambda^*(n).$$

Первая цифра каждого из чисел битовой длины m равна 1; количество чисел, имеющих кроме старшей цифры еще k , $0 \leq k \leq m-1$, ненулевых двоичных цифр, равно $\binom{m-1}{k}$ (т.е. числу сочетаний из $m-1$ по k). Поэтому искомую сложность можно переписать в виде

$$m - 2 + \frac{1}{2^{m-1}} \left(2^{m-1} + \sum_{k=1}^{m-1} k \binom{m-1}{k} \right) = m - 1 + \frac{1}{2^{m-1}} \sum_{k=1}^{m-1} k \binom{m-1}{k}.$$

Легко проверяется, что при $1 \leq k \leq m-1$ выполнено

$$k \binom{m-1}{k} = (m-1) \binom{m-2}{k-1},$$

и возможно дальнейшее упрощение выражения для сложности:

$$m - 1 + \frac{m-1}{2^{m-1}} \sum_{k=1}^{m-1} \binom{m-2}{k-1} = m - 1 + \frac{m-1}{2^{m-1}} \sum_{l=0}^{m-2} \binom{m-2}{l} = \frac{3}{2}(m-1).$$

Если рассматривать $m = \lambda(n)$ как размер входа бинарного алгоритма вычисления a^n , $n \in \mathbb{N}^+$, то сложность в среднем по числу умножений для этого алгоритма есть $\frac{3}{2}(m-1)$.

Таким образом, при больших m сложность в среднем бинарного алгоритма возведения в степень приблизительно равна трем четвертым сложности в худшем случае.

Анализ сложности в среднем для широкого ряда арифметических алгоритмов опирается на асимптотический закон распределения простых чисел, который мы приведем без доказательства.

Асимптотический закон распределения простых чисел. Для функции $\pi(n)$, значение которой равно количеству простых чисел, меньших данного натурального n , справедливо асимптотическое равенство

$$\pi(n) \sim \frac{n}{\ln n}. \quad (5.3)$$

Как следствие, вероятность того, что при выборе «наугад» одного из целых чисел $1, 2, \dots, n$ попадет простое число, асимптотически равна $\frac{1}{\ln n}$.¹

Пример 5.2. Вновь вернемся к алгоритму пробных делений, предназначенному для распознавания простоты данного $n \geq 2$ (приме-

¹ Предположение о справедливости этого закона распределения простых чисел высказывалось, в частности, Гауссом и Лежандром. Впоследствии в 1850 г. Чебышевым было доказано, что для некоторых констант c и C

$$c \frac{n}{\ln n} < \pi(n) < C \frac{n}{\ln n}$$

для всех достаточно больших n , и, более того, им было установлено, что можно положить $C = 1,10555$, $c = 0,92129$. Асимптотическое равенство (5.3) было полностью доказано в 1896 г. независимо Ж. Адамаром и Ш. Валле Пуссенном. «После доказательства неравенств Чебышева в 1850 г. речь шла, казалось бы, только о более точном нахождении и сближении постоянных c и C . Однако асимптотический закон распределения простых чисел был доказан лишь полвека спустя, в самом конце XIX века, на основании совершенно новых идей, предложенных Риманом...» [56]. В [56, гл. V] приводится элементарное доказательство неравенств Чебышева для $C = 6$ и $c = \frac{1}{3}$. Полное доказательство асимптотического закона имеется, например, в [9].

ры 1.2, 4.1). Будем рассматривать в качестве размера входа битовую длину m данного числа. На первый взгляд сложность в среднем этого алгоритма могла бы оказаться существенно меньшей, чем сложность в худшем случае, так как для многих входов затраты совсем невелики. Например, для четных входов достаточно одного деления и т.д. Для сложности в худшем случае по числу делений нами была установлена экспоненциальная оценка $\Theta(2^{m/2})$. Существует ли $d \in \mathbb{N}$ такое, что сложность в среднем алгоритма пробных делений как функция от m допускает оценку $O(m^d)$? Мы видим, что для довольно обширного множества входов размера m алгоритм пробных делений работает быстро, и предположение, что для сложности в среднем существует такое число d , выглядит правдоподобным. Но дело обстоит не так, потому что среди всех натуральных чисел доля простых (для которых алгоритм пробных делений работает долго) достаточно велика. Оценка количества $V(m)$ простых среди чисел

$$2^{m-1} \leq n < 2^m$$

может быть получена из приведенной выше теоремы о распределении простых чисел: воспользовавшись тем, что

$$\pi(2^m) \sim \frac{2^m}{(\ln 2)m}, \quad m \rightarrow \infty,$$

а также равенством $V(m) = \pi(2^m) - \pi(2^{m-1})$, получим

$$V(m) \sim \frac{1}{2 \ln 2} \frac{m-2}{m(m-1)} 2^m \sim \frac{1}{(2 \ln 2)m} 2^m. \quad (5.4)$$

Теперь уже экспоненциальность роста сложности в среднем алгоритма пробных делений выводится легко. Обозначим через $D(n)$ затраты алгоритма пробных делений применительно к натуральному числу n . Для любого простого $p \geq 2^{m-1}$, $m \geq 7$, выполнено $D(p) > 2^{m/2-1}$ (в самом деле, $D(p) = \lfloor \sqrt{p} \rfloor - 1 \geq \lfloor 2^{(m-1)/2} \rfloor - 1 > 2^{(m-1)/2} - 2$, и при $m \geq 7$ выполнено $2^{(m-1)/2} - 2 \geq 2^{m/2-1}$). Интересующая нас сложность в среднем равна $\frac{1}{2^{m-1}} \sum_{n=2^{m-1}}^{2^m-1} D(n)$. При $m \geq 7$ мы имеем

$$\frac{1}{2^{m-1}} \sum_{n=2^{m-1}}^{2^m-1} D(n) \geq \frac{1}{2^{m-1}} \sum_{\substack{2^{m-1} \leq p < 2^m \\ p \text{ — простое}}} D(p) \geq V(m) 2^{-m/2}. \quad (5.5)$$

Учитывая (5.4), находим

$$V(m) 2^{-m/2} \sim \frac{1}{(2 \ln 2)m} 2^{m/2}.$$

Это говорит о том, что исследуемая сложность по числу делений в среднем есть $\Omega\left(\frac{1}{m}2^{m/2}\right)$ и как функция от m эта сложность не может быть оценена как $O(m^d)$ ни при каком $d \in \mathbb{N}$.

Определение 5.2. Вещественная неотрицательная функция $f(m)$, определенная для целых положительных значений аргумента, называется *полиномиально ограниченной*, если существует полином $P(m)$ с вещественными коэффициентами такой, что $f(m) \leq P(m)$ для всех $m \in \mathbb{N}^+$. (Очевидно, что мы получим эквивалентное определение, если дополнительно потребуем, чтобы все коэффициенты полинома $P(m)$ были неотрицательными; другие эквивалентные варианты определения: $f(m) = O(m^d)$ для некоторого $d \in \mathbb{N}$ и $f(m) = m^{O(1)}$.)

Доказанное нами можно сформулировать так:

Пусть в качестве размера входа алгоритма пробных делений, предназначенного для проверки простоты натурального числа n , привлекается $m = \lambda(n)$. Тогда временные сложности этого алгоритма как в худшем случае, так и в среднем по числу делений не являются полиномиально ограниченными функциями от m .

Краткое обсуждение идеи такого алгоритма проверки простоты числа, который имеет полиномиально ограниченную сложность в худшем случае (тем самым, разумеется, и в среднем), будет проведено в примере 22.2. При этом речь пойдет не о сложности по числу делений, мультипликативной сложности и т.д., а о битовой сложности, что приводит к существенно более сильным утверждениям. До этого в гл. 5 будет детально рассмотрено само понятие битовой сложности.

§ 6. Сортировка и конечные вероятностные пространства.

Применение формулы полного математического ожидания

При рассмотрении сложности в среднем мы, прежде всего, должны превратить каждое из множеств входов, обладающих фиксированным размером, в вероятностное пространство. Проще работать с конечными вероятностными пространствами, но как обходиться такими пространствами, имея дело, например, с алгоритмами сортировки, входом каждого из которых, при фиксированном размере n , может быть любой массив $x_1, x_2, \dots, x_n$ с попарно различными элементами? Общий принцип заключается в том, что мы разбиваем все имеющие фиксированный размер n входы на некоторые классы, включая в один

класс входы, для данного алгоритма неразличимые: алгоритм проделывает одну и ту же последовательность операций для любого входа, принадлежащего классу (этот принцип может применяться не только при анализе сортировок). Число таких классов может оказаться конечным, даже если само множество входов размера n бесконечно. Если это не противоречит другим предположениям, то такие классы можно рассматривать как элементарные события, приписывая каждому из них некоторую вероятность.

На выполнение любой сортировки с помощью сравнений сами значения элементов $x_1, x_2, \dots, x_n$ не оказывают никакого влияния, важен лишь их относительный порядок. Поэтому в один класс естественно объединять все массивы $x_1, x_2, \dots, x_n$, имеющие один и тот же порядок элементов по отношению друг к другу. Такой класс может быть представлен перестановкой чисел $1, 2, \dots, n$ с соответствующим порядком элементов (перестановкой длины n).

Множество всех перестановок чисел $1, 2, \dots, n$ будет обозначаться через Π_n , так же будет обозначаться и вероятностное пространство, получаемое приписыванием вероятности $\frac{1}{n!}$ каждой из этих перестановок.

Рассмотрим некоторые события в вероятностном пространстве Π_n ; вероятность каждого из них, очевидно, будет равна $\frac{M}{n!}$, где M — число перестановок, составляющих событие. Эту вероятность мы будем обозначать через $P_n(\cdot)$.

Пример 6.1. Начнем с события B_n^v , $1 \leq v \leq n$, состоящего в том, что v -й элемент a_v перестановки $(a_1, a_2, \dots, a_n)$ равен v . Перестановку $(a_1, a_2, \dots, a_n) \in \Pi_n$ можно рассматривать как отображение множества $\{1, 2, \dots, n\}$ на себя, при котором 1 переходит в a_1 , 2 переходит в a_2 и т.д., и событие B_n^v состоит тогда в том, что v является неподвижной точкой перестановки. Множество, состоящее из перестановок $(a_1, a_2, \dots, a_n)$ таких, что $a_v = v$, содержит $(n-1)!$ элементов, откуда $P_n(B_n^v) = \frac{(n-1)!}{n!} = \frac{1}{n}$ при всех рассматриваемых v .

При $n > 1$ число неподвижных точек может быть любым целым из диапазона от 0 до n . Рассмотрим вопрос о среднем числе неподвижных точек перестановок длины n : найдем математическое ожидание определенной на Π_n случайной величины ξ_n , равной числу неподвижных точек перестановки a . Дополнительно определим случайные величины

$$\zeta_n^v = \begin{cases} 1, & \text{если } a_v = v, \\ 0, & \text{если } a_v \neq v, \end{cases}$$

$v = 1, 2, \dots, n$. Мы видим, что $P_n(\zeta_n^v = 1) = P_n(B_n^v) = \frac{1}{n}$, откуда $E\zeta_n^v = \frac{1}{n}$ и

$$E\xi_n = E(\zeta_n^1 + \zeta_n^2 + \dots + \zeta_n^n) = E\zeta_n^1 + E\zeta_n^2 + \dots + E\zeta_n^n = n \cdot \frac{1}{n} = 1.$$

Итак, при любом n среднее число неподвижных точек равно 1.¹

Неподвижные точки сопоставленной массиву перестановки соответствуют тем элементам массива, которые при сортировке не нуждаются в перемещениях на другие места, они и так находятся именно там, где должны находиться, и это может использоваться в некоторых видах сортировки (см. задачу 42).

Найдем теперь вероятность еще одного события, которая потребуется нам при исследовании сложностей в среднем некоторых сортировок.

Предложение 6.1. Пусть $n \in \mathbb{N}^+$. Тогда при $0 \leq u < v \leq n$ имеем

$$P_n(H_n^{u,v}) = \frac{1}{v}$$

для события $H_n^{u,v}$, состоящего в том, что в перестановке $(a_1, a_2, \dots, a_n)$ содержится ровно u элементов a_i , $1 \leq i < v$, для которых $a_i < a_v$.

Доказательство. Будем обозначать через ψ_v функцию, аргументами которой являются v любых попарно различных чисел, а значением — перестановка чисел $1, 2, \dots, v$, отражающая относительный порядок исходных чисел. Очевидно, что если p — любая перестановка чисел $1, 2, \dots, v$, то множество перестановок $(a_1, a_2, \dots, a_n)$ таких, что $\psi_v(a_1, a_2, \dots, a_v) = p$, содержит $\binom{n}{v} (n-v)! = \frac{n!}{v!}$ элементов. Заметим, что количество тех перестановок p чисел $1, 2, \dots, v$, в которых последний элемент равен $u+1$, есть $(v-1)!$. Поэтому количество перестановок, образующих событие $H_n^{u,v}$, равно $\frac{n!}{v!} (v-1)!$, т. е. $\frac{n!}{v!}$, откуда следует требуемое. $\square$

Напомним полезную формулу из курса теории вероятностей². Вновь ограничимся рассмотрением конечных вероятностных пространств элементарных событий. Пусть W — такое пространство, P — соответствующее распределение вероятностей, ξ — некоторая случайная величина (функция) на W . Пусть события $W_1, W_2, \dots, W_l$ задают полную группу событий, т. е. эти события попарно несовместны и

$$W = W_1 \cup W_2 \cup \dots \cup W_l. \quad (6.1)$$

¹ Серия вероятностных «задач о совпадении» и их решений приведена в [40].

² См. [45, гл. 1, с. 9].

Представление W в виде (6.1) с помощью некоторой полной группы событий мы будем называть *разложением* W . Пусть $E\xi$ — это значение $\sum_{w \in W} \xi(w)P(w)$, т. е. математическое ожидание случайной величины ξ , а $E(\xi|W_k)$, $k = 1, 2, \dots, l$, суть соответствующие *условные математические ожидания* этой случайной величины (при условии осуществления события W_k). Тогда, как известно, имеет место формула *полного математического ожидания*:

$$E\xi = \sum_{k=1}^l E(\xi|W_k)P(W_k). \quad (6.2)$$

Здесь дополнительно заметим, что оценивание математических ожиданий на конечном вероятностном пространстве является оцениванием числовых сумм. В этом иногда помогает простой прием, описанный в приложении В.

Пример 6.2. Исследуем сложности в среднем по числу сравнений и перемещений сортировки простыми вставками, которую, как и прежде, мы будем рассматривать в двух вариантах I_1 и I_2 (см. пример 1.4). Вначале займемся первым вариантом I_1 этой сортировки, полагая затраты равными числу сравнений. Входные массивы считаем перестановками чисел $1, 2, \dots, n$. Для исследования сложности по числу сравнений введем случайные величины $\xi_n^1, \xi_n^2, \dots, \xi_n^{n-1}$, определенные так, что значение ξ_n^i для

$$a = (a_1, a_2, \dots, a_n) \in \Pi_n$$

равно числу сравнений на i -м шаге нашей сортировки, применяемой к a . Ясно, что интересующее нас математическое ожидание есть $E(\xi_n^1 + \xi_n^2 + \dots + \xi_n^{n-1})$, и

$$\bar{T}_{I_1}(n) = E\xi_n^1 + E\xi_n^2 + \dots + E\xi_n^{n-1}. \quad (6.3)$$

Найдем $E\xi_n^i$, $1 \leq i < n$. Рассматривая события $H_n^{k,i+1}$, $k = 0, 1, \dots, i$ (см. предложение 6.1), образующие разложение пространства Π_n , и применяя формулу (6.2) полного математического ожидания, мы имеем согласно предложению 6.1

$$E\xi_n^i = \frac{1}{i+1} \sum_{k=0}^i E(\xi_n^i | H_n^{k,i+1}). \quad (6.4)$$

Если перестановка $a = (a_1, a_2, \dots, a_n)$ принадлежит событию $H_n^{k,i+1}$, то в этой перестановке перед элементом a_{i+1} имеется ровно k элемен-

тов, меньших его, и, очевидно,

$$\xi_n^i = \begin{cases} i - k + 1, & \text{если } k > 0, \\ i, & \text{если } k = 0. \end{cases}$$

Мы видим, что при выполнении события $H_n^{k,i+1}$ значение ξ_n^i определено однозначно (напомним, что значение ξ_n^i , $i = 1, 2, \dots, n-1$, равно числу сравнений на i -м шаге, а перед выполнением i -го шага элементы $a_1, a_2, \dots, a_i$ уже упорядочены по возрастанию). Поэтому

$$E(\xi_n^i | H_n^{k,i+1}) = \begin{cases} i - k + 1, & \text{если } k > 0, \\ i, & \text{если } k = 0. \end{cases}$$

В соответствии с (6.4)

$$E\xi_n^i = \frac{1}{i+1} \left(i + \sum_{k=1}^i (i - k + 1) \right) = \frac{i}{2} + 1 - \frac{1}{i+1}.$$

Используя (6.3), получаем выражение

$$n - 1 + \sum_{i=1}^{n-1} \left(\frac{i}{2} - \frac{1}{i+1} \right)$$

для искомой сложности в среднем. С помощью известной из математического анализа формулы (ее вывод имеется в приложении В)

$$\sum_{i=1}^n \frac{1}{i} = \ln n + O(1)$$

сложность $\bar{T}_{I_1}(n)$ можно записать в виде

$$\bar{T}_{I_1}(n) = \frac{(n+4)(n-1)}{4} - \ln n + O(1) \quad (6.5)$$

и, проще, в виде

$$\bar{T}_{I_1}(n) = \frac{n^2}{4} + O(n) \quad \text{или} \quad \bar{T}_{I_1}(n) \sim \frac{n^2}{4}. \quad (6.6)$$

Таким образом, сложность в среднем по числу сравнений первого варианта сортировки простыми вставками, как и сложность в худшем случае, есть величина порядка n^2 , но при больших n первая сложность примерно вдвое меньше второй. Формула (6.5) показывает, что, вообще говоря, вопреки бытовым представлениям сложность в среднем не есть среднее арифметическое затрат в худшем и в лучшем случае: такое среднее арифметическое для рассматриваемой сортировки является рациональной функцией от n и не может описываться формулой, содержащей логарифм.

Если рассмотреть вместо случайных величин $\xi_n^1, \xi_n^2, \dots, \xi_n^{n-1}$ случайные величины $\eta_n^1, \eta_n^2, \dots, \eta_n^{n-1}$, соответствующим образом отражающие затраты исследуемой сортировки по числу перемещений, то, очевидно, будем иметь

$$\xi_n^i \leq \eta_n^i \leq \xi_n^i + 1,$$

$i = 1, 2, \dots, n - 1$. Поэтому сложность в среднем по числу перемещений тоже есть $\frac{n^2}{4} + O(n)$. Нетрудно показать, что и сложности в среднем по числу сравнений и числу перемещений второго варианта сортировки простыми вставками равны $\frac{n^2}{4} + O(n)$. Сложность в среднем по суммарному числу сравнений и перемещений и для первого, и для второго варианта есть $\frac{n^2}{2} + O(n)$, поскольку для любых случайных величин ξ, η , определенных на каком-либо вероятностном пространстве, выполняется

$$E(\xi + \eta) = E\xi + E\eta. \quad (6.7)$$

Каждая из сложностей в среднем как по числу сравнений, так и по числу перемещений для двух вариантов сортировки простыми вставками (всего четыре сложности) есть $\frac{n^2}{4} + O(n)$; каждая из сложностей в среднем по общему числу сравнений и перемещений для двух вариантов этой сортировки (всего две сложности) есть $\frac{n^2}{2} + O(n)$.

Можно вспомнить, что в примере 1.4, в котором мы рассматривали для двух вариантов сортировки простыми вставками их сложности в худшем случае по суммарному числу сравнений и перемещений, наблюдалась непохожая ситуация, чему имеется, повторимся, простое объяснение: в отличие от (6.7), равенство $\max(\xi + \eta) = \max \xi + \max \eta$, вообще говоря, места не имеет.

Общая формулировка установленного нами соотношения:

Теорема 6.1. *Сложность в среднем некоторого алгоритма по суммарному числу операций нескольких различных типов равна сумме сложностей в среднем этого алгоритма по числу операций каждого типа в отдельности.*

Среди наиболее элементарных сортировок мы пока рассмотрели сложность в среднем только для сортировки простыми вставками. Сразу же можно добавить, что для сортировки выбором ее сложности по числу сравнений в среднем и в худшем случае совпадают, так как число сравнений однозначно определяется длиной n массива. Число

сравнений на i -м этапе (i -м просмотре массива) пузырьковой сортировки равно $n - i$. Не сталкиваясь с большими трудностями, можно показать (см. задачу 58), что математическое ожидание $s(n)$ числа просмотров массива есть

$$n - \sum_{k=0}^n \frac{k! k^{n-k}}{n!}. \quad (6.8)$$

Очевидно, что $s(n) < n$. С другой стороны,

$$\sum_{k=0}^n \frac{k! k^{n-k}}{n!} = \sum_{k=0}^{n-1} \frac{(n-1)! k^{n-k}}{n! (k+1)(k+2)\dots(n-1)} \leq \sum_{k=0}^n \frac{(n-1)! k}{n!} = \frac{1}{n} \sum_{k=0}^n k = \frac{n+1}{2}.$$

Поэтому $s(n) \geq n - \frac{n+1}{2} = \frac{n-1}{2}$. Мы имеем

$$\frac{n-1}{2} \leq s(n) < n$$

и $s(n) = \Theta(n)$. Из этого следует, что сложность в среднем по числу сравнений пузырьковой сортировки квадратична.

Сложности в среднем по числу сравнений пузырьковой сортировки, сортировок выбором и простыми вставками имеют одинаковый порядок со сложностями этих же сортировок в худшем случае (допускают оценку $\Theta(n^2)$).

§ 7. Пример медленного роста сложности в среднем в сравнении со сложностью в худшем случае

Следующий пример показывает, что при $n \rightarrow \infty$ сложность в среднем может расти существенно медленнее, чем сложность в худшем случае.

Пример 7.1. Рассмотрим сложность в среднем $\bar{T}_{Qs}(n)$ быстрой сортировки по числу сравнений. Как обычно, считаем равновероятными все относительные порядки элементов во входном массиве. Пусть в качестве разбивающего элемента всегда выбирается первый элемент сортируемой части массива. Часто в учебной литературе исследование поведения $\bar{T}_{Qs}(n)$ основывается на замечании, что поскольку первый элемент с одинаковой вероятностью $\frac{1}{n}$ может занимать любое место в массиве после этапа разбиения и поскольку после этапа разбиения предстоит сортировать элементы справа и элементы слева тем же алгоритмом, то выполняется соотношение

$$\bar{T}_{Qs}(n) = n - 1 + \frac{1}{n} \sum_{i=1}^n (\bar{T}_{Qs}(i-1) + \bar{T}_{Qs}(n-i)), \quad (7.1)$$

$\bar{T}_{QS}(1) = 0$. В этом рассуждении есть пробел: строго говоря, надо еще доказывать, что если разбивающий элемент занял некоторую позицию с номером i , то все относительные порядки элементов, оказавшихся слева от разбивающего (всего этих порядков $(n - i - 1)!$), равновероятны и что аналогичное утверждение верно для элементов, оказавшихся справа от разбивающего. Это можно сделать, но мы не будем на этом доказательстве останавливаться: более употребительным является рандомизированный (т. е. со случайным выбором разбивающего элемента) вариант быстрой сортировки, который будет рассмотрен в § 8 с полным выводом оценки сложности.

Так как $\sum_{i=1}^n \bar{T}_{QS}(i - 1) = \sum_{i=1}^n \bar{T}_{QS}(n - i) = \sum_{k=0}^{n-1} \bar{T}_{QS}(k)$, из (7.1) мы получаем соотношение

$$\bar{T}_{QS}(n) = n - 1 + \frac{2}{n} \sum_{k=0}^{n-1} \bar{T}_{QS}(k), \quad (7.2)$$

$\bar{T}_{QS}(0) = 0$. Домножение обеих частей равенства (7.2) на n дает

$$n\bar{T}_{QS}(n) = n(n - 1) + 2 \sum_{k=0}^{n-1} \bar{T}_{QS}(k).$$

При $n > 0$ имеем для $n - 1$:

$$(n - 1)\bar{T}_{QS}(n - 1) = (n - 1)(n - 2) + 2 \sum_{k=0}^{n-2} \bar{T}_{QS}(k).$$

Вычитая почленно последнее равенство из предпоследнего, получаем

$$n\bar{T}_{QS}(n) - (n - 1)\bar{T}_{QS}(n - 1) = 2(n - 1) + 2\bar{T}_{QS}(n - 1),$$

т. е.

$$n\bar{T}_{QS}(n) - (n + 1)\bar{T}_{QS}(n - 1) = 2(n - 1).$$

Делим последнее равенство на $n(n + 1)$ и переходим к новой неизвестной функции $t(n) = \frac{\bar{T}_{QS}(n)}{n + 1}$. Имеем для $n \geq 1$:

$$t(n) - t(n - 1) = 2 \frac{n - 1}{n(n + 1)}, \quad t(0) = 0.$$

Отсюда

$$\sum_{k=1}^n (t(k) - t(k - 1)) = \sum_{k=1}^n 2 \frac{k - 1}{k(k + 1)},$$

или

$$t(n) = t(0) + \sum_{k=1}^n 2 \frac{k - 1}{k(k + 1)}.$$

В нашем случае $t(0) = 0$ и для $n \geq 2$ получаем

$$t(n) = 2 \sum_{k=1}^n \frac{k-1}{k(k+1)} = 2 \sum_{k=1}^n \frac{1}{k+1} - 2 \sum_{k=1}^n \frac{1}{k(k+1)}. \quad (7.3)$$

Мы имеем $\sum_{k=1}^n \frac{1}{k+1} = \ln n + O(1)$; с другой стороны, $\sum_{k=1}^n \frac{1}{k(k+1)} = O(1)$, поскольку ряд $\sum_{k=1}^{\infty} \frac{1}{k(k+1)}$ сходится и его частичные суммы ограничены. Таким образом, $t(n) = 2 \ln n + O(1)$; отсюда

$$\bar{T}_{QS}(n) = (n+1)t(n) = 2n \ln n + O(n) = \Theta(n \log n). \quad (7.4)$$

В то же время, сложность в худшем случае $T_{QS}(n)$ есть величина порядка n^2 (см. задачу 18).

Число перемещений, требуемое быстрой сортировкой для конкретного массива, не превосходит числа сравнений, домноженного на некоторую небольшую константу, откуда сложность в среднем $\bar{T}'_{QS}(n)$ по числу перемещений элементов допускает оценку $\bar{T}'_{QS}(n) = \Theta(n \log n)$.

Сложность в среднем быстрой сортировки как по числу сравнений, так и по числу перемещений элементов допускает оценку $\Theta(n \log n)$.

Быстрая сортировка не использует дополнительных массивов, поэтому пространственная сложность по числу одновременно хранимых дополнительных величин, равных каким-то элементам исходного массива, ограничена (небольшой) константой и, как следствие, допускает оценку $O(1)$.

Нелишне заметить, что если быстрая сортировка реализована как рекурсивная процедура, то при ее выполнении будет использован стек отложенных заданий. Рекурсию в данном случае можно организовать так, что в стек будут попадать лишь значения индексов некоторых элементов, но не сами элементы массива. В соответствии с определением сложности объем такого стека не влияет на пространственную сложность.

Размер стека отложенных заданий также является интересной характеристикой алгоритма. Мы рассмотрим размер этого стека для некоторого типа рекурсивных сортировок, к которому относится не только быстрая сортировка, но и, например, рекурсивный вариант сортировки слияниями. Пусть сортировка организована так, что на первом ее этапе мы, следуя некоторому принципу, выделяем из мас-

сива x длины $n > 1$ две какие-то его непересекающиеся части x' , x'' , длина каждой из которых меньше n . На втором этапе эти части сортируются рекурсивно с помощью той же сортировки, и на третьем, заключительном, этапе, исходя из результатов сортировки x' и x'' , массив x представляется, уже без рекурсий, в упорядоченном виде. Если $n = 0$ или $n = 1$, то никаких действий над элементами массива не производится.

Будем использовать для такого рода сортировок рабочее название «бинарные рекурсивные сортировки». Возникающие при выполнении таких сортировок рекурсии реализуются с помощью стека отложенных заданий.

Теорема 7.1. Пусть некоторая бинарная рекурсивная сортировка такова, что первым из x' , x'' сортируется массив, имеющий не превосходящую $\frac{n}{2}$ длину. Тогда в ходе применения сортировки к массиву x длины $n > 1$ число отложенных заданий, хранящихся в стеке, никогда не превосходит $\log_2 n$.

Доказательство. Проведем индукцию по n . При $n = 2$ утверждение, очевидно, справедливо. Пусть утверждение доказано для $2, 3, \dots, n - 1$, и пусть массив x , содержащий n элементов, на первом этапе сортировки порождает два массива x' , x'' , причем длина n' первого массива x' не превосходит $\frac{n}{2}$, длина n'' второго массива x'' не превосходит $n - 1$. Пусть $n'' \geq n' \geq 1$. На протяжении сортировки массива x' , помимо отложенных заданий, связанных с сортировкой x' , в стеке будет храниться еще одно задание — сортировать x'' . Но в момент, когда сортировка массива x' завершена, в стеке не останется никаких ее следов; таким образом, по предположению индукции, число отложенных заданий в стеке никогда не превзойдет $\max\{\log_2 n' + 1, \log_2 n''\}$. В силу того, что $n' \leq \frac{n}{2}$ и $n'' \leq n - 1$, мы имеем

$$\max\{\log_2 n' + 1, \log_2 n''\} \leq \log_2 n,$$

что и требовалось. □

Полученная оценка числа отложенных заданий, хранящихся в стеке, является оценкой сверху для худшего случая и, тем самым, для этого числа в среднем тоже.

Принимая во внимание теорему 7.1, мы приходим к варианту быстрой сортировки, представляемому рекурсивной процедурой $qsort(k, l)$, обращение к которой обеспечивает сортировку части $x_k, x_{k+1}, \dots, x_l$ исходного массива $x_1, x_2, \dots, x_n$; при $k \geq l$ никаких действий не производится. Сам массив $x_1, x_2, \dots, x_n$ является глобальным по отношению

к процедуре:

```

if  $k < l$  then
   $x_k$  выбирается в качестве разбивающего элемента и выпол-
  няется разбиение  $x_k, x_{k+1}, \dots, x_l$ ; пусть  $i$  — индекс, получае-
  мый разбивающим элементом после разбиения;
  if  $i - k < l - i$  then  $qsort(k, i - 1)$ ;  $qsort(i + 1, l)$  else
     $qsort(i + 1, l)$ ;  $qsort(k, i - 1)$ 
  fi
fi.
```

Быстрая сортировка всего массива является результатом выполнения $qsort(1, n)$.

§ 8. Функция затрат рандомизированного алгоритма

Определение 8.1. Алгоритмы с элементами случайности, реализуемыми обращениями к генераторам случайных чисел, называются *рандомизированными*.

Рандомизированные алгоритмы можно разделить на вероятностные, или, что то же самое, алгоритмы типа *Монте-Карло*, и алгоритмы типа *Лас-Вегас*. Первый тип допускает, что ответ, который дает алгоритм для поставленной задачи с некоторым конкретным входом, может быть неправильным, хотя бы и с малой вероятностью; второй тип — Лас-Вегас — гарантирует правильный ответ, но (как и для алгоритмов типа Монте-Карло) время получения ответа для конкретного входа не определяется, вообще говоря, однозначно этим входом¹. Исключая специально оговариваемые редкие случаи, мы будем рассматривать только алгоритмы типа Лас-Вегас, не упоминая этого каждый раз.

Анализ сложности рандомизированного алгоритма сводится к нахождению математических ожиданий некоторых случайных величин. Но ситуация здесь отличается от той, когда множество входов фиксированного размера рассматривается как вероятностное пространство и затраты алгоритма, однозначно определенные для каждого конкретного входа, становятся случайными величинами на этом пространстве (мы шли этим путем в двух предыдущих параграфах). При исследовании рандомизированных алгоритмов вероятностное пространство, на

¹ Эта классификация является достаточно распространенной в специальной литературе по рандомизированным алгоритмам — см., например, книгу [81], — но не единственной. Например, в [37, разд. 9.2] рандомизированные алгоритмы подразделяются иначе, а именно — на алгоритмы типа Монте-Карло, типа Лас-Вегас и шервудские. Мы не будем останавливаться на этом.

котором рассматриваются случайные величины, состоит из сценариев выполнения алгоритма для фиксированного входа, и каждый сценарий определяется тем, какие случайные числа будут сгенерированы в соответствующие моменты выполнения алгоритма; за каждым таким сценарием закрепляется некоторая вероятность. В нашем контексте генератор случайных чисел можно представлять себе как функцию $random(N)$ целого положительного аргумента N , результатом выполнения которой является элемент множества $\{0, 1, \dots, N - 1\}$, но невозможно предсказать точно, каким именно будет значение этой функции, — любой из элементов указанного множества может появиться с вероятностью $1/N$.

Таким образом, затраты рандомизированного алгоритма при фиксированном входе, вообще говоря, не определяются однозначно, но зависят от сценария вычисления. При фиксированном входе мы можем рассмотреть множество всех сценариев и, приписав адекватным образом каждому из сценариев некоторую вероятность, ввести на полученном вероятностном пространстве случайную величину, значение которой для данного сценария равно соответствующим вычислительным затратам. Значение функции затрат на данном входе можно положить равным математическому ожиданию этой случайной величины (*усредненным* затратам для данного входа). А после того как определена функция затрат и принято соглашение о том, что такое размер входа, мы можем, как обычно, рассматривать сложность алгоритма — в худшем случае или же в среднем (последнее возможно, если только множество входов каждого фиксированного размера является вероятностным пространством).

Мы не будем здесь сколь-либо глубоко входить в общие проблемы теории сложности рандомизированных алгоритмов¹, а ограничимся рассмотрением двух примеров.

Пример 8.1. Массив $x_1, \dots, x_n$ назовем массивом, содержащим большинство, если больше половины его элементов имеют равные значения. Пусть над элементами массивов разрешена операция проверки равенства двух элементов, будем называть эту операцию сравнением. Пусть для данного массива, содержащего большинство, требуется найти какое-то одно i , $1 \leq i \leq n$, такое, что значение x_i принадлежит большинству значений элементов $x_1, \dots, x_n$. Один из возможных рандомизированных алгоритмов решения этой задачи состоит в том, что i выбирается случайно (распределение вероятностей

¹ Разнообразные аспекты теории рандомизированных алгоритмов со многими примерами рассмотрены в [32], [81].

равномерно), а затем проверяется, удовлетворяет ли x_i поставленному условию, — число сравнений для этой проверки равно $n - 1$. Если x_i не подходит, то все повторяется. Сложность этого рандомизированного алгоритма по числу сравнений при рассмотрении n в качестве размера входа меньше чем $2(n - 1)$. В самом деле, для фиксированного входа размера n множество всех возможных сценариев есть множество кортежей вида $(i_1, \dots, i_{m-1}, i_m)$, где $m \geq 1$ и все i_j суть целые от 1 до n , при этом $x_{i_1}, \dots, x_{i_{m-1}}$ не принадлежат большинству, а x_{i_m} принадлежит. Этим сценариям можно естественным образом приписать вероятности. Пространство сценариев разлагается на два события: первая попытка найти нужное i оказывается удачной и, соответственно, эта попытка оказывается неудачной. Пусть $p = \frac{n'}{n}$, где n' — число элементов большинства данного массива (входа). По условию $p > \frac{1}{2}$. Усредненные затраты a удовлетворяют равенству $a = p \cdot (n - 1) + (1 - p) \cdot (a + n - 1)$. Получаем $a = \frac{1}{p}(n - 1) < 2(n - 1)$. Таким образом, для любого входа размера n усредненные затраты меньше чем $2(n - 1)$. Поэтому и сложность алгоритма меньше чем $2(n - 1)$.

Пример 8.2. Вновь обратимся к быстрой сортировке. В § 7 нами установлено, что быстрая сортировка имеет сравнительно низкую сложность в среднем в предположении, что для каждого $n > 0$ все относительные порядки элементов входных массивов длины n имеют одинаковую вероятность $\frac{1}{n!}$. Но в некоторых практических задачах это предположение может оказаться безосновательным в силу того, что все входные массивы изначально оказываются, например, «почти упорядоченными». Число сравнений, затрачиваемых быстрой сортировкой на каждом таком «почти упорядоченном» массиве, будет близко к n^2 , что сведет на нет достоинство быстрой сортировки.

Однако если разбивающий элемент выбирать случайно, то при каждом фиксированном входе размера n усредненные затраты будут величиной порядка $n \log n$, что и будет показано ниже. Итак, под рандомизированной быстрой сортировкой мы будем понимать быструю сортировку со случайным выбором индекса разбивающего элемента; если надо выбрать случайное значение m из $k, k + 1, \dots, l$, то мы полагаем $m := k + \text{random}(l - k + 1)$. В том алгоритме, который записан в виде процедуры в конце § 7, вместо

x_k выбирается в качестве разбивающего элемента и выполняется разбиение $x_k, x_{k+1}, \dots, x_l$; пусть i — индекс, получаемый разбивающим элементом после разбиения;

можно написать

$$m := k + \text{random}(l - k + 1);$$

x_m выбирается в качестве разбивающего элемента и выполняется разбиение для $x_k, x_{k+1}, \dots, x_l$; пусть i — индекс, получаемый разбивающим элементом после разбиения;

это даст рандомизированный вариант быстрой сортировки.

Пусть дан массив $x_1, x_2, \dots, x_n$ попарно различных чисел. Пусть мы выбираем элемент m из множества $\{1, 2, \dots, n\}$ и вероятность выбора каждого из элементов есть $\frac{1}{n}$. Тогда место x_m в массиве $x_1, x_2, \dots, x_n$ после сортировки этого массива может оказаться любым с одной и той же вероятностью $\frac{1}{n}$. Это обстоятельство позволяет переформулировать задачу анализа сложности рандомизированной быстрой сортировки, например, следующим образом. Имеется полоска бумаги шириной в одну клетку и длиной в n клеток, $n \geq 0$, которую надо разрезать на отдельные клетки. Разрезы имеет право производить некий разрезальщик, которому надо платить за работу. При $n = 0$ или $n = 1$ разрезальщик ничего не делает и ничего не получает. В случае $n > 1$ он выбирает случайным образом (тянет из шапки билет с номером) значение i из множества $\{1, 2, \dots, n\}$ и затем вырезает из полоски i -ю клетку (рис. 8), получая за этот этап работы вознаграж-

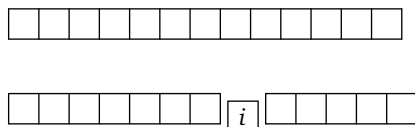


Рис. 8. Этап разрезания полоски.

дение в $n - 1$ рубль. Кроме вырезанной клетки возникают две полоски, длина каждой из которых не превышает $n - 1$, при этом не исключается равенство нулю одной из длин. С каждой из двух полосок разрезальщик продельвает ту же операцию, получая вознаграждение в соответствии с тем же принципом оплаты. Каково математическое ожидание размера вознаграждения, получаемого разрезальщиком за всю работу?

Здесь при фиксированном n множество всех возможных сценариев (будем называть их n -сценариями) — это фактически множество некоторых двоичных деревьев; на рис. 9 показано возможное представление множества всех 3-сценариев. В каждой вершине дерева

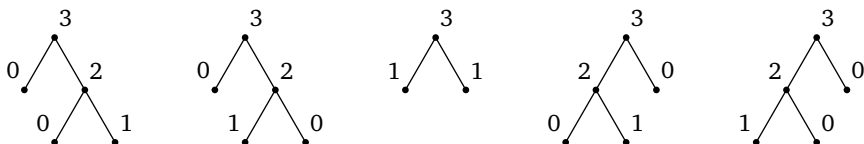
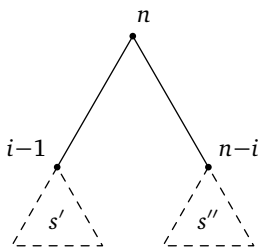


Рис. 9. Сценарии разрезания полосы из трех клеток (3-сценарии).

Рис. 10. Структура n -сценария.

записывается число клеток в полоске. При произвольном $n \geq 0$ понятие n -сценария определяется рекурсивно: 0-сценарий и 1-сценарий — это одна вершина, которой приписано число 0 или соответственно 1; пусть $n > 1$, тогда любой n -сценарий s получается выбором некоторого числа i , $1 \leq i \leq n$, некоторого $(i-1)$ -сценария s' и, независимо, некоторого $(n-i)$ -сценария s'' : из корня n исходят ребра к вершинам $i-1$ и $n-i$, к которым подклеиваются корни сценариев s' и s'' (рис. 10).

Каждому n -сценарию s естественным образом приписывается вероятность $P_n(s)$ (здесь индекс n указывает на то, что вероятность соотнесена с некоторым n -сценарием): если $n = 0$ или $n = 1$, то сценарии имеют вероятность 1, а при $n > 1$

$$P_n(s) = \frac{1}{n} P_{i-1}(s') P_{n-i}(s''). \quad (8.1)$$

Например, третий из 3-сценариев, изображенных на рис. 9, имеет вероятность $\frac{1}{3}$ (из трех клеток выбирается именно вторая), каждый из остальных — вероятность $\frac{1}{6}$.

Формула (8.1) отражает то, что после определения i выбор $(i-1)$ -сценария s' и выбор $(n-i)$ -сценария s'' независимы друг от друга. Эта формула превращает множество всех n -сценариев при фиксированном n в вероятностное пространство S_n . Пользуясь индукцией по n , проверим, например, что $\sum_{s \in S_n} P_n(s) = 1$: при $n = 0$ и $n = 1$ это очевидно, при $n > 1$ имеем

$$\begin{aligned} \sum_{s \in S_n} P_n(s) &= \sum_{i=1}^n \sum_{\substack{s' \in S_{i-1} \\ s'' \in S_{n-i}}} \frac{1}{n} P_{i-1}(s') P_{n-i}(s'') = \\ &= \frac{1}{n} \sum_{i=1}^n \left(\left(\sum_{s' \in S_{i-1}} P_{i-1}(s') \right) \left(\sum_{s'' \in S_{n-i}} P_{n-i}(s'') \right) \right) = \frac{1}{n} \sum_{i=1}^n 1 \cdot 1 = 1. \end{aligned}$$

Плата за разрезание полосы длины n на отдельные клетки полностью определяется использованным n -сценарием, это задает случайную величину χ_n на S_n , и нас интересует математическое ожидание этой величины. Для сценария s , представленного на рис. 10, мы назовем s' и s'' левым и правым *подсценариями*. При $n > 1$ определим на S_n дополнительно к χ_n еще две случайные величины χ'_n и χ''_n , положив $\chi'_n(s) = \chi_{i-1}(s')$ и $\chi''_n(s) = \chi_{n-i}(s'')$, где s' и s'' суть левый и правый подсценарии сценария s . Имеем при $n > 1$

$$\chi_n(s) = n - 1 + \chi'_n(s) + \chi''_n(s).$$

Пусть $n > 1$. Введем разложение

$$S_n = S_n^1 \cup S_n^2 \cup \dots \cup S_n^n, \quad (8.2)$$

где S_n^i — это событие «левый подсценарий данного n -сценария является $(i-1)$ -сценарием (и соответственно правый подсценарий является $(n-i)$ -сценарием)», $i = 1, 2, \dots, n$. Очевидно,

$$P_n(S_n^1) = P_n(S_n^2) = \dots = P_n(S_n^n) = \frac{1}{n},$$

поэтому по формуле полного математического ожидания получаем

$$\begin{aligned} E\chi_n &= n - 1 + \sum_{i=1}^n E(\chi_n | S_n^i) \frac{1}{n} = \\ &= n - 1 + \frac{1}{n} \sum_{i=1}^n (E(\chi'_n | S_n^i) + E(\chi''_n | S_n^i)) = \\ &= n - 1 + \frac{1}{n} \sum_{i=1}^n (E\chi_{i-1} + E\chi_{n-i}) = \\ &= n - 1 + \frac{2}{n} \sum_{i=1}^n E\chi_{i-1} = n - 1 + \frac{2}{n} \sum_{k=0}^{n-1} E\chi_k. \end{aligned}$$

Итак,

$$E\chi_n = n - 1 + \frac{2}{n} \sum_{k=0}^{n-1} E\chi_k, \quad E\chi_0 = 0. \quad (8.3)$$

Такого вида соотношения уже встречались в предыдущем параграфе, и по аналогии с (7.2) мы находим

$$E\chi_n = 2n \ln n + O(n). \quad (8.4)$$

Возвратимся к рандомизированной быстрой сортировке. Для данного массива длины n математическое ожидание затрат, измеряемых числом сравнений, полностью определяется значением n . Поэтому,

например, сложность в худшем случае, т. е. максимум математических ожиданий рассматриваемого вида для массивов длины n , есть просто математическое ожидание, найденное при рассмотрении этого n . Мы можем переписать (8.4) как соотношение для сложности $\tilde{T}_{QS}(n)$ по числу сравнений рандомизированной быстрой сортировки:

$$\tilde{T}_{QS}(n) = 2n \ln n + O(n).$$

Вновь можно заметить, что число перемещений, требуемое рандомизированной быстрой сортировкой для конкретного массива, не превосходит числа сравнений, домноженного на некоторую константу, откуда сложность в среднем $\tilde{T}'_{QS}(n)$ по числу перемещений элементов допускает оценку $\tilde{T}'_{QS}(n) = O(n \log n)$. Как и обычная быстрая сортировка, рандомизированная быстрая сортировка не использует дополнительных массивов, поэтому $\tilde{S}_{QS}(n) = O(1)$.

Сложность рандомизированной быстрой сортировки как по числу сравнений, так и по числу перемещений элементов допускает оценку $\Theta(n \log n)$. Пространственная сложность по числу одновременно хранимых дополнительных величин, равных каким-то элементам исходного массива, допускает оценку $O(1)$.

При использовании принципа «из двух заданий первым выполняется более простое», мы получаем, согласно теореме 7.1, для сложности по объему стековой памяти оценку $\log_2 n$.

Совпадению сложностей $\bar{T}_{QS}(n)$ и $\tilde{T}_{QS}(n)$ можно дать объяснение: случайный выбор разбивающего элемента, предпринимаемый на каждом этапе сортировки, эквивалентен тому, что мы меняем порядок исходного массива случайным образом и затем применяем «обычную» быструю сортировку; но равенство сложностей такого рода не обязано иметь место для других алгоритмов, — все-таки речь идет о математических ожиданиях случайных величин, определенных на разных вероятностных пространствах. Например, можно несколько ускорить рандомизированную быструю сортировку, если разбивающий элемент определять путем случайного выбора без возвращения трех элементов массива и последующего определения второго по величине из этих трех; можно показать¹, что тогда вместо $2n \ln n + O(n)$ возникнет $\frac{12}{7}n \ln n + O(n)$, и совпадения сложностей обычной и рандомизированной быстрой сортировки уже не будет.

Возвращаясь к примеру 3.1, мы видим, что, основываясь на рандомизированной быстрой сортировке, можно получить алгоритм по-

¹ Доказательство имеется, например, в [85, разд. 3.7].

строения выпуклой оболочки данных n точек координатной плоскости, имеющий сложность $O(n \log n)$ по общему числу операций.

Если использовать идею, лежащую в основе быстрой сортировки, для нахождения *медианы* массива длины n , т. е. элемента, являющегося $\left\lfloor \frac{n}{2} \right\rfloor$ -наименьшим (для которого количество меньших и больших его элементов одно и то же или, в случае четного n , различается на 1), то это дает алгоритм со сложностью в среднем $O(n)$ (см. задачу 55). Известны и алгоритмы с такого вида сложностью в худшем случае¹.

При нахождении или оценивании сложности какого-либо рандомизированного алгоритма часто не возникает необходимости детального рассмотрения всевозможных сценариев. Бывает достаточно рассмотреть сходное с (8.2) разложение пространства сценариев и применить формулу полного математического ожидания. Даже если множество всех сценариев бесконечно, разложение вида (8.2) можно выбрать конечным, — это было продемонстрировано в примере 8.1.

Теоретически нет никакого препятствия к тому, чтобы при определении функции затрат рандомизированного алгоритма взять вместо математического ожидания максимум затрат по соответствующим сценариям (при таком подходе рандомизированная быстрая сортировка имеет квадратичную сложность). Но обычно для рандомизированных алгоритмов рассматриваются усредненные затраты.

Правомерен взгляд на рандомизированные алгоритмы, при котором каждому возможному входу сопоставляется вероятностное пространство обычных детерминированных алгоритмов². Если в этой связи вновь обратиться к примеру 8.2, то можно заметить, что для разрезания полоски бумаги каждый из рассматриваемых сценариев задает конкретный детерминированный алгоритм разрезания. Мы вернемся к этому в § 18.

Подведем некоторый итог. Сложность в худшем случае определяет точную верхнюю границу возможных затрат алгоритма для входов данного размера. Нельзя исключить, что эта граница окажется достаточно высокой, а соответствующие худшему случаю входы будут встречаться редко. Если алгоритм применяется многократно к входам данного размера, то можно ожидать, что среднее арифметическое затрат будет близко к сложности в среднем нашего алгоритма. Но при этом предполагается, что входам данного размера адекватно сопоставлены некоторые вероятности, а для такого предположения должны быть достаточные основания (типичное допущение, что все

¹ См. [38], [57, разд. 7.4.9].

² См., например, [81].

входы каждого размера равновероятны, в конкретной ситуации может оказаться безосновательным).

Рассматриваемая нами сложность рандомизированных алгоритмов — это сложность в худшем случае, но она определяется на основе усредненных затрат. Эта «усредненность» заслуживает доверия, так как рандомизированные алгоритмы используют генераторы случайных чисел, в которых распределение вероятностей фиксировано и не зависит от входов.

Подчеркнем, что наше обсуждение касалось рандомизированных алгоритмов типа Лас-Вегас. Что касается алгоритмов типа Монте-Карло — предварительные сведения о них содержатся в приложении Н, — то можно отметить их использование для быстрой проверки (с небольшой вероятностью ошибки) простоты целых чисел¹. Такого рода алгоритмы основываются, в той или иной мере, на вариантах малой теоремы Ферма (см. § 22), предлагающей некоторое необходимое, но не достаточное условие простоты.

При обсуждении рандомизированных алгоритмов в нашей книжке большее внимание уделено алгоритмам типа Лас-Вегас, потому, во-первых, что именно они являются полноценными алгоритмами — их применение всегда дает правильный ответ, это может не иметь места для Монте-Карло²; во-вторых, добавим, что алгоритмы типа Монте-Карло традиционно входят как разделы в курсы численных методов³ и курсы дисциплин, связанных с криптографией⁴, где подробно обсуждаются и исследуются.

В дальнейшем тема алгоритмов типа Монте-Карло в нашей книге не затрагивается (за исключением приложения Н в конце книги), под «рандомизированным алгоритмом» понимается алгоритм типа Лас-Вегас.

Задачи

37. Верно ли, что для рассмотрения сложности в среднем некоторого алгоритма требуется задание распределения вероятностей

- а) на множестве всех допустимых входов?
- б) на каждом из множеств всех входов фиксированного размера?

¹ См., например, [21, разд. 1.3]

² Обстоятельное обсуждение сложности целого ряда алгоритмов типа Монте-Карло — не только проверки каких-то заданных свойств, но и построения разного рода объектов, — можно найти в [32, гл. 3, 4], [81]

³ См., например, [53]

⁴ См., например, [18].

38. Может ли быть так, что затраты некоторого алгоритма A для какого-то входа x превосходят значение $\bar{T}_A(n)$, где n — размер входа x ?

39. Может ли быть так, что для какого-то значения n затраты некоторого алгоритма A для любого входа размера n превосходят значение $\bar{T}_A(n)$?

40. Как говорилось при обсуждении асимптотического закона распределения простых чисел, в [56] приводится элементарное доказательство неравенств Чебышева с $C = 6$, $c = 1/3$. Добавим, что наиболее легко доказывается нижняя оценка

$$\pi(n) > \frac{1}{3} \frac{n}{\ln n}, \quad (8.5)$$

справедливая для всех $n > 1$. Доказать, что сложность в среднем алгоритма пробных делений не является полиномиально ограниченной, используя для этого только неравенство (8.5) и не прибегая к полному варианту асимптотического закона распределения простых чисел.

Указание. Предположение о том, что $V(m) < 2^{3m/4}$ для всех $m > m_0$, позволило бы, исходя из равенства

$$\pi(2^m) = \pi(2^{m_0}) + V(m_0 + 1) + V(m_0 + 2) + \dots + V(m),$$

вывести верхнюю оценку $\pi(2^m) = O(m2^{3m/4})$, что вступило бы в противоречие с неравенством $\pi(2^m) \geq c \frac{2^m}{m}$ с положительной константой c . Поэтому существует последовательность $m_1 < m_2 < \dots$ натуральных чисел таких, что $V(m_i) \geq 2^{3m_i/4}$, $i = 1, 2, \dots$. Можно рассмотреть (5.5) для $m = m_1, m_2, \dots$

41. Пусть $\bar{T}_{TD}^*(m)$ — сложность алгоритма пробных делений для проверки простоты числа n , $m = \lambda(n)$. Верно ли, что $\bar{T}_{TD}^*(m) = \Omega(2^{m/3})$?

42. Сложность по числу перемещений (обменов) в среднем для сортировки выбором при условии, что мы исключаем обмены вида $x_k \leftrightarrow x_k$, не превосходит $n - 2 + \frac{1}{n}$, где n — длина массива.

Указание. В перестановках из Π_n среднее число элементов a_i , $1 \leq i \leq n - 1$, таких, что $a_i = i$, есть $\frac{n-1}{n}$ (см. пример 6.1).

43. Пусть $T_A(n)$, $\bar{T}_A(n)$, — сложности некоторого алгоритма A в худшем случае и в среднем. Может ли быть так, что

а) $\bar{T}_A(n) = \Omega(T_A(n))$,

б) $\bar{T}_A(n) = O(T_A(n))$,

в) $\bar{T}_A(n) = o(T_A(n))$,

г) $\bar{T}_A(n) = \Theta(T_A(n))$?

44. Для некоторых алгоритмов A , B , C обозначим через $T_A(n)$, $T_B(n)$, $T_C(n)$ их сложности в худшем случае, а через $\bar{T}_A(n)$, $\bar{T}_B(n)$,

$\bar{T}_C(n)$ — сложности в среднем. Пусть $T_C(n) = T_A(n) + T_B(n)$, $\bar{T}_C(n) = \bar{T}_A(n) + \bar{T}_B(n)$, $T_A(n)$ допускает оценку $\Omega(n)$, $\bar{T}_B(n)$ допускает оценку $O(n)$. Может ли быть так, что

- а) $T_C(n) = O(n)$, $\bar{T}_C(n) = O(\log n)$;
- б) $T_C(n) = O(n)$, $\bar{T}_C(n) = O(n)$;
- в) $T_C(n) = O(n \log n)$, $\bar{T}_C(n) = \Omega(n^2)$?

45. Найти среднее число присваиваний $m := \dots$ при выполнении алгоритма поиска наименьшего элемента массива:

```

m := x1;
for i = 2 to n do
    if xi < m then m := xi fi
od

```

Предполагается, что все возможные взаимные порядки элементов в исходном массиве длины n являются равновероятными; в качестве размера входа берется n .

Указание. Рассмотреть разложение всего вероятностного пространства в объединение двух событий: последний элемент исходного массива является его наименьшим элементом (вероятность равна $\frac{1}{n}$) и соответственно последний элемент исходного массива не является его наименьшим элементом (вероятность равна $\frac{n-1}{n}$). Пусть $s(n)$ обозначает искомое математическое ожидание. Показать, что условное математическое ожидание исследуемого числа присваиваний, соответствующее первому событию, есть $s(n-1) + 1$, а условное математическое ожидание, соответствующее второму событию, есть $s(n-1)$. Пользуясь формулой полного математического ожидания, вывести отсюда рекуррентное соотношение для $s(n)$ и найти его решение, удовлетворяющее условию $s(1) = 1$.¹

46. Пользуясь дискретным аналогом формулы Ньютона—Лейбница, согласно которому $\sum_{k=n_0}^n (f(k+1) - f(k)) = f(n+1) - f(n_0)$ для $n \geq n_0$,

вычислить сумму $\sum_{k=1}^n \frac{1}{k(k+1)}$, входящую в (7.3).

47. Исходя из (7.3), пользуясь приемом оценки сумм из приложения В и результатом из предыдущей задачи, доказать неравенство $\bar{T}_{QS}(n) \leq 2n \ln n$, из которого будет следовать, что величина $O(n)$ в (7.4) отрицательна.

¹ Более подробный анализ среднего числа присваиваний при поиске наименьшего элемента массива проведен в [25, разд. 1.2.10]: установлена, в частности, оценка не только для среднего (т.е. математического ожидания), но и для дисперсии.

48. Вернемся к задаче 12. В качестве алгоритма с указанными свойствами можно предложить следующий: при наличии вагонов на правой стороне узла к операции В прибегаем лишь тогда, когда на левой стороне невозможно увеличить на 1 число вагонов с правильным чередованием цветов с помощью какой-то из операций МИМО и ИЗ; в остальных случаях выполняется одна из операций МИМО и ИЗ, если возможны обе, то первая из них. Как и в задаче 12, считаем, что черных и белых вагонов по n штук, и n — размер входа. Какова сложность по числу сортировочных операций этого алгоритма в среднем?

*Указание.*¹ Можно доказать следующие три утверждения.

1) Предположим, что в исходном расположении первый вагон на правой стороне белый. Тогда число сортировочных операций, требующихся алгоритму, равно $3n - k$, где k есть количество максимальных сплошных последовательностей черных вагонов в исходном расположении (непосредственно слева и справа от каждой такой сплошной последовательности нет черных вагонов, таков смысл «максимальности»).

2) Вновь предположим, что в исходном расположении первый вагон на правой стороне белый. Тогда количество исходных расположений, в которых имеется ровно k максимальных сплошных последовательностей черных вагонов, есть $\binom{n}{k} \binom{n-1}{k-1}$.

$$3) \sum_{k=1}^{n-1} \binom{n-1}{n-k} \binom{n-1}{k} = \binom{2n-2}{n}.$$

Из первых двух утверждений выводится, что искомая сложность в среднем есть

$$2n + 2 \cdot \frac{\sum_{k=1}^{n-1} (n-k) \binom{n}{k} \binom{n-1}{k-1}}{\binom{2n}{n}},$$

третье утверждение позволяет получить из этого, что интересующая нас сложность есть $\frac{n(5n-3)}{2n-1} = \frac{5}{2}n + \frac{1}{4} + o(1)$.

49. Имеет место равенство $\bar{T}_{QS}(n) = 2(n+1)H_n - 4n$, где $H_n = \sum_{k=1}^n \frac{1}{k}$.

Указание. Преобразовать правую часть равенства (7.3), используя решение задачи 46.

50. (Задача не связана со сложностью в среднем, но приводит к рекуррентному соотношению, которое решается сходно с (7.2).) Доказать, что для любого $l > 0$ можно, не применяя клей, построить из

¹ Решение принадлежит А. П. Фокину.

костяшек домино, длина каждой из которых равна 2, навес длины не меньше чем l (рис. 11). Предложить алгоритм конструирования таких навесов, требующий $O(e^l)$ костяшек.

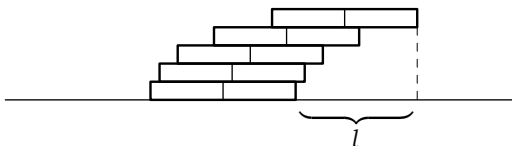


Рис. 11. Навес из костяшек домино.

Указание. Будем характеризовать навес из n костяшек неотрицательными числами $l_1, l_2, \dots, l_n$, где l_i — расстояние от правого края i -й сверху костяшки до вертикальной (пунктирной на рис. 11) линии, проходящей через правый край 1-й, т. е. самой верхней, костяшки. (Таким образом, $l_1 = 0$, $l_n = l$.) Считая, что вес каждой костяшки равен 1, можно показать, что условием равновесия является система неравенств

$$l_{i+1} \leq \frac{(l_1 + 1) + (l_2 + 1) + \dots + (l_i + 1)}{i}, \quad i = 1, 2, \dots, n - 1$$

(каждое такое неравенство означает, что центр тяжести конструкции из i верхних костяшек не выходит за правый край $(i + 1)$ -й костяшки). Самый длинный навес получается тогда, когда числа $l_1, l_2, \dots$ подчинены рекуррентному соотношению

$$l_{i+1} = \frac{(l_1 + 1) + (l_2 + 1) \dots + (l_i + 1)}{i}, \quad l_1 = 0.$$

Далее можно идти тем же путем, что и при решении соотношения (7.2) и т. д.

51. Перестановка $(a_1, a_2, \dots, a_n) \in \Pi_n$ называется *полной*, если $a_i \neq i$, $i = 1, 2, \dots, n$. Вероятность $d_n = P_n(D_n)$ события D_n , состоящего в том, что данная перестановка является полной, равна 0 при $n = 1$ и равна

$$\frac{1}{2!} - \frac{1}{3!} + \dots + \frac{(-1)^n}{n!} \quad (8.6)$$

при $n > 1$ (следствие: $\lim_{n \rightarrow \infty} d_n = \frac{1}{e}$).

Задача о значении d_n широко известна как задача о шляпах. На собрание, состоявшееся поздно вечером, n его участников пришли в шляпах и оставили их в раздевалке, а когда собрание окончилось, разобрали шляпы наугад в темноте (в раздевалке перегорела лампочка) и разошлись по домам. Какова вероятность того, что каждый пришел домой в чужой шляпе?

Указание. Для любого r такого, что $0 \leq r \leq n$, можно рассмотреть вероятность $p(n, r)$ того, что перестановка длины n имеет ровно r неподвижных точек (ровно r человек пришли домой в своих шляпах). Тогда $p(n, 0) + p(n, 1) + \dots + p(n, n) = 1$. Далее надо доказать, что $p(n, r) = \binom{n}{r} \frac{1}{n(n-1)\dots(n-r)} p(n-r, 0) = \frac{1}{r!} p(n-r, 0)$, откуда будет следовать, что

$$\frac{1}{0!} d_n + \frac{1}{1!} d_{n-1} + \frac{1}{2!} d_{n-1} + \dots + \frac{1}{(n-1)!} d_{n-1} + \frac{1}{n!} d_0 = 0.$$

С учетом $d_0 = 1$ последнее соотношение однозначно определяет все $d_1, d_2, \dots$. Остается убедиться, что подстановка значений (8.6) в левую часть этого соотношения дает 0. Подставляя $\frac{1}{0!} - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \dots + \frac{(-1)^r}{r!}$, $r = 0, 1, \dots, n$,

в левую часть этого соотношения, получим $\sum_{j=0}^n \sum_{i=0}^{n-j} \frac{(-1)^i}{j! i!}$ (роль r играет $n-j$).

Слагаемые этой суммы можно перегруппировать так, чтобы внутри группы значение $i+j$ было постоянной величиной l . Это даст

$$\sum_{l=0}^n \sum_{i=0}^l \frac{(-1)^i}{(l-i)! i!} = \sum_{l=0}^n \left(\frac{1}{l!} \sum_{i=0}^l \frac{l!}{(l-i)! i!} (-1)^i \right),$$

последний шаг доказательства очевиден.

52. Как с помощью функции $\text{random}(N)$ определить равновероятный выбор такого целого n , что $k \leq n \leq m$ для заданных целых чисел k, m ?

53. Верно ли, что определение усредненных затрат рандомизированного алгоритма требует задания распределения вероятностей

а) на множестве всех допустимых входов?

б) на каждом из множеств всех входов фиксированного размера?

54. Если измерять затраты рандомизированной быстрой сортировки количеством обращений к генератору случайных чисел, то сложность этой сортировки асимптотически равна $\frac{2}{3}n - \frac{1}{3}$.

Указание. Для этой сложности выполнено соотношение, аналогичное (8.3), но слагаемое $n-1$ в правой части заменяется на 1. Можно воспользоваться решением задачи 46. Нижняя граница суммирования и начальное значение для искомой величины должны быть изменены. (Например, заметим, что при $n=1$ не потребуется обращений к генератору случайных чисел.)

55. Идея, лежащая в основе быстрой сортировки, может быть использована для нахождения m -го наименьшего элемента среди $a_1, a_2, \dots, a_n$, $1 \leq m \leq n$ (самый маленький элемент — это 1-й наименьший, следующий элемент — 2-й наименьший и т. д.). Разбиение массива с использованием случайно выбранного разбивающего элемента

порождает два массива длины $i - 1$ и $n - i$ при некотором $1 \leq i \leq n$. Если $m = i$, то поиск закончен, если $m \leq i - 1$, то далее рекурсивно находится m -й наименьший в первом массиве, а если $m > i$, то рекурсивно находится $(m - i)$ -й наименьший во втором массиве. Можно ввести сложность по числу сравнений при рассмотрении двух компонент n, m размера входа. Беря максимум этих сложностей по всем m таким, что $1 \leq m \leq n$, определяем сложности $T(n)$, $\bar{T}(n)$ по числу сравнений в худшем случае и в среднем с использованием n в качестве размера входа. Показать, что $T(n) = \Theta(n^2)$ и $\bar{T}(n) < 4n$.

Указание. Возьмем $U(n) = \max_{1 \leq k \leq n} \bar{T}(k)$. Очевидно, что $U(n)$ не убывает при возрастании n и что $\bar{T}(n) \leq U(n)$. Доказывается неравенство

$$U(n) \leq \frac{1}{n}(U(n-1) + \max\{U(1), U(n-2)\} + \dots \\ \dots + \max\{U(n-2), U(1)\} + U(n-1)) + n - 1,$$

из которого, в силу неубывания $U(n)$, следует

$$U(n) \leq \frac{2}{n} \left(U(n-1) + U(n-2) + \dots + U\left(\left\lfloor \frac{n}{2} \right\rfloor\right) \right) + n.$$

Отсюда с помощью индукции выводится, что $U(n) < 4n$ для всех $n \geq 1$.

56. В инверсионном векторе $(b_1, b_2, \dots, b_n)$ произвольной перестановки $(a_1, a_2, \dots, a_n) \in \Pi_n$ каждая компонента b_i равна количеству целых j таких, что $1 \leq j < i$ и $a_j > a_i$. Например, инверсионный вектор перестановки $(2, 4, 3, 1, 6, 5)$ — это $(0, 0, 1, 3, 0, 1)$. Показать, что каждому целочисленному вектору $(b_1, b_2, \dots, b_n)$, для которого $0 \leq b_i < i$, $i = 1, 2, \dots, n$, соответствует некоторая перестановка длины n , для которой этот вектор является инверсионным.

Указание. Очевидно, что если $b_n = k$, $1 \leq k < n$, то $a_n = n - k$, и это соответствие приводит к алгоритму построения $(a_1, a_2, \dots, a_n)$, применимому к любому вектору b , удовлетворяющему оговоренным условиям: просматриваем компоненты вектора b , продвигаясь от последней к первой, находим значение соответствующей компоненты перестановки a и удаляем найденное значение из множества M , первоначально равного $\{1, 2, \dots, n\}$; при этом значение a_i , $i = n, n - 1, \dots, 1$, определяется как $(b_i - 1)$ -й наибольший в множестве M (самый большой элемент — это первый наибольший, следующий по величине элемент — это второй наибольший и т.д.).

57. Показать, что один этап пузырьковой сортировки понижает на единицу значение каждой неотрицательной компоненты инверсионного вектора перестановки (см. предыдущую задачу) и не изменяет нулевые компоненты. Показать, что вероятность того, что значение

максимума компонент инверсионного вектора выбранной наугад перестановки длины n равно k , $0 \leq k < n$, есть $\frac{k^{n-k}k!}{n!}$.

Указание ко второй части задачи. Количество векторов $(b_1, b_2, \dots, b_n)$, для каждого из которых $b_i \in \mathbb{N}^+$, $0 \leq b_i < i$, $i = 1, 2, \dots, n$, и $\max\{b_1, b_2, \dots, b_n\} \leq k$, $1 \leq k \leq n-1$, равно $k!k^{n-k-1}$, так как b_i может принимать любое значение между 0 и $i-1$ при $i \leq k$ и любое значение между 0 и $k-1$ для $k < i \leq n$.

58. Показать, что математическое ожидание числа этапов пузырьковой сортировки совпадает с (6.8).

Указание. Пользуясь решением предыдущей задачи, легко получить, что это математическое ожидание есть

$$\frac{1}{n!} \sum_{k=1}^n k(k^{n-k}k! - (k-1)^{n-k+1}(k-1)!),$$

что равно

$$\frac{1}{n!} \left(\sum_{k=1}^n (k^{n-k+1}k! - (k-1)^{n-k+2}(k-1)!) - \sum_{k=1}^n (k-1)^{n-k+1}(k-1)! \right).$$

В упрощении последнего выражения поможет дискретный аналог формулы Ньютона—Лейбница (см. задачу 46).

59. Используя идею решения задачи 55, предложить рандомизированный алгоритм восстановления перестановки длины n по ее инверсионному вектору (см. задачу 56), имеющий сложность в среднем по числу сравнений $O(n^2)$.

60. Пусть $(a_1, a_2, \dots, a_n)$ — произвольная перестановка длины n . Ее преобразование

for $i = n$ downto 2 do $j := 1 + \text{random}(i)$; $a_i \leftrightarrow a_j$ od

может дать любую перестановку длины n с вероятностью $\frac{1}{n!}$. (Получаем алгоритм построения случайных перестановок с равномерным распределением вероятностей.)

Здесь существенно, что $1 + \text{random}(n)$ принимает случайное значение, принадлежащее множеству $\{1, \dots, n\}$ (см. задачу 52).

Этот алгоритм был предложен Р. Фишером и Ф. Ятсом¹. Его сложность по числу обменов, очевидно, равна $n-1$, пространственная же сложность равна нулю. Мы обозначаем далее этот алгоритм начальными буквами FY фамилий его авторов. Алгоритм FY может быть применен не только к перестановке чисел $1, \dots, n$, но и к произволь-

¹ См. [26, с. 171].

ному упорядоченному набору каких-то n попарно различных элементов (можно называть эти наборы n -кортежами), такое применение приводит к изменению порядка следования элементов. Любой конкретный новый порядок возможен с вероятностью $\frac{1}{n!}$.

Указание. Пусть $a = (a_1, \dots, a_n)$, $b = (b_1, \dots, b_n)$ — два n -кортежа, различающиеся не более, чем порядком элементов. Используя индукцию, можно показать, что вероятность совпадения $FY(a)$ с b равна $\frac{1}{n!}$. Очевидно, что утверждение верно для $n = 1$. Пусть $n > 1$. Предположим, что оно верно для $n - 1$. Выполнение FY можно представить двухэтапно:

- 1) $j := 1 + \text{random}(n)$; $a_n \leftrightarrow a_j$;
- 2) FY применяется к $n - 1$ -кортежу $\tilde{a}$, получившемуся на первом этапе заменой в $(a_1, \dots, a_{n-1})$ элемента a_j на a_n при $j \neq n$; при $j = n$ алгоритм FY применяется к $(a_1, \dots, a_{n-1})$.

Далее работают предположение индукции и равенство $\frac{1}{n} \frac{1}{(n-1)!} = \frac{1}{n!}$.

61. Пусть 1 и 0 генерируются с вероятностями p и $1 - p$ соответственно. Каково математическое ожидание числа обращений к этому генератору до первого появления единицы?

Указание.

$$\sum_{s=0}^{\infty} (s+1)(1-p)^s p = -p \frac{d}{dp} \left(\sum_{s=1}^{\infty} (1-p)^s \right) = -p \frac{d}{dp} \frac{1}{1-(1-p)} = \frac{1}{p}.$$

Другой путь получения ответа $\frac{1}{p}$ см. в [40, решение задачи 4].

62. Предположим, что у нас нет другого генератора случайных чисел, кроме генератора, в результате обращения к которому появляется 0 или 1 с одинаковой вероятностью $\frac{1}{2}$ (аналог подбрасывания монеты), и пусть p , $0 \leq p \leq 1$, — заданное вещественное число. Каким образом с помощью этого генератора можно определить генератор $randp$, в результате обращения к которому появляется 0 или 1 с вероятностями соответственно p и $1 - p$ (незначительные отклонения допустимы)? Сложность в среднем алгоритма получения одного случайного числа с помощью $randp$ должна быть меньше 2 (затраты определяются числом обращений к изначально имеющемуся генератору).

Указание. Представить p (возможно, с небольшой погрешностью) в виде конечной суммы вида

$$\frac{a_1}{2} + \frac{a_2}{2^2} + \dots + \frac{a_k}{2^k},$$

где a_i — это 0 или 1 для всех i , а k — некоторое целое положительное число.

Будем использовать s для обозначения искомого числа из множества $\{0, 1\}$. Если как-то сгенерировать «равномерно распределенное» случайное вещественное число $0 \leq b \leq 1$ и окажется, что $b \leq p$, то можно положить $s = 1$, иначе $s = 0$. При реализации этого плана будем с помощью имеющегося генератора порождать двоичные цифры $\beta_1, \beta_2, \dots$ числа b . Процесс останавливается, когда найдено первое такое i , что $\alpha_i \neq \beta_i$, или же будет обнаружено, что $\alpha_i = \beta_i$ для всех $i = 1, \dots, k$. Оценка математического ожидания числа обращений к генератору случайных чисел проводится сходно с получением оценки в указании к задаче 61 при $p = \frac{1}{2}$. Но в данном случае для математического ожидания возникает не бесконечный ряд неотрицательных членов, а его частичная сумма, что дает в итоге значение меньшее, чем 2.

Добавим, что если исходное число p является рациональным, заданным отношением целых $\frac{n}{d}$, $d > 0$, $0 \leq n \leq d$, то процедура $random(N)$ дает приемлемое простое решение:

if $random(d+1) \leq n$ then $s := 1$ else $s := 0$

(так как $random(d+1)$ есть некоторое случайное целое, $r \in \{0, \dots, d\}$, то имеем $\frac{r}{d} \leq \frac{n}{d}$ при $r \leq n$).

63. Пусть изначально у нас имеется генератор случайных вещественных чисел, принадлежащих отрезку $[0, 1]$, и вероятность появления числа, принадлежащего отрезку $[a, b]$, $0 \leq a \leq b \leq 1$, есть $b - a$. Пусть даны неотрицательные вещественные числа $\alpha_0, \alpha_1, \dots, \alpha_{N-1}$ такие, что $\alpha_0 + \alpha_1 + \dots + \alpha_{N-1} = 1$. Как определить генератор чисел, принадлежащих множеству $\{0, 1, \dots, N-1\}$, такой, что вероятность появления числа k , $0 \leq k \leq N-1$, равна α_k ?

64. Предположим, что у нас нет другого генератора случайных чисел, кроме генератора, в результате обращения к которому появляется 0 с вероятностью p или 1 с вероятностью $1 - p$, причем о p мы ничего не знаем, кроме того, что $p \neq 0$ и $p \neq 1$. Как с помощью этого генератора можно сконструировать генератор, в результате обращения к которому появляется 0 или 1 с одинаковой вероятностью $1/2$?

Указание. Порождая подряд две цифры с помощью имеющегося генератора, мы получаем комбинации 0, 1 и 1, 0 с одинаковой ненулевой вероятностью. Комбинации 0, 0 и 1, 1 игнорируются.

65. (Продолжение предыдущей задачи.) Чему равно математическое ожидание числа обращений к изначально имеющемуся генератору случайных чисел при построении последовательности пар до появления 0, 1 или 1, 0? Найти сложность в среднем алгоритма получения k «равновероятных» нулей и единиц с помощью сконструированного генератора (затраты определяются количеством обращений к из-

начально имеющемуся генератору; чтобы воспользоваться решением задачи 61, потребуется определить вероятность того, что очередной парой будет 0, 1 или 1, 0). Можно ли указать значения p , для которых эта сложность имеет минимальное и, соответственно, максимальное значение?

66. Предложенную в указании к задаче 64 идею обобщить на ситуацию, когда необходимо сконструировать генератор $\text{random}(N)$, $N \geq 1$, описанный в § 8. Найти сложность в среднем алгоритма получения k равновероятных случайных чисел из отрезка $[0, N - 1]$ (затраты определяются числом обращений к изначально имеющемуся генератору).

67. Сохранится ли для сложности в среднем формула $2n \ln n + O(n)$, если в задаче о разрезании полосы клетчатой бумаги на отдельные клетки (см. пример 8.2) считать, что плата за вырезание одной случайно выбранной клетки равна не $n - 1$, а n рублей? Тот же вопрос, если эта плата составляет n^2 рублей.

68. Верно ли, что $\bar{T}_{QS}(n) = O(n^2)$?

Ответ. Да, верно, так как $n \log_2 n = O(n^2)$. Верно также, что $\bar{T}_{QS}(n) = O(n^k)$ при любом $k > 1$.

69. Известное утверждение (теорема Дирихле, 1849 г.), что два случайных числа $x, y \in \mathbb{N}^+$ с вероятностью $P = \frac{6}{\pi^2}$ оказываются взаимно простыми, имеет тот смысл, что если ввести в рассмотрение число $M(n)$, равное количеству пар (x, y) таких, что x и y взаимно просты и, дополнительно, $1 \leq x \leq n$, $1 \leq y \leq n$, то предел $\lim_{n \rightarrow \infty} \frac{M(n)}{n^2}$ существует и равен $\frac{6}{\pi^2}$. Предполагая (не обосновывая и не вникая в то, можно ли это обосновать; это соответствует «физическому уровню строгости»), что множество $\mathbb{N}^+$ может быть превращено в вероятностное пространство так, что случайное $x \in \mathbb{N}^+$ кратно фиксированному $d \in \mathbb{N}^+$ с вероятностью $\frac{1}{d}$, можно предложить несколько довольно простых доказательств равенства $P = \frac{6}{\pi^2}$. Для этого можно воспользоваться тем, что

$$\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}$$

(доказано Эйлером в 1734 г.) или свойством дзета-функции Римана, согласно которому

$$\sum_{n=1}^{\infty} \frac{1}{n^s} = \prod_{p \text{ — простое}} \frac{1}{1 - p^{-s}}$$

и которое справедливо, например, для всех целых $s > 1$, и, в частности, для $s = 2$.

В упомянутых выше предположениях доказать равенство $P = \frac{6}{\pi^2}$, используя следующие подходы.

а) Для произвольного $d \in \mathbb{N}^+$ вычислить вероятность $\varphi(d)$ того, что для случайных $x, y \in \mathbb{N}^+$ выполнено $d = \text{нод}(x, y)$. Из равенства $P = 1 - \sum_{d=2}^{\infty} \varphi(d)$ определить P .¹

б) Для произвольного простого p вычислить вероятность $\psi(p)$ того, что по крайней мере одно из случайных $x, y \in \mathbb{N}^+$ не делится на p . Из равенства $P = \prod_{p - \text{простое}} \psi(p)$ определить P .²

Указания. а) $\varphi(d) = \frac{P}{d^2}$; б) $\psi(p) = 1 - \frac{1}{p^2}$.

Для того чтобы сделать доказательство «настоящим», надо для произвольного $n \in \mathbb{N}^+$ детально рассмотреть ситуацию $1 \leq x \leq n$, $1 \leq y \leq n$ и перейти к пределу при $n \rightarrow \infty$, что потребует более кропотливой работы³.

¹ См. [26, разд. 4.5.2].

² См. [71, гл. I, разд. 3, прим. 3.3].

³ См. [26, разд. 4.5.2, упр. 10].

Последовательность получаемых остатков убывает (остаток меньше делителя), при этом все остатки — неотрицательные целые. Не существует убывающей бесконечной последовательности, элементами которой являются неотрицательные целые числа, поэтому выполнение алгоритма Евклида (будем обозначать его буквой E) завершается для любых натуральных a_0, a_1 , и число делений с остатком не пре-

восходит a_1 . Обозначив через $C_E(a_0, a_1)$ исследуемое число делений с остатком, получаем

$$C_E(a_0, a_1) \leq a_1 \quad (9.2)$$

(упрощая запись, мы пишем $C_E(a_0, a_1)$ вместо $C_E^T(a_0, a_1)$). Это говорит о том, что если a_1 рассматривается как размер входа или если a_0, a_1 рассматриваются как две компоненты размера входа, то сложность алгоритма Евклида по числу делений не превосходит a_1 . Как мы увидим, эта оценка является весьма грубой, но, привлекая сходные рассуждения, можно получать и более тонкие оценки.

Формализуем те соображения, которые привели нас к этой первой оценке. Каждый шаг алгоритма имеет дело с парой неотрицательных целых (a_{i-1}, a_i) и при условии $a_i \neq 0$ перерабатывает эту пару в (a_i, a_{i+1}) , где a_{i+1} равно остатку от деления a_{i-1} на a_i . На множестве пар неотрицательных целых чисел k, l мы определяем функцию $L(k, l)$, принимающую неотрицательные целые значения. Эта функция такова, что когда при выполнении алгоритма Евклида мы переходим от пары (a_{i-1}, a_i) к паре (a_i, a_{i+1}) , значения функции убывают: $L(a_{i-1}, a_i) > L(a_i, a_{i+1})$. Так как функция целочисленна, то ее значение убывает с каждым шагом по крайней мере на единицу. Отсюда следует, что общее число шагов не превзойдет значения функции L на исходной паре чисел. Мы рассмотрели функцию $L(k, l) = l$, и это привело нас к выводу, что число шагов не превзойдет значения $L(a_0, a_1) = a_1$.

Для обоснования того, что число делений в алгоритме Евклида растет медленнее, чем a_1 , было бы достаточно найти соответствующую функцию $L(k, l)$, значения которой при больших k, l оказываются существенно меньшими, чем l . Эта функция по-прежнему должна быть определена для любых неотрицательных целых k, l , $k \geq l$, принимать неотрицательные целые значения и убывать при замене пары k, l парой l, r , где r — остаток от деления k на l . Хотя бы одна пара целых неотрицательных k, l , $k \geq l$, должна обращать $L(k, l)$ в нуль, иначе вместо $L(k, l)$ можно взять функцию $L'(k, l) = L(k, l) - 1$ и т. д.

Предложение 9.1. Пусть $k, l \in \mathbb{N}^+$, $k > l$, и пусть r — остаток от деления k на l . Тогда

(i) $\lambda(k) > \lambda(r)$, где, как обычно, $\lambda(\cdot)$ — битовая длина данного числа;

(ii) функция

$$L(k, l) = \lambda(k) + \lambda(l) - 2 \quad (9.3)$$

такова, что $L(k, l) > L(l, r)$.

Доказательство. (i) Имеем $k = ql + r$, $q \geq 1$, откуда $k \geq l + r > 2r$. Следовательно, $\lambda(k) > \lambda(r)$.

(ii) Из $\lambda(k) > \lambda(r)$ следует $\lambda(k) + \lambda(l) - 2 > \lambda(l) + \lambda(r) - 2$. $\square$

Очевидно, что функция (9.3) неотрицательна. Таким образом, справедлива оценка

$$C_E(a_0, a_1) \leq \lambda(a_0) + \lambda(a_1) - 2. \quad (9.4)$$

Но если a_0 очень велико в сравнении с a_1 , то $\lambda(a_0) + \lambda(a_1) - 2 > a_1$. Тем не менее, теперь для $C_E(a_0, a_1)$ уже легко выводится хорошая оценка. Пусть число делений с остатком больше 1. Имеем

$$C_E(a_0, a_1) = C_E(a_1, a_2) + 1 \leq \lambda(a_1) + \lambda(a_2) - 1 \leq 2\lambda(a_1) - 1.$$

Оценка

$$C_E(a_0, a_1) \leq 2\lambda(a_1) - 1, \quad (9.5)$$

очевидно, верна и в случае единственного деления: $C_E(a_0, a_1) = 1$ при том, что $\lambda(a_1) \geq 1$.

Если взять a_1 за размер входа (a_0, a_1) , то для сложности по числу делений будем иметь

$$T_E(a_1) = \max_{a_0 \geq a_1} C_E(a_0, a_1) \leq 2\lambda(a_1) - 1 = 2\lfloor \log_2 a_1 \rfloor + 2 - 1 \leq 2 \log_2 a_1 + 1.$$

Доказанное нами можно переформулировать так:

Если рассматривать a_1 как размер входа a_0, a_1 алгоритма Евклида, то для сложности $T_E(a_1)$ по числу делений выполнено неравенство

$$T_E(a_1) \leq 2 \log_2 a_1 + 1. \quad (9.6)$$

Эта же оценка имеет место и при рассмотрении двух компонент a_0, a_1 размера входа.

Для достаточно больших a_1 верхняя оценка (9.6) существенно лучше, чем $T_E(a_1) \leq a_1$. (Несколько более точная, чем (9.6), оценка дается в задаче 76.)

Рассматривая далее оценки сложности по числу делений алгоритма Евклида, мы будем использовать значение a_1 в качестве размера входа (значение a_0 может быть очень большим в сравнении с a_1 , но первое же деление с остатком приведет к a_1, a_2 , где $a_2 < a_1$).

Известно, что алгоритм Евклида допускает разнообразные обобщения и расширения. Прежде всего, вместе с $\text{нод}(a_0, a_1)$ можно находить целые s, t такие, что

$$sa_0 + ta_1 = \text{нод}(a_0, a_1). \quad (9.7)$$

Чтобы это показать, обратимся к (9.1). Пусть уже известны $a_0, a_1, \dots, a_i, 1 \leq i \leq n-1$, и пусть для них найдены множители $s_0, t_0; s_1, t_1; \dots; s_i, t_i$ такие, что

$$s_0 a_0 + t_0 a_1 = a_0, \quad s_1 a_0 + t_1 a_1 = a_1, \quad \dots, \quad s_{i-1} a_0 + t_{i-1} a_1 = a_{i-1}, \quad s_i a_0 + t_i a_1 = a_i.$$

После выполнения очередного деления с остатком $a_{i-1} = q_i a_i + a_{i+1}$ имеем $a_{i+1} = a_{i-1} - q_i a_i$ и

$$(s_{i-1} - q_i s_i) a_0 + (t_{i-1} - q_i t_i) a_1 = a_{i+1}.$$

Можем положить

$$s_{i+1} = s_{i-1} - q_i s_i, \quad t_{i+1} = t_{i-1} - q_i t_i, \quad i = 1, \dots, n-1; \quad (9.8)$$

при этом

$$s_0 = 1, \quad t_0 = 0; \quad s_1 = 0, \quad t_1 = 1. \quad (9.9)$$

Решение поставленной задачи дают числа

$$s = s_n, \quad t = t_n.$$

В процессе применения этого алгоритма к числам $a_0 = 39, a_1 = 15$ возникают остатки 9, 6, 3, 0 и соответствующие ненулевым остаткам пары множителей суть 1, -2; -1, 3; 2, -5. Имеем $2 \cdot 39 + (-5) \cdot 15 = 3 = \text{нод}(39, 15)$.

Этот алгоритм мы назовем *расширенным* алгоритмом Евклида и будем его обозначать буквами ЕЕ, от его английского названия extended Euclidean — расширенный евклидов. Каждый шаг расширенного алгоритма Евклида содержит три мультипликативных операции — одно деление с остатком и два умножения.

Если рассматривать a_1 как размер входа a_0, a_1 расширенного алгоритма Евклида, то для мультипликативной сложности $T_{\text{ЕЕ}}(a_1)$ этого алгоритма имеем $T_{\text{ЕЕ}}(a_1) \leq 6 \log_2 a_1 + 3$.

Расширенный алгоритм Евклида дает возможность решать в целых числах линейные уравнения с целыми коэффициентами (см. задачи 80, 81), он также играет существенную роль в модулярной арифметике (см. § 22).

Если дополнить расширенный алгоритм Евклида еще одним шагом, то получатся s_{n+1}, t_{n+1} такие, что

$$s_{n+1} a_0 + t_{n+1} a_1 = 0. \quad (9.10)$$

Например, для $a_0 = 39, a_1 = 15$ имеем $s_5 = -5, t_5 = 13$.

Можно непосредственно проверить, что выполнены равенства $s_2 = 1$, $t_2 = -q_1$, $s_3 = -q_2$, $t_3 = 1 + q_1q_2$. Получаем

$$|s_2| \leq |s_3|, \quad |t_2| < |t_3|. \quad (9.11)$$

Можно также показать, что

$$|s_3| < |s_4| < \dots < |s_{n+1}|, \quad |t_3| < |t_4| < \dots < |t_{n+1}|. \quad (9.12)$$

Несколько труднее, но также возможно доказать, что $|s_i|$ и $|t_i|$ взаимно просты при $i = 1, 2, \dots, n+1$ (см. задачу 82). Из (9.10) и взаимной простоты s_{n+1} и t_{n+1} следует

$$|s_{n+1}| = \frac{a_1}{\text{НОД}(a_0, a_1)}, \quad |t_{n+1}| = \frac{a_0}{\text{НОД}(a_0, a_1)}.$$

При $n \geq 2$ это вместе с (9.11), (9.12) дает

$$|s_n| \leq a_1, \quad |t_n| < a_0. \quad (9.13)$$

Эти неравенства пригодятся нам в дальнейшем.

Пример 9.2. Бинарный поиск места (т.е. значения p , $1 \leq p \leq n+1$, — как объяснено в приложении А) элемента y в упорядоченном массиве из n элементов $x_1 < x_2 < \dots < x_n$:

```
p := 1; q := n + 1;
while p < q do
  r := ⌊ (p + q) / 2 ⌋; if xr < y then p := r + 1 else q := r fi
od
```

Обозначим этот алгоритм буквами BS от его английского названия binary search — бинарный поиск. Будем считать число элементов сегмента массива *длиной* этого сегмента (при рассмотрении задач сортировки и поиска мы называем *сегментом* массива любую упорядоченную часть $x_s < x_{s+1} < \dots < x_{t-1} < x_t$ данного массива). Легко видеть, что от сегмента длины k мы переходим к сегменту длины $\left\lfloor \frac{k}{2} \right\rfloor$ или $\left\lfloor \frac{k}{2} \right\rfloor - 1$. Это говорит о том, что с каждым шагом алгоритма функция $L(k) = \lambda(k)$, где положительное k является длиной сегмента, рассматриваемого на данном шаге, убывает по крайней мере на единицу, пока не приходим к сегменту, содержащему один элемент, после чего еще одно сравнение полностью решает задачу. Отсюда сложность бинарного поиска не превосходит $\lambda(n) = \lfloor \log_2 n \rfloor + 1$. Мы видим также, что если y меньше минимального элемента рассматриваемого сегмента длины $k > 1$, то длина сегмента на следующем шаге будет равна $\left\lfloor \frac{k}{2} \right\rfloor$. Поэтому если изначально $y < x_1$, то в ходе бинарного

поиска будут рассматриваться сегменты длины

$$n, \left\lfloor \frac{n}{2} \right\rfloor, \left\lfloor \frac{\left\lfloor \frac{n}{2} \right\rfloor}{2} \right\rfloor, \dots, 1$$

соответственно (битовая длина каждого следующего элемента последовательности на единицу меньше битовой длины предыдущего), и в целом потребуется в точности $\lambda(n) = \lfloor \log_2 n \rfloor + 1$ сравнений. Используя утверждение, содержащееся в задаче 3, мы можем сформулировать установленное свойство бинарного поиска так:

Сложность $T_{BS}(n)$ бинарного поиска места элемента в массиве длины n по числу сравнений равна $\lfloor \log_2 n \rfloor + 1$, или, что то же самое, $\lceil \log_2(n+1) \rceil$.

Выражение $\lceil \log_2(n+1) \rceil$ для указанной сложности иногда бывает удобнее, чем $\lfloor \log_2 n \rfloor + 1$, потому что оно имеет смысл и в вырожденном случае $n = 0$.

Бинарный поиск находит широчайшее применение при поиске информации в разнообразных таблицах. Укажем здесь еще одно его применение, касающееся вычислительной геометрии: он позволяет быстро определять, принадлежит ли произвольная точка Q выпуклому n -угольнику, заданному вершинами $P_1, P_2, \dots, P_n$. Можно легко построить какую-нибудь внутреннюю точку O данного n -угольника. В силу его выпуклости точка Q — внутренняя, если и только если Q и O лежат в одной полуплоскости относительно любой из прямых $P_1P_2, \dots, P_{n-1}P_n, P_nP_1$. Это соображение приводит к имеющему временную сложность $\Theta(n)$ алгоритму. Но допустим, что проведены n добавочных лучей (рис. 12), исходящих из точки O и проходящих через

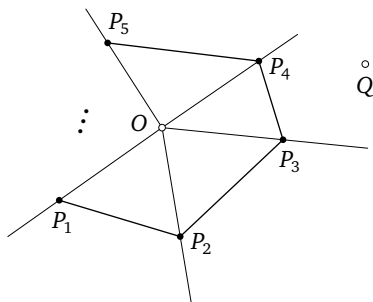


Рис. 12. Точка Q лежит между двумя лучами, проведенными из внутренней точки O многоугольника через его вершины.

вершины (считаем, что $O \neq Q$, иначе мы сразу бы заключили, что Q принадлежит многоугольнику). Можно установить, какому из углов $\angle P_1OP_2, \dots, \angle P_{n-1}OP_n, \angle P_nOP_1$ принадлежит точка Q : если углы пронумерованы в указанном порядке, то бинарным поиском определяется номер m угла, $1 \leq m \leq n$; при этом если Q лежит на одном из проведенных лучей, то из двух значений m берется любое. Узнав m , мы проверяем согласованность расположения точек O и Q по отношению к прямой, являющейся продолжением той стороны многоугольника, концы которой лежат на сторонах m -го угла.

Теперь заметим, что в самом проведении лучей $OP_1, OP_2, \dots, OP_n$ нет необходимости: сравнение $\angle P_1OQ$ с $\angle P_1OP_i$ требует ограниченно-го числа операций и в том случае, когда нам лишь известны координаты точек O, Q, P_1, P_i .

Основывающийся на бинарном поиске алгоритм распознавания принадлежности точки выпуклому n -угольнику имеет сложность $O(\log n)$ по общему числу операций и пространственную сложность $O(1)$.

Полезным для решения ряда задач является то обстоятельство, что если точка не принадлежит данному выпуклому n -угольнику, то с помощью этого алгоритма мы дополнительно определяем одну из сторон n -угольника, которая видна из этой точки (рис. 13).

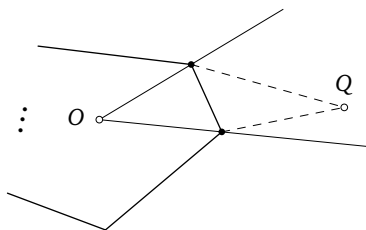


Рис. 13. Для точки Q , не принадлежащей данному выпуклому n -угольнику, находим сторону n -угольника, которая видна из Q .

Пример 9.3. Установим число этапов слияния при сортировке, предложенной Дж. фон Нейманом (которая является одним из вариантов сортировки слияниями). При сортировке фон Неймана шаг за шагом происходят переброски элементов исходного массива в дополнительный массив и наоборот, и каждая переброска сопровождается слиянием соседних сегментов массива (рис. 14). От числа сегмен-

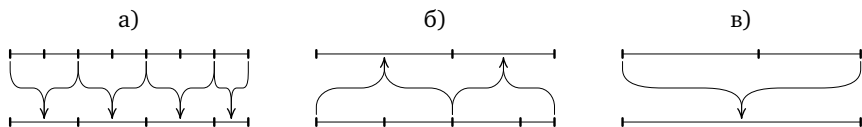


Рис. 14. Три последовательных шага сортировки фон Неймана, удлиняющих упорядоченные участки (сегменты): а) переход от семи сегментов к четырем; б) переход от четырех сегментов к двум; в) переход от двух сегментов к одному (массив полностью упорядочен).

тов k (первоначально $k = n$) происходит переход к $k' = \left\lceil \frac{k}{2} \right\rceil$. Если при некотором целом $m \geq 1$ выполнено $2^{m-1} < k \leq 2^m$, то после одного этапа будем иметь $2^{m-2} < k' \leq 2^{m-1}$, из этого получаем, что функция $L(k) = \lceil \log_2 k \rceil$, где k — текущее число построенных сегментов, убывает с каждым шагом ровно на единицу. Таким образом, число этапов слияния, или число перебросок сортируемых элементов из массива в массив, равно $\lceil \log_2 n \rceil$. Это позволяет утверждать следующее:

Сложность сортировки фон Неймана по числу сравнений меньше, чем $n \lceil \log_2 n \rceil$; сложность по числу присваиваний равна $n \lceil \log_2 n \rceil$.

В самом деле, каждый этап слияния, сопровождаемый переброской элементов из массива в массив, требует n присваиваний. Число сравнений при каждом слиянии по крайней мере на единицу меньше, чем длина получаемого упорядоченного сегмента: известно, что число тех сравнений, которые априори могут потребоваться для слияния двух упорядоченных массивов, содержащих соответственно u и v элементов, $u \leq v$, лежит в диапазоне от u до $u + v - 1$ (обе указанные границы достижимы).

Вновь обращаясь к примеру 3.1, мы видим, что, основываясь на сортировке фон Неймана, можно получить алгоритм построения выпуклой оболочки данных n точек координатной плоскости, имеющий сложность $O(n \log n)$ в худшем случае по общему числу операций.

Некоторое различие между примером 9.1 и примерами 9.2, 9.3 состоит в том, что в примере 9.1 мы определяли L как функцию самого входа алгоритма и из полученной оценки для затрат выводили оценку для сложности (переходили от входа как такового к его размеру), а в 9.2, 9.3 мы изначально рассматривали L как функцию размера. В этих двух последних примерах значения функции L возникали из сравнений размера n с элементами некоторой последовательности s_m ,

$m = 0, 1, \dots$ (в обоих случаях было $s_m = 2^m$). В примере 9.2 удобно было считать, что $L(n) = m$, если $s_{m-1} \leq n < s_m$, а в примере 9.3 — если $s_{m-1} < n \leq s_m$.

Типичный итерационный алгоритм содержит цикл, в котором набор v начальных величин преобразуется в набор v' , затем v' преобразуется в v'' и т. д. Функция L , которую мы подбираем, должна принимать неотрицательные значения и быть определенной либо на самих наборах величин, либо на их размерах; в обоих случаях функция должна убывать по ходу выполнения алгоритма:

$$L(v) > L(v') > L(v'') > \dots$$

в первом случае, или

$$L(\|v\|) > L(\|v'\|) > L(\|v''\|) > \dots$$

во втором. Значение $L(v)$ или соответственно $L(\|v\|)$ дает верхнюю границу для числа шагов цикла¹.

Заметим попутно, что, исходя из установленной в примере 9.2 сложности бинарного поиска, мы сразу можем получить верхнюю оценку

$$T_B(n) \leq (n - 1) \lceil \log_2 n \rceil \quad (9.14)$$

для сложности по числу сравнений сортировки бинарными вставками; мы используем для обозначения этой сортировки букву B , от английского слова *binary* — бинарный. По числу перемещений сложность этой сортировки квадратична, здесь эта сортировка уступает сортировке фон Неймана. Пространственную сложность сортировки бинарными вставками можно считать равной нулю (все делается на том же месте), а для сортировки фон Неймана эта сложность есть n .

Пример 9.4. Число итераций численных алгоритмов тоже часто оценивают сравнением размеров данных, соответствующих текущим итерациям, с элементами последовательности, которая может иметь, например, вид q^n или q^{2^n} ($q < 1$), $n = 0, 1, \dots$, и т. д. Этим способом получаются оценки числа итераций для классических методов (алгоритмов) приближенного решения уравнений. Вспомним две такие оценки.

¹ Таковую функцию можно назвать *функцией Ляпунова* цикла, подразумевая аналогию с функцией, убывающей вдоль решения обыкновенного дифференциального уравнения (аналогия принадлежит С. С. Лаврову, см. [34, разд. 3.1.3]). Это не единственная аналогия между дифференциальными уравнениями и циклическими алгоритмами и программами. Например, понятие первого интеграла, или закона сохранения, сопоставимо с понятием инварианта цикла и т. д.

Пусть корень уравнения $f(x) = 0$ разыскивается на отрезке $[a, b]$, на котором функция $f(x)$ непрерывна, $f(a)f(b) \leq 0$. Ошибка не должна превышать данного $\varepsilon > 0$. При $\varepsilon \rightarrow 0$ и фиксированных $f(x)$, a , b число итераций метода делений пополам есть

$$\log_2 \frac{1}{\varepsilon} + O(1). \quad (9.15)$$

Метод же Ньютона (касательных) требует

$$O\left(\log \log \frac{1}{\varepsilon}\right) \quad (9.16)$$

итераций при соблюдении ограничений на функцию $f(x)$, обеспечивающих сходимость метода.

§ 10. Качество оценок

После вывода оценки сложности алгоритма часто возникает вопрос о том, насколько эта оценка хороша и нельзя ли входящие в оценку константы и функции заменить другими так, чтобы оценка стала более точной, и, как следствие, несущей больше информации об исследуемом алгоритме.

Пример 10.1. В примере 9.1 для сложности $T_E(a_1)$ алгоритма Евклида по числу делений мы легко получили верхнюю оценку a_1 , а затем, после некоторых усилий, пришли к логарифмической верхней оценке (9.6). Отвлекаясь от значений констант, перейдем от (9.6) к

$$T_E(a_1) = O(\log a_1). \quad (10.1)$$

Нами пока не доказано, что оценка (10.1) является точной, и, как следствие, что не верна, скажем, оценка $O(\sqrt{\log a_1})$ или $O(\log \log a_1)$ и т. д. Чтобы доказать точность (10.1), достаточно подобрать для алгоритма Евклида последовательность входов

$$(a_0^{(1)}, a_1^{(1)}), \quad (a_0^{(2)}, a_1^{(2)}), \quad \dots$$

таких, что последовательность $a_1^{(n)}$ (последовательность размеров входов) неограниченно возрастает и $C_E(a_0^{(n)}, a_1^{(n)}) = \Omega(\log a_1^{(n)})$ при $n \rightarrow \infty$; тогда, очевидно, будем иметь $T_E(a_1^{(n)}) = \Omega(\log a_1^{(n)})$.

Фактически, нам нужна последовательность таких входов возрастающего размера, для каждого из которых алгоритм Евклида работает медленно. Подойдет, например, последовательность входов (F_{n+1}, F_n) , $n = 0, 1, \dots$, где $F_0, F_1, F_2, \dots$ — числа Фибоначчи, определя-

емые равенствами $F_0 = 0$, $F_1 = 1$ и правилом «каждое следующее число равно сумме двух предыдущих», т. е. рекуррентным соотношением $F_{n+2} = F_{n+1} + F_n$. Из определения чисел Фибоначчи следует, что применение алгоритма Евклида к F_{n+1}, F_n при $n \geq 1$ требует n делений (все частные будут равны единице), поэтому $C_E(F_{n+1}, F_n) = n$.

По индукции доказывается, что для всех неотрицательных n справедлива формула Бине:

$$F_n = \frac{1}{\sqrt{5}}(\phi^n - (-\phi)^{-n}), \quad (10.2)$$

где

$$\phi = \frac{1 + \sqrt{5}}{2} = 1,61803\dots$$

(деление отрезка в отношении $1 : \phi$ называют *золотым сечением*). Так как $\phi > 1$, то из (10.2) получаем

$$\phi^n = \sqrt{5}F_n(1 + o(1)), \quad (10.3)$$

и, поскольку $\log_\phi(1 + o(1)) = o(1)$, с помощью логарифмирования (10.3) по основанию ϕ имеем

$$C_E(F_{n+1}, F_n) = n = \log_\phi F_n + O(1) = \Omega(\log F_n),$$

откуда следует, что оценка (10.1) точная.

Мы также докажем следующее утверждение:

$$T_E(a_1) > \frac{1}{4} \log_\phi a_1 + \gamma, \quad (10.4)$$

где γ — некоторая константа. Это вместе с (10.1) даст

$$T_E(a_1) = \Theta(\log a_1). \quad (10.5)$$

Приступая к доказательству, мы перейдем от функции $C_E(a_0, a_1)$, имеющей два аргумента, к функции одного аргумента. Каждому входу (a_0, a_1) , $a_0 \geq a_1 > 0$, мы сопоставим рациональное число $r = \frac{a_0}{a_1} \geq 1$, которое однозначно определяет число шагов алгоритма Евклида для a_0, a_1 . (Пусть $\frac{a_0}{a_1} = \frac{\tilde{a}_0}{\tilde{a}_1}$, $\tilde{a}_0$ и $\tilde{a}_1$ взаимно просты, $a_0 = u\tilde{a}_0$, $a_1 = u\tilde{a}_1$; тогда остатки $a_2, a_3, \dots$ и $\tilde{a}_2, \tilde{a}_3, \dots$, возникающие в ходе применения алгоритма Евклида к a_0, a_1 и соответственно к $\tilde{a}_0, \tilde{a}_1$, отличаются лишь множителем u .) Будем обозначать это число шагов через $C'_E(r)$. Таким образом, если $r = \frac{a_0}{a_1}$, то $C'_E(r) = C_E(a_0, a_1)$.

Для дальнейшего будет полезным следующее свойство чисел Фибоначчи:

$$F_n^2 - F_{n-1}F_{n+1} = (-1)^n, \quad n = 1, 2, \dots \quad (10.6)$$

(доказывается индукцией по n). Рассмотрим вложенные отрезки $\Phi_1 \supset \Phi_2 \supset \Phi_3 \supset \dots$ числовой прямой:

$$\Phi_n = \left[\frac{F_{2n}}{F_{2n-1}}, \frac{F_{2n+1}}{F_{2n}} \right], \quad n = 1, 2, \dots$$

Соотношение (10.6) позволяет установить, что ни один из этих отрезков не пуст и что длина отрезка Φ_n равна $\frac{1}{F_{2n-1}F_{2n}}$. Очевидно, что $\Phi_1 = [1, 2]$ (рис. 15). Далее можно доказать, что если рациональное

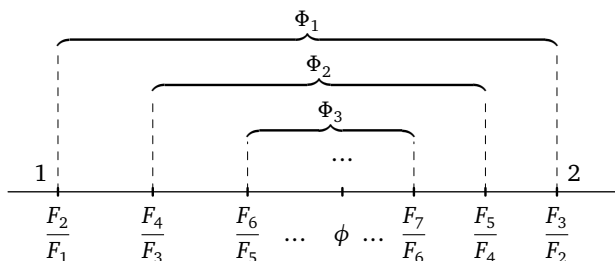


Рис. 15. Расположение чисел $\frac{F_{n+1}}{F_n}$ и интервалов Φ_n , $n = 1, 2, \dots$

число $\frac{a_0}{a_1}$ принадлежит Φ_n , $n > 2$, то деление a_0 на a_1 дает ненулевой остаток a_2 , и $\frac{a_1}{a_2}$ принадлежит Φ_{n-1} . В самом деле, мы имеем

$$1 < \frac{F_{2n}}{F_{2n-1}} \leq \frac{a_0}{a_1} \leq \frac{F_{2n+1}}{F_{2n}} < 2.$$

Следовательно, частное от деления a_0 на a_1 с остатком равно 1, т. е. $a_2 = a_0 - a_1$. Далее,

$$\frac{a_0 - a_1}{a_1} = \frac{a_0}{a_1} - 1,$$

поэтому

$$\frac{F_{2n}}{F_{2n-1}} - 1 \leq \frac{a_2}{a_1} \leq \frac{F_{2n+2}}{F_{2n}} - 1.$$

Но

$$\frac{F_{2n}}{F_{2n-1}} - 1 = \frac{F_{2n} - F_{2n-1}}{F_{2n-1}} = \frac{F_{2n-2}}{F_{2n-1}},$$

и, аналогично, $\frac{F_{2n+1}}{F_{2n}} - 1 = \frac{F_{2n-1}}{F_{2n}}$; мы имеем, следовательно, $\frac{F_{2n-2}}{F_{2n-1}} \leq \frac{a_2}{a_1} \leq \frac{F_{2n-1}}{F_{2n}}$ и

$$\frac{F_{2n}}{F_{2n-1}} \leq \frac{a_1}{a_2} \leq \frac{F_{2n-1}}{F_{2n-2}}. \quad (10.7)$$

Получаем

$$\frac{a_1}{a_2} \in \left[\frac{F_{2n}}{F_{2n-1}}, \frac{F_{2n-1}}{F_{2n-2}} \right] \subset \left[\frac{F_{2n-2}}{F_{2n-3}}, \frac{F_{2n-1}}{F_{2n-2}} \right] = \Phi_{n-1}.$$

Отрезки $\Phi_2, \Phi_3, \dots$ не содержат целых чисел, поэтому из доказанного можно заключить, что если $r \in \Phi_n \cap \mathbb{Q}$, $n > 2$, то $C'_E(r) \geq n$.

Пусть теперь фиксировано $a_1 \in \mathbb{N}^+$. Рассмотрим множество

$$M = \left\{ r : r = \frac{a_0}{a_1}, a_0 = a_1, a_1 + 1, \dots \right\}.$$

Точки, соответствующие элементам множества M , расположены на правой полуоси, начиная с 1, с шагом $\frac{1}{a_1}$; хотя бы одна из этих точек непременно принадлежит отрезку Φ_n , коль скоро $\frac{1}{a_1}$ меньше дли-

ны отрезка Φ_n , т. е. $\frac{1}{a_1} \leq \frac{1}{F_{2n-1}F_{2n}}$, или, что то же самое, $a_1 \geq F_{2n-1}F_{2n}$.

Пусть N — самое большое значение n , для которого выполняется это неравенство (такое N существует, так как единственной точкой, принадлежащей бесконечному числу отрезков рассматриваемого семейства, является $\phi \notin \mathbb{Q}$). Тогда $F_{2N}F_{2N+1} > a_1$. Из формулы Бине следует, что

$$F_{2N}F_{2N+1} = \frac{1}{5} \phi^{4N} (1 - (-\phi)^{-4N}) (\phi - (-\phi)^{-4N-1}),$$

откуда

$$F_{2N}F_{2N+1} < c \phi^{4N},$$

где c — некоторая положительная константа. Таким образом, $c \phi^{4N} > a_1$ и

$$N > \frac{1}{4} \log_{\phi} a_1 - \log_{\phi} c.$$

Имеем $C'_E(r) \geq N - 1 > \frac{1}{4} \log_{\phi} a_1 - \log_{\phi} c - 1$ по крайней мере для одного рационального числа $r \in M$, откуда следует соотношение (10.4)¹.

Для перехода к размеру входа $m = \lambda(a_1)$ заметим, что если m фиксировано, то для любого a_1 с такой битовой длиной выполнено

¹ Приведенный выше и дополненный в указании к задаче 79 элементарный вывод оценки (10.4) принадлежит М. Оналбекову; в [26, разд. 4.5.3, упр. 38] со ссылкой на статью Я. Микусинского (1954 г.) приведена оценка, фактически совпадающая с (10.4), но ее вывод опирается на теорию цепных дробей и не столь элементарен и нагляден.

$\log_2 a_1 < m$. Поэтому из (9.6) следует, что при любом таком a_1 число делений меньше $2m + 1$. Выполняется также $\log_2 a_1 \geq m - 1$, и из (10.4) мы выводим, что число делений не меньше, чем $c_1 m + c_2$ для некоторых положительных констант c_1, c_2 . Отсюда $T_E^*(m) = \Theta(m)$.

Число шагов расширенного алгоритма Евклида (см. пример 9.1) совпадает с числом шагов обычного алгоритма Евклида. Мы получаем следующее:

Если рассматривать a_1 как размер входа (a_0, a_1) расширенного алгоритма Евклида, то для мультипликативной сложности $T_{EE}(a_1)$ этого алгоритма выполнено $T_{EE}(a_1) = \Theta(\log a_1)$. При рассмотрении $m = \lambda(a_1)$ в качестве размера входа (a_0, a_1) расширенного алгоритма Евклида имеем для мультипликативной сложности оценку $T_{EE}^*(m) = \Theta(m)$. Все это сохраняет силу и для сложности $T_E^*(m)$ «обычного» алгоритма Евклида.

Запись алгоритма Евклида в форме (9.1) далека от записи в виде «приличной» компьютерной программы не только в смысле использованной символики, но, главное, в смысле предписываемого расточительного расхода памяти (использован массив, и при буквальном следовании алгоритму (9.1) мы бы имели $S_E(a_1) = \Theta(\log a_1)$). С точки зрения программирования более приемлемой будет, например, запись

```

a := a0; b := a1;
while b > 0 do
    d := rem(a, b); a := b; b := d
od

```

(rem — знак операции нахождения остатка). После выполнения алгоритма имеем $a = \text{нод}(a_0, a_1)$. Использовано только три дополнительных переменных (a, b и d), и мы получаем $S_E(a_1) = 3$.

Оценку (10.4) можно усилить, показав, что найдется константа γ' такая, что

$$T_E(a_1) > \frac{1}{2} \log_\phi a_1 + \gamma' \quad (10.8)$$

(см. указание к задаче 79).

Может быть также доказано (см. задачу 76), что $T_E(a_1) < \log_\phi a_1 + c$, где c — некоторая константа (заметим, что $\log_\phi 2 = 1,44... < 2$).

Рассуждения, сходные с приведенными при доказательстве неравенства (10.4), показывают, что для каждого a_0 можно подобрать a_1 , $a_0 \geq a_1$, так, что будет выполняться

$$C_E(a_0, a_1) > \frac{1}{4} \log_\phi a_0 + \beta, \quad (10.9)$$

где β — некоторая константа. В данном случае можно рассмотреть вложенные отрезки $\Psi_1 \supset \Psi_2 \supset \Psi_3 \supset \dots$:

$$\Psi_n = \left[\frac{F_{2n}}{F_{2n+1}}, \frac{F_{2n-1}}{F_{2n}} \right], \quad n = 1, 2, \dots$$

(при $n > 1$ отрезок Ψ_n не содержит чисел вида $\frac{1}{m}$, $m \in \mathbb{N}^+$); если a_0 не делится нацело на a_1 и $\frac{a_1}{a_0} \in \Psi_n$, то $\frac{a_2}{a_1} \in \Psi_{n-1}$ и т. д. Мы воспользуемся этим в § 21.

Пример 10.2. Сортировка массива длины n бинарными вставками выполняется за $n - 1$ шаг, причем на l -м шаге число сравнений не превосходит $\lceil \log_2(l + 1) \rceil$ и, следовательно, не превосходит $\lceil \log_2 n \rceil$ на любом из шагов, — это приводит к оценке (9.14). Возникает вопрос, не сможем ли мы получить существенно лучшей оценки сверху, рассмотрев сумму $\sum_{l=0}^{n-1} \lceil \log_2(l + 1) \rceil$, которая совпадает со значением $T_B(n)$, т. е. сложности по числу сравнений (такое число сравнений достигается, например, в случае, когда изначально массив имеет обратную упорядоченность: $x_1 > x_2 > \dots > x_n$). Замена переменной суммирования дает

$$T_B(n) = \sum_{k=1}^n \lceil \log_2 k \rceil. \quad (10.10)$$

Для получения простой асимптотической оценки $T_B(n)$ воспользуемся формулой Стирлинга $n! \sim n^n e^{-n} \sqrt{2\pi n}$, или, что то же самое,

$$n! = n^n e^{-n} \sqrt{2\pi n} (1 + o(1)), \quad (10.11)$$

которая после логарифмирования принимает вид

$$\log_2 n! = n \log_2 n - n \log_2 e + \frac{1}{2} \log_2 n + \frac{1}{2} \log_2 \pi + \frac{1}{2} + o(1)$$

(принято во внимание, что $\log_2(1 + o(1)) = o(1)$). Так как $1 < \log_2 e < 2$, то

$$n \log_2 n - 2n < \log_2 n! < n \log_2 n - n \quad (10.12)$$

для всех достаточно больших n ; в частности, это означает, что

$$\log_2 n! = n \log_2 n + O(n).$$

Имеем

$$\sum_{k=1}^n \lceil \log_2 k \rceil = \sum_{k=1}^n \log_2 k + O(n) = \log_2 n! + O(n) = n \log_2 n + O(n).$$

Отсюда получаем оценку

$$T_B(n) = n \log_2 n + O(n), \quad (10.13)$$

из которой следует, в частности, что $T_B(n) \sim n \log_2 n$.

Таким образом, тщательный анализ суммы (10.10) не дает оснований утверждать, что оценка (9.14) может быть существенно улучшена, но зато позволяет получить асимптотику $T_B(n)$.

В § 14 будет показано, что из того, что сложность по числу сравнений некоторой сортировки не превосходит $n \log_2 n + cn$, где c — некоторая константа, следует, что эта сложность есть $n \log_2 n + O(n)$. Из этого будет следовать, что для сложности по числу сравнений сортировки фон Неймана выполнено $T_{vN}(n) = n \log_2 n + O(n)$; мы обозначаем эту сортировку буквами vN , от фамилии von Neumann.

§ 11. Завершимость работы алгоритма

Если выполнение циклического (итерационного) алгоритма не завершается для некоторого входа v и при этом на каждом шаге выполняются какие-то из учитываемых операций, то временные затраты бесконечны для этого входа. Поэтому анализ сложности алгоритма прямо или косвенно включает исследование завершимости.

Установление завершимости может быть рассмотрено и как самостоятельная задача. Когда функция L подбирается для выяснения зависимости числа шагов алгоритма от размера входа задачи, то при наличии выбора более предпочтительной является функция с медленным ростом (имеется в виду рост при увеличении размера входа); даже если при этом усложняется доказательство того, что L обладает всеми нужными свойствами, мы зато получаем более точную оценку числа шагов алгоритма. Но иногда речь может идти только о доказательстве завершимости работы алгоритма — тогда характер роста функции L отходит на второй план. То, что выполнение алгоритма Евклида завершается, было доказано в самом начале обсуждения этого алгоритма без обращения к функциям с логарифмическим ростом.

Пример 11.1. Алгоритм, заданный оператором цикла

```
while  $n > 3$  do
  if  $2 \mid n$  then  $n := \frac{n}{2} + 1$  else  $n := n + 1$  fi
od
```

(запись $2 \mid n$ означает, что n делится на 2) завершает свою работу для любого натурального n . Для доказательства достаточно рассмотреть

функцию $n - (-1)^n$, принимающую при $n > 3$ неотрицательные целые значения, и убедиться в ее убывании при $n > 3$ в ходе выполнения алгоритма.

В доказательствах завершимости, основанных на подборе подходящей убывающей целочисленной функции, используется следующее свойство множества $\mathbb{N}$ неотрицательных целых чисел:

Не существует бесконечной убывающей последовательности $n_0 > n_1 > n_2 > \dots$ элементов множества $\mathbb{N}$.

В некоторых ситуациях естественно выбирать функцию L со значениями в множествах, отличных от $\mathbb{N}$, но обладающих аналогичным свойством. В качестве такого множества может, например, выступать множество конечных последовательностей фиксированной длины k неотрицательных целых чисел с лексикографическим (словарным) отношением порядка. Можно показать, что в этом множестве не существует бесконечных убывающих последовательностей¹.

Установление завершимости выполнения алгоритма иногда оказывается очень трудной задачей.

Пример 11.2. Завершимость для любого натурального n выполнения алгоритма

```
while  $n > 1$  do
  if  $2 \mid n$  then  $n := \frac{n}{2}$  else  $n := 3n + 1$  fi
od
```

не доказана и не опровергнута по сей день, хотя на это направлялись серьезные усилия².

Замечание. Известно, что общая задача проверки завершимости алгоритма для предлагаемых входных данных алгоритмически неразрешима. В теории алгоритмов, возникшей в связи с результатами А. Тьюринга 1936 г.³, принято говорить не о завершимости, а о *применимости* алгоритма. В книге А. А. Маркова и Н. М. Нагорного⁴ результат Тьюринга подан, фактически, в следующем виде: при заданном произвольном непустом алфавите Λ может быть предъявлен такой алгоритм F_Λ , что невозможен алгоритм, который бы по произ-

¹ Доказательство можно найти в [13, разд. 2.3]. Задачи 98, 99 в конце этой главы нашей книги заимствованы из книги [13]. Они могут быть решены, в частности, исходя из упомянутого общего свойства.

² См. [79].

³ См. [86].

⁴ [39, § 48].

вольному слову w в алфавите Λ проверял бы завершимость выполнения F_Λ для w . В § 49, п. 1.2 этой же книги отмечено, что это справедливо и для однобуквенного алфавита Λ , откуда выводится, например, существование перечислимых, но не разрешимых подмножеств множества натуральных чисел.

В случае рандомизированных алгоритмов мы сталкиваемся (см. ниже пример 11.3 и первую часть примера 11.4) с возможностью того, что алгоритм завершается с вероятностью 1, но математическое ожидание времени от начала выполнения до полного завершения алгоритма бесконечно. Поэтому утверждения о завершимости возможны в двух формах: завершимость с вероятностью 1 и конечность ожидаемого времени выполнения. Вторая форма является, вообще говоря, более сильной.

Пример 11.3. В теории вероятностей подробно рассматривается задача о блуждании частицы по прямой: за один шаг частица перемещается на 1 вправо или на то же расстояние влево, направление же движения на каждом шаге выбирается случайно, вероятности выбора каждого из направлений одинаковы и равны $\frac{1}{2}$. Пусть в начальный момент частица находится в точке 0, и на прямой отмечена некоторая точка m с целой координатой. Доказывается, что с вероятностью 1 частица через некоторое время попадает в точку m . Попытаемся теперь вычислить математическое ожидание a времени (общего числа шагов), которое потребуется для достижения точки m . Легко убедиться, что уже в случае $m = \pm 1$ это среднее бесконечно. Пусть, например, $m = 1$. Используем формулу полного математического ожидания: если шаг сделан вправо, то за этот один шаг мы достигаем цели, если же шаг сделан влево, то придется дожидаться момента, когда частица вернется в точку 0 и тем самым повторится начальная ситуация:

$$a = 1 \cdot \frac{1}{2} + 2a \cdot \frac{1}{2}.$$

Если бы a было конечным, это бы дало $0 = \frac{1}{2}$. Поэтому среднее бесконечно.

Рассмотрим с этой точки зрения задачу 26. Итак, путник столкнулся со стеной, простирающейся бесконечно в обе стороны. Имеется дверь в этой стене, но путник не знает ни расстояния до двери, ни направления к ней. Если путник использует алгоритм поиска двери, состоящий в бросании перед каждым шагом монеты для определения направления (вправо или влево) этого шага, то он с вероятностью 1

найдет дверь, но математическое ожидание числа шагов равно бесконечности, коль скоро в начальный момент путник не стоит прямо перед дверью. Иными словами, если p_k ($k \geq 1$) есть вероятность того, что после k шагов путник впервые окажется перед дверью, то

$$\sum_{k=1}^{\infty} p_k = 1 \quad \text{и} \quad \sum_{k=1}^{\infty} kp_k = \infty.$$

Пример 11.4. Обратимся к системам автоматизированного обучения. При использовании известного педагогического приема — задания вопросов вразбивку — с каждым выбранным системой вопросом может быть связана следующая цепочка действий: вопрос обучаемому; прием и проверка ответа; сообщение, правилен ли ответ и объявление правильного ответа, если был дан неправильный ответ. Нетрудно привести примеры учебных тем, когда такой подход выглядит разумно: таблица умножения, исторические даты, иностранные слова (здесь же — неправильные глаголы), значения иероглифов, название созвездий (показываются картинки), определение на слух музыкальных интервалов и т. д. Прообразом одного из алгоритмов выбора вопросов служит способ заучивания ответов с помощью колоды карточек, на лицевой стороне каждой из которых написан вопрос, а на обороте — ответ. Из колоды выбирается наугад карточка, и делается попытка ответить на вопрос. Если это не удастся, ответ прочитывается на обороте. Затем карточка возвращается в колоду, и все повторяется. На примере колоды карточек предлагаемый рандомизированный алгоритм (называемый алгоритмом кратных карт¹) может быть интерпретирован так: выбранная карточка, содержащая вопрос с известным обучаемому ответом, уже не возвращается в колоду; если же обучаемый не смог дать правильный ответ, то кроме того, что в колоду возвращается эта карточка, туда добавляется еще один ее экземпляр. Сеанс обучения заканчивается, когда в колоде не остается карточек.

Пусть вопросы занумерованы числами $1, 2, \dots, n$. Каждый этап сеанса обучения характеризуется кратностями $m_1, m_2, \dots, m_n$ вопросов (не исключается равенство нулю каких-то кратностей); пусть $m = m_1 + m_2 + \dots + m_n$. Если $m = 0$, то сеанс заканчивается, иначе выбирается очередной вопрос; вероятность того, что будет выбран i -й вопрос, должна равняться $\frac{m_i}{m}$.

¹ См. [4].

Сеанс обучения становится бесконечным, если, например, с некоторого момента обучаемый начинает давать только неправильные ответы. Но можно интересоваться вероятностями и средним временем ожидания некоторых событий в течение этого бесконечного сеанса. Пусть в некоторый момент только один вопрос, скажем, вопрос с номером 1, имеет кратность 1, а все остальные — кратности большие, чем 1. Предположим, что начиная с этого момента обучаемый стабильно дает неправильные ответы на предлагаемые вопросы. Найдем вероятность того, что рано или поздно обучаемый получит вопрос с номером 1, а также математическое ожидание времени, проходящего до появления этого вопроса (время измеряется количеством заданных вопросов). Пусть m — суммарная кратность, а n — общее число вопросов, $m > n \geq 2$. Вероятность получить вопрос с номером 1 после i вопросов с другими номерами равна $\frac{1}{m}$, если $i = 0$, и равна

$$\frac{m-1}{m} \cdot \frac{m}{m+1} \cdots \frac{m+i-2}{m+i-1} \cdot \frac{1}{m+i} = \frac{m-1}{(m+i-1)(m+i)},$$

если $i \geq 1$. Поэтому вероятность получить рано или поздно вопрос с номером 1 равна

$$\frac{1}{m} + (m-1) \sum_{i=1}^{\infty} \frac{1}{(m+i-1)(m+i)}. \quad (11.1)$$

Так как

$$\frac{1}{(m+i-1)(m+i)} = \frac{1}{m+i-1} - \frac{1}{m+i},$$

то для частичной суммы

$$s_N = \sum_{i=1}^N \frac{1}{(m+i-1)(m+i)}$$

имеем

$$s_N = \frac{1}{m} - \frac{1}{m+N},$$

и $\lim_{n \rightarrow \infty} s_N = \frac{1}{m}$, откуда значение выражения (11.1) (искомая вероятность) есть

$$\frac{1}{m} + \frac{m-1}{m} = 1.$$

Математическое ожидание времени есть

$$(m-1) \sum_{i=1}^{\infty} \frac{i}{(m+i-1)(m+i)}. \quad (11.2)$$

При этом ряд $\sum_{i=1}^{\infty} \frac{i}{(m+i-1)(m+i)}$ расходится: члены этого ряда положительны, и i -й член есть $\Theta\left(\frac{1}{i}\right)$ (является величиной порядка $\frac{1}{i}$). Отсюда математическое ожидание (11.2) равно бесконечности.

Пусть теперь u вопросов (будем считать, что это вопросы с номерами $1, 2, \dots, u$) имеют кратность 1, а остальные $n - u$ вопросов — кратность большую, чем 1, и пусть $u \geq 2$. Оказывается, что при стабильных неправильных ответах обучаемого на задаваемые вопросы математическое ожидание времени, проходящего до получения всех, кроме какого-то одного, вопросов с номерами $1, 2, \dots, u$, есть конечная величина. Докажем конечность среднего времени ожидания какого-нибудь одного вопроса с номером от 1 до u , далее можно применить индукцию. Вероятность получить такого рода вопрос после i вопросов с номерами из диапазона от $u+1$ до n полностью определяется значениями u, m, i :

$$\frac{m-u}{m} \cdot \frac{m-u+1}{m+1} \cdots \frac{m-u+i-1}{m+i-1} \cdot \frac{u}{m+i}.$$

Эта вероятность при $i > u$ равна

$$\frac{u(m-u)(m-u+1)\dots(m-1)}{(m-u+i)(m-u+i+1)\dots(m+i-1)(m+i)}.$$

Поэтому искомое математическое ожидание есть сумма некоторого конечного числа слагаемых и ряда

$$\sum_{i=u+1}^{\infty} \frac{u(m-u)(m-u+1)\dots(m-1)}{(m-u+i)(m-u+i+1)\dots(m+i-1)(m+i)} (i+1),$$

или

$$u(m-u)(m-u+1)\dots(m-1) \sum_{i=u+1}^{\infty} \frac{i+1}{(m-u+i)(m-u+i+1)\dots(m+i)}.$$

Упрощая, получаем

$$u(m-u)(m-u+1)\dots(m-1) \sum_{j=1}^{\infty} \frac{j+u+1}{(j+m)(j+m+1)\dots(j+m+u)}.$$

Ряд, входящий в это выражение, сходится, так как j -й член ряда есть $\Theta\left(\frac{1}{j^u}\right)$ и $u \geq 2$.

Это рассуждение сохраняет силу и в том случае, когда вопросы с номерами $1, 2, \dots, u$ являются одним и тем же вопросом, что поз-

воляет отметить следующую характерную черту алгоритма кратных карт:

Если изначально все вопросы имеют кратность 1 и обучаемый дает на все вопросы неправильные ответы, то математическое ожидание времени до наступления момента, к которому обучаемый получит хотя бы по одному разу каждый из вопросов, равно бесконечности, хотя вероятность наступления такого момента равна 1. Если же изначально все вопросы имели большие единицы кратности (например, все они имели кратность 2), то обсуждаемое математическое ожидание конечно.

§ 12. Вложенные циклы: дополнительные примеры

При анализе сложности вложенных циклов мы естественно приходим к вычислению и оцениванию кратных сумм.

Пример 12.1. Число обменов при транспонировании $(n \times n)$ -матрицы (a_{ij})

```
for i = 1 to n - 1 do
  for j = i + 1 to n do  $a_{ij} \leftrightarrow a_{ji}$  od
od
```

есть $\sum_{i=1}^{n-1} \sum_{j=i+1}^n 1 = \sum_{i=1}^{n-1} (n-i) = \frac{n^2-n}{2}$. Аналогично, проверка кососимметричности данной $(n \times n)$ -матрицы требует в худшем случае сравнений в количестве $\sum_{i=1}^{n-1} \sum_{j=i}^n 1 = \sum_{i=1}^{n-1} (n-i+1) = \frac{(n+1)(n-2)}{2}$ и т.д.

Легкое получение ответов в примере 12.1 объясняется тем, что в выписанных формальным путем суммах нижние границы суммирования не превосходили верхних; в таких случаях всегда, например, $\sum_{j=p}^q 1 = q - p + 1$. Но, получая сумму этим формальным путем из оператора цикла, можно столкнуться с тем, что $p > q$ (оператор цикла затратит 0 шагов), и ответ $q - p + 1$ будет неправильным. Соответствующий пример дается в задаче 102.

Подходя к асимптотическим оценкам, отметим один простой, но полезный факт.

Предложение 12.1. Пусть Λ — один из символов O , Θ , Ω . Пусть $a(k)$, $b(k)$ определены для всех $k \in \mathbb{N}^+$ и принимают положительные

значения (не обязательно целые); пусть $a(k) = \Lambda(b(k))$, $k \rightarrow \infty$. Тогда $\sum_{k=1}^n a(k) = \Lambda\left(\sum_{k=1}^n b(k)\right)$, $n \rightarrow \infty$. Утверждение остается справедливым и при замене верхней границы суммирования n на $\varphi(n)$ для любой функции $\varphi(n)$, определенной для всех $n \in \mathbb{N}^+$ и принимающей значения в $\mathbb{N}^+$.

Доказательство. Докажем первую часть утверждения для $\Lambda = \Theta$, остальные случаи доказываются аналогично. В силу замечания, сделанного в § 2 после предложения 2.1,

$$c_1 b(k) \leq a(k) \leq c_2 b(k)$$

для всех $k \in \mathbb{N}^+$ и некоторых $c_1, c_2 > 0$. Отсюда

$$c_1 \sum_{k=1}^n b(k) \leq \sum_{k=1}^n a(k) \leq c_2 \sum_{k=1}^n b(k).$$

□

Пример 12.2. Проведем асимптотический анализ следующего алгоритма, оценивая общее число выполняемых операций в зависимости от исходного значения n :

```

l := 0;
for i = 1 to  $\lfloor \sqrt{n^3 + 1} \rfloor$  do
  k := i;
  while k > 1 do l := l + k; k :=  $\left\lfloor \frac{k}{3} \right\rfloor$  od
od

```

Для внутреннего цикла, выполняемого при начальном значении k , равном значению i , число шагов и общее число операций допускают оценку $\Theta(\log i)$ (можно представить себе, что k записано в троичной системе счисления, тогда каждый шаг удаляет одну цифру из записи k). Таким образом, затраты на выполнение внешнего

цикла представляют собой $\sum_{k=1}^{\varphi(n)} a(k)$, где $\varphi(n) = \lfloor \sqrt{n^3 + 1} \rfloor$, $a(k) = \Theta(\log k)$, и по предложению 12.1 допускают оценку $\Theta\left(\sum_{k=1}^{\varphi(n)} \log k\right)$

или $\Theta(\log(\varphi(n)!))$. Так как $\varphi(n) \rightarrow \infty$ при $n \rightarrow \infty$, то можно применить соотношение $\log(\varphi(n)!) = \Theta(\varphi(n) \log \varphi(n))$, которое является следствием формулы Стирлинга (см. (10.11)). Это дает нам оценку $\Theta(\lfloor \sqrt{n^3 + 1} \rfloor \log \lfloor \sqrt{n^3 + 1} \rfloor)$ для интересующих нас затрат. После упрощения получаем $\Theta(n^{3/2} \log n)$. В этом примере затраты однозначно определяются по n , поэтому полученная оценка является асимптотической оценкой сложности рассматриваемого алгоритма при использовании n в качестве размера входа.

Оценку $O(\lfloor \sqrt{n^3 + 1} \rfloor \log \lfloor \sqrt{n^3 + 1} \rfloor)$ и, тем самым, $O(n^{3/2} \log n)$ можно было бы получить не прибегая к формуле Стирлинга, исходя из того, что сумма конечного числа неотрицательных слагаемых не превосходит произведения наибольшего слагаемого на число слагаемых суммы.

Для Θ этот прием не работает, как показывает пример суммы $\sum_{i=1}^n 2^i$.

§ 13. Нецелые размеры входа и непрерывные оценочные функции

Предложение 13.1. Пусть для некоторого алгоритма A можно указать входы $v_0, v_1, \dots$, размеры которых ограничены снизу и сверху положительными константами c_1 и c_2 :

$$c_1 \leq \|v_i\| \leq c_2, \quad i = 0, 1, \dots,$$

и в то же время затраты $C_A^T(v_i)$ неограниченно возрастают при $i \rightarrow \infty$. Тогда не существует такой непрерывной на отрезке $[c_1, c_2]$ функции $f(x)$ вещественной переменной, что $T_A(x) \leq f(x)$ для всех значений x размера входа, принадлежащих отрезку $[c_1, c_2]$.

Доказательство. Очевидно, что $T_A(\|v_i\|) \geq C_A^T(v_i)$, и поэтому последовательность $T_A(\|v_i\|)$, $i = 0, 1, \dots$, не ограничена. В то же время, любая функция f , непрерывная на $[c_1, c_2]$, ограничена на этом отрезке. $\square$

Сформулируем еще одно столь же легко доказываемое утверждение (доказательство опускаем).

Предложение 13.2. Пусть A — некоторый алгоритм, и $f(x)$ — функция вещественной переменной, определенная на некотором интервале I . Пусть $T_A(x) \leq f(x)$ всякий раз, когда $x \in I$ и значение $T_A(x)$ определено. Пусть $x_0 \in I$ и для любых $\varepsilon > 0$, $N > 0$ найдется такой вход v алгоритма A , что $|x_0 - \|v\|| < \varepsilon$ и $C_A^T(v) > N$. Тогда $f(x)$ разрывна в точке x_0 .

Пример 13.1. Вновь рассмотрим алгоритм Евклида, используя теперь в качестве размера входа (a_0, a_1) рациональное число $\frac{a_0}{a_1}$, которое, как указывалось в примере 9.1, однозначно определяет соответствующее количество делений с остатком (сложность $T_E'(\frac{a_0}{a_1})$), соответствующая этому размеру, совпадает с затратами $C_E(a_0, a_1)$). В силу формулы Бине (10.2) имеем

$$\lim_{n \rightarrow \infty} \frac{F_{n+1}}{F_n} = \phi,$$

при этом $C_E(F_{n+1}, F_n) = n$. Поэтому если функция вещественной переменной $f(x)$ такова, что $C_E(a_0, a_1) \leq f\left(\frac{a_0}{a_1}\right)$, то, в силу предложения 13.2, $f(x)$ разрывна в точке ϕ . Мы воспользовались тем, что для любого $\varepsilon > 0$ и любого натурального N найдется рациональное число u/v такое, что $\left|\phi - \frac{u}{v}\right| < \varepsilon$ и $C_E(u, v) \geq N$.

Можно доказать, что здесь дело не в числе ϕ — то же самое справедливо для любого $x_0 > 1$. Пусть $0 < \varepsilon < x_0 - 1$. Докажем, что в рациональных точках ε -окрестности $V_{x_0, \varepsilon}$ числа x_0 функция $T'_E(r)$ не является ограниченной. Предположим противное: пусть $r = \frac{u_0}{u_1} \in V_{x_0, \varepsilon}$ — рациональное число, на котором достигается максимум. Пусть частные, возникающие в процессе применения алгоритма Евклида к u_0, u_1 , равны $q_1, q_2, \dots, q_n$:

$$u_{i-1} = q_i u_i + u_{i+1}, \quad i = 1, 2, \dots, n, \quad u_{n+1} = 0.$$

Выберем $\delta > 0$ столь малым, что $V_{r, \delta} \subset V_{x_0, \varepsilon}$, и возьмем любое $s \in \mathbb{Q} \setminus \mathbb{Z}$ такое, что $\left|\frac{u_{n-1}}{u_n} - s\right| < \delta$ (легко заметить, что $\frac{u_{n-1}}{u_n} = q_n \in \mathbb{Z}$). Пусть l, m — такие целые, что $s = \frac{l}{m}$. Используя $q_1, q_2, \dots, q_n$, найдем $v_0, v_1, \dots, v_n \in \mathbb{N}^+$, для которых

$$v_{i-1} = q_i v_i + v_{i+1}, \quad i = 1, 2, \dots, n,$$

и $v_{n-1} = l, v_n = m$. Индукцией по k легко доказать, что для $k = 0, 1, \dots, n-1$

$$\left|\frac{v_{n-k-1}}{v_{n-k}} - \frac{u_{n-k-1}}{u_{n-k}}\right| < \delta. \quad (13.1)$$

В самом деле, для $k = 0$ это очевидно; остается показать, что при $0 < k < n-1$ из (13.1) следует

$$\left|\frac{v_{n-k-2}}{v_{n-k-1}} - \frac{u_{n-k-2}}{u_{n-k-1}}\right| < \delta. \quad (13.2)$$

Имеем

$$\begin{aligned} \frac{v_{n-k-2}}{v_{n-k-1}} &= \frac{q_{n-k-1} v_{n-k-1} + v_{n-k}}{v_{n-k-1}} = q_{n-k-1} + \frac{v_{n-k}}{v_{n-k-1}}, \\ \frac{u_{n-k-2}}{u_{n-k-1}} &= \frac{q_{n-k-1} u_{n-k-1} + u_{n-k}}{u_{n-k-1}} = q_{n-k-1} + \frac{u_{n-k}}{u_{n-k-1}}, \end{aligned}$$

откуда

$$\left|\frac{v_{n-k-2}}{v_{n-k-1}} - \frac{u_{n-k-2}}{u_{n-k-1}}\right| = \left|\frac{v_{n-k}}{v_{n-k-1}} - \frac{u_{n-k}}{u_{n-k-1}}\right| = \frac{|u_{n-k-1} v_{n-k} - u_{n-k} v_{n-k-1}|}{u_{n-k-1} v_{n-k-1}}.$$

Неравенство (13.1) можно переписать в виде

$$\frac{|u_{n-k-1}v_{n-k} - u_{n-k}v_{n-k-1}|}{u_{n-k}v_{n-k}} < \delta;$$

это неравенство влечет

$$\frac{|u_{n-k-1}v_{n-k} - u_{n-k}v_{n-k-1}|}{u_{n-k-1}v_{n-k-1}} < \delta,$$

так как $u_{n-k-1}v_{n-k-1} > u_{n-k}v_{n-k}$. Неравенство (13.2) доказано, и индукция завершена. Следовательно, $\frac{v_0}{v_1} \in V_{s,\delta}$ и, тем самым, $\frac{v_0}{v_1} \in V_{x_0,\varepsilon}$. Но $T'_E\left(\frac{v_0}{v_1}\right) > n$, так как $s = \frac{v_{n-1}}{v_n} \notin \mathbb{Z}$. Это противоречит предположению о максимальнойности значения $T'_E\left(\frac{u_0}{u_1}\right)$.

В силу предложения 13.2 мы получаем следующее:

Пусть размер каждого входа (a_0, a_1) алгоритма Евклида определен как $r = a_0/a_1$ (рациональное число ≥ 1), и в соответствии с этим рассматривается сложность $T'_E(r)$ этого алгоритма по числу делений. Пусть функция $f(x)$ вещественной переменной, определенная всюду на бесконечном полуинтервале $I = [1, \infty)$, такова, что $T'_E(r) < f(r)$ при всех рациональных $r \geq 1$. Тогда $f(x)$ является разрывной в любой принадлежащей I точке.

Нелишне заметить, что при целочисленном размере входа условие предложения 13.1 может выполняться, если только для некоторого целого n , $c_1 \leq n \leq c_2$, сложность $T_A(n)$ бесконечна.

Приводившиеся ранее оценки (9.15), (9.16) для числа итераций методов деления пополам и касательных используют, фактически, нецелый размер $1/\varepsilon$. Положение спасается тем, что при малом изменении ε не происходит катастрофического увеличения числа итераций. Эту ситуацию можно было бы считать типичной, а пример 13.1 искусственным, если бы упомянутые возможные трудности не приводили бы иногда к реальным ошибкам. В приложении С рассмотрена известная алгоритмическая проблема, связанная с алгебраическими числами («проблема орбит»), для которой в свое время было предложено и опубликовано неверное решение, и причина ошибки была в том, что авторы проглядели ту негативную возможность, которая показана в примере 13.1.

В ситуациях, подобных описанной в примере 13.1, мы не можем использовать в оценках вида $T_A(x) < f(x)$, $T_A(x) = O(f(x))$, $T_A(x) = \Theta(f(x))$ в качестве функции $f(x)$, скажем, логарифм, степенную и показательную функции, полиномы и т. д. Во многих случаях это означает, что размер как функция входа выбран неудачно. В качестве

размера входа алгоритма надо, по возможности, выбирать такие величины, исходя из которых сложность алгоритма может быть выражена или оценена с помощью простых аналитических функций. Это позволит составить представление об изменении сложности при возрастании размера входа. Появление функций, поведение которых трудно себе вообразить, лишает возможности получения такой информации.

Задачи

70. Функцию $L(s)$ не обязательно выбирать целочисленной. Достаточно, чтобы ее значения были неотрицательными и каждый шаг алгоритма понижал ее значение на величину, ограниченную снизу некоторым положительным числом d . Как с помощью такой функции оценить число шагов?

71. Имеется прямоугольная матрица с вещественными элементами. Разрешены два вида операций: умножение любой строки на -1 и умножение любого столбца на -1 . Последовательностью таких операций можно добиться того, что суммы элементов в любой строке и любом столбце будут неотрицательными.

72. Бинарный алгоритм Евклида основан на том, что если a_0 и a_1 четны, то $\text{нод}(a_0, a_1) = 2 \text{ нод}\left(\frac{a_0}{2}, \frac{a_1}{2}\right)$; если a_0 четно, но a_1 нечетно, то $\text{нод}(a_0, a_1) = \text{нод}\left(\frac{a_0}{2}, a_1\right)$; если a_0 нечетно, но a_1 четно, то $\text{нод}(a_0, a_1) = \text{нод}\left(a_0, \frac{a_1}{2}\right)$; если a_0 и a_1 нечетны, то можно перейти к вычислению $\text{нод}(a_0 - a_1, a_1)$ при $a_0 \geq a_1$ и к вычислению $\text{нод}(a_0, a_1 - a_0)$ при $a_0 < a_1$. Вычисления заканчиваются, как только на некотором этапе вычитание даст нуль. Выполнение этого алгоритма завершается для любых исходных a_0, a_1 , для которых $a_0 \geq a_1 > 0$. Для числа шагов алгоритма как функции от a_0, a_1 справедлива оценка $O(\log a_0)$.

Указание. По крайней мере на одном из двух последовательных шагов этого алгоритма по крайней мере одно из чисел делится на два.

73. Пусть в качестве размера входа алгоритма Евклида, применяемого к целым a_0, a_1 , таким, что $a_0 \geq a_1 > 0$, используется число a_0 . Верно ли, что тогда сложность алгоритма Евклида по числу делений с остатком есть $\Theta(\log a_0)$?

74. Существует ли такой полином $p(x, y)$ с целыми коэффициентами, что $p(x, y) = \text{нод}(x, y)$ для любых целых значений x, y , не равных нулю одновременно?

Указание. Пусть $p(x, y)$ существует. Для полинома $p(x, 2)$ одной переменной x равенство $p(x, 2) = 2$ выполняется для бесконечного множества целых x , откуда $p(x, 2) = 2$ тождественно.

75. Пусть в ходе применения алгоритма Евклида к числам a_0, a_1 , $a_0 > a_1$, возникают ненулевые остатки $a_2, a_3, \dots, a_n$. Тогда $a_{n-t} \geq F_{t+2}$, $t = 0, 1, \dots, n-1$. Отсюда следует справедливость следующего предложения, обычно именуемого в литературе теоремой Ламе. Пусть в ходе применения алгоритма Евклида к числам a_0, a_1 , $a_0 > a_1$, возникают ненулевые остатки $a_2, a_3, \dots, a_n$. Тогда $a_1 \geq F_{n+1}$.

76. Пусть в ходе применения алгоритма Евклида к числам a_0, a_1 , $a_0 > a_1$, возникают ненулевые остатки $a_2, a_3, \dots, a_n$. Тогда $n < \log_\phi(a_1 + 1) + \log_\phi \sqrt{5} - 1$, и, как следствие, $n < \log_\phi a_1 + c$ для некоторой константы c .

Указание. См. предыдущую задачу.

77. Для чисел Фибоначчи $F_1, F_2, \dots$ справедливы соотношения:

а) $\lambda(F_n) \leq n$;

б) $F_n \leq 2^{n-1}$.

78. Привести доказательство равенства (10.6).

79. Доказать оценку (10.8).

Указание. В § 10 при рассмотрении примера 13.1 уже доказано, что если $\frac{a_0}{a_1} \in \Phi_n$, $n > 2$, то выполнены неравенства (10.7). Вывести отсюда, что $\frac{F_{2n-2}}{F_{2n-3}} \leq \frac{a_2}{a_3} \leq \frac{F_{2n-1}}{F_{2n-2}}$, т. е. $\frac{a_2}{a_3} \in \Phi_{n-1}$, $n > 2$.

80. Разработать алгоритм, распознающий существование у уравнения

$$ax + by = c, \quad a, b, c \in \mathbb{Z}, \quad (13.3)$$

решения в неотрицательных целых числах, и исследовать мультипликативную сложность этого алгоритма.

Указание. Уравнение (13.3) тогда и только тогда имеет решение в целых числах, когда $d \mid c$, где $d = \text{нод}(a, b)$.

81. Если x_0, y_0 — одно из целочисленных решений уравнения (13.3), то все решения описываются формулами

$$x = x_0 + \frac{b}{d}t, \quad y = y_0 - \frac{a}{d}t, \quad t = 0, \pm 1, \pm 2, \dots,$$

$d = \text{нод}(a, b)$.

Указание. Если x_0, y_0 и x_1, y_1 — два различных решения уравнения (13.3), то $x_0 - x_1, y_0 - y_1$ — решение уравнения $\frac{a}{d}x + \frac{b}{d}y = 0$.

82. Числа s_i, t_i , получаемые в ходе выполнения расширенного алгоритма Евклида, являются взаимно простыми при $i = 0, 1, \dots, n+1$.

Указание. Индукцией по i доказать равенство

$$\begin{pmatrix} -q_1 & 1 \\ 1 & 0 \end{pmatrix} \cdots \begin{pmatrix} -q_{i-1} & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} t_i & t_{i-1} \\ s_i & s_{i-1} \end{pmatrix}$$

для $i = 2, 3, \dots, n-1$ и рассмотреть определители обеих его частей.

83. При некоторых значениях n число сравнений, затрачиваемых при бинарном поиске, не определяется однозначно исходя лишь из значения n (например, зная лишь, что $n = 6$, мы не можем указать точное число сравнений). Но существует бесконечно много n таких, для которых это значение определяется единственным образом и равно $\lfloor \log_2 n \rfloor + 1$.

84. Бинарный поиск места элемента y в упорядоченном массиве $x_1 < x_2 < \dots < x_n$ требует $\lfloor \log_2(n+1) \rfloor$ сравнений в лучшем случае. Как следствие, для значений n вида $2^k - 1$, $k = 1, 2, \dots$, и только для них число сравнений однозначно определено.

Указание. Пусть $\mu(n)$ — минимум числа сравнений при бинарном поиске места элемента. Имеем $\mu(1) = 1$, и по индукции можно доказать, что при $n \geq 2$ выполнено $\mu(n) \geq \mu(n-1)$ и $\mu(n) = \mu\left(\left\lfloor \frac{n-1}{2} \right\rfloor\right) + 1$. Отсюда следует, что $\mu(n)$ равно числу положительных элементов последовательности

$$n, \left\lfloor \frac{n-1}{2} \right\rfloor, \left\lfloor \frac{\left\lfloor \frac{n-1}{2} \right\rfloor - 1}{2} \right\rfloor, \dots \quad (13.4)$$

Заметим, что $\lambda\left(\left\lfloor \frac{n-1}{2} \right\rfloor\right) = \lambda(n) - 1$, если $n \neq 2^k$, и $\lambda\left(\left\lfloor \frac{n-1}{2} \right\rfloor\right) = \lambda(n) - 2$, если $n = 2^k$, $k > 0$. Если $n = 2^k - 1$, $k > 0$, то число положительных элементов (13.4) равно $\lambda(n)$; в остальных случаях в последовательности (13.4) имеется в точности один элемент вида 2^k , $k > 0$, и число положительных элементов рассматриваемой последовательности равно $\lambda(n) - 1$.

85. *Опорная прямая* к многоугольнику P , проходящая через точку Q , внешнюю по отношению к многоугольнику, — это прямая, которой принадлежит по крайней мере одна вершина многоугольника P , и при этом все вершины этого многоугольника лежат в одной полуплоскости по отношению к этой прямой (рис. 16). Предложить имеющий сложность в $O(n)$ по общему числу операций алгоритм проведения двух опорных прямых к данному выпуклому n -угольнику из данной точки.

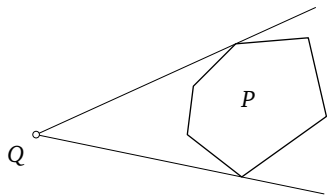


Рис. 16. Опорные прямые к многоугольнику.

Указание. Сначала найдем сторону n -угольника, которая видна из точки Q (см. пример 9.2). Это даст две соседние вершины n -угольника. Через точку Q и каждую из этих вершин проведем прямые. Затем сдвигаемся от каждой из этих двух вершин в сторону, противоположную другой вершине, пока увеличивается угол с вершиной Q .

86. Сложность алгоритма, схематически описанного в указании к предыдущей задаче, есть $\Theta(n)$. (Вместе с тем, возможно построение опорных прямых со сложностью $\Theta(\log n)$ — для этого надо применить некий вариант бинарного поиска для нахождения каждой из тех двух вершин, через которые проходят искомые опорные прямые.)

87. Предложить имеющий сложность $O(n_1 + n_2)$ по общему числу операций алгоритм построения выпуклой оболочки объединения двух выпуклых многоугольников, имеющих, соответственно, n_1 и n_2 вершин, считая $n_1 + n_2$ размером входа.

Указание. Один из известных алгоритмов этого построения основывается на том, что, взяв некоторую точку Q , принадлежащую первому многоугольнику, мы, в том случае, когда эта точка принадлежит и второму многоугольнику, можем путем слияния двух упорядоченных последовательностей получить последовательность всех вершин обоих многоугольников, упорядоченных по величине углов, под которыми видны эти вершины по отношению к какой-нибудь фиксированной прямой, проходящей через Q . После этого, как в алгоритме Грэхема (см. пример 3.1), можно удалить вмятые вершины. Если Q не принадлежит второму многоугольнику, то проведем из Q две опорные прямые ко второму многоугольнику. Пусть S и T — вершины второго многоугольника, через которые проходят опорные прямые (если опорная прямая проходит через несколько вершин, то из них выбирается наиболее удаленная от Q). Исключим из рассмотрения все принадлежащие треугольнику QST вершины второго многоугольника, кроме вершин S и T (рис. 17). Оставшиеся вершины рассматриваем в порядке возрастания углов; сливаем, как и в первом случае, две упорядоченные последовательности, а затем удаляем вмятые вершины.

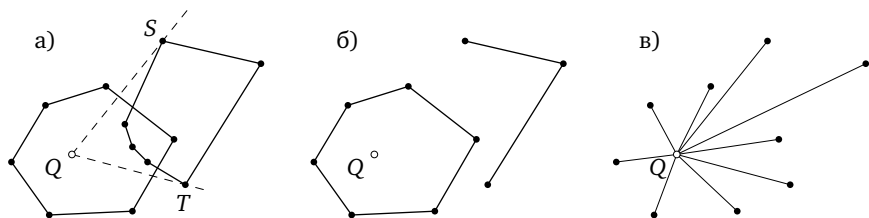


Рис. 17. Этапы построения выпуклой оболочки объединения двух выпуклых многоугольников.

88. Пусть алгоритм A перерабатывает шаг за шагом набор величин v в v' , затем v' в v'' и т.д. Пусть на этих наборах определена неотрицательная целочисленная функция L такая, что если на каком-то шаге алгоритма имело место $2^{m-1} < L(v) \leq 2^m$ при некотором целом $m \geq 1$, то при $m = 1$ выполнение алгоритма законче-

но, иначе следующий шаг дает такое v' , для которого выполняется $2^{m-2} < L(v') \leq 2^{m-1}$. Выразить через $L(v)$ общее число шагов алгоритма для набора v исходных величин.

Указание. Эта же ситуация имеет место для алгоритма бинарного поиска места элемента в упорядоченном массиве.

89. Является ли сложность $T_{BS}(n)$ бинарного алгоритма поиска места элемента в упорядоченном массиве возрастающей функцией при $n \rightarrow \infty$?

90. Является ли сложность $T_{RS}(n)$ бинарного алгоритма вычисления a^n неубывающей функцией при $n \rightarrow \infty$?

91. Верно ли, что сложность по числу сравнений сортировки массива из n элементов бинарными вставками есть $n \log_2 n + O(1)$?

Указание. Рассмотреть (9.14) при $n = 2^k$, $k = 0, 1, \dots$

92. Пусть $T'_{VN}(n)$, $T''_{VN}(n)$ — сложности сортировки фон Неймана по числу сравнений и перемещений. Верно ли, что

а) $T'_{VN}(n) \leq \frac{1}{2} T''_{VN}(n)$ для всех достаточно больших n ;

б) $T'_{VN}(n) - 1 = T''_{VN}(n)$ для всех $n \geq 2$?

93. Верно ли, что число сравнений, требуемых алгоритмом бинарного поиска места элемента в упорядоченном массиве длины n , определяется однозначно по n для бесконечного множества значений n ?

94. Нахождение медианы массива из n элементов с помощью операции сравнения $<$ требует не менее, чем $n - 1$ сравнение.

95. Обратимся к известной школьной задаче: имеются 3^m монет, из которых одна фальшивая (более легкая); с помощью чашечных весов без гирь надо за m взвешиваний определить фальшивую монету. Сравнение по весу двух групп по 3^{m-1} монет сужает диапазон последующего поиска до 3^{m-1} монет, поэтому в итоге будет затрачено m взвешиваний, как и требуется. Для числа монет $n = 3^m$ мы имеем алгоритм, использующий $\log_3 n$ взвешиваний. (Здесь мы рассматриваем n в качестве размера исходной группы монет.) Можно обобщить этот алгоритм на произвольное число монет n так, что число взвешиваний будет ограничено близкой к $\log_3 n$ функцией: сравниваем по весу две группы, в каждой из которых по $\lceil n/3 \rceil$ монет, это сужает диапазон дальнейшего поиска до не превосходящего $\lceil n/3 \rceil$ числа монет; затем действуем тем же способом.

Получить верхнюю границу $\lceil \log_3 n \rceil$ для числа взвешиваний сопоставлением размеров групп с элементами последовательности 1, 3, 9, ... подобно тому, как при анализе сортировки фон Неймана

количество упорядоченных сегментов сопоставлялось с элементами последовательности 1, 2, 4, ...

96. Изменим алгоритм поиска фальшивой монеты (задача 95): будем сравнивать по весу две группы, в каждой из которых по $\lfloor n/3 \rfloor$ монет, и т.д.

а) Показать, что этому алгоритму может потребоваться число взвешиваний, которое превосходит $\lceil \log_3 n \rceil$.

б) Получить верхнюю границу $\lceil \log_3 n \rceil + 2$ для числа взвешиваний сопоставлением размеров групп с элементами последовательности 1 + 2, 3 + 2, 9 + 2, ... подобно тому, как в при анализе бинарного поиска размеры сегментов сопоставлялись с элементами последовательности 1, 2, 4, ...

97. Вернемся к начальному варианту алгоритма определения фальшивой монеты (задача 95). При некоторых значениях n количество взвешиваний не определяется однозначно исходя лишь из значения n (например, при $n = 10$), хотя и существует бесконечно много n таких, для которых это значение определяется единственным образом.

98. Каждый день бизнесмен дает черту одну монету, и в обмен получает любой набор монет по своему выбору, но меньшего достоинства. Видов монет конечное число; менять или получать деньги в другом месте бизнесмен по условиям сделки не может. Если у бизнесмена не остается монет достоинством выше минимального, он проигрывает. На таких условиях рано или поздно бизнесмен терпит поражение, каков бы ни был первоначальный запас его монет.

99. Имеется конечная последовательность нулей и единиц. Разрешается найти в ней группу 01 и заменить на 100...00 (при этом можно написать сколько угодно нулей). Доказать, что это преобразование не может быть произведено бесконечно много раз.

Указание. Количество k единиц не меняется в ходе указанных преобразований последовательности. Каждому этапу преобразований соответствует вектор $(n_1, \dots, n_k)$ номеров единиц в последовательности после выполнения этапа. Рассмотреть последовательность этих векторов.

100. Алгоритм, приведенный в примере 11.1, затрачивает $O(\log n)$ шагов.

101. Вернемся к алгоритму кратных карт (пример 11.4). Если изначально все вопросы имели большие единицы кратности и обучаемому удалось дать ряд правильных ответов на один из вопросов, понизив его кратность до единицы, причем этого ему удалось добиться на фоне плохих ответов на все остальные вопросы, то дальнейшее рас-

смотрение этого вопроса будет как бы отложено до тех пор, пока обучаемый не усвоит еще по крайней мере один вопрос, сведя и его кратность к единице. Алгоритм препятствует слишком быстрому изъятию из рассмотрения отдельных вопросов.

Переформулировать последнее утверждение в терминах вероятностей и доказать его.

102. Найти число операций прибавления единицы к r при выполнении алгоритма для заданного натурального n

```

 $r := 0;$ 
for  $i = 1$  to  $n$  do
  for  $j = i + 1$  to  $n$  do
    for  $k = i + j - 1$  to  $n$  do  $r := r + 1$  od
  od
od

```

(иначе: определить, чему равно r). Формальное построение и последующее столь же формальное вычисление суммы по аналогии с проделанными в примере 12.1 привело бы к неправильному ответу (причина указывалась в § 12 в замечании к примеру 12.1).

103. Исследовать зависимость от целого $n > 0$ общего числа арифметических операций, требуемых алгоритмом

```

 $l := 0;$ 
for  $i = 1$  to  $n$  do
  for  $j = 1$  to  $\lfloor \log_2 i \rfloor$  do  $l := l + i + j$  od
od

```

104. Исследовать зависимость от $n > 0$ общего числа арифметических операций, требуемых алгоритмом

```

 $l := 0;$ 
for  $i = 1$  to  $\lfloor \sqrt{n} \rfloor$  do
   $z := i;$ 
  while  $z > 1$  do  $l := l + 1; z := \frac{z}{7}$  od
od

```

(значения z не обязательно целые).

Глава 4

Нижняя граница сложности алгоритмов некоторого класса. Оптимальные алгоритмы

https://t.me/it_books/2

§ 14. Понятие нижней границы сложности

Хорошо известно, что существуют алгоритмически неразрешимые задачи. Но если даже задача алгоритмически разрешима, то возможно, что сложность каждого из алгоритмов ее решения будет в определенном смысле достаточно высокой.

Пример 14.1. Рассмотрим задачу поиска наименьшего элемента с помощью сравнений в массиве длины n . Для того чтобы иметь основания отсеять элемент массива как заведомо не наименьший, необходимо, чтобы этот элемент участвовал хотя бы в одном сравнении и оказался большим. Мы должны отсеять $n - 1$ элемент, а в каждом сравнении ровно один элемент оказывается большим. Это означает, что потребуется не менее $n - 1$ сравнения.

Введем основное понятие этой главы.

Определение 14.1. Пусть $\mathcal{A}$ — фиксированный класс алгоритмов решения некоторой задачи. Пусть принято соглашение о том, в чем измеряются затраты алгоритмов и что считается размером входа, и пусть n — обозначение размера входа. Функция $f(n)$ называется *нижней границей* сложности принадлежащих $\mathcal{A}$ алгоритмов, если для сложности $T_A(n)$ любого $A \in \mathcal{A}$ мы имеем $T_A(n) \geq f(n)$ при всех допустимых значениях размера входа n . (Если используются несколько компонент $n_1, n_2, \dots, n_k$ размера входа, то нижняя граница — это, соответственно, функция аргументов $n_1, n_2, \dots, n_k$.)

Это определение имеет смысл как для сложности в худшем случае, так и для сложности в среднем. Далее в примерах этого и следующего параграфов мы рассматриваем сложность в худшем случае; к сложности в среднем мы вернемся в § 17. Всюду будет рассматриваться вре-

менная сложность. Мы можем сформулировать обнаруженный при рассмотрении примера 14.1 факт следующим образом.

Предложение 14.1. *Функция $f(n) = n - 1$ является нижней границей сложности алгоритмов поиска наименьшего элемента массива длины n с помощью сравнений.*

(Иными словами, не существует такого алгоритма выбора наименьшего элемента массива длины n с помощью сравнений, сложность которого хотя бы для одного значения n была меньше чем $n - 1$.)

В роли класса $\mathcal{A}$ здесь выступает класс алгоритмов поиска наименьшего элемента массива с помощью сравнений. Если бы помимо сравнений были допустимы какие-то еще операции, то, возможно, $f(n) = n - 1$ уже не годилось бы в качестве нижней границы. Например, если было бы разрешено пользоваться операцией выбора наименьшего из нескольких элементов, то задачу можно было бы решить одной операцией.

Пример 14.2. Рассмотрим класс алгоритмов сортировки с помощью сравнений. Если алгоритм работает для массивов любой длины, то, разумеется, можно рассмотреть этот алгоритм применительно к массивам некоторой фиксированной длины n . Любая сортировка с помощью сравнений может быть для каждого конкретного n изображена бинарным деревом. В корне и внутренних вершинах находятся выполняемые сравнения, в листьях выписаны результаты сортировки. Априори в исходном массиве возможен любой порядок элементов, поэтому дерево будет иметь $n!$ листьев. Если взять, например, сортировку простыми вставками, то при $n = 2$ ее можно изобразить деревом, представленным на рис. 18а. При $n = 3$ дерево принимает вид, показанный на рис. 18б. Сложность в худшем случае для каждого n равна высоте соответствующего дерева (напомним, что высотой листа называется число ребер в том единственном пути, который ведет от корня дерева к этому листу; высотой дерева называется максимум высот его листьев). Если высота некоторого бинарного дерева равна h , то, очевидно, оно содержит не более 2^h листьев (максимальное возможное число листьев достигается в случае полного бинарного дерева, которое имеет ровно 2^h листьев). Поэтому если $T(n)$ — это временная сложность некоторой сортировки сравнениями, то должно выполняться неравенство

$$2^{T(n)} \geq n!,$$

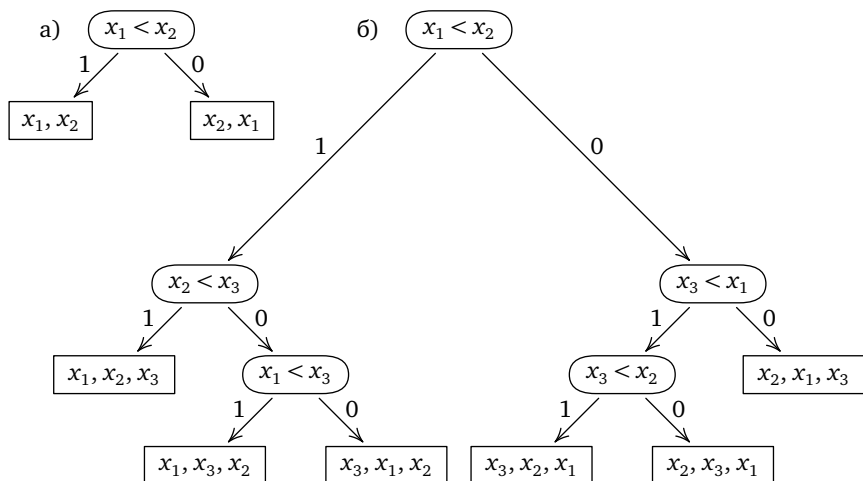


Рис. 18. Дерево сортировки простыми вставками для случаев а) $n = 2$ и б) $n = 3$.

откуда $T(n) \geq \log_2 n!$; так как значение $T(n)$ — целое число (мы измеряем сложность числом сравнений), то $T(n) \geq \lceil \log_2 n! \rceil$. Доказанное можно сформулировать в виде предложения.

Предложение 14.2. Функция $f(n) = \lceil \log_2 n! \rceil$ является нижней границей сложности алгоритмов сортировки массивов длины n с помощью сравнений.

Отсюда можно вывести, например, следующее предложение.

Предложение 14.3. Пусть сложность $T(n)$ некоторой сортировки по числу сравнений не превосходит $n \log_2 n + cn$, где c — некоторая константа. Тогда $T(n) = n \log_2 n + O(n)$ и, как следствие, $T(n) \sim n \log_2 n$.

Доказательство. В силу предложения 14.2 и формулы (10.12) для всех достаточно больших n мы имеем $T(n) \geq \lceil \log_2 n! \rceil > n \log_2 n - 2n$, и, следовательно, $n \log_2 n - 2n < T(n) \leq n \log_2 n + cn$. Отсюда следует требуемое. $\square$

Это позволяет, например, заключить, что для сложности по числу сравнений сортировки фон Неймана имеет место оценка $T_{\text{vN}}(n) = n \log_2 n + O(n)$ и асимптотическое равенство $T_{\text{vN}}(n) \sim n \log_2 n$, — мы уже видели, что $T_{\text{vN}} < n \lceil \log_2 n \rceil < n \log_2 n + n$.

Добавим, что никакая сортировка не может иметь сложность $T(n)$ по числу сравнений, допускающую оценку $\alpha n \log_2 n + o(n \log n)$, $\alpha < 1$ (в частности, оценку $\alpha n \log_2 n + O(n)$), — это следует из того, что для всех достаточно больших n выполнено $T(n) > n \log_2 n - 2n$.

Пример 14.3. Обратимся к задаче поиска места элемента в упорядоченном массиве длины n .

Предложение 14.4. Функция $f(n) = \lceil \log_2(n+1) \rceil$ является нижней границей сложности алгоритмов поиска места элемента в упорядоченном массиве длины n с помощью сравнений.

Доказательство. Любой алгоритм поиска места элемента в упорядоченном массиве фиксированной длины n может быть изображен бинарным деревом. Число листьев такого дерева есть $n+1$. Далее рассуждаем так же, как при доказательстве предложения 14.2. $\square$

Пример 14.4. Рассмотрим задачу вычисления a^n с помощью умножений (пример 4.2). Будем считать n размером входа. Затраты будем измерять количеством умножений.

Предложение 14.5. Функция $f(n) = \lceil \log_2 n \rceil$ является нижней границей сложности алгоритмов вычисления a^n с помощью умножений.

Доказательство. На каждом этапе алгоритма мы имеем вычисленный набор степеней

$$a^{l_1}, a^{l_2}, \dots, a^{l_k}, \quad (14.1)$$

при этом на начальном этапе набор состоит из одного элемента a^1 . Выполнив одно умножение, мы, очевидно, не можем увеличить максимальный показатель $l = \max\{l_1, l_2, \dots, l_k\}$ более чем вдвое. Поэтому если определить на наборах вида (14.1) функцию L , равную $\log_2 n - \log_2 l$, то в результате одного шага (одного умножения) эта функция не может уменьшиться больше чем на 1. В то же время значение этой функции на исходном наборе равно $\log_2 n$, а на итоговом наборе она заведомо неположительна. $\square$

§ 15. Оптимальные алгоритмы

Нижняя граница сложности класса алгоритмов определяется не единственным образом. Например, $f(n) = 0$ всегда является нижней границей, равно как и любая отрицательная функция. Чем больше найденная нижняя граница, тем она нетривиальнее и ценнее. Сигналом о том, что мы не сможем построить нижнюю границу бóльшую, чем

уже имеющаяся у нас нижняя граница $f(n)$, может служить, например, наличие $A \in \mathcal{A}$, для которого $T_A(n) = f(n)$. С такой ситуацией мы сталкиваемся в предыдущем параграфе в примерах 14.1 и 14.3. Известный нам алгоритм поиска наименьшего элемента и алгоритм бинарного поиска места элемента в упорядоченном массиве имеет каждый сложность, совпадающую с найденной нижней границей. Эти алгоритмы являются оптимальными в смысле следующего определения.

Определение 15.1. Пусть $\mathcal{A}$ — класс алгоритмов решения некоторой задачи. Пусть принято соглашение о том, в чем измеряются затраты алгоритмов и что считается размером входа, и пусть n — обозначение размера входа. Алгоритм $A \in \mathcal{A}$ называется *оптимальным* в $\mathcal{A}$, если $T_A(n)$ является нижней границей сложности алгоритмов из $\mathcal{A}$.

Пример 15.1. При получении нижней границы сложности и доказательстве оптимальности иногда бывает полезным привлечение функций на наборах тех величин, которые возникают в процессе выполнения алгоритма, например, на наборах значений переменных, используемых алгоритмом.

Предложение 15.1. Функция $f(n) = \left\lceil \frac{3n}{2} \right\rceil - 2$ является нижней границей сложности алгоритмов одновременного выбора наибольшего и наименьшего элементов массива длины n с помощью сравнений.

Доказательство. Каждый этап выполнения произвольного алгоритма V , основанного на сравнениях и предназначенного для поиска наибольшего и наименьшего элементов массива, может быть охарактеризован четверкой (A, B, C, D) подмножеств множества исходных элементов $\{x_1, x_2, \dots, x_n\}$, где

- A состоит из всех тех элементов, которые вообще не сравнивались;
- B состоит из всех тех элементов, которые участвовали в некоторых сравнениях и всегда оказывались большими;
- C состоит из всех тех элементов, которые участвовали в некоторых сравнениях и всегда оказывались меньшими;
- D состоит из всех тех элементов, которые участвовали в некоторых сравнениях, иногда оказываясь большими, а иногда — меньшими.

Пусть a, b, c, d — количество элементов множеств A, B, C, D соответственно. Исходная ситуация характеризуется равенствами $a = n$, $b = c = d = 0$. После завершения алгоритма должны выполняться равен-

ства $a = 0$, $b = c = 1$, $d = n - 2$. После первого сравнения на протяжении всего выполнения алгоритма постоянно будут иметь место неравенства $b \geq 1$, $c \geq 1$.

Все сравнения, совершаемые при выполнении алгоритма V , можно разбить на типы, обозначаемые $AA, AB, AC, AD, BB, BC, BD, CC, CD, DD$, например: сравнение принадлежит типу AB , если один из сравниваемых элементов берется из A , другой — из B , и т. д. Исходя из этого, можно выписать все возможные изменения четверки (a, b, c, d) под действием сравнений разных типов:

$$\begin{aligned} AA: & (a - 2, b + 1, c + 1, d), \\ AB: & (a - 1, b, c + 1, d) \mid (a - 1, b, c, d + 1), \\ AC: & (a - 1, b + 1, c, d) \mid (a - 1, b, c, d + 1), \\ AD: & (a - 1, b + 1, c, d) \mid (a - 1, b, c + 1, d), \\ BB: & (a, b - 1, c, d + 1), \\ BC: & (a, b - 1, c - 1, d + 2) \mid (a, b, c, d), \\ BD: & (a, b - 1, c, d + 1) \mid (a, b, c, d), \\ CC: & (a, b, c - 1, d + 1), \\ CD: & (a, b, c - 1, d + 1) \mid (a, b, c, d), \\ DD: & (a, b, c, d) \end{aligned}$$

(вертикальная черточка здесь означает «или»). Рассмотрим функцию $L(a, b, c, d) = \frac{3}{2}a + b + c - 2$. Для начальной и конечной стадий имеем $L(n, 0, 0, 0) = \frac{3}{2}n - 2$ и $L(0, 1, 1, n - 2) = 0$ соответственно.

Каждое сравнение типов AA, BB, CC понижает значение L на 1, т. е. дает $\Delta L = -1$. Выпишем все возможные значения ΔL при выполнении сравнений различных типов:

$$\begin{aligned} AA, BB, CC: & -1, \\ BD, CD: & -1 \mid 0, \\ AB, AC: & -\frac{3}{2} \mid -\frac{1}{2}, \\ BC: & -2 \mid 0, \\ AD: & -\frac{1}{2}, \\ DD: & 0. \end{aligned}$$

Сравнения, относящиеся к типам AB, AC, BC , могут приводить к $\Delta L < -1$, но это не может быть гарантировано никаким специальным выбором элементов из соответствующих множеств четверки

(A, B, C, D) , даже если принимать во внимание результаты всех сравнений, в которые были вовлечены конкретные элементы этих множеств. Например, сравнение типа AB в том случае, когда выбранный элемент из A оказывается больше выбранного элемента из B , преобразует (a, b, c, d) в $(a - 1, b, c, d + 1)$, что дает $\Delta L < -1$. Но результаты предшествующих сравнений не дают оснований для отметания возможности того, что выбранный элемент из B равен $\max\{x_1, x_2, \dots, x_n\}$ (ибо этот элемент оказался большим во всех сравнениях, в которые он был вовлечен). Но тогда будет выполнено $\Delta L = -\frac{1}{2}$. Аналогичным образом дело обстоит для сравнений, принадлежащих типам AC, BC . Поэтому, рассматривая поведение алгоритма V в худшем случае, надо считать, что на всех этапах $\Delta L \geq -1$. Но тогда для достижения равенства $L(a, b, c, d) = 0$ потребуется не менее $\lceil L(n, 0, 0, 0) \rceil$ сравнений. Это значит, что общее число сравнений в худшем случае не меньше, чем $\left\lceil \frac{3n}{2} \right\rceil - 2$. $\square$

Представленный в приложении А алгоритм одновременного поиска наибольшего и наименьшего элементов массива, требующий в худшем случае не более $\left\lceil \frac{3n}{2} \right\rceil - 2$ сравнений, является оптимальным¹.

Наряду с алгоритмами поиска наименьшего элемента, бинарного поиска места элемента в упорядоченном массиве и алгоритма одновременного поиска наибольшего и наименьшего элементов массива примером оптимального алгоритма может служить алгоритм («схема») Горнера вычисления значения полинома².

Для произвольного класса $\mathcal{A}$ оптимальный алгоритм может и не существовать: если, например, $\mathcal{A}$ содержит ровно два алгоритма, A_1, A_2 , и при этом A_1 имеет низкую сложность при четных значениях целочисленного размера входа n и высокую — при нечетных, а A_2 — наоборот, то, очевидно, ни A_1 , ни A_2 не будут оптимальными в $\mathcal{A}$.

Ясно, что если оптимальные алгоритмы в классе $\mathcal{A}$ существуют, то их сложности совпадают: из неравенств $T_{A_1}(n) \geq T_{A_2}(n)$ и $T_{A_2}(n) \geq T_{A_1}(n)$ следует, что $T_{A_1}(n) = T_{A_2}(n)$. Несколько более содержательный пример можно найти в задаче 108.

¹ Этот алгоритм и идея использования четверок (A, B, C, D) в доказательстве его оптимальности были предложены И. Полом в [82], но при этом убывающие функции в его доказательстве не привлекались, из-за чего потребовались дополнительные словесные мотивации.

² См., например, [26, разд. 4.6.4, упр. 38], [83].

В отличие от поиска наименьшего элемента, поиска места в упорядоченном массиве и одновременного поиска наибольшего и наименьшего элементов, задача сортировки не столь проста: те алгоритмы, которыми пользуются на практике для ее решения, не являются, строго говоря, оптимальными (см. приложение Е).

Заметим при этом, что для каждого конкретного n за конечное число шагов может быть найдено наименьшее число сравнений, достаточное в худшем случае для сортировки n элементов, а вместе с ним и алгоритм сортировки с такой сложностью. Это следует из того, что при каждом конкретном n алгоритм сортировки может быть представлен бинарным деревом. Можно порождать одно за другим все бинарные деревья с высотой, не превосходящей $\lceil \log_2 n! \rceil$, в вершинах каждого такого дерева различными способами расставлять выражения вида $x_i < x_j$, где $i, j \leq n$, а в листьях — различные перестановки длины n . Для каждого размеченного таким способом дерева нужно проверить, действительно ли каждая ветвь приводит именно к тому порядку, который указан в листе, и что все возможные порядки охвачены. Из всех «правильных» деревьев выбирается имеющее наименьшую высоту. Оно определяет оптимальный алгоритм для заданного n .

Таким образом, мы имеем алгоритм, который, исходя из $n > 0$, строит оптимальный по числу сравнений алгоритм сортировки массивов из n элементов (алгоритм строит алгоритм). Этот алгоритм построения оптимального алгоритма сортировки требует огромной работы. Этот пример еще раз показывает, что понятие сложности по числу операций требует осторожного обращения, — объем неучитываемых операций может превзойти разумные пределы.

Бинарный алгоритм возведения в степень n также не является оптимальным по числу умножений при использовании n в качестве размера входа (см. задачу 28), хотя и легко показать, что для случая $n = 2^k$ при рассмотрении k в качестве размера входа оптимальность имеет место: из предложения 14.5 следует, что возведение в степень $n = 2^k$ не может быть выполнено с затратами меньшими, чем k умножений, а бинарный алгоритм обходится в точности этим числом умножений.

В общем же случае, как и многие известные алгоритмы сортировки, бинарный алгоритм возведения в степень оптимален лишь в некотором асимптотическом смысле, о чем пойдет речь в следующем параграфе.

Если затраты для каждого входа являются целыми числами (например, они являются количеством выполненных операций), то для

фиксированного размера n_0 входа в рассматриваемом классе $\mathcal{A}$ алгоритмов решения некоторой задачи P существует такой, сложность которого при $n = n_0$ есть минимум сложностей всех алгоритмов из $\mathcal{A}$. Можно определить такой минимум для любого значения n , получаемую таким способом функцию от n иногда называют *сложностью задачи P* по отношению к классу алгоритмов $\mathcal{A}$. Важно, что при разных n минимумы могут доставляться разными алгоритмами из $\mathcal{A}$ (см. задачу 108). В дальнейшем мы не будем касаться этих вопросов.

В завершение этого этапа разговора об оптимальности и о нижних границах, мы упомянем некоторые факты, устанавливаемые в рамках теории Ф. П. Рамсея¹, сущность которой выражают словами типа «упорядоченные конфигурации присутствуют в любой большой структуре». Часто эту теорию поясняют примером так называемой задачи о гостях: какое минимальное число гостей надо пригласить, чтобы либо по меньшей мере 3 гостя были знакомы между собой, либо нашлись 3 гостя попарно не знакомых друг с другом? Не слишком сложно показывается, что это число равно 6. (Если в этой задаче вместо количества 3 взять 4, то ответом будет 18, а если взять 5, то известно только то, что нужное число гостей находится между 45 и 49. Вместе с этим получен точный ответ 25 для несколько измененного вопроса: либо найдутся 3 знакомых, либо 5 незнакомых между собой.²

В контексте теории Рамсея известна также следующая теорема Скереша—Эрдеша³ (через $|A|$ обозначено общее число элементов конечной последовательности A).

Теорема 15.1. Пусть $A = (a_1, \dots, a_n)$ — последовательность из n различных вещественных чисел. Если $|A| = n \geq sr + 1$ для некоторых целых положительных s, r , то либо A имеет возрастающую подпоследовательность из $s + 1$ элементов, либо убывающую подпоследовательность из $r + 1$ элементов (или и то, и другое).

Доказательство. Сопоставим каждому элементу a_i из A пару «оценок» (x_i, y_i) , где x_i — число элементов самой длинной возрастающей подпоследовательности, оканчивающейся на a_i , а y_i — количество элементов самой длинной убывающей подпоследовательности, начинающаяся с a_i . Обратим внимание, что никакие два элемента не имеют одинаковые оценки, т. е. что $(x_i, y_i) \neq (x_j, y_j)$ всякий раз, когда $i \neq j$. Действительно, если у нас есть $\dots a_i \dots a_j \dots$, то либо $a_i < a_j$ и самую длинную возрастающую подпоследовательность, оканчивающую-

¹ См., например, [19]

² См. [73, pp. 52—53]

³ См., например, ([75, ch. 4.2])

юся на a_i , можно расширить, добавив a_j (так что $x_i < x_j$) или $a_i > a_j$, и самой длинной убывающей подпоследовательности, начинающейся с a_j , может предшествовать a_i (так что $y_i > y_j$).

Теперь нарисуем сетку из n^2 ячеек (= квадратов). Поместим каждое a_i в ячейку с координатами (x_i, y_i) . Каждый элемент множества A можно поместить в какую-то ячейку, так как $1 \leq x_i, y_i \leq n$ для всех $i = 1, \dots, n$. Кроме того, ни в одной ячейке не может быть более одного элемента, потому что $(x_i, y_i) \neq (x_j, y_j)$ всякий раз, когда $i \neq j$. Поскольку $|A| = n \geq sr + 1$, у нас больше элементов, чем ячеек в нижнем левом прямоугольнике $r \times s$ сетки. Поэтому некоторый элемент a_i будет лежать вне этой части сетки. Но это означает, что либо $x_i \geq s + 1$ или $y_i \geq r + 1$ (или оба), — это именно то, что нам нужно. $\square$

§ 16. Асимптотические нижние границы. Алгоритм, оптимальный по порядку сложности

Известно не очень много алгоритмов, оптимальных в смысле определения 15.1. Рассмотрим дополнительно понятие алгоритма, оптимального по порядку сложности.

Определение 16.1. Пусть $\mathcal{A}$ — класс алгоритмов решения некоторой задачи. Пусть принято соглашение о том, в чем измеряются затраты алгоритмов и что считается размером входа, и пусть n — обозначение размера входа. Функция $f(n)$ называется *асимптотической нижней границей сложности* принадлежащих $\mathcal{A}$ алгоритмов, если для сложности $T_A(n)$ любого $A \in \mathcal{A}$ мы имеем $T_A(n) = \Omega(f(n))$. (Если используются несколько компонент $n_1, n_2, \dots, n_k$ размера входа, то асимптотическая нижняя граница — это, соответственно, функция аргументов $n_1, n_2, \dots, n_k$.) Алгоритм $A \in \mathcal{A}$ называется *оптимальным по порядку сложности* в $\mathcal{A}$, если $T_A(n)$ является асимптотической нижней границей сложности алгоритмов из $\mathcal{A}$.

Очевидно, что любая неотрицательная нижняя граница сложности является и асимптотической нижней границей, и что любой оптимальный алгоритм является оптимальным по порядку сложности.

Ниже в примере 16.4 мы увидим, что может существовать несколько оптимальных по порядку сложности в $\mathcal{A}$ алгоритмов. Однако сложности таких алгоритмов оказываются величинами одного порядка.

Теорема 16.1. Пусть $A, B \in \mathcal{A}$ оптимальны по порядку сложности в $\mathcal{A}$. Тогда $T_A(n) = \Theta(T_B(n))$.

Доказательство. Так как алгоритм A оптимален по порядку сложности, имеем $T_B(n) = \Omega(T_A(n))$ или, что то же самое, $T_A(n) = O(T_B(n))$, что вместе с $T_A(n) = \Omega(T_B(n))$ (алгоритм B оптимален по порядку сложности) дает $T_A(n) = \Theta(T_B(n))$. $\square$

Укажем достаточное условие оптимальности алгоритма по порядку сложности.

Теорема 16.2. Если функция $f(n)$ является асимптотической нижней границей сложности принадлежащих классу $\mathcal{A}$ алгоритмов и если алгоритм $A \in \mathcal{A}$ таков, что $T_A(n) = O(f(n))$, то этот алгоритм оптимален в $\mathcal{A}$ по порядку сложности и $T_A(n) = \Theta(f(n))$.

Доказательство. Для произвольного $B \in \mathcal{A}$ имеем $T_B(n) = \Omega(f(n))$; учитывая, что $T_A(n) = O(f(n))$, получаем $f(n) = \Omega(T_A(n))$ и $T_B(n) = \Omega(T_A(n))$, т. е. алгоритм A оптимален по порядку сложности.

Из $T_A(n) = \Omega(f(n))$ и $T_A(n) = O(f(n))$ получаем $T_A(n) = \Theta(f(n))$. $\square$

Пример 16.1. Из предложения 14.5 следует, что функция $f(n) = \log_2 n$ является асимптотической нижней границей сложности алгоритмов вычисления a^n с помощью умножений (эта функция даже является нижней границей в смысле определения 14.1). Для сложности бинарного алгоритма вычисления a^n мы имеем $T_{RS}(n) = O(\log n)$ (см. пример 4.2). Из теоремы 16.2 теперь получаем:

Бинарный алгоритм возведения в степень оптимален по порядку сложности в классе алгоритмов вычисления a^n с помощью умножений.

Иногда, хотя не часто, из совершенно элементарных соображений удается найти такую нижнюю границу сложности, что далее по теореме 16.2 оказывается возможным обосновать оптимальность некоторого известного алгоритма по порядку сложности. Рассмотрим два примера из теории графов.

Пример 16.2. Вопрос о существовании и построении эйлерова цикла данного ориентированного графа сформулирован в задаче 25. Остановимся на случае связного графа. Очевидно, что если избрать число ребер $|E|$ в качестве размера входа, то $|E|$ является нижней границей сложности класса всех алгоритмов построения эйлеровых циклов, так как, во-первых, любой эйлеров цикл содержит по определению все ребра графа и на любое ребро уйдет по крайней мере одна операция, и, во-вторых, для любого натурального n существует граф с $|E| = n$, содержащий эйлеров цикл. Поэтому тот алгоритм со сложностью $O(|E|)$, который требуется дать в задаче 25, будет оптимальным по порядку сложности.

Пример 16.3. Рассмотрим задачу построения минимального остовного (покрывающего) дерева данного связного графа, каждое ребро которого имеет некоторый неотрицательный вес. Имеется в виду дерево:

- 1) которое охватывает все вершины данного графа,
- 2) все ребра которого являются ребрами данного графа,
- 3) которое среди всех деревьев, обладающих свойствами 1), 2), имеет минимальный суммарный вес ребер¹.

Будем считать, что граф не содержит кратных ребер. Пример графа с соответствующим остовным деревом дан на рис. 19. Для построения минимального остовного дерева известны остроумные алгоритмы, например, алгоритм Р. Прима² с оценкой сложности

$$O(|E| + |V| \cdot \log |V|). \quad (16.1)$$

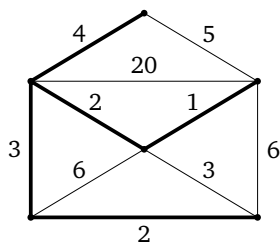


Рис. 19. Граф с приписанными ребрам весами и его остовное дерево.

В этой оценке $|V|$ и $|E|$ рассматриваются как две компоненты размера входа. Не обсуждая вопрос, является или нет алгоритм Прима оптимальным по порядку сложности при таком выборе компонент размера входа и не рассматривая детали алгоритма Прима, мы покажем, что этот алгоритм оптимален по порядку сложности при использовании $|V|$ как размера входа. Так как среди всех графов без кратных ребер, имеющих $|V|$ вершин, наибольшее число ребер

$$\frac{|V|(|V| - 1)}{2}$$

имеет полный граф, то оценка (16.1) имеет следствием оценку $O(|V|^2)$. С другой стороны, асимптотическая нижняя граница $\Omega(|V|^2)$ для алгоритмов такого рода получается тривиально: эту оценку допускает число ребер полного графа, а на каждое ребро графа алгоритм затрачивает по крайней мере одну операцию (он не может не принять в расчет вес какого-либо ребра).

Если использовать для входов алгоритмов построения остовного дерева размер $|V|$, то $|V|^2$ даст асимптотическую нижнюю границу сложности этих алгоритмов, и алгоритм Прима будет оптималь-

¹ См. [28, гл. 24].

² Этот алгоритм описан, например, в [28, разд. 24.2].

ным по порядку сложности (его сложность, соответствующая этому размеру входа, есть $\Theta(|V|^2)$).

Из этого примера видно, что само свойство оптимальности находится в зависимости от выбора размера входа¹.

В следующем примере, касающемся сортировки массивов, рассматривается сложность в худшем случае по числу сравнений.

Пример 16.4. Аналогично тому, как это сделано в примере 16.1, мы получаем, что любая сортировка, сложность которой допускает оценку $O(\log n!)$, является оптимальной по порядку сложности. В силу формулы Стирлинга мы имеем $n \log_2 n = O(\log n!)$, откуда следует, что любая сортировка, сложность которой по числу сравнений допускает оценку $O(n \log n)$, является оптимальной по порядку сложности.

Сортировка бинарными вставками и сортировка фон Неймана являются оптимальными по порядку сложности по числу сравнений.

(Позднее мы установим оценку $O(n \log n)$ и для сложности рекурсивной сортировки слияниями.) Эти сортировки имеют разные сложности как функции от n : например, $T_{\text{vN}}(5) = 9$, $T_{\text{B}}(5) = 8$. Но, повторим, их сложности являются величинами одного порядка при $n \rightarrow \infty$.

Резюмируем наши наблюдения.

Предложение 16.1. *Необходимым и достаточным условием оптимальности сортировки по порядку сложности является справедливость оценки $O(n \log n)$ для сложности этой сортировки. Если для сложности некоторой сортировки справедлива оценка $O(n \log n)$, то справедлива и оценка $\Theta(n \log n)$.*

Доказательство. Достаточность уже установлена выше. Оставшаяся часть доказательства следует, например, из теоремы 16.1 и оптимальности по порядку сложности сортировки бинарными вставками. $\square$

Заметим при этом, что для сортировок бинарными вставками и фон Неймана справедливо значительно более сильное утверждение, чем утверждение об их оптимальности по порядку сложности. Для их сложностей и сложности $T_{\text{opt}}(n)$ оптимального алгоритма имеет место асимптотическая эквивалентность:

$$T_{\text{B}}(n) \sim T_{\text{vN}}(n) \sim T_{\text{opt}}(n) \sim n \log_2 n \sim \log_2 n!.$$

¹ Этот пример сообщил автору Е. В. Зима.

Коснемся нижних границ пространственной сложности алгоритмов сортировки. Если из всех этих алгоритмов выделить класс таких, которые используют для перемещения элементов обмены ($\leftrightarrow$), а не присваивания ($:=$), то для этого класса единственной нижней границей, если говорить о функциях, принимающих неотрицательные значения, будет тождественно равная нулю функция, так как существуют, например, пузырьковая сортировка, сортировки простыми и бинарными вставками, не использующие дополнительных переменных того типа, к которому относятся элементы сортируемого массива. (То, что требуются переменные для хранения значений индексов элементов, несущественно при изучении сложности сортировки.) Алгоритмы, использующие присваивания для перемещения сортируемых элементов, не могут, естественно, обойтись без дополнительного места для хранения хотя бы одной величины того же типа, что и элементы сортируемого массива, и одной из нижних границ будет функция, тождественно равная единице. Если рассматривать все алгоритмы сортировки вместе, то вновь единственная неотрицательная целочисленная граница — это 0. В соответствии с этим определяются как оптимальные, так и оптимальные по порядку пространственной сложности алгоритмы.

Для произвольного класса $\mathcal{A}$ оптимальный по порядку сложности алгоритм может и не существовать: если $\mathcal{A}$ содержит всего два алгоритма A_1, A_2 и при этом A_1 имеет низкую сложность при четных значениях целочисленного размера входа n и высокую — при нечетных, а A_2 — наоборот, то, очевидно, ни A_1 , ни A_2 не будут оптимальными в $\mathcal{A}$ по порядку сложности (представим себе, что $T_{A_1} = (1 + (-1)^n)n$, $T_{A_2} = (1 - (-1)^n)n$). В то же время, если допустить, что $T_{A_1} = (2 + (-1)^n)n$, $T_{A_2} = (2 - (-1)^n)n$, то $T_{A_1}(n) = \Theta(T_{A_2}(n))$ (обе функции имеют порядок n), и это говорит о возможности существования оптимального по порядку сложности алгоритма при отсутствии оптимального.

§ 17. Нижняя граница сложности в среднем

При рассмотрении сложности в среднем мы основываемся на тех же понятиях нижней границы, оптимального и оптимального по порядку сложности алгоритма, которые были введены для сложности в худшем случае.

Пример 17.1. Вновь обратимся к классу алгоритмов сортировки. Будем, как и в § 6, рассматривать вероятностное пространство Π_n .

Предложение 17.1. Функция $\log_2 n!$ является нижней границей сложности в среднем для алгоритмов сортировки массивов длины n с помощью сравнений.

Прежде всего докажем вспомогательное утверждение.

Лемма 17.1. В любом двоичном дереве с m листьями сумма высот всех листьев не меньше $m \log_2 m$.

Доказательство. Пусть непусто множество M всех двоичных деревьев, для которых сумма высот всех листьев меньше $m \log_2 m$. Выберем в M какое-нибудь из деревьев, имеющих наименьшую сумму высот, и обозначим его U (сумму высот всех листьев будем обозначать через $H(U)$). Это дерево не может состоять из одного лишь корня, потому что для такого дерева обсуждаемое неравенство выполнено. Далее, из корня этого дерева не может выходить только одно ребро, потому что для дерева, получающегося из исходного удалением корня и этого ребра, обсуждаемое неравенство не выполняется, и такое новое дерево имеет сумму высот меньшую, чем дерево U (противоречие со способом выбора дерева U). Поэтому из корня дерева U выходит два ребра, концы которых являются корнями двух деревьев U_1 и U_2 , для которых выполнено обсуждаемое неравенство. Пусть эти деревья имеют, соответственно, m_1 и m_2 листьев, $m_1, m_2 > 0$. Имеем соотношение для сумм высот всех листьев

$$H(U) = H(U_1) + H(U_2) + m \geq m_1 \log_2 m_1 + m_2 \log_2 m_2 + m.$$

Отсюда получаем

$$H(U) \geq m_1 \log_2 m_1 + (m - m_1) \log_2 (m - m_1) + m$$

при некотором m_1 таком, что $1 \leq m_1 \leq m - 1$.

Легко показать, что при фиксированном m функция $f(x) = x \log_2 x + (m - x) \log_2 (m - x)$ достигает минимума на отрезке $[1, m - 1]$ в точке (не обязательно целой) $m/2$. Поэтому

$$H(U) \geq \frac{m}{2} \log_2 \frac{m}{2} + \frac{m}{2} \log_2 \frac{m}{2} + m = m \log_2 m.$$

Но это противоречит способу выбора U . Лемма доказана. $\square$

Доказательство предложения 17.1. Рассмотрим какое-либо дерево сортировки массива длины n . Это дерево имеет ровно $n!$ листьев, каждый лист соответствует перестановке элементов исходного массива. Появление на выходе рассматриваемой сортировки любой из этих перестановок имеет одну и ту же вероятность $\frac{1}{n!}$, количество сравнений, приводящее к этой перестановке — это высота соответствующего

этой перестановке листа. Поэтому математическое ожидание числа сравнений равно произведению суммы высот всех листьев на $\frac{1}{n!}$.

Согласно лемме 17.1 сумма высот всех листьев должна быть не меньше, чем $n! \log_2 n!$. Отсюда математическое ожидание числа сравнений не меньше, чем $\log_2 n!$. $\square$

Доказанные ранее теорема 16.2 и предложение 16.1 справедливы в равной мере и для сложности в худшем случае, и для сложности в среднем. Принимая это во внимание, мы получаем следствие предложения 17.1:

Сортировка бинарными вставками и сортировка фон Неймана, а также быстрая сортировка являются оптимальными по порядку сложности в среднем по числу сравнений.

К этому списку позднее мы добавим и рекурсивную сортировку слияниями. Наиболее же существенно упоминание в этом списке быстрой сортировки. С одной стороны, как мы знали и раньше, эта сортировка очень удобна и имеет низкую пространственную сложность, с другой — мы видим теперь, что и в смысле временной сложности в среднем эта сортировка в определенном смысле может быть отнесена к наилучшим.

Можно показать существование оптимальной в среднем сортировки: для каждого фиксированного n в множестве всех деревьев сортировки массивов длины n можно рассмотреть подмножество деревьев, имеющих наименьшую сумму H высот всех листьев (тогда и $H/n!$ будет иметь наименьшее значение), и взять какое-нибудь из деревьев этого подмножества. Определяя сортировку этим способом для всех n мы получаем оптимальную сортировку. Для любой оптимальной в среднем сортировки выполнено

$$\bar{T}_{\text{opt}}(n) \sim \log_2 n! \sim n \log_2 n,$$

так как, например, мы имеем $T_{\text{vN}}(n) \sim \log_2 n!$ и

$$T_{\text{vN}}(n) \geq \bar{T}_{\text{vN}}(n) \geq \bar{T}_{\text{opt}}(n) \geq \log_2 n!.$$

Таким образом, функция $\bar{T}_{\text{opt}}(n)$ при $n \rightarrow \infty$ «зажата» между двумя величинами порядка $\log_2 n!$; как следствие, $\bar{T}_{\text{opt}}(n) = \Theta(\log_2 n!)$.

В этом примере, как и во всем этом параграфе, мы не касаемся рандомизированных алгоритмов, о которых будет говориться в § 18.

Как мы ранее наблюдали в некоторых примерах, рассмотрение функции L , подобранной надлежащим образом, и изучение изменения ее значений в ходе выполнения алгоритма нередко позволяет

получать хорошие оценки (в частности, нижние границы) сложности в худшем случае. Рассмотрение такого рода функций может приводить к цели и при исследовании сложности в среднем. В задаче 57 функцию L можно определить как максимум значений компонент инверсионного вектора перестановки. Эта функция является случайной величиной на Π_n , при этом $\Delta L = -1$ на каждом шаге алгоритма. В других ситуациях начальное значение L может определяться однозначно, тогда как ΔL является случайной величиной. Сформулируем теорему, полезную в этих ситуациях.

Теорема 17.1. Пусть $\xi_1, \xi_2, \dots$ — последовательность неотрицательных случайных величин на некотором вероятностном пространстве. Пусть числовая последовательность $h_1, h_2, \dots$ такова, что для каждого $k \geq 1$ выполнено $E(\xi_k | \xi_1, \xi_2, \dots, \xi_{k-1}) \leq h_k$ при всех значениях $\xi_1, \xi_2, \dots, \xi_{k-1}$. Зафиксируем неотрицательное число q и введем целочисленную случайную величину

$$\tau = \min \left\{ n : \sum_{k=1}^n \xi_k \geq q \right\}.$$

Пусть $\tau < \infty$ всюду на рассматриваемом вероятностном пространстве. Тогда $E\left(\sum_{k=1}^{\tau} h_k\right) \geq q$.

Доказательство приведено в приложении D.

Для того чтобы воспользоваться этой теоремой, можно опять попытаться определить некоторую неотрицательную функцию L , отражающую степень «недорешенности» рассматриваемой задачи: значение L равно нулю, если алгоритм доработал до конца и задача решена. Считаем, что всем входам фиксированного размера сопоставляется одно и то же значение функции L . После выбора такой функции L мы должны показать, что последовательность величин $E(-\Delta L)$, соответствующих идущим друг за другом этапам алгоритма, ограничена сверху некоторой известной числовой последовательностью $h_1, h_2, \dots$; тогда случайная величина τ , равная для данного входа общему количеству этапов, приводящих к завершению алгоритма, такова, что $E\left(\sum_{k=1}^{\tau} h_k\right) \geq q$, где q — значение функции L , сопоставляемое входам алгоритма рассматриваемого фиксированного размера. Это неравенство можно использовать, чтобы оценить снизу значение $E\tau$. Конечность величины τ следует из завершимости алгоритма для любого входа.

Пример 17.2. Вернемся к задаче одновременного выбора наибольшего и наименьшего элементов массива длины n , $n \geq 2$, с помощью сравнений (пример 15.1). Вероятностным пространством здесь вновь будем считать Π_n . Каждая перестановка из Π_n отражает, как обычно, взаимный порядок элементов исходного массива. Все перестановки считаются равновероятными.

Вначале рассмотрим эту задачу на некотором «интуитивном» уровне, подобно рассмотрению пузырьковой сортировки непосредственно перед теоремой 17.1.

Обратимся к множествам A, B, C, D , типам сравнений $AA, AB, \dots, DD$, количествам a, b, c, d элементов множеств A, B, C, D (см. доказательство предложения 15.1) и функции $L(a, b, c, d) = \frac{3}{2}a + b + c - 2$; равенство $L = 0$ является необходимым и достаточным условием того, что искомые элементы найдены.

Если при четном n ограничиться сравнениями типов AA, BB, CC , то сравнений потребуется в точности $\frac{3}{2}n - 2$: для этих типов сравнения $E\Delta L = -1$. Если же n нечетно, то потребуется выполнить хотя бы одно сравнения типа AB, AC или AD . Можно показать, что математическое ожидание величины, на которую изменится значение L после такого сравнения, удовлетворяет неравенству $E\Delta L \geq -1 + \frac{1}{2n}$. При нечетном n , таким образом, среднее число сравнений, обеспечивающее заключительное нулевое значение функции L , будет не меньше, чем сумма $\frac{3}{2}n - 2 - \left(-1 + \frac{1}{2n}\right)$ и 1 (первое выписанное слагаемое — это нижняя граница среднего числа сравнений типов AA, BB, CC , второе — равная 1 нижняя граница числа сравнений, принадлежащих типам AB, AC, AD). И мы готовы заключить, что функция

$$f(n) = \begin{cases} \frac{3}{2}n - 2, & \text{если } n \text{ четно,} \\ \frac{3}{2}n - 2 + \frac{1}{2n}, & \text{если } n \text{ нечетно,} \end{cases} \quad (17.1)$$

является нижней границей сложности в среднем алгоритмов одновременного выбора наибольшего и наименьшего элементов массива длины n , $n \geq 2$, с помощью сравнений.

Чтобы здесь воспользоваться теоремой 17.1 для получения гарантии правильности сделанного предположения о нижней границе, мы полагаем, что случайные величины $\xi_1, \xi_2, \dots$ для произвольного алгоритма одновременного выбора наибольшего и наименьшего элементов равны значениям $-\Delta L$ на последовательных шагах этого ал-

горитма, об определении функции L будет сказано ниже. Значения $\xi_1, \xi_2, \dots$ зависят от конкретной перестановки из Π_n , отражающей взаимный порядок элементов в исходном массиве. После того, как алгоритм закончил работу, значения ξ_k при последующих значениях k равны нулю.

Предложение 17.2. Функция (17.1) является нижней границей сложности в среднем алгоритмов одновременного выбора наибольшего и наименьшего элементов массива длины n , $n \geq 2$, с помощью сравнений.

Доказательство. Вновь обратимся к множествам A, B, C, D , к рассмотренным в доказательстве предложения 15.1 типам $AA, AB, \dots, DD$ сравнений, количествам a, b, c, d элементов множеств A, B, C, D и функции $L(a, b, c, d) = \frac{3}{2}a + b + c - 2$ (равенство $L = 0$ является необходимым и достаточным условием того, что искомые элементы найдены).

Пусть в процессе выбора наибольшего и наименьшего элементов массива уже произведены некоторые сравнения элементов этого массива. При каждом из этих сравнений получался некоторый результат «истина» или «ложь», и эти результаты нам известны. Если процесс выбора наибольшего и наименьшего элементов еще не завершен, то следующим шагом вновь будет некоторое сравнение одного из типов $AA, AB, \dots, DD$. Сравнения AB, AC и BC представляют особый интерес, так как можно бы предположить, что здесь получится $E\Delta L < -1$. Но на самом деле это неравенство не имеет места ни при одном из этих трех типов сравнений.

Тип BC. Покажем невозможность такого выбора индексов i и j , основанного только на результатах уже произведенных сравнений, что $x_i \in B$, $x_j \in C$ и при этом с вероятностью, не меньшей половины, выполняется $x_i < x_j$.

Пусть сделан некий выбор индексов i и j , при котором $x_i \in B$, $x_j \in C$. Рассмотрим множество M всех массивов $y_1, y_2, \dots, y_n$, которые получаются перестановками элементов массива $x_1, x_2, \dots, x_n$ и при этом сравнения элементов с теми же самыми индексами, которые использовались в уже произведенных сравнениях элементов массива $x_1, x_2, \dots, x_n$, дают для массивов из M результаты, совпадающие с полученными для $x_1, x_2, \dots, x_n$. Представим множество M в виде объединения двух непересекающихся подмножеств M_1 и M_2 , где в M_1 входят те массивы, для которых $y_i < y_j$, а в M_2 — те, для которых $y_j < y_i$. Покажем, что в M_1 больше элементов, чем в M_2 . Операция обмена $y_i \leftrightarrow y_j$ определяет взаимно однозначное отображение на себя множества всех массивов, которые получаются перестановками элемен-

тов массива $x_1, x_2, \dots, x_n$. Обратным к этому отображению является оно само. В силу определения множеств B и C это отображение переводит элементы множества M_1 в элементы множества M_2 . Следовательно, в M_1 элементов не больше, чем в M_2 . Но в M_2 найдется массив, который при этом отображении не переходит в массив из M_1 и, более того, вообще покидает множество M (чтобы убедиться в этом, достаточно заметить, что в M_2 имеется такой массив $y_1, y_2, \dots, y_n$, для которого $y_j = \min\{y_1, y_2, \dots, y_n\}$). Поэтому в M_1 элементов меньше, чем в M_2 . Если m_1, m_2 — количество элементов множеств M_1 и M_2 , то имеем

$$\frac{m_1}{m_1 + m_2} < \frac{m_2}{m_1 + m_2}$$

и, следовательно,

$$\frac{m_1}{m_1 + m_2} < \frac{1}{2}.$$

Таким образом, вероятность того, что i -й элемент меньше j -го, есть $\frac{1}{2} - \varepsilon$, $\varepsilon > 0$. Получаем

$$E\Delta L = -2\left(\frac{1}{2} - \varepsilon\right) = -1 + 2\varepsilon > -1.$$

Тип АВ. Аналогично предыдущему можно показать, что при любом способе выбора индексов таких, что $x_i \in A$, $x_j \in B$, вероятность выполнения неравенства $x_i > x_j$ есть $\frac{1}{2} - \varepsilon$, $\varepsilon > 0$. Получаем

$$E\Delta L = -\frac{3}{2}\left(\frac{1}{2} - \varepsilon\right) - \frac{1}{2}\left(\frac{1}{2} + \varepsilon\right) = -1 + \varepsilon > -1.$$

Тип АС. Рассматривается аналогично типу АВ.

В дальнейшем будет полезна оценка ε для сравнений типов АВ и АС. Рассмотрим для определенности АВ. Очевидно, что чем больше сравнений к рассматриваемому моменту прошел элемент $x_j \in B$, тем меньше вероятность того, что элемент $x_i \in A$ окажется больше него. Наибольшая же вероятность получится в случае, когда x_j сравнивался только с одним элементом, например с x_s , и x_s — это наименьший элемент исходного массива. Вычислим вероятность, с которой в этом случае $x_i > x_j$. Итак, если упорядочить все элементы, кроме x_i, x_j , то x_s окажется наименьшим (первым). Общее число способов, которыми к этой упорядоченной совокупности можно добавить x_i, x_j , с учетом $x_s < x_j$ равно $n(n-2)$ (для добавления x_j существует $n-2$ способа, затем добавляем x_i , и для этого существует n способов). Из них $\binom{n}{2} - 1$ соответствуют неравенству $x_i < x_j$ (возможен любой выбор двух среди n мест, кроме двух первых).

Интересующая нас вероятность равна

$$\frac{n(n-2) - \binom{n}{2} + 1}{n(n-2)} = \frac{1}{2} - \frac{1}{2n}.$$

Это дает нам $\varepsilon \geq \frac{1}{2n}$, и, таким образом,

$$E\Delta L \geq -1 + \frac{1}{2n}$$

для AB и AC . Если при четном n удастся обойтись сравнениями типов AA, BB, CC , то сравнений потребуется в точности $\frac{3}{2}n - 2$. Если n нечетно, то для решения задачи обязательно потребуется произвести хотя бы одно сравнение типа AB, AC или AD . Сравнение типа AD дает $\Delta L = -\frac{1}{2} > -1 + \frac{1}{2n}$.

Можно убедиться и в том, что для сравнений типов AB и AC выполняется $E\Delta L \geq -1 + \frac{1}{2n}$, если $E\Delta L$ рассматривается как условное математическое ожидание при известных значениях ΔL для всех предыдущих сравнений, предписываемых алгоритмом. Эти известные значения определяют некоторое событие V в вероятностном пространстве Π_n . Событие V может быть представлено как сумма $V_1 + V_2 + \dots + V_l$ попарно несовместных событий, где каждое V_k состоит из тех элементов Π_n , для которых все предыдущие сравнения образуют некоторую фиксированную последовательность сравнений элементов с полностью определенными для данного k индексами, и при этом возникающие значения ΔL совпадают с известными из условия. В силу доказанного выше $E(\Delta L|V_k) \geq -1 + \frac{1}{2n}$ для каждого $k \leq l$. Отсюда по формуле полного математического ожидания $E(\Delta L|V) \geq -1 + \frac{1}{2n}$.

В сумме $\sum_{k=1}^{\tau} h_k$, о которой идет речь в теореме 17.1, мы можем положить $h_k = 1$ для всех значений k , кроме какого-то одного значения k_0 , для которого $h_{k_0} = 1 - \frac{1}{2n}$ (при этом $\xi_i = -\Delta L$ с ΔL , вычисленным для i -го сравнения, $i = 1, 2, \dots$). Это дает нам

$$E\left(\sum_{k=1}^{\tau} h_k\right) = E(\tau - 1) + 1 - \frac{1}{2n},$$

и с помощью следующего из теоремы 17.1 соотношения

$$E\left(\sum_{k=1}^{\tau} h_k\right) \geq \frac{3}{2}n - 2$$

мы находим

$$E(\tau - 1) + 1 - \frac{1}{2n} \geq \frac{3}{2}n - 2.$$

После упрощения получаем

$$E\tau \geq \frac{3}{2}n - 2 + \frac{1}{2n}.$$

Таким образом, все доказано и для четных, и для нечетных n . $\square$

Если существует алгоритм решения рассматриваемой задачи, требующий числа сравнений, которое равно значению функции $f(n)$, определенной формулой (17.1), то, по только что доказанному предложению, этот алгоритм является оптимальным в среднем по числу сравнений. Укажем такой алгоритм. Если n четно, то действуем в соответствии с алгоритмом, обсуждавшимся в примере 15.1. Пусть $n = 2k + 1$, $k \geq 0$. Находим

$$m_i = \min\{x_{2i-1}, x_{2i}\}, \quad M_i = \max\{x_{2i-1}, x_{2i}\},$$

$i = 1, 2, \dots, k$, и

$$m = \min\{m_1, m_2, \dots, m_k\},$$

что потребует $n - 2$ сравнений. Без дополнительных затрат можно найти s , $1 \leq s \leq 2k$, такое, что $m = m_s$. Далее сравниваем x_n (т.е. x_{2k+1}) с M_s . Согласно рассуждениям, проведенным выше при анализе сравнений типа АВ, вероятность того, что $x_n > M_s$, равна $\frac{1}{2} - \frac{1}{2n}$. Если $x_n > M_s$, то результатом работы алгоритма будет

$$\max\{M_1, \dots, M_{s-1}, M_{s+1}, \dots, M_k, x_n\}, \quad m,$$

в противном случае —

$$\max\{M_1, M_2, \dots, M_k\}, \quad \min\{m, x_n\}.$$

Среднее число сравнений при нечетном n равно

$$(n - 2) + \left(\frac{1}{2} - \frac{1}{2n}\right) + 2\left(\frac{1}{2} + \frac{1}{2n}\right) + \left(\frac{n-1}{2} - 1\right) = \frac{3}{2}n - 2 + \frac{1}{2n},$$

что и требовалось.

Существует оптимальный в среднем алгоритм одновременного нахождения наибольшего и наименьшего элементов массива длины n , $n \geq 2$, с помощью сравнений. Этот алгоритм имеет сложность (17.1).

§ 18. Нижние границы сложности рандомизированных алгоритмов. Принцип Яо

Этот параграф начнем с формулировки и обсуждения одного факта из теории игр. С произвольной квадратной числовой матрицей $M = (m_{ij})$ порядка k связывается *матричная игра* двух игроков, которых мы назовем I и J . Один раунд игры состоит в том, что I и J независимо друг от друга называют соответственно целые i и j из диапазона от 1 до k , и после этого J выплачивает своему сопернику I сумму в m_{ij} рублей (при $m_{ij} < 0$ сумма в $-m_{ij}$ рублей выплачивается игроком I игроку J). *Стратегией* называется вектор $p = (p_1, p_2, \dots, p_k)$ с вещественными неотрицательными компонентами, в сумме дающими единицу; стратегии $(1, 0, \dots, 0)$, $(0, 1, \dots, 0)$, $\dots$, $(0, 0, \dots, 1)$ называются *чистыми*. Допустим, что игроки независимо избирают для себя некоторые стратегии — обозначим их соответственно p и q , — и затем в последовательных раундах игрок I называет число i с вероятностью p_i , а игрок J называет число j с вероятностью q_j . Математическое ожидание выигрыша игрока I равно

$$\sum_i \sum_j m_{ij} p_i q_j,$$

эту величину мы будем обозначать $M(p, q)$. Допустим также, что вместо проведения большого числа раундов игры каждый из игроков, основываясь на известной матрице M , один раз выбирает, независимо от другого игрока, для себя некоторую стратегию и сообщает ее судье, который по полученным от игроков стратегиям (векторам p и q) вычисляет $M(p, q)$, умножает его на количество предпологавшихся раундов и объявляет полученное число выигрышем игрока I . Игрок I заинтересован в нахождении такой стратегии, которая максимизирует $M(p, q)$, а у игрока J цели прямо противоположные. Одна из теорем раздела теории игр, посвященного матричным играм¹, утверждает, что для игроков существуют *оптимальные* стратегии p^*, q^* такие, что $M(p^*, q^*)$ является одновременно значением величин

$$\max_p \min_q M(p, q) \quad \text{и} \quad \min_q \max_p M(p, q).$$

¹ См., например, [11, теорема 2.1].

В дальнейшем нас не будут интересовать оптимальные стратегии, однако равенство

$$\max_p \min_q M(p, q) = \min_q \max_p M(p, q) \quad (18.1)$$

окажется для нас очень ценным.

При фиксированной стратегии p значение $\min_q M(p, q)$ равно минимальному значению среди

$$\sum_i m_{i1}p_i, \quad \sum_i m_{i2}p_i, \quad \dots, \quad \sum_i m_{ik}p_i,$$

т. е. $\min_j \sum_i m_{ij}p_i$ (в качестве стратегии q , на которой достигается минимум, выступает некоторая чистая стратегия). Аналогичным образом $\max_p M(p, q) = \max_i \sum_j m_{ij}q_j$. Это позволяет переписать равенство (18.1) в виде

$$\max_p \min_j \sum_i m_{ij}p_i = \min_q \max_i \sum_j m_{ij}q_j, \quad (18.2)$$

откуда следует, что для любых стратегий $p = (p_1, p_2, \dots, p_k)$ и $q = (q_1, q_2, \dots, q_k)$ выполнено

$$\min_j \sum_i m_{ij}p_i \leq \max_i \sum_j m_{ij}q_j. \quad (18.3)$$

То, что исходная матрица квадратная, не играло никакой роли в выводе последней формулы, каждая из переменных i и j могла пробегать свое множество значений. Можно даже не нумеровать строки и столбцы исходной матрицы, а дать им какие-то имена. Для матриц с такого рода именованием строк и столбцов чаще используют название *таблица*.

Вернемся теперь от матричных игр к анализу сложности алгоритмов. Пусть имеется конечное множество $\mathcal{A}$ алгоритмов решения некоторой задачи и конечное множество X входов фиксированного размера, и для них составлена числовая таблица, элементы которой проиндексированы элементами множества X (первый индекс) и множества $\mathcal{A}$ (второй индекс). В качестве элемента таблицы с индексами x, A берется величина затрат (например, временных) $C_A(x)$. Если рассмотреть какие-либо распределения вероятностей на X и $\mathcal{A}$, то в силу (18.3) будем иметь

$$\min_{A \in \mathcal{A}} E_X C_A(x) \leq \max_{x \in X} E_{\mathcal{A}} C_A(x), \quad (18.4)$$

где символами E_X и $E_{\mathcal{A}}$ обозначены математические ожидания, связанные с заданными (произвольными) распределениями вероятностей на X и $\mathcal{A}$.

Будем рассматривать рандомизированный алгоритм как конечное множество детерминированных («обычных») алгоритмов с некоторым распределением вероятностей на нем, — о возможности такого взгляда на рандомизированные алгоритмы говорилось в конце § 8. Тогда можно определить усредненные затраты этого рандомизированного алгоритма для каждого входа. Фиксируя размер входа, мы можем рассмотреть сложность в худшем случае такого рандомизированного алгоритма $\mathcal{A}$ как максимум усредненных затрат по входам данного размера, эта сложность записана в правой части неравенства (18.4). Левая же часть этого неравенства может быть интерпретирована как наименьшая из сложностей в среднем алгоритмов класса $\mathcal{A}$, для определения этой сложности используется некоторое распределение вероятностей на X .

Формула (18.4) ведет к минимаксному принципу, предложенному А. Яо¹.

Теорема 18.1 (принцип Яо). Пусть для размера входа зафиксировано некоторое значение n , и пусть все входы размера n образуют конечное множество X , на котором задано некоторое распределение вероятностей. Пусть $\mathcal{A}$ — конечное множество алгоритмов, применимых к элементам множества X , и пусть на $\mathcal{A}$ тоже задано некоторое распределение вероятностей. Множество $\mathcal{A}$ с этим распределением можно рассматривать как единый рандомизированный алгоритм, считая его сложностью $T(n)$ усредненные затраты в худшем случае. Тогда, если найти некоторую нижнюю границу $f(n)$ сложностей в среднем (в соответствии с распределением вероятностей на множестве X) всех детерминированных алгоритмов класса $\mathcal{A}$, то, независимо от конкретного вида распределений на X и $\mathcal{A}$, будет выполняться неравенство $f(n) \leq T(n)$.

Пример 18.1. Рассмотрим произвольную рандомизированную сортировку массивов фиксированной длины n , которую, по крайней мере теоретически, можно задать как конечное множество детерминированных алгоритмов сортировки массивов длины n с приписанными этим детерминированным алгоритмам вероятностями, в сумме дающими единицу. (Предполагаем, что это вероятностное пространство алгоритмов не зависит от конкретного входа, т.е. конкретного массива длины n .) Тогда в соответствии с принципом Яо слож-

¹ См. [81, разд. 2.2.2], [88].

ность этой рандомизированной сортировки не может быть меньше чем $\log_2 n!$, так как при равномерном распределении вероятностей на Π_n для сложности в среднем детерминированных алгоритмов сортировки мы имеем в силу предложения 17.1 нижнюю границу $\log_2 n!$ при любом n .¹ Это говорит о том, что рандомизированная быстрая сортировка (пример 8.2) оптимальна по порядку сложности в классе рандомизированных сортировок. (Каждый из рассмотренных в примере 8.2 сценариев задает некоторый детерминированный алгоритм.)

Задачи

105. Для обсуждавшихся в задаче 12 алгоритмов сортировки вагонов функция $3n - 1$ является нижней границей их сложности, откуда следует, что тот алгоритм, который требуется указать в задаче 12, является оптимальным в рассматриваемом классе.

106. Какие из следующих функций являются нижними границами временной сложности алгоритмов выбора наименьшего из элементов данного массива длины n с помощью операции сравнения:

а) n ; б) $n - 1$; в) $n - 2$; г) $\log_2 n$; д) $\log_{3/2} n$?

107. Для сложности класса обсуждавшихся в задаче 26 алгоритмов поиска двери в стене функция n является асимптотической нижней границей, откуда следует, что тот алгоритм, который требуется указать в задаче 26, является оптимальным по порядку сложности в рассматриваемом классе.

108. (Продолжение предыдущей задачи.) Для поиска двери можно указать класс $\mathcal{A}$ алгоритмов, каждый из которых определяется некоторым целым $k \geq 2$: путник делает k шагов вправо, и если дверь не найдена, то возвращается назад и делает k шагов влево, и опять возвращается назад, если дверь не найдена; затем делает k^2 шагов вправо, возвращаясь назад в случае неуспеха, затем k^2 шагов влево и т.д. Таким образом, $\mathcal{A} = \{A_2, A_3, \dots\}$, и для сложности по числу шагов имеем $T_{A_k}(n) = 3n$, если $k = n$, и $T_{A_k}(n) > 3n$, если $k \neq n$. Как следствие, в классе $\mathcal{A}$ нет оптимального алгоритма, но каждый алгоритм из этого класса является оптимальным в нем по порядку сложности.

Указание. Пусть m таково, что $k^{m-1} < n \leq k^m$. Возможно, что потребуется число шагов, равное $2(k + k^2 + \dots + k^m) + 2(k + k^2 + \dots + k^{m-1}) + n$.

109. Нарисовать дерево быстрой сортировки для случая $n = 3$, считая, что первый элемент всегда выбирается в качестве разбивающего.

¹ Дополнительные примеры применения принципа Яо можно найти в [81, гл. 2].

110. Согласно теореме 15.1 в произвольной перестановке чисел $1, \dots, n^2 + 1$ ($n \geq 1$) можно так выбрать, не меняя порядка следования, $n + 1$ элемент, что полученная конечная последовательность этих элементов является либо возрастающей, либо убывающей. Показать, что при замене $n^2 + 1$ на n^2 это утверждение становится неверным.

Указание. Рассмотреть перестановку чисел, записанную в виде последовательности блоков $B_n, B_{n-1}, \dots, B_1$ по n чисел в каждом:

$$B_k = \{(k-1)n + 1, (k-1)n + 2, \dots, (k-1)n + n\}$$

(последний элемент блока B_k равен, очевидно, kn), $k = n, n-1, \dots, 1$.

111. Какой максимальной длины может быть последовательность различных вещественных чисел, не содержащая ни возрастающей ни убывающей последовательности из k элементов? Для найденного значения показать на примерах $k = 2$, $k = 3$ возможность отсутствия подпоследовательностей требуемого вида.

112. Предложить имеющий по числу сравнений сложность $O(n^2)$ алгоритм выбора самой длинной возрастающей подпоследовательности последовательности $a_1, \dots, a_n$ попарно различных вещественных чисел.

Указание. Шаг за шагом строим подпоследовательности $S_1, S_2, \dots, S_n$, где S_k — одна из наиболее длинных возрастающих подпоследовательностей, последним элементом которой служит a_k , $k = 1, \dots, n$. Вместе с S_k имеет смысл хранить и число элементов этой подпоследовательности.¹

Можно показать, что такой алгоритм имеет сложность $\Theta(n^2)$. Известны алгоритмы со сложностью $O(n \log n)$.²

Аналогичным образом можно организовать алгоритм выбора самой длинной убывающей подпоследовательности. При $n = k^2 + 1$ по крайней мере одна из подпоследовательностей — возрастающая или убывающая — будет иметь согласно теореме Секереша—Эрдеша не меньшую, чем $k + 1$ длину.

113. Дать другое доказательство предложения 14.2. Рассмотреть последовательности из нулей и единиц, соответствующие результатам всех проводимых сравнений в процессе применения сортировки к массивам длины n (каждому массиву сопоставляется своя последовательность).

Указание. Если некоторый алгоритм сортировки требует не более $h(n)$ сравнений, то длина каждой из последовательностей не превосходит $h(n)$, и общее число последовательностей не превосходит $2^{h(n)}$.

114. Аналогично предыдущей задаче, дать другое доказательство предложения 14.4.

¹ Идея этого решения сообщена автору А. А. Разборовым.

² См. https://en.wikipedia.org/wiki/Longest_increasing_subsequence.

115. Как уже отмечалось, n является нижней границей сложности как по числу аддитивных операций, так и по числу умножений для класса алгоритмов, вычисляющих значение полинома $a_n x^n + \dots + a_1 x + a_0$ с помощью операций сложения, вычитания и умножения (при рассмотрении числа n как размера входа $x, a_0, a_1, \dots, a_n$). Означает ли это, что при вычислении любого фиксированного полинома $p(x)$ степени n при заданном значении переменной x потребуется не менее n аддитивных операций и n умножений? Указать такие нижние границы сложности (по числу аддитивных операций и по числу операций умножения) алгоритмов вычисления полинома $\sum_{k=0}^n \binom{n}{k} x^k$, которые растут при $n \rightarrow \infty$ существенно медленнее, чем n .

116. Имеется плитка шоколада размера $k \times l$ клеток, которую требуется разломать на отдельные клетки. Каждый разлом применяется к одному куску плитки и производится по прямой линии. Затраты каждого алгоритма решения этой задачи применительно к конкретной плитке измеряются числом потребовавшихся разломов. Указать оптимальный алгоритм решения этой задачи и выписать его сложность. Числа k, l считаются компонентами размера входа алгоритмов.

117. В наборе $a = (a_1, \dots, a_n)$, состоящем из чисел 1 и -1 , за один шаг можно изменять знаки всех элементов за исключением знака какого-то одного явно указываемого элемента.

а) При каких n набор $a_1 = \dots = a_n = -1$ можно за несколько шагов преобразовать в набор $a_1 = \dots = a_n = 1$?

б) Если n таково, что это преобразование возможно, то чему равно наименьшее число шагов, за которое оно может быть выполнено?

в) Какова эта последовательность шагов?

Частичный ответ. а) При любом четном n можно предъявить последовательность шагов; один шаг при любом нечетном n не изменяет четности или нечетности числа отрицательных элементов набора; б) наименьшее число равно n — два последовательных шага либо не изменяют число отрицательных элементов, либо уменьшают его на 2.

118. Если условно изобразить элементы массива длины n точками на плоскости, соединяя отрезками те из них, которые непосредственно сравнивались в процессе некоторого алгоритма поиска, — скажем, поиска наименьшего элемента, одновременного поиска наибольшего и наименьшего элементов, поиска k -го по величине элемента (в частности, поиска медианы, т. е. $\left\lceil \frac{n}{2} \right\rceil$ -го элемента), — то получающийся граф должен быть связным, иначе мы бы ничего не могли сказать о том, как соотносятся между собой элементы из разных компонент.

Показать, что для всех алгоритмов поиска, сопоставляющих указанным (неявным) образом связные графы конкретным массивам, $n - 1$ является нижней границей сложности по числу сравнений (как в худшем случае, так и в среднем); в частности, это верно для алгоритмов поиска медианы.

Указание. Дерево с n вершинами имеет $n - 1$ ребро.

119. Рассмотрим класс алгоритмов поиска k первых (меньших) по величине элементов массива без установления порядка между найденными элементами и класс алгоритмов поиска k -го по величине элемента. Как обычно, имеется в виду поиск с помощью сравнений. Доказать, что любая нижняя граница сложности по числу сравнений алгоритмов первого класса является нижней границей и для второго класса.

120. Дать доказательство предложения 14.1, построенное по той же схеме, что и доказательство предложения 15.1.

Указание. Рассмотреть тройки множеств (A, B, C) , включая в A на каждом этапе алгоритма все те элементы, которые еще не сравнивались, в B — все те, которые участвовали в сравнениях и всегда оказывались меньшими, в C — все те, которые участвовали в сравнениях и в некоторых оказывались большими.

121. Доказать содержащееся в доказательстве предложения 15.1 утверждение, что после первого сравнения постоянно будет выполнено $b \geq 1$, $c \geq 1$.

122. Может ли сортировка со сложностью в худшем случае $\Omega(n \log n)$ по числу перемещений элементов быть оптимальной по порядку сложности по числу сравнений?

123. Пусть S — некоторый алгоритм сортировки массивов, и n обозначает число элементов входного массива. Пусть $T_S(n)$, $\bar{T}_S(n)$ — сложности алгоритма S как число сравнений в худшем случае и в среднем. Возможно ли, что

$$\begin{aligned} \text{а) } \bar{T}_S(n) &= \frac{3}{2}T_S(n) + O(n), & \text{б) } \bar{T}_S(n) &= 2T_S(n) + O(n^2), \\ \text{в) } \bar{T}_S(n) &= 3T_S(n) + O(n \log n), & \text{г) } \bar{T}_S(n) &= \frac{3}{2}T_S(n) + O(\log n)? \end{aligned}$$

124. В классе алгоритмов вычисления a^n с помощью умножений при рассмотрении n в качестве размера входа существует оптимальный алгоритм. То же самое — при рассмотрении $m = \lambda(n)$ в качестве размера входа.

125. Бинарный алгоритм возведения в степень n не является оптимальным по числу умножений и при использовании $m = \lambda(n)$ в качестве размера входа.

Указание. Рассмотреть алгоритм, который для всех $n \neq 15$ работает как бинарный алгоритм, а при $n = 15$ — несколько быстрее (см. задачу 28), и по-

казать, что такой алгоритм при $m = 4$ имеет сложность меньшую, чем бинарный.

126. Какие из следующих функций являются нижними границами временной сложности алгоритмов вычисления a^n с помощью операций умножения:

а) $n - 1$; б) $\log_2 n$; в) $2 \log_2 n$; г) $\frac{n}{2}$; д) $\log_2 n - 1$; е) $\log_2 n + 1$?

127. (Продолжение задачи 125.) Является ли оптимальным по порядку сложности бинарный алгоритм возведения в степень n при использовании $m = \lambda(n)$ в качестве размера входа?

128. Рассмотренный в задаче 95 алгоритм поиска фальшивой монеты, требующий в худшем случае не более $\lceil \log_3 n \rceil$ взвешиваний, является оптимальным для худшего случая.

129. Нижней границей сложности любого алгоритма деления отрезка на n равных частей с помощью циркуля и линейки (см. задачу 27) при рассмотрении числа n в качестве размера входа является, в частности, $\frac{n-1}{2}$.

130. (Продолжение задачи 27 и предыдущей задачи.) Обратимся к построению с помощью циркуля и линейки отрезка, длина которого равна $1/n$ от длины исходного отрезка, считая данное натуральное число n размером входа. Является ли $\log n$ асимптотической нижней границей для алгоритмов решения этой задачи?

Указание. Для этой задачи можно предложить алгоритм решения, сложность которого растет медленнее, чем $\log n$. Зафиксируем некоторый отрезок e как единичный. Достаточно уметь быстро построить отрезок длины pe , — мы будем говорить «построить число p ». Далее с помощью теоремы Фалеса можно будет решить основную задачу.

Используются три факта:

1) как сумма, так и произведение двух отрезков могут быть построены со сложностью $O(1)$ (построение произведения основывается на теореме о длине перпендикуляра, опущенного в прямоугольном треугольнике из вершины прямого угла на гипотенузу, причем эта теорема применяется дважды);

2) каждое натуральное число n может быть записано в факториальной системе счисления:

$$a_1 1! + a_2 2! + \dots + a_{\lceil \Gamma^{-1}(n) \rceil} \lceil \Gamma^{-1}(n) \rceil!, \quad a_i < i;$$

3) $\Gamma^{-1}(n) = o(\log n)$, где Γ^{-1} — функция, обратная к Γ -функции (сама Γ -функция при $n \rightarrow \infty$ растет быстрее экспоненциальной функции, откуда следует соотношение $\Gamma^{-1}(n) = o(\log n)$).

Это ведет к алгоритму, сложность которого есть $o(\log n)$.¹

¹ См. [6].

131. Отсутствие указания основания логарифма в (16.1) является корректным.

132. Функция $\log_2(n+1)$ является нижней границей сложности в среднем алгоритмов поиска места элемента в упорядоченном массиве длины n с помощью сравнений (считаем, что в диапазоне от 1 до $n+1$ место может быть любым с одинаковой вероятностью $\frac{1}{n+1}$).

Указание. Для доказательства воспользоваться леммой 17.1.

133. (Продолжение предыдущей задачи.) Алгоритм бинарного поиска является оптимальным в среднем по порядку сложности в классе алгоритмов поиска места элемента с помощью сравнений.

134. Алгоритм, описанный в задаче 48, является оптимальным как в худшем случае, так и в среднем.

135. Пусть $\bar{T}_{QS}(n)$ и $\bar{T}_{opt}(n)$ — сложности в среднем быстрой сортировки и оптимальной в среднем сортировки. Верно ли, что $\bar{T}_{QS}(n) \sim \bar{T}_{opt}(n)$? Если нет, то можно ли подобрать константу c такую, что $\bar{T}_Q(n) \sim c\bar{T}_{opt}(n)$?

136. Утверждение теоремы 17.1 перестает быть верным, если его изменить, удалив условие, что $\tau < \infty$ всюду на рассматриваемом вероятностном пространстве: при выполнении всех остальных условий может оказаться, что $\tau = \infty$ при том, что $\sum_{k=1}^{\infty} h_k$ имеет конечное значение¹.

Указание. Рассмотреть постоянные величины $\xi_k = \frac{1}{k(k+1)}$, $k = 1, 2, \dots$, взяв $h_k = \xi_k$ для всех k ; при любом $q \geq 1$ получается противоречие с измененным утверждением.

137. Функция $\log_2(n+1)$ является нижней границей сложности рандомизированных алгоритмов поиска места элемента в упорядоченном массиве длины n с помощью сравнений. (Предполагается, что каждый из рассматриваемых рандомизированных алгоритмов может быть задан как конечное множество детерминированных алгоритмов поиска места элемента в упорядоченном массиве длины n , с приписанными этим детерминированным алгоритмам вероятностями, в сумме дающими единицу; при этом такое вероятностное пространство алгоритмов не должно зависеть от конкретного входа размера n .)

¹ Эта задача сообщена автору А. В. Бернштейном.

Глава 5

Битовая сложность

§ 19. Битовые операции

Каждый алгоритм строится на основе некоторого фиксированного набора базовых операций, и, согласно определениям из § 1, сложность по числу операций алгоритма рассматривается при допущении, что затратность операций не зависит от размеров операндов. С позиций компьютерной арифметики это допущение вполне приемлемо, если каждый из операндов умещается в одном слове памяти. Но оно перестает быть приемлемым, если разрешаются операнды любой длины и вычисления проводятся в рамках арифметики многократной точности (арифметики длинных чисел). Здесь анализ сложности должен исходить из других принципов.

Приемлемой является, например, следующая модель как системы допущений, принимаемых нами при анализе алгоритмов. Число представляется набором бит, являющихся содержимым его двоичных разрядов (знак числа — тоже бит, например, обычно знаки $+$ и $-$ изображаются соответственно нулем и единицей), и все арифметические операции сводятся, в конечном счете, к битовым операциям. В качестве битовых операций могут выступать булевы операции $\vee$, $\wedge$, $\neg$ (другая нотация: *or*, *and*, *not*) при интерпретации $1 = \text{«истина»}$, $0 = \text{«ложь»}$ встречающихся бит; эти операции считаются равнозатратными. Все биты, участвующие в записи числа, считаются равнодоступными.

Можно смотреть на все это так, что ячейки памяти машины содержат по одному биту каждая, а в остальном действует принцип РАМ — ячеек бесконечно много и они в любой момент равнодоступны. Это — *битовая модель вычислений* (сокращение БМ может расшифровываться как «битовая модель» и как «битовая машина»).

Для анализа сложности с использованием БМ нет необходимости переписывать алгоритмы, детализируя их до уровня битовых операций. Вместо этого можно рассматривать привлекаемые алгоритмом

базовые арифметические операции как основанные на битовых вычислениях процедуры, полагая временные и пространственные затраты алгоритма равными числу затрачиваемых битовых операций и, соответственно, требующихся дополнительных бит памяти. В качестве размера входа часто рассматривается либо суммарная битовая длина всего входа (всех входных данных), либо максимум этих длин. В некоторых случаях вместо битовых длин целых чисел берутся сами эти числа. Возможно и использование многокомпонентного размера входа.

Определение 19.1. Пусть выбран какой-то размер входа. Рассмотрение каждой из базовых операций алгоритма A как процедуры, основанной на битовых вычислениях, и измерение затрат на выполнение A для каждого конкретного входа в битовых операциях или, соответственно, в хранимых дополнительных битах, приводит к *битовой сложности* (временной или пространственной) алгоритма A .

Для нахождения битовой сложности какого-либо алгоритма, построенного на основных арифметических операциях над целыми числами, надо знать битовые сложности самих основных арифметических операций. Мы исследуем школьные алгоритмы сложения, вычитания, умножения и деления «столбиком» неотрицательных целых чисел в двоичной системе.

В процессе сложения «столбиком» мы шаг за шагом вычисляем двоичные цифры суммы, продвигаясь от младших разрядов к старшим. При этом в старшие разряды каждый раз переносится 0 или 1. Таким образом, на каждом шаге мы работаем с тремя битами (на начальном шаге, когда рассматриваются самые младшие разряды, бит переноса полагаем равным нулю). Для сложения многозначных чисел нам нужны две битовые процедуры трех аргументов (два аргумента — это цифры одноименных разрядов двух данных чисел, третий — это бит переноса из предшествующих разрядов), одна из процедур дает цифру соответствующего разряда суммы, вторая — новый бит переноса в старшие разряды. Мы не станем рассматривать этот вопрос более подробно, потому что и без деталей ясно, что затраты по построению каждого разряда суммы ограничены сверху некоторой константой.

Алгоритм сложения «столбиком» будем обозначать буквами Add , от английского слова *addition* — сложение. Обозначим через a и b операнды алгоритма ($a, b \in \mathbb{N}^+$), через m_1, m_2 — их битовые длины: $m_1 = \lambda(a)$, $m_2 = \lambda(b)$.

Лемма 19.1. Количество $C_{\text{Add}}^T(a, b)$ битовых операций, затрачиваемых при сложении a и b «столбиком», удовлетворяет неравенствам

$$\min\{m_1, m_2\} \leq C_{\text{Add}}^T(a, b) \leq c(\max\{m_1, m_2\} + 1), \quad (19.1)$$

где c — некоторая положительная константа.

Доказательство. Принимая во внимание замечание относительно ограниченности затрат на построение каждого разряда суммы, а также то, что битовая длина суммы $a + b$ не может превышать $\max\{m_1, m_2\} + 1$, получаем оценку $C_{\text{Add}}^T(a, b) \leq c(\max\{m_1, m_2\} + 1)$. Пусть $m' = \min\{m_1, m_2\}$. Очевидно, что m' младших цифр суммы $a + b$ получаются выполнением битовых операций над соответствующими цифрами обоих слагаемых и битами переноса (в то время как часть цифр старших разрядов большего из слагаемых может быть просто перенесена в старшие разряды суммы). Это дает нам неравенство $\min\{m_1, m_2\} \leq C_{\text{Add}}^T(a, b)$. $\square$

В дальнейшем будем полагать

$$m = \max\{m_1, m_2\}. \quad (19.2)$$

Предложение 19.1. Пусть $T_{\text{Add}}^*(m)$ — сложность по числу битовых операций алгоритма сложения «столбиком» при использовании t в качестве размера входа. Тогда $T_{\text{Add}}^*(m) = \Theta(m)$.

Доказательство. При фиксированном m мы можем выбрать a и b так, что $m_1 = m_2 = m$, тогда $\min\{m_1, m_2\} = \max\{m_1, m_2\} = m$; далее применяем лемму 19.1. $\square$

Алгоритм сложения «столбиком» позволяет получать сумму a и b на месте максимального из этих двух чисел, в этом случае для битовой пространственной сложности имеем $S_{\text{Add}}^*(m) = O(1)$. Если для суммы чисел используется дополнительное место (чтобы не изменять данные слагаемые), то, соответственно, $S_{\text{Add}}^*(m) = m + O(1)$.

Утверждения леммы 19.1 и предложения 19.1 верны, как легко убедиться, и для операции вычитания (при вычитании «столбиком» в старших разрядах занимается 0 или 1; битовая длина разности не превосходит m).

Формула $T_{\text{Add}}^*(m) = \Theta(m)$ говорит о том, что алгоритм сложения «столбиком» при рассмотрении m в качестве размера входа является оптимальным по порядку битовой сложности: мы не можем игнорировать содержимое разрядов, и поэтому m является нижней границей битовой сложности алгоритмов сложения. Для алгоритмов умножения и деления «столбиком» в дальнейшем будет показано, что их

сложность имеет порядок m^2 , и в этой ситуации уже ниоткуда не следует, что не может существовать алгоритмов с лучшей сложностью, и такие алгоритмы действительно существуют (мы об этом еще будем говорить). Поэтому умножение и деление «столбиком» мы будем называть, как это делается в некоторых книгах, *наивным* умножением и делением соответственно, используя обозначения NM и ND (от английских слов naïve multiplication/division — наивное умножение/деление). Сложение «столбиком» будем просто называть сложением.

Пример 19.1. Допустим, что для умножения двух целых положительных чисел a и b мы используем сложения:

$$a + a, \quad (a + a) + a, \quad \dots, \quad \underbrace{(a + \dots + a)}_{b-1} + a. \quad (19.3)$$

Исследуем битовую сложность такого «сверхнаивного» умножения, считая m размером входа. В силу леммы 19.1 при $m_1 = m_2 = m$ на каждое сложение уйдет не менее m битовых операций. Учитывая, что $b > 2^{m-1}$, получаем для исследуемой сложности оценку $\Omega(2^m m)$. С другой стороны, $ab < 2^{2m}$, поэтому результат каждого шага в (19.3) имеет не превосходящую $2m$ битовую длину. Это означает, что число битовых операций на каждом из шагов не превосходит $c \cdot 2m$, где c — константа. Отсюда получаем оценку $O(2^m m)$, и в итоге — оценку $\Theta(2^m m)$.

Наряду с размером входа (19.2) часто рассматривают две компоненты m_1, m_2 размера входа.

Предложение 19.2. Если рассмотреть две компоненты $m_1 = \lambda(a)$, $m_2 = \lambda(b)$ размера входа, то битовая сложность $T_{\text{Add}}^{**}(m_1, m_2)$ алгоритма сложения будет допускать оценку $\Theta(\max\{m_1, m_2\})$.

Доказательство. Пусть, например, $\max\{m_1, m_2\} = m_1$. Тогда оценка $T_{\text{Add}}^{**}(m_1, m_2) = \Omega(\max\{m_1, m_2\})$ подтверждается наличием входа $a = 2^{m_1} - 1$, $b = 2^{m_2} - 1$. Оценка же $T_{\text{Add}}^{**}(m_1, m_2) = O(\max\{m_1, m_2\})$ следует из леммы 19.1. $\square$

Битовая сложность рассматривается не только в связи с арифметическими задачами. Например, булевы матрицы смежности, которые служат одним из стандартных средств представления графов без кратных ребер, являются битовыми матрицами, и целому ряду алгоритмов решения задач на графах естественным образом сопоставляется битовая сложность. Об этом будет идти разговор в § 23. Но прежде,

в § 20 и 21, мы займемся битовой сложностью наивного умножения и деления.

Еще одно замечание в заключение этого параграфа. Битовый анализ может учесть мельчайшие затраты, связанные с выполнением алгоритма. Вопрос в том, нужен ли настолько детализированный подход, коль скоро компьютеры выполняют операции над основными структурами данных на уровне слов, длина же слова — это, как правило, 64 бита. В связи с этим иногда вместо битовой рассматривается так называемая *словесная сложность*¹. Но принципиальной разницы между битовой и словесной сложностью нет. Можно сказать так, что битовая сложность предполагает использование системы счисления с основанием 2, а словесная — с основанием 64. Анализ словесной сложности не является, в сравнении с анализом битовой сложности, ни более легким, ни более информативным.

§ 20. Наивная арифметика: умножение

Предложение 20.1. Пусть $T_{\text{NM}}^*(m)$ — временная битовая сложность наивного умножения при использовании m в качестве размера входа, а $T_{\text{NM}}^{**}(m_1, m_2)$ — временная битовая сложность этого же алгоритма при использовании m_1, m_2 в качестве двух компонент размера входа. Тогда

$$T_{\text{NM}}^*(m) = \Theta(m^2), \quad T_{\text{NM}}^{**}(m_1, m_2) = \Theta(m_1 m_2). \quad (20.1)$$

Доказательство. Затраты наивного умножения a на b связаны с суммированием m_2 чисел $n_1, n_2, \dots, n_{m_2}$, где каждое n_i равно 0 или $a \cdot 2^{i-1}$ в зависимости от соответствующей цифры в двоичной записи b . Несложной индукцией доказывается, что для $s_i = n_1 + n_2 + \dots + n_i$, $i = 1, 2, \dots, m_2$, выполнено $\lambda(s_i) \leq m_1 + i$. Если $n_{i+1} \neq 0$, то последние i цифр числа n_{i+1} суть нули, и при вычислении $s_{i+1} = s_i + n_{i+1}$ мы не притрагиваемся к этим цифрам, а преобразуем s_i в s_{i+1} сложением двух чисел, первое из которых образовано всеми разрядами числа s_i , кроме i последних (согласно сказанному, битовая длина этого числа не превосходит m_1), второе же слагаемое есть a , и его битовая длина равна m_1 . Число битовых операций, требующихся для преобразования s_i в s_{i+1} , не превосходит $c(m_1 + 1)$ для некоторой положительной константы c . В том случае, когда все цифры числа b равны 1, это число при любом i не меньше, чем m_1 , по лемме 19.1.

¹ См., например, книгу [71]. В литературе на английском языке, в частности в книге [71], используется термин «word complexity».

Поэтому битовые затраты, связанные с наивным умножением a на b , не превосходят $c(m_1 + 1)m_2$, а при $b = 2^{m_2} - 1$ (двоичная запись этого b состоит только из единиц) затраты не меньше, чем $m_1 m_2$. Это дает оценку $T_{\text{NM}}^{**}(m_1, m_2) = \Theta(m_1 m_2)$.

Рассмотрим теперь m как размер входа. Так как $m_1 \leq m$ и $m_2 \leq m$, то затраты не превосходят $c(m + 1)m$, это дает оценку $T_{\text{NM}}^*(m) = O(m^2)$. С другой стороны, если $m_1 = m_2 = m$ и $a = b = 2^m - 1$, то затраты не могут быть меньше, чем m^2 , и, следовательно, $T_{\text{NM}}^*(m) = \Omega(m^2)$. Таким образом, $T_{\text{NM}}^*(m) = \Theta(m^2)$. $\square$

Для пространственной битовой сложности наивного умножения мы имеем, очевидно, оценки

$$S_{\text{NM}}^*(m) = 2m + O(1), \quad S_{\text{NM}}^{**}(m_1, m_2) = m_1 + m_2 + O(1).$$

Для перехода в верхних оценках сложности от $\lambda(k)$ к $\log_2 k$, $k \in \mathbb{N}^*$, часто оказывается удобным простое неравенство $\lceil \log_2(k + 1) \rceil \leq \leq 2 \log_2 k$, справедливое для всех $k \geq 2$:

$$\lceil \log_2(k + 1) \rceil = \lfloor \log_2 k \rfloor + 1 \leq \log_2 k + 1 \leq 2 \log_2 k.$$

Например,

$$m_1 \leq 2 \log_2 a, \quad m_2 \leq 2 \log_2 b$$

и

$$m_1 m_2 \leq 4 \log_2 a \log_2 b \quad (20.2)$$

для всех $a, b \geq 2$. Имеем для достаточно больших m_1, m_2 и некоторой положительной константы c

$$C_{\text{NM}}^T(a, b) \leq T_{\text{NM}}^{**}(m_1, m_2) \leq c m_1 m_2 \leq 4c \log_2 a \log_2 b. \quad (20.3)$$

Мы можем написать $C_{\text{NM}}^T(a, b) = O(\log a \log b)$, считая, что выражение под знаком O рассматривается как функция двух переменных a, b , при этом $a, b \rightarrow \infty$; считаем также, что логарифмы имеют общее основание (подобные предположения будут подразумеваться и в дальнейшем). Остается заметить, что битовая временная сложность при использовании самих a, b в качестве компонент размера входа совпадает с битовыми временными затратами. Это позволяет получить из оценки (20.2) оценку $T_{\text{NM}}(a, b) = O(\log a \log b)$. Итак:

При использовании самих положительных целых a, b в качестве компонент размера входа сложность наивного умножения a на b по числу битовых операций допускает оценку $O(\log a \log b)$.

Выше наивное умножение (умножение «столбиком») понималось так, что если какая-то цифра числа b равна нулю, то, несмотря на

это, построение соответствующих n_i и s_i требует не менее m_1 битовых операций. При таком взгляде сложность будет величиной $\Theta(\log a \log b)$. Но можно считать, что в рассматриваемом случае битовые затраты на получение s_i не зависят от a и ограничены константой. Тогда оценка $\Omega(\log a \log b)$ места не имеет: если $a = 2^{k_1}$, $b = 2^{k_2}$, где k_1, k_2 — положительные целые, то битовые затраты будут ограничены линейной функцией от k_1, k_2 .

Пример 20.1. Покажем, что битовая временная сложность алгоритма вычисления $n!$ с помощью пошаговых наивных умножений

$$2 \cdot 3, (2 \cdot 3) \cdot 4, \dots, (2 \cdot 3 \dots (n-1)) \cdot n$$

при использовании n в качестве размера входа допускает оценку $O((n \log n)^2)$. В силу (20.3) рассматриваемая сложность не превосходит $cf(n)$, где c — некоторая положительная константа, а значение $f(n)$ равно

$$\log_2 2 \log_2 3 + \log_2(2 \cdot 3) \log_2 4 + \dots + \log_2(2 \cdot 3 \dots (n-1)) \log_2 n.$$

Имеем

$$f(n) \leq \log_2(2 \cdot 3 \dots (n-1))(\log_2 3 + \log_2 4 + \dots + \log_2 n) \leq \log_2^2(n!).$$

Из $\log_2(n!) = O(n \log n)$ получаем $\log_2^2(n!) = O((n \log n)^2)$, и $f(n) = O((n \log n)^2)$.

Пример 20.2. Мы упоминали о том, что для алгоритма, входом которого является несколько целых чисел, можно в некоторых случаях определять размер входа как суммарную битовую длину всех этих чисел. Предположим, что имеется несколько целых чисел $a_1, a_2, \dots, a_n$, каждое из которых ≥ 2 , и с помощью пошаговых наивных умножений

$$a_1 a_2, (a_1 a_2) a_3, \dots, (a_1 a_2 \dots a_{n-1}) a_n \quad (20.4)$$

вычисляется произведение $A = a_1 a_2 \dots a_n$. Пусть суммарная битовая длина чисел равна M , и число M рассматривается как размер входа алгоритма (20.4). Покажем, что тогда битовая сложность этого алгоритма допускает оценку $O(M^2)$, — примечательно, что число n , т.е. общее количество сомножителей, в этой оценке не появляется.

В силу предложения 20.1 и неравенства (20.3) битовые затраты на вычисление произведения $a_1 a_2 \dots a_n$ не превосходят $cF(a_1, a_2, \dots, a_n)$, где c — некоторая положительная константа, а значение $F(a_1, a_2, \dots, a_n)$ равно

$$\log_2 a_1 \log_2 a_2 + \log_2(a_1 a_2) \log_2 a_3 + \dots + \log_2(a_1 a_2 \dots a_{n-1}) \log_2 a_n$$

(использовано предположение, что $a_1, a_2, \dots, a_n \geq 2$). Мы легко получаем

$$F(a_1, a_2, \dots, a_n) \leq \log_2(a_1 \dots a_{n-1})(\log_2 a_2 + \dots + \log_2 a_n)$$

и

$$F(a_1, a_2, \dots, a_n) \leq (\log_2 a_1 + \log_2 a_2 + \dots + \log_2 a_n)^2.$$

Последнее неравенство позволяет перейти к рассмотрению M в качестве размера входа, так как, очевидно,

$$F(a_1, a_2, \dots, a_n) \leq (\lceil \log_2(a_1+1) \rceil + \lceil \log_2(a_2+1) \rceil + \dots + \lceil \log_2(a_n+1) \rceil)^2 = M^2,$$

что дает нам требуемое.

Доказанное утверждение справедливо и в том случае, когда среди $a_1, a_2, \dots, a_n$ имеются единицы. Очевидно, что умножение на 1 требует не более M битовых операций. Если среди $a_1, a_2, \dots, a_n$ имеются k единиц, то из $M \geq k$ следует $(M - k)^2 + Mk \leq M^2$.

§ 21. Наивная арифметика: деление с остатком

Обратимся к наивному делению с остатком. Здесь мы считаем, что $m = m_1 \geq m_2$.

Предложение 21.1. Пусть $T_{\text{ND}}^*(m)$ — сложность по числу битовых операций наивного деления при использовании m в качестве размера входа. Пусть $T_{\text{ND}}^{**}(m_1, m_2)$ — сложность по числу битовых операций этого же алгоритма при использовании m_1, m_2 в качестве двух компонент размера входа. Тогда

$$T_{\text{ND}}^*(m) = \Theta(m^2), \quad T_{\text{ND}}^{**}(m_1, m_2) = \Theta((m_1 - m_2 + 1)m_2). \quad (21.1)$$

Доказательство. Наивное деление a на b сводится к ряду вычитаний числа b , и в каждом вычитании битовая длина уменьшаемого либо равна, либо на единицу превосходит m_2 . Битовые затраты на каждое такое вычитание не могут превзойти, как мы знаем, $c(m_2 + 1)$. Количество всех таких вычитаний не превосходит $m_1 - m_2 + 1$, откуда $T_{\text{ND}}^{**}(m_1, m_2) = O((m_1 - m_2 + 1)(m_2 + 1))$ и, после упрощения, $T_{\text{ND}}^{**}(m_1, m_2) = O((m_1 - m_2 + 1)m_2)$. С другой стороны, если, например, $a = 2^{m_1} - 1$, $b = 2^{m_2 - 1}$, то число вычитаний равно $m_1 - m_2 + 1$, и, как следствие, битовые затраты на наивное деление будут не меньше, чем $(m_1 - m_2 + 1)m_2$. Поэтому $T_{\text{ND}}^{**}(m_1, m_2) = \Theta((m_1 - m_2 + 1)m_2)$.

Оценка $T_{\text{ND}}^*(m) = O(m^2)$ следует теперь из неравенства $m^2 \geq m_1 m_2 \geq (m_1 - m_2 + 1)m_2$. Если взять $a = 2^m - 1$, $b = 2^{\lfloor m/2 \rfloor}$, то битовые затраты

на наивное деление будут не меньше чем $\left(m - \left\lfloor \frac{m}{2} \right\rfloor + 1\right) \left(\left\lfloor \frac{m}{2} \right\rfloor + 1\right)$, и, как следствие, $T_{\text{ND}}^*(m) = \Omega(m^2)$. Получаем $T_{\text{ND}}^*(m) = \Theta(m^2)$. $\square$

Мы не пишем $T_{\text{ND}}^{**}(m_1, m_2) = \Theta((m_1 - m_2)m_2)$ из-за возможности равенства $m_1 = m_2$ (из которого не следует, что $a = b$). Эта возможность указывает и на то, что неверно соотношение $T_{\text{ND}}^{**}(m_1, m_2) = \Theta(m_1 m_2)$.

Предложение 21.2. При использовании самих положительных целых a, b , $a \geq b > 1$, в качестве компонент размера входа, сложность наивного деления a на b допускает оценку $O(\log a \log b)$. При использовании a в качестве размера входа имеем оценку $O(\log^2 a)$.

Доказательство. Как уже говорилось, число битовых операций, требуемых наивным делением, не превосходит произведения

$$c(\lceil \log_2 a \rceil + 1) - (\lceil \log_2 b \rceil + 1) + 1)(\lceil \log_2 b \rceil + 1)$$

(c — положительная константа), которое не превосходит в свою очередь

$$c(\log_2 a - \log_2 b + 2)(\log_2 b + 1).$$

Каждое слагаемое, возникающее в результате раскрытия скобок в последнем выражении, есть $O(\log a \log b)$ при $a, b \rightarrow \infty$, $a \geq b$. Отсюда следует первая часть утверждения. Вторая часть следует из того, что $\log_2 a \log_2 b \leq \log_2^2 a$ при $a \geq b$. $\square$

Пример 21.1. Пусть n и k — положительные целые числа, $k \geq 2$, заданные в двоичной системе счисления. Покажем, что при избрании n в качестве размера входа n, k (равно как и при избрании n, k в качестве двух компонент размера этого входа) битовая сложность обычного алгоритма построения k -ичной записи числа n с помощью наивных делений с остатком допускает оценку $O(\log^2 n)$.

В самом деле, при построении k -ичной записи n шаг за шагом рассматриваются числа $q_0, q_1, \dots, q_t$, $t = \lfloor \log_k n \rfloor + 1$, где $q_0 = n$, $q_i = \left\lfloor \frac{q_{i-1}}{k} \right\rfloor$, $i = 1, 2, \dots, t$. Очевидно, что $q_i \leq n$, в силу чего общие затраты всех шагов для данных n и k допускают оценку $O(\log n \log k \log_k n)$. Очевидно, $\log k \log_k n = \log n$ (например, $\log_2 k \log_k n = \log_2 n$), поэтому оценка для затрат переписывается в виде $O(\log^2 n)$; при указанных выше размерах входа эта оценка является и оценкой сложности.

Вновь вернемся к обозначению m для битовой длины максимального из двух данных целых положительных чисел, и пусть m рассматривается как размер входа a, b алгоритмов умножения и деления

с остатком. Для сложностей наивного умножения и деления мы получили оценку $\Theta(m^2)$. Вместе с этим, для умножения и деления известны алгоритмы, сложности которых при $m \rightarrow \infty$ растут существенно медленнее, чем m^2 : первый алгоритм такого рода для умножения был предложен в 1962 г. А. А. Карацубой (верхняя оценка $O(m^{\log_2 3})$). Алгоритм, предложенный в 2007 г. М. Фюрером, имеет битовую сложность $O(m \log m f(m))$, где $f(m)$ — некоторая функция, растущая медленнее, чем $\log \log m$ и, более того, медленнее, чем любой повторный логарифм. Существуют алгоритмы деления, сложность которых допускает такие же оценки (см. задачу 208 к главе 7). Алгоритм Карацубы будет рассмотрен нами в § 27, алгоритм Фюрера в этом курсе подробно не рассматривается; дополнительно несколько слов об алгоритмах умножения будет сказано в конце § 27.

Пример 21.2. Исследуем битовую сложность алгоритма Евклида. В качестве размера входа можно рассмотреть большее число a_0 , но ни в коем случае не a_1 : если фиксировано a_1 , то, неограниченно увеличивая a_0 , мы будем неограниченно увеличивать битовые затраты, связанные с первым делением с остатком (a_0 на a_1). Поэтому при выборе a_1 в качестве размера входа битовая сложность этого алгоритма тождественно равна бесконечности — здесь битовая сложность существенно отличается от сложности по числу делений. Можно считать a_0 размером входа, а можно рассмотреть две компоненты a_0, a_1 размера входа. Если m_0, m_1 — битовые длины данных чисел a_0, a_1 , то в качестве размера входа можно рассмотреть $m = \max\{m_0, m_1\} = m_0$ или же m_0, m_1 . В настоящем примере мы будем использовать m .

Прежде всего покажем, что для алгоритма Евклида

$$a_{i-1} = q_i a_i + a_{i+1}, \quad i = 1, 2, \dots, n, \quad a_{n+1} = 0,$$

выполняется

$$a_0 \geq q_1 q_2 \dots q_n. \quad (21.2)$$

В самом деле,

$$a_0 > q_1 a_1, \quad a_1 > q_2 a_2, \quad \dots, \quad a_{n-2} > q_{n-1} a_{n-1}, \quad a_{n-1} = q_n a_n,$$

и получаем $a_0 \geq q_1 q_2 \dots q_n a_n$, откуда следует (21.2).

Для построения a_{i+1} делением a_{i-1} на a_i с остатком требуется не более $c(\lfloor \log_2 q_i \rfloor + 1)(\lfloor \log_2 a_i \rfloor + 2)$ битовых операций, где c — положительная константа. Это дает общую оценку сверху

$$c \sum_{i=1}^n (\lfloor \log_2 q_i \rfloor + 1)(\lfloor \log_2 a_i \rfloor + 2).$$

Написанная сумма не превосходит

$$c(\lfloor \log_2 a_1 \rfloor + 2) \sum_{i=1}^n (\lfloor \log_2 q_i \rfloor + 1).$$

При целом $a_1 > 1$ выполнено $\lfloor \log_2 a_1 \rfloor + 2 \leq 3 \log_2 a_1$ и, используя (21.2), получаем

$$\begin{aligned} c(\lfloor \log_2 a_1 \rfloor + 2) \sum_{i=1}^n (\lfloor \log_2 q_i \rfloor + 1) &\leq 3c(\log_2 a_1) \left(n + \sum_{i=1}^n \log_2 q_i \right) = \\ &= 3c(\log_2 a_1)(n + \log_2(q_1 q_2 \dots q_n)) \leq 3c(\log_2 a_1)(n + \log_2 a_0). \end{aligned}$$

Как мы знаем, $n \leq 2 \log_2 a_1 + 1$ (см. (9.6)); при $a_1 > 1$, очевидно, можем написать $n \leq 3 \log_2 a_1$. Это дает нам оценку сверху

$$3c(\log_2 a_1)(6 \log_2 a_1 + \log_2 a_0)$$

для числа битовых операций при $a_1 > 1$. Учитывая, что $a_0 \geq a_1$, мы получаем отсюда следующее.

Количество битовых операций, затрачиваемых при применении алгоритма Евклида к a_0, a_1 , допускает оценку

$$O(\log a_0 \log a_1), \quad (21.3)$$

как следствие — оценку $O(\log^2 a_0)$, и оценку $O(m^2)$ при $m = \lambda(a_0)$.

Приведенные оценки получены в предположении, что для построения остатков используется наивное деление, имеющее квадратичную сложность. Может ли быть так, что привлечение имеющего меньшую битовую сложность алгоритма деления с остатком приведет к существенно меньшим битовым затратам алгоритма Евклида? Ответ отрицательный: нетрудно, например, показать, что если a_0 берется в качестве размера входа, то для битовой сложности алгоритма Евклида выполняется оценка $\Omega(\log^2 a_0)$. В самом деле, в примере 10.1 было показано, что для каждого a_0 можно подобрать меньшее его a_1 такое, что для числа делений с остатком выполняется неравенство (10.9). Если обозначить это число делений через n , то будем иметь $n = \Omega(\log a_0)$, при этом

$$\lambda(a_0), \lambda(a_1), \dots, \lambda(a_n)$$

является невозрастающей конечной последовательностью натуральных чисел, и в силу предложения 9.1 никакое значение не встречается в ней более двух раз. Так как деление с остатком a_{i-1} на a_i требует не менее $\lambda(a_i)$ битовых операций, то общее число битовых операций не может быть меньше, чем

$$1 + 1 + 2 + 2 + \dots + k + k = k(k + 1)$$

при $k = \left\lfloor \frac{n+1}{2} \right\rfloor$ и $n = \Omega(\log a_0)$. Следовательно, общее число битовых операций есть $\Omega(\log^2 a_0)$.

Вместе с ранее установленной оценкой $O(\log^2 a_0)$ это дает оценку $\Theta(\log^2 a_0)$. Как следствие, имеем оценку $\Theta(m^2)$ для битовой сложности алгоритма Евклида при избрании битовой длины m числа a_0 в качестве размера входа. Итак:

Если алгоритм Евклида основывается на некотором алгоритме деления с остатком, битовые затраты которого применительно к делимому v и делителю w допускают оценку $\Omega(\log w)$, то при рассмотрении большего входного числа a_0 в качестве размера входа битовая сложность алгоритма Евклида допускает оценку $\Theta(\log^2 a_0)$. При рассмотрении $m = \lambda(a_0)$ в качестве размера входа имеем оценку $\Theta(m^2)$.

Оказывается, что полученные оценки сохраняют силу и при рассмотрении расширенного алгоритма Евклида (пример 9.1) — обстоятельство, имеющее большое значение для модулярной арифметики (§ 22).

Предложение 21.3. Пусть расширенный алгоритм Евклида основывается на некоторых алгоритмах деления с остатком и умножения, битовые затраты каждого из которых применительно к целым v и w допускают оценку $O(\log v \log w)$. Тогда, при рассмотрении большего входного числа a_0 в качестве размера входа, битовая сложность расширенного алгоритма Евклида допускает оценку $\Theta(\log^2 a_0)$, а при рассмотрении битовой длины m большего числа a_0 в качестве размера входа — оценку $\Theta(m^2)$.

Доказательство. Исходя из вычислительных формул (9.8) и принимая во внимание (9.12), (9.13), мы заключаем, что битовые затраты, дополнительные к затратам собственно алгоритма Евклида («нерасширенного») на вычисление s_n, t_n , допускают оценку $O(\log a_0 \log a_1)$ — это обосновывается так же, как оценка (21.3). Поэтому сложность добавочной части расширенного алгоритма Евклида допускает оценку $O(\log^2 a_0)$ и оценку $O(m^2)$. Остается заметить, что, например, $\Theta(m^2) + O(m^2) = \Theta(m^2)$. $\square$

§ 22. Модулярная арифметика

Многие арифметические алгоритмы основываются на вычислениях по некоторому целому модулю k , или, как говорят, на *модулярных вычислениях*; соответственно, говорят о *модулярной арифметике*. Да-

лее в этом параграфе мы считаем k фиксированным целым положительным числом.

Целые a и b называют *сравнимыми по модулю k* и пишут

$$a \equiv b \pmod{k}, \quad (22.1)$$

если $k \mid (a - b)$, т.е. если k является делителем разности $a - b$. Соотношение вида (22.1) называется *сравнением*. Изложение основ теории сравнений можно найти в любом учебнике алгебры или теории чисел¹. Мы приведем основные элементарные факты этой теории, не останавливаясь на их доказательстве:

(i) *бинарное отношение сравнимости по модулю k является отношением эквивалентности на множестве целых чисел $\mathbb{Z}$* ;

(ii) *если*

$$a_1 \equiv b_1 \pmod{k} \quad \text{и} \quad a_2 \equiv b_2 \pmod{k},$$

то

$$a_1 \pm a_2 \equiv b_1 \pm b_2 \pmod{k} \quad \text{и} \quad a_1 a_2 \equiv b_1 b_2 \pmod{k};$$

(iii) *каждое целое число a сравнимо по модулю k в точности с одним целым числом из множества $\{0, 1, \dots, k-1\}$.*

В силу (i) множество $\mathbb{Z}$ разбивается на классы эквивалентности по модулю k (кратко: классы по модулю k), и, согласно (ii), эти классы образуют кольцо, если считать, что сумма двух классов — это класс, содержащий сумму каких-либо чисел, взятых по одному из складываемых классов, а произведение — это, соответственно, класс, содержащий произведение каких-либо чисел, взятых по одному из перемножаемых классов. Нулем этого кольца является класс, содержащий число 0. Для введенного кольца используется обозначение $\mathbb{Z}_k$, его называют *кольцом вычетов по модулю k* (считаем, что вычет — это другое название класса эквивалентности по модулю k).

Любое множество $\{a_0, a_1, \dots, a_{k-1}\}$ чисел, взятых по одному из каждого класса по модулю k , называется *полной системой представителей* по модулю k .

Множество

$$\mathbb{I}_k = \{0, 1, \dots, k-1\},$$

указанное в (iii), называется *канонической* (или *начальной*) *полной системой представителей* по модулю k . Иногда используется и так называемая *симметричная* полная система представителей:

$$\mathbb{S}_k = \left\{ \left\lceil \frac{-k+1}{2} \right\rceil, \dots, -1, 0, 1, \dots, \left\lfloor \frac{k}{2} \right\rfloor \right\},$$

¹ См., например, [22, § 42, 43], [29, гл. 4, § 4, п. 2] и [41, гл. 4].

которая симметрична (при игнорировании знаков) только при нечетном k : например, $\mathbb{S}_3 = \{-1, 0, 1\}$ и $\mathbb{S}_4 = \{-1, 0, 1, 2\}$.

Пусть для изображения элементов кольца $\mathbb{Z}_k$ используется система $\mathbb{I}_k$. Операциям сложения и умножения в $\mathbb{Z}_k$ соответствуют обычные сложение и умножение в $\mathbb{Z}$ с последующей заменой полученного результата остатком от его деления на k . Если некоторому элементу кольца $\mathbb{Z}_k$ соответствует число a системы $\mathbb{I}_k$, то противоположному (обратному по сложению) элементу будет соответствовать число $k - a$, если $a \neq 0$, и число 0, если $a = 0$.

Исследуем битовую сложность операций в кольце $\mathbb{Z}_k$ при указанном способе изображения его элементов. Будем считать размером входа битовую длину m числа k и ограничимся верхними асимптотическими оценками. Для операций сложения и нахождения противоположной величины имеем, очевидно, оценку $O(m)$. Использование операции наивного умножения целых чисел приводит к оценке $O(m^2)$ для сложности умножения в $\mathbb{Z}_k$. Эта оценка, разумеется, сохраняется и при использовании более быстрого умножения и деления в $\mathbb{Z}$.

Как известно, в случае простого k кольцо $\mathbb{Z}_k$ является полем. При составном k это кольцо полем не является и, более того, содержит делители нуля: если $k = pq$, $p > 1$, $q > 1$, то произведение элементов $\mathbb{Z}_k$, изображаемых числами $p, q \in \mathbb{I}_k$, равно нулю. Допустим, что k является простым и примем для этого простого числа обозначение p , по-прежнему считая его битовую длину равной m . Нахождение обратного к элементу кольца $\mathbb{Z}_p$, представленному целым ненулевым числом $a \in \mathbb{I}_p$, может быть выполнено с помощью расширенного алгоритма Евклида. Так как p и a , очевидно, взаимно просты, то с помощью расширенного алгоритма Евклида можно найти s и t такие, что $sp + ta = 1$. Получаем $ta \equiv 1 \pmod{p}$, т.е. обратным к классу, содержащему a , является класс, содержащий t . В силу (9.13) имеем $|t| < p$; если $t < 0$, то заменяем t на $t + p$.

Пример 22.1. Для $p = 13$, $a = 5$ получаем с помощью расширенного алгоритма Евклида $2 \cdot 13 + (-5) \cdot 5 = 1$, и отсюда $8 = -5 + 13$ является обратным для 5 в $\mathbb{Z}_{13}$ (проверка подтверждает это: $5 \cdot 8 \equiv 1 \pmod{13}$).

Как следствие предложения 21.3 получаем такое утверждение.

Предложение 22.1. Пусть расширенный алгоритм Евклида основывается на алгоритмах деления с остатком и умножения, битовые затраты каждого из которых применительно к целым v и w допускают оценку $O(\log v \log w)$. Тогда битовая сложность нахождения обратного в поле $\mathbb{Z}_p$ с помощью расширенного алгоритма Евклида до-

пускает оценку $O(\log^2 p)$ при использовании числа p в качестве размера входа n , соответственно, оценку $O(m^2)$ при использовании в этом качестве битовой длины m числа p .

Несомненное достоинство алгоритма обращения, основанного на расширенном алгоритме Евклида, состоит в том, что с его помощью обращение возможно всякий раз, когда обращаемое число и модуль взаимно просты, а не только тогда, когда модуль — простое число. Например, можно определить a такое, что $5a \equiv 1 \pmod{6}$ — среди чисел, входящих в $\mathbb{I}_6$, значением a является 5. В этом смысле применимость этого алгоритма шире, чем, например, алгоритма, основанного на малой теореме Ферма.

Малая теорема Ферма. Пусть p — простое число, a — произвольное целое. Тогда $a^p \equiv a \pmod{p}$.

(Путь доказательства показан в задаче 148.) Если a не делится на p , то согласно этой теореме a^{p-2} является обратным к a по простому модулю p , в частности, если p — простое число и $a \in \{1, \dots, p-1\}$, то

$$a^{p-1} \equiv 1 \pmod{p},$$

иначе говоря —

$$a^{p-2} \cdot a \equiv 1 \pmod{p}.$$

Но это, вообще говоря, неверно для составных p . Например, неверно, что $2^2 \cdot 2 \equiv 1 \pmod{4}$.

С помощью расширенного алгоритма Евклида возможно обращение по модулю k любого числа a , взаимно простого с k ; если $a \in \mathbb{I}_k$, то это обращение имеет битовую сложность $O(\log^2 k)$ или же $O(m^2)$, коль скоро в качестве размера входа используется $m = \lambda(k)$.

Пример 22.2. Уже говорилось, что вопрос о возможности распознавания простоты со сложностью $O(m^d)$ с некоторым $d \in \mathbb{N}$ представляет большой интерес и что, скажем, алгоритм пробных делений (примеры 1.2, 4.1, 5.2) не обладает указанным свойством даже при рассмотрении не битовой сложности, а сложности по числу делений, причем как в худшем случае, так и в среднем.

Если для некоторого $a \in \mathbb{Z}$ мы обнаружим, что не выполняется сравнение

$$a^n \equiv a \pmod{n}, \tag{22.2}$$

то по малой теореме Ферма из этого можно сразу заключить, что n не является простым. Если фиксировано число a такое, что $1 < a < n$, то использование бинарного алгоритма возведения в степень с заме-

ной результата каждого из умножений остатком от деления на n позволяет проверить выполнение сравнения (22.2), затратив $O(\log^3 n)$, или, что то же самое, $O(m^3)$ битовых операций, где $m = \lambda(n)$. Однако это не позволяет утверждать, что мы можем на основе этого проверять простоту n за $O(m^3)$ битовых операций, так как существует бесконечно много составных n таких, что (22.2) выполняется для любых a . Это так называемые числа Кармайкла (наименьшее из которых равно $561 = 3 \cdot 11 \cdot 17$).

Напомним, что алгоритм со сложностью $O(m^d)$, где m — размер входа, $d \in \mathbb{N}$, называют алгоритмом с полиномиально ограниченной сложностью или полиномиальным. В задаче распознавания простоты числа за размер входа берется количество m двоичных цифр числа n или же $\log_2 n$. После долгих поисков, в которых принимали участие многие исследователи, алгоритм с полиномиально ограниченной битовой сложностью был в 2002 г. предложен тремя индийскими математиками — М. Агравалом, Н. Кайалом и Н. Саксеной¹. Этот алгоритм получил в литературе название AKS по первым буквам фамилий авторов. Мы обсудим лишь главную идею алгоритма AKS, основанием которого служит следующее необходимое и достаточное условие простоты числа n .

Предложение 22.2. Пусть $a \in \mathbb{Z}$, $n \in \mathbb{N}$ взаимно просты, тогда n является простым, если и только если выполнено полиномиальное сравнение

$$(x - a)^n \equiv x^n - a \pmod{n}, \quad (22.3)$$

которое надо понимать так, что числовые коэффициенты при одинаковых степенях x в полиномах, расположенных в левой и правой частях, сравнимы по модулю n .

Доказательство. Пусть n — простое. Если $n = 2$, то выполнение (22.3) проверяется непосредственно, и остается рассмотреть случай нечетного простого n . Коэффициенты при x^k в левой и правой частях для случая $1 < k < n$ равны соответственно $(-1)^k \binom{n}{k} a^{n-k}$ и 0. При этом $\binom{n}{k}$ делится на n , так как n простое (см. задачу 147). Коэффициенты при x^n в обеих частях (22.3) равны 1, свободные члены соответственно равны $(-a)^n$ и $-a$. В силу нечетности n достаточно показать, что $a^n \equiv a \pmod{n}$, а это сравнение выполняется в силу малой теоремы Ферма. Мы доказали, что если n — простое, то выполнено (22.3).

¹ См. [60].

Пусть теперь n — составное. Пусть, далее, простое q и $l \in \mathbb{N}^+$ таковы, что $q^l \mid n$ и при этом неверно, что $q^{l+1} \mid n$. Имеем

$$\binom{n}{q} = \frac{(n-q+1)(n-q+2)\dots n}{q!}.$$

Так как $q \mid n$, то произведение $(n-q+1)(n-q+2)\dots(n-1)$ не делится на q ; при этом один из входящих в n множителей, равных q , сократится со знаменателем. Поэтому $\binom{n}{q}$ не делится на q^l . Помимо этого, q^l взаимно просто с a^{n-q} , так как a и q взаимно просты. Поэтому коэффициент при x^q в левой части (22.3), равный $(-1)^{n-q}a^{n-q}\binom{n}{q}$, не делится на q^l , следовательно, не делится на n и поэтому не сравним с 0 по модулю n . Мы доказали, что если n — составное, то сравнение (22.3) не имеет места. $\square$

Из предложения 22.2 следует, что для проверки простоты числа n можно взять любое a , взаимно простое с n (подходит, например, 1), и проверить (22.3). Но прямая такая проверка потребует $\Omega(n) = \Omega(2^m)$ битовых операций, так как раскрытие скобок в левой части (22.3) дает $n+1$ слагаемое. Алгоритм AKS предписывает проверку (22.3) по двум модулям

$$(x-a)^n \equiv x^n - a \pmod{x^r - 1, n}, \quad (22.4)$$

$r > 0$. Сравнение (22.4) понимается в том смысле, что все коэффициенты остатка от деления полинома $(x-a)^n - x^n + a$ на $x^r - 1$ (они будут целыми числами) делятся на n . Вычисление остатков от деления $(x-a)^n$ и x^n на $x^r - 1$ производится с помощью бинарного алгоритма возведения в степень, результат каждого умножения сразу же заменяется остатком от деления на $x^r - 1$. Но если взять произвольное $r \in \mathbb{N}^+$, то может оказаться, что (22.4) выполняется и при некоторых составных n . Число r должно подбираться специальным образом, и авторы показывают, что подходящее для целей алгоритма r всегда существует и может быть выбрано без существенных затрат так, что $r = O(m^5)$; после выбора r сравнение (22.4) надо проверить для «небольшого» числа «легко определяемых» a — количество этих a есть $O(\sqrt{r}m)$. Итоговая сложность алгоритма допускает оценку $O(m^{11})$. Заметим попутно, что в литературе по теории сложности часто используются оценки вида $\tilde{O}(f(m))$ ($\tilde{O}$ называется « O мягкое»), за таким обозначением скрывается $O(f(m) \log^d m)$, где d — положительное число, значение которого не уточняется. Авторы показывают, что битовая сложность их алгоритма допускает оценку $\tilde{O}(m^{21/2})$, добав-

для при этом, что если верны некоторые известные в теории чисел гипотезы (для которых имеются серьезные основания полагать, что они верны), то можно взять $r = O(m^2)$, и в этом случае сложность алгоритма в целом есть $\tilde{O}(m^6)$.

На этом мы завершим разговор об алгоритме AKS, добавив лишь, что, например, в криптографии практикуются вероятностные (типа Монте-Карло) алгоритмы проверки простоты, имеющие меньшую сложность, но не исключающие появления, хоть и с малой вероятностью, неверного ответа. В нашем курсе такого рода алгоритмы не рассматриваются¹.

§ 23. Булева арифметика

Для алгоритмов булевой арифметики сложность по числу булевых операций и битовая сложность — это одно и то же. Аналогично дело обстоит и с пространственной булевой (= битовой) сложностью.

Пример 23.1. Пусть дан ориентированный граф $G = (V, E)$, $n = |V|$ без кратных ребер. Пусть $C = (c_{ij})$ — булева матрица смежности (см. пример 3.2 графа G : $c_{ij} = 1$, если и только если i -я вершина соединена ребром с j -й, $i, j = 1, 2, \dots, n$). Требуется построить для G его матрицу связности $D = (d_{ij})$: $d_{ij} = 1$, если и только если $i = j$ или i -я вершина соединена путем из одного или более ребер с j -й, $i, j = 1, 2, \dots, n$. Если рассматривать G как граф некоторого бинарного отношения на множестве из n элементов, то задачу можно интерпретировать как задачу построения транзитивно-рефлексивного замыкания данного бинарного отношения. Соответственно, граф, для которого D является матрицей смежности, называют транзитивно-рефлексивным замыканием графа G , а саму матрицу D — транзитивно-рефлексивным замыканием матрицы C . Для матрицы D используется обозначение C^* . На рис. 20 показан ориентированный граф и его транзитивно-рефлексивное замыкание. Соответствующие матрицы C и C^* выглядят следующим образом:

$$C = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}, \quad C^* = \begin{pmatrix} 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 \\ 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{pmatrix}.$$

¹ По поводу этих алгоритмов проверки простоты числа см. [28, гл. 33], [12, гл. 4], [18, разд. 2.6].

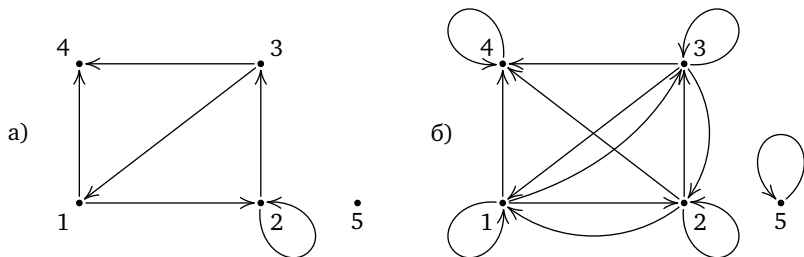


Рис. 20. а) Ориентированный граф; б) его транзитивно-рефлексивное замыкание.

Нам понадобятся произведения булевых матриц, которые определяются как обычно: например, произведение F квадратных матриц A и B порядка n определяется формулой

$$f_{ij} = \bigvee_{k=1}^n (a_{ik} \wedge b_{kj}), \quad i, j = 1, 2, \dots, n.$$

Элемент с индексами i, j для краткости будем называть (i, j) -элементом матрицы.

Легко видеть, что $C^2 = C \wedge C$ — это матрица, для которой (i, j) -элемент равен 1, если и только если i -я вершина соединена с j -й путем длины 2 (т.е. состоящим из двух ребер). Аналогично, C^k — матрица, для которой (i, j) -элемент равен 1, если и только если i -я вершина соединена с j -й путем длины k . Пусть I_n — матрица, для которой (i, j) -элемент равен 1, если и только если $i = j$. Тогда

$$C^* = I_n \vee C \vee C^2 \vee \dots \vee C^{n-1}.$$

Индукцией по k несложно доказать, что для любой квадратной булевой матрицы C и натурального k выполнено

$$I_n \vee C \vee C^2 \vee \dots \vee C^k = (I_n \vee C)^k. \quad (23.1)$$

Это дает нам

$$C^* = (I_n \vee C)^{n-1}. \quad (23.2)$$

Тривиальный способ умножения булевых $(n \times n)$ -матриц требует $\Theta(n^3)$ битовых операций. Если используется именно этот способ, то оценка числа операций для формулы (23.2) есть $\Theta(n^4)$. Применение бинарного алгоритма RS (пример 4.2) для возведения в степень булевой матрицы C позволяет перейти к оценке

$$\Theta(n^3 \log n). \quad (23.3)$$

Если нежелательно связывать основанный на формуле (23.2) алгоритм построения транзитивно-рефлексивного замыкания с конкретным способом умножения матриц, то верхнюю оценку числа битовых операций можно записать как

$$n + B(n)(\lambda(n) + \lambda^*(n) - 2), \quad (23.4)$$

где $B(n)$ — сложность используемого алгоритма умножения булевых $(n \times n)$ -матриц; слагаемое n отражает расход на сложение с матрицей I_n .

Рассмотренный пример высвечивает связь задачи построения транзитивно-рефлексивного замыкания ориентированного графа с задачей умножения булевых матриц. Эта связь оказывается очень тесной, о чем мы еще будем говорить в § 28.

В следующем примере, однако, задача построения транзитивно-рефлексивного замыкания решается без использования умножения булевых матриц.

Пример 23.2. Исследуем сложность алгоритма С. Уоршелла, предназначенного для построения матрицы C^* . В процессе применения этого алгоритма матрица C^* строится за $n = |V|$ шагов, после k -го шага получается матрица $D^{(k)} = (d_{ij}^{(k)})$, и $d_{ij}^{(k)} = 1$, если и только если i -я вершина соединена таким путем с j -й, номера всех промежуточных вершин которого не выходят за пределы множества $\{1, 2, \dots, k\}$. Вначале полагаем $D^{(0)} = I_n \vee C$. Затем для каждого $k = 1, 2, \dots, n$ находим $D^{(k)}$:

$$d_{ij}^{(k)} = d_{ij}^{(k-1)} \vee (d_{ik}^{(k-1)} \wedge d_{kj}^{(k-1)}). \quad (23.5)$$

После этого $C^* = D^{(n)}$.

Формула (23.5) отражает то, что путь из i -й вершины в j -ю, все промежуточные вершины которого принадлежат $\{1, 2, \dots, k\}$, существует, если и только если реализуется хотя бы одна из следующих возможностей:

- имеется путь из i -й вершины в j -ю, все промежуточные вершины которого принадлежат $\{1, 2, \dots, k-1\}$;
- имеются пути из i -й вершины в k -ю и из k -й в j -ю, все промежуточные вершины которых принадлежат $\{1, 2, \dots, k-1\}$.

На вычисление $D^{(0)}$ уходит n битовых операций, вычисление каждой из матриц $D^{(k)}$, $1 \leq k \leq n$, требует $2n^2$ операций. Итого, требуется $2n^3 + n$ операций.

Алгоритм Уоршелла вычисляет матрицу C^* , т. е. строит транзитивно-рефлексивное замыкание ориентированного графа G с n вершинами, заданного матрицей смежности, затрачивая $2n^3 + n$ битовых операций.

Алгоритм Уоршелла можно легко реализовать так, что хранить в памяти будут только $D^{(k-1)}$ и $D^{(k)}$. Но на самом деле алгоритм Уоршелла позволяет производить все вычисления, расходуя всего n^2 бит под хранение матричных элементов, причем это будут те самые n^2 бит, которые занимала исходная матрица. Если мы вычислим $D = I_n \vee C$, а затем n раз обновим матрицу D (каждый раз всю целиком, не оставляя незатронутых элементов), используя на k -м этапе обновления матрицы D присваивания

$$d_{ij} := d_{ij} \vee (d_{ik} \wedge d_{kj}),$$

$k = 1, 2, \dots, n$, то получим искомую матрицу связности. В самом деле, индукцией по k можно доказать, что после k -го этапа обновления получаем матрицу, равную $D^{(k)}$, при этом индуктивный переход от $k - 1$ к k основывается на том, что имеют место очевидные соотношения

$$d_{ik}^{(k)} = d_{ik}^{(k-1)}, \quad d_{kj}^{(k)} = d_{kj}^{(k-1)},$$

$k = 1, 2, \dots, n$, т. е. при обновлении элемента d_{ij} на k -м этапе не имеет никакого значения, был ли уже обновлен какой-то из элементов d_{ik} , d_{kj} или нет.

Если строить матрицу C^* на месте исходной матрицы C , то алгоритм можно изобразить так:

```

for  $l = 1$  to  $n$  do  $c_{ll} := 1$  od;
for  $k = 1$  to  $n$  do
  for  $i = 1$  to  $n$  do
    for  $j = 1$  to  $n$  do
       $c_{ij} := c_{ij} \vee (c_{ik} \wedge c_{kj})$ 
    od
  od
od
```

Подводя итог рассмотрению алгоритма Уоршелла, заметим, что, как и алгоритм Евклида в расширенной версии, он дает довольно редкий пример, когда низкие временные затраты сочетаются с малым расходом памяти.

Алгоритм Уоршелла построения транзитивно-рефлексивного замыкания ориентированного графа имеет битовую временную слож-

ность $2n^3 + n$, где $n = |V|$ — число вершин графа. Пространственная сложность этого алгоритма ограничена константой.

При рассмотрении $|V|, |E|$ в качестве двух компонент размера входа сказанное сохраняет силу, так как затраты алгоритма Уоршелла не зависят от числа ребер заданного входного графа.

Задачи

138. Исследовать битовую сложность вычисления величины $1 + 2 + \dots + n$ последовательными сложениями. Размером входа считать битовую длину m целого числа n .

139. Для временной битовой сложности «сверхнаивного» умножения (пример 19.1) при использовании компонент m_1, m_2 верна оценка сверху $O(2^{\max\{m_1, m_2\}} \max\{m_1, m_2\})$, а оценка $\Theta(2^{\max\{m_1, m_2\}} \max\{m_1, m_2\})$ неверна.

Указание. Неверна оценка $\Omega(2^{\max\{m_1, m_2\}} \max\{m_1, m_2\})$: она противоречит оценке $O(2^{m_2} m_1)$, так как для любого $N > 0$ существуют $m_1, m_2 > N$ такие, что $m_1 = 2^{m_2}$.

140. Определение чисел Фибоначчи, данное в § 10, позволяет вычислить F_n , выполнив $n - 1$ сложение. Битовая сложность этого алгоритма допускает оценку $O(n^2)$ при рассмотрении n в качестве размера входа.

Указание. См. задачу 77.

141. Обосновать использованный в доказательстве предложения 21.1 переход от оценки $T_{\text{ND}}^{**}(m_1, m_2) = O((m_1 - m_2 + 1)(m_2 + 1))$ к $T_{\text{ND}}^{**}(m_1, m_2) = O((m_1 - m_2 + 1)m_2)$.

142. Можно ли усилить предложение 21.2, заменив приведенные там оценки $O(\log a \log b)$, $O(\log^2 a)$ на $\Theta(\log a \log b)$, $\Theta(\log^2 a)$?

143. Рассмотрим $m = \lambda(n)$ в качестве размера входа алгоритма вычисления факториала (пример 20.1). Верна ли для соответствующей временной битовой сложности оценка $O(2^m m)$?

144. К числу, первоначально равному нулю, прибавляется шаг за шагом по единице до получения значения $2^n - 1$, $n \geq 0$. При этих сложениях потребуется $2^n - n - 1$ переносов единицы в старший разряд.

145. Пусть p — простое, a и b — произвольные целые, k — натуральное. Тогда $(a + b)^{p^k} \equiv a^{p^k} + b^{p^k} \pmod{p}$.

146. а) Аддитивная группа кольца $\mathbb{Z}_k$ является циклической, в качестве образующей этой группы может выступать любой класс, содержащий число l , взаимно простое с k .

б) Пусть p — простое, тогда мультипликативная группа поля $\mathbb{Z}_p$, т. е. группа всех ненулевых элементов по умножению, является циклической.

147. Если число p — простое, а k — неотрицательное целое, меньшее p , то $\binom{p}{k} \equiv 0 \pmod{p}$.

Указание. Предварительно показать, что p не делит $k!$.

148. Индукцией по a доказать малую теорему Ферма.

Указание. Использовать равенство $a^p = (1 + (a - 1))^p$ и предыдущую задачу.

149. Имеет место следующая китайская теорема об остатках: пусть $k_1, k_2, \dots, k_n$ — попарно взаимно простые натуральные числа (модули); тогда для любых $b_1, b_2, \dots, b_n \in \mathbb{Z}$ найдется $f \in \mathbb{N}$ такое, что $f \equiv b_i \pmod{k_i}$, $i = 1, 2, \dots, n$. (Задача восстановления числа по остаткам обсуждалась в древних китайских математических трактатах.) Дать конструктивное доказательство этой теоремы, т. е. доказательство, содержащее в себе алгоритм построения подходящего f (каждый такой алгоритм может быть назван китайским алгоритмом).

Указание. Пусть $k = k_1 k_2 \dots k_n$ и $g_i = k/k_i$, $i = 1, 2, \dots, n$. При всех $i = 1, 2, \dots, n$ числа k_i и g_i взаимно просты, поэтому расширенным алгоритмом Евклида можно определить h_i так, что $h_i g_i \equiv 1 \pmod{k_i}$ и положить $f = \sum_{j=1}^n b_j h_j g_j$.

150. Исходя из того, что значение полинома $f(x)$ в точке $x = a$ равно по теореме Безу остатку от деления $f(x)$ на $x - a$, установить параллелизм между интерполяционной формулой Лагранжа и формулой для значения f , данной в указании к задаче 149.

151. Битовая сложность китайского алгоритма, эскизно обрисованного в указании к задаче 149 и основывающегося на расширенном алгоритме Евклида, наивном делении и наивном умножении, допускает оценку $O(\log^2 k)$, если в качестве размера входа рассмотреть k , и оценку $O(m^2)$, если в качестве размера входа рассмотреть битовую длину m числа k .

Указание. По поводу сложности вычисления k см. пример 20.2; сложность этапа вычисления всех g_i допускает оценку $O\left(\sum_{i=1}^n \log k \log k_i\right)$ и т. д.

152. Обобщение китайской теоремы об остатках (см. задачу 149). Пусть натуральные числа $k_1, k_2, \dots, k_n$ не обязательно попарно взаимно просты. Утверждение, что для любых целых $b_1, b_2, \dots, b_n$ найдется натуральное f такое, что $f \equiv b_i \pmod{k_i}$, $i = 1, 2, \dots, n$, справедливо, если и только если $b_i \equiv b_j \pmod{\text{нод}(k_i, k_j)}$ для всех пар k_i, k_j .

153. Верно ли, что для битовой сложности в среднем алгоритма пробных делений справедлива оценка $\Omega\left(\left(\frac{4}{3}\right)^m\right)$?

154. Битовая сложность в среднем алгоритма наивного умножения двух неотрицательных целых чисел a и b допускает оценку $\Omega(m^2)$, а тем самым — и $\Theta(m^2)$, где $m = \max\{\lambda(a), \lambda(b)\}$. Считаем, что при фиксированной битовой длине входного числа все входные числа равновероятны.

Указание. Битовые затраты при наивном умножении a на b не могут быть меньше, чем $c\lambda(a)\lambda^*(b)$. Два случая

$$\lambda(a) = m, \quad \lambda(b) \leq m \quad \text{и} \quad \lambda(a) \leq m, \quad \lambda(b) = m$$

приводят к двум вероятностным пространствам V_1, V_2 , состоящим из соответствующих пар (a, b) с равномерными распределениями вероятностей. Определить значения вероятностей событий $\lambda(a) = k$ и $\lambda(b) = k$ для произвольного $0 \leq k \leq m$ для каждого из пространств V_1, V_2 и найти математические ожидания случайных величин $\xi(a, b) = \lambda(a)$, $\eta(a, b) = \lambda^*(b)$. В силу независимости ξ и η можно воспользоваться равенством $E(\xi(a, b)\eta(a, b)) = (E(\xi(a, b)))(E\eta(a, b))$.

155. Привести полное доказательство равенств (23.1), (23.2).

156. Основываясь на (23.2) и бинарном алгоритме возведения в степень, построение C^* можно выполнить с пространственной сложностью $3n^2 + O(1)$.

157. Искомая матрица C^* будет вычисляться правильно и после увеличения показателя степени в правой части (23.2).

158. Воспользовавшись утверждением, содержащимся в задаче 157, заменим показатель степени в правой части (23.2) на 2^l , где l — наименьшее целое, такое что $2^l \geq n - 1$. Как изменится верхняя оценка (23.4) при переходе к этой новой версии алгоритма?

159. Исследовать пространственную сложность описанного в задаче 158 алгоритма.

160. Какой наибольшей временной сложностью может обладать алгоритм умножения булевых матриц, или, другими словами, какую верхнюю границу должна допускать функция $B(n)$, чтобы временные затраты описанного в задаче 158 алгоритма допускали бы оценку $O(n^3)$?

161. Можно ли первую строку приведенного в § 23 псевдокода алгоритма Уоршелла переместить в конец этого псевдокода?

Указание. Рассмотрим алгоритм в сокращенном виде, удалив эту строку из псевдокода. Влияют ли значения диагональных элементов исходной матрицы на значения внедиагональных элементов матрицы-результата?

162. Описать версию алгоритма Уоршелла для вычисления кратчайших путей между вершинами графа G , считая, что вместо матрицы смежности дана матрица, содержащая длины соответствующих ребер. Если какие-то вершины не соединены, то длина ребра равна ∞ . Этот вариант алгоритма Уоршелла предложен Р. Флойдом и называется алгоритмом Флойда. Провести анализ сложности этого алгоритма по числу операций сложения и операций нахождения минимума (раздельно) и анализ пространственной сложности. (В этой задаче мы не рассматриваем битовую сложность.)

163. Алгоритм Флойда (см. предыдущую задачу) основан на вычислениях в множестве $\mathbb{R} \cup \{\infty\}$ с помощью операций $+$ и $\min$. Пусть $a, b \in \mathbb{R} \cup \{\infty\}$. Введем обозначения

$$a \odot b = a + b, \quad a \oplus b = \min\{a, b\}.$$

Операции $\odot, \oplus$ приводят к иным вычислениям, нежели обычные операции умножения $\cdot$ и сложения $+$ (в последние годы такие вычисления стали называть «тропическими»¹). Показать, что

а) 0 служит нейтральным элементом (единицей) по отношению к $\odot$, а ∞ играет аналогичную роль (т. е., в этом случае, роль нуля) по отношению к $\oplus$.

б) обе операции ассоциативны и операция $\odot$ дистрибутивна по отношению к $\oplus$.

(Можно, сохранив определение $a \odot b$, положить $a \oplus b = \max\{a, b\}$ и рассматривать эти операции на множестве $\mathbb{R} \cup \{-\infty\}$; ассоциативность и дистрибутивность сохраняются. Какими будут нейтральные элементы?)

¹ См., например, [7, с. 11].

Глава 6

Рекуррентные соотношения как средство анализа сложности алгоритмов

§ 24. Простейшие рекуррентные уравнения

Рекуррентные соотношения (уравнения и неравенства) играют существенную роль в анализе алгоритмов. Наиболее известный и простой вид таких соотношений — это линейные рекуррентные уравнения с постоянными коэффициентами.

Пример 24.1. Вернемся к задаче 144, в которой надо доказать, что если к числу, первоначально равному нулю, прибавляется шаг за шагом по единице до достижения значения $2^n - 1$, $n \geq 0$, то в целом потребуется $2^n - n - 1$ переносов единиц в старшие разряды. Коль скоро заранее указан ответ, для доказательства можно использовать индукцию. Но формулу $2^n - n - 1$ можно вывести, исходя лишь из условия задачи. Обозначим через $s(n)$ исследуемое количество переносов и заметим, что если прибавлением единиц уже получено число $2^{n-1} - 1$ (на это потребуется $s(n-1)$ переносов), то очередное прибавление единицы потребует $n - 1$ переносов и приведет к числу 2^{n-1} , двоичная запись которого есть $10...0$ (количество нулей после единицы равно $n - 1$). Далее для достижения числа $11...1$ (n единиц) потребуется еще $s(n-1)$ переносов. Получаем рекуррентное уравнение $s(n) = 2s(n-1) + n - 1$, или

$$s(n) - 2s(n-1) = n - 1, \quad (24.1)$$

при этом $s(0) = 0$. Характеристическое уравнение¹, соответствующее рекуррентному уравнению (24.1), имеет вид $\lambda - 2 = 0$. Общее решение однородного уравнения $s(n) - 2s(n-1) = 0$ есть $c2^n$. Правую часть уравнения (24.1) можно записать в виде квазиполинома $(n-1)1^n$. Значение 1 не является корнем характеристического уравнения, поэтому

¹ О методе решения линейных рекуррентных уравнений с постоянными коэффициентами см. в приложении F.

(24.1) обладает частным решением вида $an + b$; подставляя это выражение вместо $s(n)$ в (24.1), получаем $an + b - 2(a(n - 1) + b) = n - 1$, и, приравнявая в левой и правой частях коэффициенты при первой и нулевой степенях n , имеем $a = b = -1$. Получаем общее решение уравнения (24.1): $s(n) = c2^n - n - 1$. Подбираем значение константы c так, чтобы выполнялось $s(0) = 0$; для этого должно выполняться $c \cdot 1 - 1 = 0$, т. е. $c = 1$. Итак, потребуется $2^n - n - 1$ переносов единиц в старшие разряды.

Алгоритм, который содержит прямые или косвенные (т. е. при посредстве других алгоритмов) обращения к себе, как известно, называется *рекурсивным*. В следующем примере обсуждается использование рекуррентных уравнений в анализе рекурсивных алгоритмов и указывается один неудачный вид рекурсии.

Пример 24.2. Пусть некоторый рекурсивный алгоритм укладывается в схему

$$Y_n = U(Y_{n-1}, Y_{n-2}), \quad (24.2)$$

с заданными Y_0, Y_1 , где U — какая-то функция, и для нас не важно, являются ли Y_i числами, множествами или же какими-то другими объектами. Нахождение Y_n для данного n с помощью простейшего циклического алгоритма потребует $n - 1$ обращений к U . Рекурсивный же алгоритм, предполагающий независимое вычисление Y_{n-1} и Y_{n-2} , потребует $y(n)$ обращений, где $y(n)$ удовлетворяет уравнению $y(n) = y(n - 1) + y(n - 2) + 1$, или

$$y(n) - y(n - 1) - y(n - 2) = 1, \quad (24.3)$$

при этом $y(0) = y(1) = 0$.

Правая часть уравнения (24.3) является квазиполиномом 1^n , и 1 не принадлежит множеству корней характеристического уравнения

$$\lambda^2 - \lambda - 1 = 0. \quad (24.4)$$

Следовательно, существует частное решение вида $c1^n$, где c — полином нулевой степени, т. е. константа; подстановка в (24.3) дает $c = -1$. Корнями уравнения (24.4) служат числа

$$\phi = \frac{1 + \sqrt{5}}{2}, \quad \tilde{\phi} = \frac{1 - \sqrt{5}}{2}$$

(входящие в формулу Бине (10.2), которая сама может быть выведена с помощью общего метода решения линейных рекуррентных

уравнений с постоянными коэффициентами). Общее решение уравнения (24.3) может быть записано в виде $y(n) = C_1 \phi^n + C_2 \tilde{\phi}^n - 1$. Начальные условия $y(0) = y(1) = 0$ приводят нас к системе

$$\begin{aligned} C_1 + C_2 &= 1, \\ \frac{1 + \sqrt{5}}{2} C_1 + \frac{1 - \sqrt{5}}{2} C_2 &= 1. \end{aligned}$$

Получаем

$$C_1 = \frac{\phi}{\sqrt{5}}, \quad C_2 = -\frac{\tilde{\phi}}{\sqrt{5}}. \quad (24.5)$$

Таким образом,

$$y(n) = \frac{1}{\sqrt{5}} \phi^{n+1} - \frac{1}{\sqrt{5}} \tilde{\phi}^{n+1} - 1 = F_{n+1} - 1.$$

Отсюда следует ряд асимптотических формул:

$$y(n) = \frac{1}{\sqrt{5}} \phi^{n+1} + O(1), \quad y(n) \sim \frac{1}{\sqrt{5}} \phi^{n+1}, \quad y(n) = \Theta(\phi^n) \quad (24.6)$$

и т.д. Формулу $y(n) = \Theta(\phi^n)$ можно было бы получить без рассмотрения начальных значений: ясно, что $y(n)$ возрастает при увеличении n , поэтому $C_1 \neq 0$.

Причина избыточной сложности рекурсивного варианта алгоритма в сравнении с циклическим абсолютно ясна. Построение Y_n требует вычисления Y_{n-1} и Y_{n-2} , а Y_{n-1} в свою очередь потребует построения Y_{n-2} и Y_{n-3} . Даже не проследивая ход вычислений слишком далеко, мы видим, что требуются два независимых вычисления Y_{n-2} , при этом вычисление Y_{n-2} по тем же причинам будет производиться не лучшим образом.

При $k > 2$ рекурсия вида

$$Y_n = U(Y_{n-1}, \dots, Y_{n-k}) \quad (24.7)$$

(например, с заданными $Y_0, Y_1, \dots, Y_{k-1}$) еще более разорительна, коль скоро эта рекурсия предписывает независимое вычисление $Y_{n-1}, Y_{n-2}, \dots, Y_{n-k}$. Нахождение Y_n с помощью простейшего циклического алгоритма потребует $n - k + 1$ обращений к U . Если при рекурсивном нахождении их требуется $y(n)$, то

$$y(n) - y(n-1) - \dots - y(n-k) = 1, \quad (24.8)$$

$y(0) = y(1) = \dots = y(k-1) = 0$. Можно асимптотически описать рост $y(n)$, не решая полностью рекуррентного уравнения (24.8).

Предложение 24.1. Пусть рекурсивный алгоритм организован по схеме (24.7), пусть $k \geq 2$ и пусть для получения значения функции U необходимы значения всех ее k аргументов. Тогда количество вычислений значений этой функции при нахождении Y_n есть $\Theta(\alpha_k^n)$, и α_k удовлетворяет неравенствам $2 - \frac{1}{k} \leq \alpha_k < 2$, $k = 2, 3, \dots$

В приложении G можно найти полное доказательство этого предложения. Оно основывается на исследовании расположения на комплексной плоскости корней алгебраических уравнений

$$\lambda^k - \lambda^{k-1} - \dots - \lambda - 1 = 0,$$

$k = 2, 3, \dots$, т.е. характеристических уравнений для рекуррентных уравнений (24.8).

Случай, когда $k = 1$, исключителен в том отношении, что 1 здесь будет корнем кратности 1 характеристического уравнения $\lambda - 1 = 0$, и если рассмотреть правую часть как квазиполином 1^n , то получим, что (24.8) при $k = 1$ имеет частное решение в виде полинома первой степени. Подстановкой $an + b$ в уравнение находим, что, например, n является таким решением. Общее решение соответствующего однородного уравнения есть $C1^n$, т.е. C . Получили общее решение $C + n$ исходного неоднородного уравнения. Начальное условие $y(0) = 0$ дает $C = 0$, получаем вполне ожидаемое решение $y(n) = n$.

Мы видим, что анализ сложности простейших рекурсивных алгоритмов часто приводит к линейным рекуррентным уравнениям с постоянными коэффициентами (однородным или с правыми частями в виде суммы квазиполиномов). В таких случаях оказывается возможным найти явные формулы для сложности алгоритма. Асимптотические формулы иногда удается получить непосредственно из общего решения, не прибегая к вычислению значений входящих в него произвольных постоянных.

Правда, дело не всегда обстоит просто, — иногда возникают уравнения с переменными коэффициентами или нелинейные уравнения и неравенства, и об этом мы еще будем говорить. Линейные уравнения с постоянными коэффициентами могут играть вспомогательную роль в исследовании решений уравнений и неравенств более общего вида.

§ 25. Об одном классе нелинейных рекуррентных соотношений

Одним из источников появления рекурсии в алгоритмах служит привлечение стратегии «разделяй и властвуй»: вычислительная задача

с размером входа n разбивается на $s > 1$ задач того же типа, что и исходная, и при этом размер входа каждой из возникших новых задач примерно равен $\frac{n}{s}$ («разделяй»); эти задачи решаются рекурсивно, т. е. с применением этой же стратегии, а затем полученные решения соединяются в решение исходной задачи («властвуй»). Для входов маленьких размеров — как правило, 0 или 1 — задачи решаются каким-то простым способом, без рекурсии. При анализе сложности таких алгоритмов возникают специфические нелинейные рекуррентные соотношения в виде уравнений и неравенств. В общем случае эти соотношения довольно трудны для решения, но для многих конкретных задач удается исследовать сложность алгоритмов сведением рекуррентных соотношений к хорошо изученным линейным рекуррентным уравнениям первого порядка с постоянными коэффициентами и квазиполиномиальными правыми частями.

Пример 25.1. Обратимся к сортировке слияниями в ее рекурсивном варианте и исследуем ее сложность $T_{MS}(n)$ по числу сравнений (n — длина массива). Функция $T_{MS}(n)$ подчиняется рекуррентному неравенству

$$T_{MS}(n) \leq \begin{cases} 0, & \text{если } n = 1, \\ T_{MS}\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T_{MS}\left(\left\lceil \frac{n}{2} \right\rceil\right) + n - 1, & \text{если } n > 1. \end{cases} \quad (25.1)$$

В самом деле, для любого массива длины $n > 1$ сортировка первых его $\left\lfloor \frac{n}{2} \right\rfloor$ элементов потребует не более $T_{MS}\left(\left\lfloor \frac{n}{2} \right\rfloor\right)$ сравнений (так как $T_{MS}\left(\left\lfloor \frac{n}{2} \right\rfloor\right)$ — это максимальное число сравнений, которое может потребоваться рассматриваемой сортировке при работе с массивом длины $\left\lfloor \frac{n}{2} \right\rfloor$); аналогично, сортировка последних $\left\lceil \frac{n}{2} \right\rceil$ элементов потребует не более чем $T_{MS}\left(\left\lceil \frac{n}{2} \right\rceil\right)$ сравнений, а слияние потребует сравнений в количестве, не превосходящем $\left\lfloor \frac{n}{2} \right\rfloor + \left\lceil \frac{n}{2} \right\rceil - 1 = n - 1$. В любом, а значит, и в худшем случае потребуется не более $T_{MS}\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T_{MS}\left(\left\lceil \frac{n}{2} \right\rceil\right) + n - 1$ сравнений.

Рекуррентные неравенства, схожие с неравенством (25.1), характерны для многих алгоритмов, построенных на стратегии «разделяй и властвуй». Мы прервем на некоторое время рассмотрение этого примера и обсудим общую ситуацию с неравенствами возникающего типа.

Предложение 25.1. (i) Пусть вещественная функция f натурального аргумента удовлетворяет неравенству

$$f(n) \leq \begin{cases} u, & \text{если } n = 1, \\ v f\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + w f\left(\left\lceil \frac{n}{2} \right\rceil\right) + \varphi(n), & \text{если } n > 1, \end{cases} \quad (25.2)$$

где u, v, w — неотрицательные вещественные числа, причем $v + w \geq 1$, а функция φ — неотрицательная неубывающая. Тогда

$$f(n) \leq t(\lceil \log_2 n \rceil), \quad (25.3)$$

где

$$t(k) = \begin{cases} u, & \text{если } k = 0, \\ (v + w)t(k - 1) + \varphi(2^k), & \text{если } k > 0. \end{cases} \quad (25.4)$$

(ii) Пусть вещественная функция f натурального аргумента удовлетворяет неравенству, получающемуся из (25.2) заменой знака $\leq$ на $\geq$:

$$f(n) \geq \begin{cases} u, & \text{если } n = 1, \\ v f\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + w f\left(\left\lceil \frac{n}{2} \right\rceil\right) + \varphi(n), & \text{если } n > 1, \end{cases} \quad (25.5)$$

с прежними предположениями относительно $u, v, w, \varphi(n)$. Тогда

$$f(n) \geq t(\lfloor \log_2 n \rfloor), \quad (25.6)$$

где t — решение рекуррентного уравнения (25.4).

Доказательство. Установим прежде всего, что вещественная функция F натурального аргумента, удовлетворяющая равенству

$$F(n) = \begin{cases} u, & \text{если } n = 1, \\ v F\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + w F\left(\left\lceil \frac{n}{2} \right\rceil\right) + \varphi(n), & \text{если } n > 1, \end{cases} \quad (25.7)$$

является неубывающей, т. е. при $n \geq 2$ выполнено

$$F(n) \geq F(n - 1). \quad (25.8)$$

Проведем индукцию по n . При $n = 2$ неравенство выполнено, так как

$$F(2) = (v + w)u + \varphi(2) \geq u = F(1).$$

Пусть $n > 2$ и неравенство (25.8) выполнено для $2, 3, \dots, n - 1$; докажем, что тогда оно верно и для n . Воспользуемся тем, что при $n > 2$

$$n > \left\lfloor \frac{n}{2} \right\rfloor \geq \left\lfloor \frac{n-1}{2} \right\rfloor \geq 1, \quad n > \left\lceil \frac{n}{2} \right\rceil \geq \left\lceil \frac{n-1}{2} \right\rceil \geq 1.$$

Имеем

$$\begin{aligned} F(n) &= vF\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + wF\left(\left\lceil \frac{n}{2} \right\rceil\right) + \varphi(n) \geq \\ &\geq vF\left(\left\lfloor \frac{n-1}{2} \right\rfloor\right) + wF\left(\left\lceil \frac{n-1}{2} \right\rceil\right) + \varphi(n-1) = F(n-1). \end{aligned}$$

Неравенство (25.8) доказано. Обратимся теперь к утверждениям (i), (ii).

(i) Вновь используем индукцию и докажем, что

$$f(n) \leq F(n) \quad (25.9)$$

для $n \geq 1$. Очевидно, $f(1) \leq u = F(1)$. Пусть $n > 1$ и неравенство (25.9) выполнено для $1, 2, \dots, n-1$; докажем, что тогда оно верно и для n . В силу (25.2) мы имеем при $n > 1$

$$f(n) \leq vf\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + wf\left(\left\lceil \frac{n}{2} \right\rceil\right) + \varphi(n), \quad (25.10)$$

и так как при $n > 1$ выполняются неравенства $\left\lfloor \frac{n}{2} \right\rfloor \leq n-1$ и $\left\lceil \frac{n}{2} \right\rceil \leq n-1$, то по предположению индукции

$$vf\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + wf\left(\left\lceil \frac{n}{2} \right\rceil\right) + \varphi(n) \leq vF\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + wF\left(\left\lceil \frac{n}{2} \right\rceil\right) + \varphi(n). \quad (25.11)$$

Правая часть последнего неравенства есть $F(n)$. Это доказывает (25.9).

Свойства (25.8), (25.9) дают нам

$$f(n) \leq F(n) \leq F(2^{\lceil \log_2 n \rceil}) \quad (25.12)$$

для $n \geq 1$. Исследуем функцию $t(k) = F(2^k)$, $k = 0, 1, \dots$ В силу (25.7)

$$F(2^k) = \begin{cases} u, & \text{если } k = 0, \\ (v+w)F(2^{k-1}) + \varphi(2^k), & \text{если } k > 0, \end{cases}$$

и, следовательно, $t(k)$ удовлетворяет линейному рекуррентному уравнению (25.4). Отсюда и из (25.12) следует требуемое.

(ii) Доказательство получается заменой знака $\leq$ на $\geq$ в (25.9), (25.10) и (25.11); вместо (25.12) получаем

$$f(n) \geq F(n) \geq F(2^{\lfloor \log_2 n \rfloor}),$$

откуда следует (25.6). □

Определение 25.1. Рекуррентное уравнение (25.4), включающее в себя начальное условие $t(0) = u$, мы назовем уравнением, ассоциированным с рекуррентными неравенствами (25.2) и (25.5).

Исходя из рекуррентного неравенства вида (25.2) (например, из (25.1)), можно написать соответствующее ассоциированное уравнение, найти его решение и, воспользовавшись первой частью предложения 25.1, получить для $f(n)$ верхнюю оценку (25.3). Аналогичным образом, с помощью второй части предложения 25.1 можно получить нижнюю оценку (25.6). Если же возникает соотношение со знаком $=$ вместо $\geq$ или $\leq$, то мы можем рассматривать его как систему двух рекуррентных неравенств (25.2), (25.5). Тогда получим оценки

$$t(\lfloor \log_2 n \rfloor) \leq f(n) \leq t(\lceil \log_2 n \rceil). \quad (25.13)$$

Возвращаясь к примеру 25.1, можно доказать (см. задачу 179), что для сложности по числу сравнений рекурсивной сортировки слияниями выполнено

$$T_{\text{MS}}(n) = \begin{cases} 0, & \text{если } n = 1, \\ T_{\text{MS}}\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T_{\text{MS}}\left(\left\lceil \frac{n}{2} \right\rceil\right) + n - 1, & \text{если } n > 1. \end{cases} \quad (25.14)$$

Верхняя и нижняя оценки могут быть получены описанным способом; здесь ассоциированным уравнением будет

$$t(k) = \begin{cases} 0, & \text{если } k = 0, \\ 2t(k-1) + 2^k - 1, & \text{если } k > 0, \end{cases} \quad (25.15)$$

его решение $t(k) = (k-1)2^k + 1$ (правая часть в $t(k) - 2t(k-1) = 2^k - 1$ является суммой квазиполиномов 2^k и -1 ; первый из них, т. е. 2^k , интересен тем, что 2 является корнем характеристического уравнения $\lambda - 2 = 0$). Мы получаем

$$(\lfloor \log_2 n \rfloor - 1)2^{\lfloor \log_2 n \rfloor} + 1 \leq T_{\text{MS}}(n) \leq (\lceil \log_2 n \rceil - 1)2^{\lceil \log_2 n \rceil} + 1. \quad (25.16)$$

Пространственная сложность этой сортировки есть $n + O(1)$, ибо слияние упорядоченных массивов выполняется с получением результата на новом месте.

§ 26. Асимптотические оценки решений рекуррентных неравенств

Рассмотренный метод оценивания решений рекуррентных неравенств приводит к достаточно точным оценкам, но требует многоэтапной работы. Часто бывает достаточно получить описание роста решения в виде простой асимптотической оценки, для вывода которой не нужно определять численные значения коэффициентов,

входящих в соответствующее частное решение ассоциированного рекуррентного уравнения. Этот путь приводит к ряду общих теорем. Для них в разных литературных источниках используются синонимичные названия «теорема о рекуррентном неравенстве», «основная теорема о рекуррентных оценках» и т. д.¹ Мы приведем две теоремы такого рода.

Теорема 26.1 (о рекуррентном неравенстве, случай $\leq$). Пусть неотрицательная вещественная функция f натурального аргумента удовлетворяет неравенству

$$f(n) \leq \begin{cases} u, & \text{если } n = 1, \\ vf\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + wf\left(\left\lceil \frac{n}{2} \right\rceil\right) + cn^d, & \text{если } n > 1, \end{cases} \quad (26.1)$$

где u, v, w, d — неотрицательные вещественные числа, причем $v + w \geq 1$ и c — положительное вещественное число. Тогда

$$f(n) = \begin{cases} O(n^d \log n), & \text{если } d = \log_2(v + w), \\ O(n^d), & \text{если } d > \log_2(v + w), \\ O(n^{\log_2(v+w)}), & \text{если } d < \log_2(v + w). \end{cases} \quad (26.2)$$

Доказательство. Решение ассоциированного уравнения

$$t(k) = \begin{cases} u, & \text{если } k = 0, \\ (v + w)t(k - 1) + c(2^d)^k, & \text{если } k > 0, \end{cases}$$

имеет вид

$$t(k) = c_0(v + w)^k + (c_1k + c_2)(2^d)^k = c_0(2^{\log_2(v+w)})^k + (c_1k + c_2)(2^d)^k \quad (26.3)$$

с некоторыми конкретными c_0, c_1, c_2 , причем $c_1 \neq 0$, если и только если $2^d = v + w$, или, что то же самое, если $d = \log_2(v + w)$. При этом если $c_1 = 0$, то $c_2 \neq 0$.

Рассмотрим отдельно три случая.

Случай $d = \log_2(v + w)$. Имеем $t(k) = (c_1k + (c_0 + c_2))(2^d)^k$. Для достаточно больших значений k выполняется

$$t(k) < Ck(2^d)^k = Ck(2^k)^d,$$

где C — некоторое положительное число. Используя предложение 25.1(i), заключаем, что

$$f(n) < C[\log_2 n](2^{\lceil \log_2 n \rceil})^d,$$

¹ В литературе на английском языке — «master theorem» (мастер-теорема).

откуда

$$f(n) < C(\log_2 n + 1)(2 \cdot 2^{\log_2 n})^d = C2^d n^d (\log_2 n + 1),$$

и, таким образом, $f(n) = O(n^d \log n)$.

Случай $d > \log_2(v + w)$. Имеем $t(k) = c_0(v + w)^k + c_2(2^d)^k$. Для достаточно больших значений k будет выполняться

$$t(k) < C(2^d)^k = C(2^k)^d,$$

где C — некоторое положительное число. С помощью предложения 25.1(i) получаем

$$f(n) < C(2^{\lceil \log_2 n \rceil})^d < C(2^{\log_2 n + 1})^d = C2^d n^d,$$

откуда $f(n) = O(n^d)$.

Случай $d < \log_2(v + w)$. Вновь имеем $t(k) = c_0(v + w)^k + c_2(2^d)^k$, но теперь для достаточно больших значений k будет выполняться

$$t(k) < C(v + w)^k = C(2^{\log_2(v+w)})^k = C(2^k)^{\log_2(v+w)},$$

где C — некоторое положительное число. С помощью предложения 25.1(i) получаем

$$f(n) < C(2^{\lceil \log_2 n \rceil})^{\log_2(v+w)} \leq C(2^{\log_2 n + 1})^{\log_2(v+w)} = C(v + w)n^{\log_2(v+w)}.$$

Приходим к оценке $f(n) = O(n^{\log_2(v+w)})$. □

Похожая теорема может быть доказана и для случая неравенства со знаком $\geq$ вместо знака $\leq$.

Теорема 26.2 (о рекуррентном неравенстве, случай $\geq$). Пусть вещественная функция $f(n)$ натурального аргумента удовлетворяет неравенству

$$f(n) \geq \begin{cases} u, & \text{если } n = 1, \\ uf\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + wf\left(\left\lceil \frac{n}{2} \right\rceil\right) + cn^d, & \text{если } n > 1, \end{cases}$$

где u, v, w, d — неотрицательные вещественные числа, причем $u + w \geq 1$; c — положительное вещественное число. Тогда

$$f(n) = \begin{cases} \Omega(n^d \log n), & \text{если } d = \log_2(v + w), \\ \Omega(n^{\log_2(v+w)}), & \text{если } d > \log_2(v + w), \\ \Omega(n^d), & \text{если } d < \log_2(v + w). \end{cases} \quad (26.4)$$

Доказательство отличается от доказательства теоремы 26.1 лишь тем, что в случаях $d \neq \log_2(v + w)$ вместо выбора в (26.3) слагаемого,

содержащего степень двойки с большим показателем, мы выбираем то слагаемое, которое содержит степень двойки с меньшим показателем. Вместо части (i) предложения 25.1 пользуемся его частью (ii). Мы учли также, что c_0 в (26.3) может быть равным 0. $\square$

Пример 26.1. Для исследования сложности $\tilde{T}_{MS}(n)$ по числу перемещений рассмотрим рекуррентное уравнение

$$\tilde{T}_{MS}(n) = \begin{cases} 0, & \text{если } n = 1, \\ \tilde{T}_{MS}\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + \tilde{T}_{MS}\left(\left\lceil \frac{n}{2} \right\rceil\right) + n, & \text{если } n > 1 \end{cases} \quad (26.5)$$

(при слиянии массивов, содержащих соответственно $\left\lfloor \frac{n}{2} \right\rfloor$ и $\left\lceil \frac{n}{2} \right\rceil$ элементов, требуется n перемещений элементов). Уравнение (26.5) представим как систему двух неравенств со знаками соответственно $\leq$ и $\geq$. Применяя к ним теоремы 26.1 и 26.2 ($v + w = 2$, $\log_2(v + w) = d = 1$), из первой теоремы получаем $\tilde{T}_{MS}(n) = O(n \log n)$, из второй — $\tilde{T}_{MS}(n) = \Omega(n \log n)$. В итоге имеем $\tilde{T}_{MS}(n) = \Theta(n \log n)$. Аналогичным образом представим в виде системы двух неравенств уравнение (25.14). Заменим в правой части $n - 1$ на n в случае $\leq$ и на $\frac{n}{2}$ в случае $\geq$. Очевидно, что полученные неравенства являются следствиями исходных. С помощью теорем 26.1 и 26.2 (вновь $v + w = 2$, $\log_2(v + w) = d = 1$) получаем $T_{MS}(n) = O(n \log n)$ и $T_{MS}(n) = \Omega(n \log n)$. В итоге имеем $T_{MS}(n) = \Theta(n \log n)$.

Сложность рекурсивной сортировки слияниями как по числу сравнений, так и по числу перемещений элементов массива допускает оценку $\Theta(n \log n)$.

Пример 26.2. Вернемся к алгоритму RS бинарного возведения в степень n (пример 4.2), который легко можно записать рекурсивно, различая случаи нечетного и четного n . Применяя теоремы 26.1 и 26.2 к

$$T_{RS}(n) \leq \begin{cases} 0, & \text{если } n = 1, \\ T_{RS}\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 2, & \text{если } n > 1, \end{cases} \quad (26.6)$$

и

$$T_{RS}(n) \geq \begin{cases} 0, & \text{если } n = 1, \\ T_{RS}\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 1, & \text{если } n > 1 \end{cases} \quad (26.7)$$

(теперь $v + w = 1$, $\log_2(v + w) = d = 0$), получим $T_{RS}(n) = O(\log n)$ и $T_{RS}(n) = \Omega(\log n)$. В итоге получаем уже известную нам асимптотическую оценку $T_{RS}(n) = \Theta(\log n)$.

Пример 26.3. Известен алгоритм построения выпуклой оболочки объединения двух выпуклых многоугольников, имеющих соответственно n_1 и n_2 вершин, со сложностью $O(n_1 + n_2)$ по общему числу операций, $n_1 + n_2$ рассматривается как размер входа этого алгоритма (см. задачу 87). Стратегия «разделяй и властвуй» позволяет исходя из этого описать алгоритм построения выпуклой оболочки данного множества n точек, сложность которого по общему числу операций при выборе n в качестве размера входа определяется соотношением

$$T(n) = \begin{cases} 0, & \text{если } n = 1, \\ T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + O(n) & \text{при } n \rightarrow \infty. \end{cases}$$

От этого соотношения мы переходим к неравенству

$$T(n) \leq \begin{cases} 0, & \text{если } n = 1, \\ T\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + T\left(\left\lceil \frac{n}{2} \right\rceil\right) + cn, & \text{если } n > 1, \end{cases}$$

где c — некоторое неизвестное нам положительное число; мы пользуемся здесь тем, что если $g(n) = O(h(n))$ и $h(n) > 0$ при $n > 1$, то $g(n) \leq ch(n)$ для некоторой константы c и всех $n > 1$.

Применяем теорему 26.1 ($v + w = 2$, $\log_2(v + w) = 1$, $d = 1$). Это дает $T(n) = O(n \log n)$. Мы имеем, таким образом, еще один алгоритм построения выпуклой оболочки n заданных точек, по общему числу операций имеющий сложность $O(n \log n)$.

Теоремы о рекуррентных неравенствах в некоторых книгах формулируются иначе (см. задачу 195). Иногда¹ в условии этой теоремы предполагается, что исследуемая сложность является неубывающей функцией от n . Перед тем как применять теорему в таком ее виде к сложности конкретного алгоритма, необходимо доказывать неубывание этой сложности. Такое неубывание не является самоочевидным фактом для алгоритмов, построенных по стратегии «разделяй и властвуй». Например, для бинарного алгоритма вычисления a^n это не так: $T_{RS}(7) = 4$, $T_{RS}(8) = 3$. В теоремах 26.1 и 26.2 неубывание не предполагается.

Еще одно замечание. В § 9 мы получали формулы и оценки для сложностей алгоритма бинарного поиска места элемента, сортировки фон Неймана и т. д. путем сравнения возникающих величин с последовательностью 2^l , $l = 0, 1, \dots$; этот прием во многом аналогичен использованию соотношений вида (25.2), (25.5), но в том лишь частном случае, когда одно из чисел v, w равно нулю.

¹ См., например, [5].

§ 27. Упрощающее добавление нулей

Ряд алгоритмов целочисленной арифметики и теории матриц оказывается достаточным (без нанесения сколь-либо существенного ущерба идее алгоритма и его качеству) описать для случая, когда размер входа есть число вида 2^k . При использовании стратегии «разделяй и властвуй» это облегчает и описание алгоритма, и анализ его сложности. С теоретической точки зрения для задачи умножения двух целых чисел a и b мы можем предполагать, что битовая длина каждого из данных чисел равна 2^k , где

$$k = \max\{\lceil \log_2 \lambda(a) \rceil, \lceil \log_2 \lambda(b) \rceil\}, \quad (27.1)$$

так как всегда возможно добавить спереди любого из данных чисел некоторое количество нулей. Если речь идет об умножении квадратных матриц A и B произвольного порядка n , то мы можем добавить к матрицам несколько нулевых строк и столбцов так, чтобы сделать их порядки равными $2^{\lceil \log_2 n \rceil}$:

$$\begin{pmatrix} A & 0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} B & 0 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} AB & 0 \\ 0 & 0 \end{pmatrix} \quad (27.2)$$

(нулями обозначены нулевые блоки соответствующего размера). Несмотря на переход от начальных данных к более громоздким, некоторые из алгоритмов, основанных на стратегии «разделяй и властвуй» и использующих этот переход, имеют существенно меньшую сложность, чем наивные алгоритмы.

Предложение 27.1. Пусть вещественная функция f натурального аргумента такова, что $f(n) = f(2^{\lceil \log_2 n \rceil})$ для всех $n \in \mathbb{N}^+$ и

$$f(2^k) \leq \begin{cases} u, & \text{если } k = 0, \\ wf(2^{k-1}) + \varphi(2^k), & \text{если } k > 0, \end{cases}$$

при $k \in \mathbb{N}$, где u, w — вещественные числа, причем $u \geq 0, w \geq 1$, а $\varphi(n)$ — неотрицательная функция, определенная для всех $n \in \mathbb{N}^+$. Тогда при всех $n \in \mathbb{N}^+$ выполняется неравенство

$$f(n) \leq \begin{cases} u, & \text{если } n = 1, \\ wf\left(\left\lceil \frac{n}{2} \right\rceil\right) + \varphi(2^{\lceil \log_2 n \rceil}), & \text{если } n > 1. \end{cases} \quad (27.3)$$

Доказательство. Легко видеть, что

$$\left\lceil \log_2 \left\lceil \frac{n}{2} \right\rceil \right\rceil = \lceil \log_2 n \rceil - 1. \quad (27.4)$$

В самом деле, если $2^{k-1} < n \leq 2^k$, $k > 1$, то $2^{k-2} < \left\lceil \frac{n}{2} \right\rceil \leq 2^{k-1}$, т. е. если $\lceil \log_2 n \rceil = k$, то $\left\lceil \log_2 \left\lceil \frac{n}{2} \right\rceil \right\rceil = k - 1$. Для случая $n = 2^{\lceil \log_2 n \rceil}$ неравенство (27.3) выполнено по условию, для остальных случаев используем равенства $f(n) = f(2^{\lceil \log_2 n \rceil})$, $f\left(\left\lceil \frac{n}{2} \right\rceil\right) = f(2^{\lceil \log_2 \lceil n/2 \rceil \rceil}) = f(2^{\lceil \log_2 n \rceil - 1})$. $\square$

Теорема 27.1. Пусть вещественная функция f натурального аргумента такова, что $f(n) = f(2^{\lceil \log_2 n \rceil})$ для всех $n \in \mathbb{N}^+$ и

$$f(2^k) \leq \begin{cases} u, & \text{если } k = 0, \\ wf(2^{k-1}) + c(2^k)^d, & \text{если } k > 0, \end{cases}$$

при $k \in \mathbb{N}$, где $u, d \geq 0$, $c > 0$, $w \geq 1$. Тогда выполнено

$$f(n) = \begin{cases} O(n^d \log n), & \text{если } d = \log_2 w, \\ O(n^d), & \text{если } d > \log_2 w, \\ O(n^{\log_2 w}), & \text{если } d < \log_2 w. \end{cases}$$

Доказательство следует из предложения 27.1 и теоремы 26.1. $\square$

Подобно тому, как доказательство теоремы 26.1 было преобразовано в доказательство теоремы 26.2, из приведенного выше доказательства мы можем получить доказательство следующей теоремы

Теорема 27.2. Пусть вещественная функция f натурального аргумента такова, что $f(n) = f(2^{\lceil \log_2 n \rceil})$ для всех $n \in \mathbb{N}^+$ и

$$f(2^k) \geq \begin{cases} u, & \text{если } k = 0, \\ wf(2^{k-1}) + c(2^k)^d, & \text{если } k > 0, \end{cases}$$

где $u, d \geq 0$, $c > 0$, $w \geq 1$. Тогда выполнено

$$f(n) = \begin{cases} \Omega(n^d \log n), & \text{если } d = \log_2 w, \\ \Omega(n^d), & \text{если } d \neq \log_2 w. \end{cases}$$

Перейдем к примерам.

Пример 27.1 (умножение Карацубы). Пусть a и b — целые положительные числа битовой длины $m = 2^k$. Положив $l = 2^{k-1}$, можем записать

$$a = e2^l + f, \quad b = g2^l + h,$$

где e, f, g, h — целые числа битовой длины l . В статье 1962 г.¹ А. А. Карацубы и Ю. П. Офмана содержится замечательное наблюдение, поз-

¹ См. [24].

воляющее вычислить произведение ab , выполнив всего три умножения чисел половинной длины, несколько сдвигов (домножений на 2^m и 2^l) и несколько аддитивных операций над числами битовой длины $\leq 2m$. Так, например,

$$ab = eg2^{2l} + ((e + f)(g + h) - eg - fh)2^l + fh, \quad (27.5)$$

тогда как обычное раскрытие скобок в $(e2^l + f)(g2^l + h)$ требует выполнения четырех таких умножения:

$$ab = eg2^{2l} + (eh + fg)2^l + fh. \quad (27.6)$$

Мы видим, что формула (27.5) использует произведения eg , fh , $(e + f)(g + h)$, а формула (27.6) — произведения eg , eh , fg , fh .

Небольшая проблема, которая выше была замаскирована словами «половинная длина», состоит в том, что битовая длина любого из чисел $e + f$, $g + h$, входящих в произведение $(e + f)(g + h)$, может оказаться равной $l + 1$, а не l . Но если

$$e + f = e_12^l + f_1, \quad g + h = g_12^l + h_1,$$

где e_1, g_1 — однобитовые числа (0 или 1), то

$$(e + f)(g + h) = e_1g_12^{2l} + (e_1h_1 + g_1f_1)2^l + f_1h_1. \quad (27.7)$$

Произведение f_1h_1 вычисляется рекурсивным обращением к алгоритму, произведения e_1g_1, e_1h_1, g_1f_1 , как и все сложения и сдвиги, требуют $O(l)$ операций.

Равенство (27.5) и предположение, что $m = 2^k$, приводят к рекурсивному алгоритму, названному алгоритмом Карацубы, умножения целых положительных чисел (будем обозначать этот алгоритм буквами КМ: первая из этих букв — начальная в фамилии автора алгоритма, вторая — в английском слове multiplication — умножение). Предположение $m = 2^k$ приводит к следующему соотношению для битовой сложности умножения Карацубы:

$$T_{\text{КМ}}(m) \leq \begin{cases} 1, & \text{если } m = 1, \\ 3T_{\text{КМ}}\left(\frac{m}{2}\right) + cm, & \text{если } m > 1, \end{cases} \quad (27.8)$$

где c — некоторая положительная константа.

Умножение Карацубы при произвольном входе $a, b \in \mathbb{N}^+$ размера $m = \max\{\lambda(a), \lambda(b)\}$ предполагает, что сначала мы находим $k = \lceil \log_2 m \rceil$, затем добавляем спереди каждого из a, b некоторое количество нулей так, чтобы битовая длина каждого из сомножителей стала равной 2^k , а после этого используем рекурсивный алгоритм, основанный на (27.5).

Мы можем применить теорему 27.1 ($w = 3$, $d = 1$), так как при произвольном $m \in \mathbb{N}^+$ выполняется $T_{\text{KM}}(m) = T_{\text{KM}}(2^{\lceil \log_2 m \rceil})$. Получаем

$$T_{\text{KM}}(m) = O(m^{\log_2 3}), \quad (27.9)$$

при этом $\log_2 3 = 1,58\dots$

Для $m > 1$ мы имеем

$$T_{\text{KM}}(m) > G(m), \quad (27.10)$$

где функция G натурального аргумента такова, что $G(m) = G(2^{\lceil \log_2 m \rceil})$ для всех $m \in \mathbb{N}^+$ и

$$G(2^k) = \begin{cases} 1, & \text{если } k = 0, \\ 3G(2^{k-1}), & \text{если } k > 0, \end{cases}$$

откуда

$$G(m) = 3^{\lceil \log_2 m \rceil} \geq 3^{\log_2 m} = m^{\log_2 3}.$$

Получаем

$$T_{\text{KM}}(m) = \Theta(m^{\log_2 3}). \quad (27.11)$$

При использовании m , равного максимальной из битовых длин двух данных чисел $a, b \in \mathbb{N}^+$, в качестве размера входа битовая сложность умножения Карацубы допускает оценку

$$T_{\text{KM}}(m) = \Theta(m^{\log_2 3}),$$

в то время как алгоритм наивного умножения имеет сложность $\Theta(m^2)$ (при этом $\log_2 3 = 1,58\dots$)¹.

Историко-библиографическая справка. У статьи [24] два автора — А. А. Карацуба и Ю. П. Офман, но результаты авторов в статье разделены, формулы быстрого умножения отнесены к результатам А. А. Карацубы. Возможность быстрого умножения рассматривается в статье как следствие возможности быстрого возведения в квадрат:

$$ab = \frac{1}{4}((a+b)^2 - (a-b)^2), \quad (27.12)$$

и А. А. Карацубой предложен алгоритм такого возведения. Дадим поддержку из статьи (изменив лишь обозначения, — будем придерживаться обозначений, введенных выше в тексте этого параграфа): «Формула

$$(e2^l + f)^2 = 2^{l-4}(e+f)^2 + (2^{2l} - 2^{l-4})e^2 + (1 - 2^{l-4})f^2 \quad (27.13)$$

¹ История создания алгоритма Карацубы и публикации о нем в 1962 г. сообщены [24] увлекательно рассказана в статье [23] самого А. А. Карацубы; особенно богат яркими историческими деталями раздел 6 этой статьи.

показывает, что возведение в квадрат $2l$ -значных чисел сводится к трем возведениям в квадрат l -значных чисел и операциям (сложение, умножение на степень двойки), осуществление которых можно произвести весьма экономно...»

Формула (27.5) получается из (27.12) и (27.13). Она показывает, что умножение двух $2l$ -значных чисел сводится к трем умножениям l -значных чисел и — заимствуем слова из статьи Карацубы и Офмана — операциям (сложение, умножение на степень двойки), осуществление которых можно произвести весьма экономно.

Стратегия добавления нулей особенно характерна для исследований, в которых главной целью служит преодоление некоторого сложностного барьера; последнее было и остается важным стимулом развития теории сложности.

Пример 27.2. Алгоритм Штрассена умножения двух квадратных числовых матриц A и B порядка n , являющегося степенью двойки, основан на том, что если $n = 2l$ и

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix}, \quad B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix},$$

где все A_{ij}, B_{ij} — квадратные матрицы порядка l , то матрицу

$$C = AB = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$$

можно получить, выполнив семь умножений квадратных матриц порядка l (при том, что потребовалось бы восемь таких умножений при использовании простейшего алгоритма, основанного на определении произведения матриц):

$$\begin{aligned} X_1 &= (A_{11} + A_{22})(B_{11} + B_{22}), & X_5 &= (A_{11} + A_{12})B_{22}, \\ X_2 &= (A_{21} + A_{22})B_{11}, & X_6 &= (A_{21} - A_{11})(B_{11} + B_{12}), \\ X_3 &= A_{11}(B_{12} - B_{22}), & X_7 &= (A_{12} - A_{22})(B_{21} + B_{22}), \\ X_4 &= A_{22}(B_{21} - B_{11}), \end{aligned}$$

далее используются только аддитивные операции:

$$\begin{aligned} C_{11} &= X_1 + X_4 - X_5 + X_7, & C_{12} &= X_3 + X_5, \\ C_{21} &= X_2 + X_4, & C_{22} &= X_1 + X_3 - X_2 + X_6. \end{aligned}$$

В правильности этого можно убедиться прямой проверкой.

Равенство $n = 2^k$ создает возможность рекурсивного применения алгоритма. (Обратим внимание, что алгоритм не использует коммута-

тативность умножения.) Алгоритм Штрассена будем обозначать начальными буквами St фамилии его автора.

В приведенных формулах использовано восемнадцать сложений матриц порядка l . Сложение двух матриц порядка l требует l^2 сложений чисел. В предположении, что $n = 2^k$, $k \in \mathbb{N}$, имеем для общего числа операций — сложений и умножений чисел:

$$T_{\text{St}}(n) = \begin{cases} 1, & \text{если } n = 1, \\ 7T_{\text{St}}\left(\frac{n}{2}\right) + 18\left(\frac{n}{2}\right)^2, & \text{если } n > 1. \end{cases} \quad (27.14)$$

Если $n \in \mathbb{N}^+$ произвольно, то вначале к матрицам A и B добавляются нулевые строки и столбцы (см. (27.2)) так, чтобы порядки матриц стали равными $2^{\lceil \log_2 n \rceil}$, а затем применяется описанный рекурсивный алгоритм. Рассматривая равенство (27.14) как систему двух неравенств со знаками $\leq$ и $\geq$ и применяя теоремы 27.1 и 27.2, получаем следующий результат.

Сложность $T_{\text{St}}(n)$ по числу арифметических операций алгоритма Штрассена перемножения двух числовых матриц порядка n допускает оценку $T_{\text{St}}(n) = \Theta(n^{\log_2 7})$, в то время как алгоритм, непосредственно следующий из определения произведения матриц, имеет сложность $\Theta(n^3)$ (при этом $\log_2 7 = 2,80735\dots$).

Что касается булевых матриц, то алгоритм Штрассена не может быть непосредственно применен для их умножения по той, например, причине, что этим алгоритмом используется вычитание, для которого нет аналога в булевой арифметике. Но матрицы A и B порядка n , состоящие из нулей и единиц, можно перемножить как целочисленные. Каждый элемент такого произведения не превосходит n , он равен нулю, если и только если соответствующий элемент произведения булевых матриц равен нулю. Для того, чтобы в процессе применения алгоритма Штрассена к целочисленным матрицам не возникало больших промежуточных значений, здесь можно все вычисления проводить по модулю $n + 1$, т. е. проводить вычисления не в кольце $\mathbb{Z}$, а в кольце $\mathbb{Z}_{n+1}$. Если $M(n)$ — некоторая верхняя граница числа битовых операций, затрачиваемых при выполнении одной операции сложения, вычитания или умножения в $\mathbb{Z}_{n+1}$, то сложность модифицированного таким способом алгоритма Штрассена будет допускать оценку $O(n^{\log_2 7} M(n))$. Наивное умножение в $\mathbb{Z}_{n+1}$ дает $M(n) = O(\log^2 n)$. Таким образом, $M(n)$ растет очень медленно в сравнении с остальными затратами. Так как $\log_2 7 = 2,81\dots$, мы можем использовать для сложности алгоритма Штрассена, модифицированного на булев слу-

чай, например, оценку $O(n^{2,82})$ или оценку $\tilde{O}(n^{\log_2 7})$, — смысл « O мягкого» объяснялся в конце § 22.

Применение алгоритма Штрассена и арифметики по модулю $n + 1$ дает алгоритм умножения двух булевых матриц порядка n , битовая сложность которого допускает оценки $\tilde{O}(n^{\log_2 7})$, $O(n^{2,82})$.

Мы ограничились рассмотрением применения стратегии «разделяй и властвуй» для случая, когда в результате этапа разделения возникают две задачи, и каждый из размеров входа примерно вдвое меньше изначального размера. Иногда разделение приводит к трем и более задачам.

В 1963 г. А. Л. Тоом обобщил идею умножения Карацубы¹. Пусть s — большее единицы целое. Предполагая, что битовая длина m каждого из сомножителей имеет вид s^k , $k \in \mathbb{N}$, последовательность двоичных цифр каждого из сомножителей можно разбить на s групп по s^{k-1} цифр. Тоом показал, что умножение исходных чисел сводится к $2s - 1$ умножениям чисел битовой длины s^{k-1} (в умножении Карацубы $s = 2$, $2s - 1 = 3$), остальные затраты — сложения, сдвиги — будут ограничены функцией cm , где c — зависящая от выбора s константа. Здесь этап разделения приводит к $2s - 1$ задачам. Для умножения Тоома (ТМ) неравенство

$$T_{\text{ТМ}}^{(s)}(m) \leq \begin{cases} 1, & \text{если } m = 1, \\ (2s - 1)T_{\text{ТМ}}^{(s)}\left(\frac{m}{s}\right) + cm, & \text{если } m > 1, \end{cases} \quad (27.15)$$

выполненное в случае $m = s^k$, $k \in \mathbb{N}$, и равенство

$$T_{\text{ТМ}}^{(s)}(m) = T_{\text{ТМ}}^{(s)}(s^{\lceil \log_s m \rceil}),$$

выполненное при произвольном $m \in \mathbb{N}^+$, приводят к оценке $T_{\text{ТМ}}^{(s)}(m) = O(m^{\log_s(2s-1)})$ для битовой сложности алгоритма, использующего разбиение на s частей. Может быть также показано, что

$$T_{\text{ТМ}}^{(s)}(m) = \Theta(m^{\log_s(2s-1)}). \quad (27.16)$$

Очевидно,

$$\log_s(2s - 1) = \log_s 2s \left(1 - \frac{1}{2s}\right) = 1 + \log_s 2 + \log_s \left(1 - \frac{1}{2s}\right).$$

Отсюда

$$\lim_{s \rightarrow \infty} \log_s(2s - 1) = 1.$$

¹ См. [51], [5].

Это означает, что для любого $\varepsilon > 0$ можно найти целое $s \geq 2$ такое, что умножение Тоома с разделением на s частей (битовая длина числа предполагается равной s^k , $k \in \mathbb{N}$) будет иметь битовую сложность, допускающую оценку $\Theta(m^{1+\delta})$ при некотором δ таком, что $0 < \delta \leq \varepsilon$; разумеется, для битовой сложности этого алгоритма справедлива оценка $O(m^{1+\varepsilon})$.

Скажем коротко об основной идее алгоритма Тоома, приводящей к неравенству (27.15). Если, как предполагалось, битовая длина каждого из сомножителей a, b есть s^k , $k \in \mathbb{N}$, то последовательность двоичных цифр каждого из сомножителей a, b можно разбить на s групп по s^{k-1} цифр:

$$a_{s-1}, \dots, a_1, a_0; \quad b_{s-1}, \dots, b_1, b_0.$$

Сами сомножители a, b суть значения полиномов

$$A(x) = a_{s-1}x^{s-1} + \dots + a_1x + a_0, \quad B(x) = b_{s-1}x^{s-1} + \dots + b_1x + b_0$$

в точке $x_0 = 2^{s^{k-1}}$. Полином $C(x)$, равный произведению $A(x)B(x)$, есть полином степени не выше чем $2s - 2$ (мы не утверждаем, что эта степень равна $2s - 2$, так как возможно, что к изначально заданным целочисленным сомножителям спереди дописывались нули), и достаточно знать значения $C(x)$ в $2s - 1$ точках (узлах интерполяции) для того, чтобы затем, например, с помощью интерполяционной формулы Лагранжа найти значение

$$C(2^{s^{k-1}}), \tag{27.17}$$

равное ab . Тоом показал, что если в качестве узлов интерполяции $x_1, x_2, \dots, x_{2s-1}$ взять числа

$$-(s-1), -(s-2), \dots, -1, 0, 1, \dots, s-2, s-1,$$

рекурсивно с помощью рассматриваемого алгоритма найти

$$A(x_i), \quad B(x_i), \quad C(x_i) = A(x_i)B(x_i), \quad i = 1, 2, \dots, 2s-1,$$

и затем, пользуясь интерполяционной формулой Лагранжа, найти значение (27.17), то это приведет к (27.15), (27.16). Такое использование интерполяции включает в себе требуемое обобщение алгоритма Карацубы.

В 1972 г. Шёнхаге и Штрассен, основываясь на идее Тоома использования интерполяции полиномов в алгоритмах умножения целых чисел, получили алгоритм умножения, битовая сложность которого допускает оценку $O(m \log m \log \log m)$, мы уже упоминали этот алгоритм в § 21. Функция $m \log m \log \log m$ растет медленнее, чем $m^{1+\delta}$ при любом $\delta > 0$. Улучшение достигнуто за счет привлечения интер-

поляции специального вида, участвующей в так называемом быстром преобразовании Фурье¹. Шёнхаге на основе этого алгоритма умножения предложил алгоритм² нахождения $\text{нод}(a_0, a_1)$, сложность которого допускает оценку $O(m \log^2 m \log \log m)$, где m — битовая длина большего числа a_0 (в примере 21.2 было показано, что алгоритм Евклида имеет сложность $\Theta(m^2)$). И как мы уже говорили, в 2007 г. М. Фюрер опубликовал алгоритм со сложностью $O(m \log m f(m))$, где $f(m)$ — некоторая функция, растущая медленнее, чем $\log \log m$ и, более того, медленнее, чем любой повторный логарифм³.

Возник вопрос, можно ли продвинуться дальше, — существует ли, например, алгоритм умножения, имеющий сложность $O(m \log m)$? В 2018 г. Дж. Харли и Й. ван дер Ховен объявили, что ими найден такой алгоритм. В переводе на русский название их публикации в интернете [72] — «Целочисленное умножение за время $O(n \log n)$ »; в этой статье для наибольшей из битовых длин двух сомножителей авторы используют обозначение n .

При всем сказанном преимущества по времени выполнения рассмотренных алгоритмов перед наивным умножением и алгоритмом Евклида проявляются лишь при очень больших значениях m .

С умножением матриц положение таково, что обобщения алгоритма Штрассена в духе обобщения, предложенного Тоомом для алгоритма Карацубы, до сих пор не найдено. В 2014 г. Ф. Ле Галль опубликовал алгоритм⁴, имеющий сложность $O(n^{2,3728642})$, где n — порядок перемножаемых квадратных матриц. Этот результат остается рекордным по сей день. Существует ли для любого $\varepsilon > 0$ алгоритм умножения матриц со сложностью $O(n^{2+\varepsilon})$ — это открытый вопрос.

Касаясь идеи алгоритма Штрассена, прежде всего отметим, что доказана невозможность алгоритма умножения (2×2) -матриц, требующего менее семи умножений. Для случая некоммутативного кольца, к которому принадлежат элементы матриц, это сделано в [72], затем в [87] этот результат был перенесен на коммутативный случай. Если существуют какие-то очень быстрые способы перемножения (3×3) - или (4×4) -матриц, то, возможно, удалось бы прийти к алгоритму, применимому к квадратным матрицам произвольного размера n , имеющему сложность

$$O(n^\omega), \quad \omega < \log_2 7. \quad (27.18)$$

¹ Об алгоритме Шёнхаге—Штрассена см., например, [8, разд. 7.5].

² См., например, [8, разд. 8.10].

³ См. [68, 69].

⁴ См. [70].

Можно установить (см. задачу 197), что для (3×3) -матриц подошло бы 21 умножение, но 22 умножения не позволяют получить (27.18), так как $\log_3 22 = 2,81358809... > \log_2 7$. Вместе с этим, алгоритм, в основе которого 23 умножения для (3×3) -матриц, рассматривался в [78].

Остается неясным, можно ли умножать (3×3) -матрицы, затрачивая менее двадцати трех умножений в том кольце, к которому принадлежат элементы матриц. Компьютерные вычисления, обсуждавшиеся в [54], позволили найти более тысячи новых методов с 23 умножениями, но не дали к настоящему моменту метода с 22 умножениями, не говоря уже о меньшем числе (21, например).

Задачи

164. Игра «Ханойские башни». К горизонтальной доске приделаны три вертикальных столбика. На первый нанизано n дисков, диаметры которых убывают вверх (рис. 21). Требуется, перекладывая диски

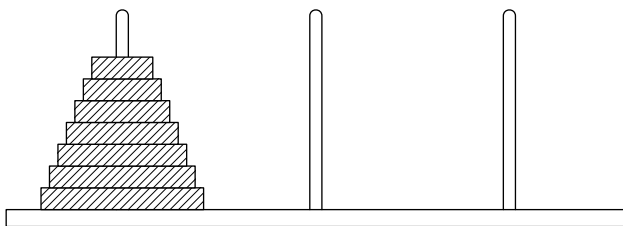


Рис. 21. Игра «Ханойские башни». Исходное положение.

по одному, расположить их в прежнем порядке на третьем столбике. Ограничение состоит в том, что больший диск никогда не может располагаться над меньшим. Разработать рекурсивный алгоритм решения этой задачи и указать его временную сложность по числу перекладываний дисков.

165. (Продолжение предыдущей задачи.) Требуется определить сложность алгоритма перекладывания дисков в игре «Ханойские башни» при условии, что затраты на перекладывание со столбика на столбик i -го по величине диска равны а) i ; б) i^2 ; в) 2^i .

166. Электрическое соединение таково, что для горения лампы нужно, чтобы все входящие в соединение n выключателей находились в состоянии «включено». У каждого выключателя есть кнопка, нажатие которой меняет состояние выключателя — «включено» на «выключено» и наоборот. Внешне состояния выключателя не опозна-

ются. Отсутствуют сведения о состояниях выключателей в исходный момент времени, но лампа не горит. Требуется нажатиями на кнопки добиться ее горения. Предложить алгоритм выполнения поставленного задания, требующий в худшем случае $2^n - 1$ нажатий.

Указание. Естественными для алгоритма будут рекурсивные вызовы — переходы к рассмотрению задания с меньшим значением n . Например, могут быть полезными два вызова алгоритма с $n - 1$. (Такой алгоритм дополнительно способен распознавать ситуацию «что-то не так», когда никакая комбинация состояний выключателей не обеспечивает горения лампы.) Для числа нажатий в худшем случае y_n возникает рекуррентное уравнение вида $y_n = ay_{n-1} + b$, $y_1 = c$. Подобрать нужные a, b, c и найти решение.

167. Для обсуждавшихся в предыдущей задаче алгоритмов включения лампочки функция $2^n - 1$ является нижней границей их сложности: не существует алгоритма, требующего в худшем случае менее $2^n - 1$ нажатий.

168. Сформулировать правило нахождения частного решения рекуррентного уравнения $a_d y(n) + a_{d-1} y(n-1) + \dots + a_0 y(n-d) = f(n)$ для случая, когда $f(n)$ принимает постоянное значение c .

169. Пусть функция $f(n)$ определена для всех $n \in \mathbb{N}^+$ и пусть a — ненулевое число. Любое определенное для всех $n \in \mathbb{N}$ решение рекуррентного уравнения $y(n) - ay(n-1) = f(n)$ имеет вид $y(n) = a^n \left(y(0) + \sum_{k=1}^n \frac{f(k)}{a^k} \right)$.

Указание. Индукция по n .

170. Найти общее решение рекуррентного уравнения

$$\text{а) } y(n) - 2y(n-1) + y(n-2) = -\frac{n+1}{4}2^n,$$

$$\text{б) } y(n) - y(n-1) - 2y(n-2) = 6(-1)^n.$$

171. Для чисел Фибоначчи выполнено равенство $\sum_{i=1}^n F_i = F_{n+2} - 1$, $n = 1, 2, 3, \dots$

Указание. Решением рекуррентного уравнения $y(n) - y(n-1) = F_n$, $y(0) = 1$, является $y(n) = F_{n+2}$.

172. Верно ли, что для любого полинома $p(n)$ найдется полином $q(n)$ (оба с вещественными коэффициентами) такой, что $q(n+1) - q(n) = p(n)$, и если да, то как различаются степени полиномов $p(n)$ и $q(n)$?

173. Найти множество всех вещественных последовательностей, удовлетворяющих рекуррентному уравнению $y(n+1) + y(n) = 0$.

174. а) Арифметическая прогрессия $v_n = a + (n - 1)d$, $n = 1, 2, \dots$, при $d \neq 0$ не удовлетворяет никакому линейному однородному уравнению $fv_{n+1} + gv_n = 0$ первого порядка с постоянными коэффициентами.

б) Эта прогрессия удовлетворяет однородному уравнению 2-го порядка $v_{n+2} - 2v_{n+1} + v_n = 0$.

в) Для любого целого $m > 1$ существует линейное однородное уравнение порядка m с постоянными коэффициентами, которому удовлетворяет эта прогрессия.

175. Вернемся к алгоритму (24.2). Пусть $1 \leq l \leq n$. Сколько раз при вычислении Y_n будет вычислено Y_l ?

Ответ. F_{n-l+1} .

176. Рассмотрим построение множества V_n всех целых чисел, десятичная запись каждого из которых содержит только цифры 1 и 2, а сумма цифр равна заданному числу $n \geq 1$. Непосредственно видно, что $V_1 = \{1\}$, $V_2 = \{11, 2\}$, $V_3 = \{111, 21, 12\}$. В общем случае V_n является, очевидно, объединением двух непересекающихся подмножеств, в первое из которых входят все числа из V_n , оканчивающиеся цифрой 1, во второе — цифрой 2. Если вычеркнуть последнюю цифру, то при $n \geq 2$ первое подмножество превратится в V_{n-1} , второе — в V_{n-2} . Это наблюдение приводит к рекурсивному алгоритму V_{REC} построения V_n : если $n = 1$ или $n = 2$, то результатом будет $\{1\}$ или соответственно $\{11, 2\}$; иначе надо найти V_{n-1} (рекурсивное обращение к алгоритму) и приписать справа ко всем числам этого множества единицу, затем найти V_{n-2} (еще одно рекурсивное обращение) и приписать справа ко всем числам этого множества двойку, после чего построить объединение двух получившихся множеств.

Установить, чему равно число элементов V_n . Определить количество преобразований чисел при построении множества V_n по описанному алгоритму (каждое из этих преобразований состоит в приписывании справа к числу единицы или двойки) и количество операций объединения множеств.

177. Назовем перестановку $z_1, z_2, \dots, z_n$ чисел $1, 2, \dots, n$ критической, если на ней достигается максимум числа сравнений, требуемых рекурсивной сортировкой слияниями для массивов длины n . Пусть $n > 1$, а $x_1, \dots, x_{\lfloor n/2 \rfloor}$ и $y_1, \dots, y_{\lfloor n/2 \rfloor}$ суть некоторые критические перестановки длины $\left\lfloor \frac{n}{2} \right\rfloor$ и $\left\lceil \frac{n}{2} \right\rceil$. Тогда числа $z_1, z_2, \dots, z_n$, равные соответственно

$$2x_1, \dots, 2x_{\lfloor n/2 \rfloor}, 2y_1 - 1, \dots, 2y_{\lfloor n/2 \rfloor} - 1$$

образуют критическую перестановку длины n (рис. 22).

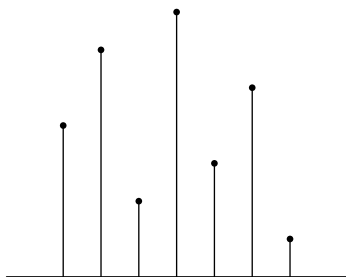


Рис. 22. Случай, когда при $n = 7$ для сортировки слияниями потребуется максимальное число сравнений.

178. Как выглядят критические перестановки длины 6, 7, 8 и 9, полученные с помощью предложенного в предыдущей задаче подхода?

179. Имеет место равенство (25.14).

Указание. См. задачу 177.

180. Продолжение задачи 177, в которой, фактически, в рекурсивной форме описан алгоритм построения некоторой критической перестановки длины n . Исследовать сложности описанного алгоритма построения некоторой критической перестановки длины n по числу умножений на два и по числу вычитаний единицы. Предложить нерекурсивный алгоритм (например, можно рассмотреть последовательное построение критических перестановок, длины которых суть соответственно $1, 2, \dots, n$) и исследовать его сложности по названным операциям.

181. Чему равно наименьшее n вида 2^k , для которого число сравнений при рекурсивной сортировке слияниями превосходит $\lceil \log_2 n! \rceil$?

182. Написать ассоциированное уравнение для (26.5), найти его решение и получить верхнюю и нижнюю оценки вида (25.13) для $\tilde{T}_{MS}(n)$.

183. Вернемся к бинарному алгоритму вычисления a^n . Для его сложности по числу умножений имеем (26.6), (26.7). Написать ассоциированные уравнения, найти их решения и получить верхнюю и нижнюю оценки для $T_{RS}(n)$.

184. Оценки, полученные при решении предыдущей задачи, являются следствием оценки $T_{RS}(n) = \lambda(n) + \lambda^*(n) - 2$.

185. Для сложности алгоритма бинарного поиска места элемента в упорядоченном массиве обосновать рекуррентное неравенство

$$T_{BS}(n) \leq \begin{cases} 1, & \text{если } n = 1, \\ T_{BS}\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + 1, & \text{если } n > 1, \end{cases}$$

не прибегая к известному явному выражению для $T_{BS}(n)$. Из этого рекуррентного неравенства получить оценки для $T_{BS}(n)$ как с помощью предложения 25.1, так и с помощью теорем о рекуррентном неравенстве.

186. Верно ли, что сложность (число сравнений в худшем случае) $T_{BS}(n)$ бинарного поиска места элемента в упорядоченном массиве длины n удовлетворяет рекуррентному соотношению

$$\text{а) } T_{BS}(n) = \begin{cases} 1, & n = 1, \\ T_{BS}\left(\left\lceil \frac{n}{2} \right\rceil\right), & n > 1; \end{cases} \quad \text{б) } T_{BS}(n) < \begin{cases} 2, & n = 1, \\ 2T_{BS}\left(\left\lceil \frac{n}{2} \right\rceil\right), & n > 1; \end{cases}$$

187. Предложить алгоритм построения пересечения n полуплоскостей

$$a_i x + b_i y + c_i \geq 0, \quad i = 1, 2, \dots, n,$$

имеющий временную сложность $\Theta(n \log n)$ по общему числу операций.

Указание. Пересечением будет выпуклый многоугольник (возможно, неограниченный). Опираясь на алгоритм Шеймоса—Хоя (задача 22), использовать стратегию «разделяй и властвуй».

188. Пусть $t(k)$, $k = 0, 1, \dots$, удовлетворяет рекуррентному уравнению

$$t(k) = \begin{cases} \alpha, & \text{если } k = 0, \\ \beta t(k-1) + \gamma \mu^k, & \text{если } k > 0, \end{cases}$$

в котором $\alpha, \beta, \gamma \neq 0$ и $\beta > \mu$. Можно ли утверждать, что соотношение $t(k) = \Theta(\beta^k)$ обязательно выполняется?

189. По сравнению с теоремами 26.1, 26.2 предложение 25.1 представляет собой более сильное утверждение (меньше ограничений на правую часть $\varphi(n)$). Используя это предложение, получить асимптотические оценки для

$$f(n) = \begin{cases} 0, & \text{если } n = 1, \\ f\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + f\left(\left\lceil \frac{n}{2} \right\rceil\right) + \varphi(n), & \text{если } n > 1, \end{cases}$$

при $\varphi(n) = \log_2 n$ и $\varphi(n) = n \log_2 n$.

190. Рекуррентное неравенство вида (26.1) может иметь более одного решения, однако во всех трех случаях, предусмотренных формулой (26.2), по крайней мере для одного решения соответствующая оценка из (26.2) является точной.

191. Показать, что следствием (27.12) и (27.13) является формула

$$ab = (e2^l + f)(g2^l + h) = (2^{2l} + 2^l)eg + 2^l(e - f)(h - g) + (2^l + 1)fh,$$

которая дает, в некотором смысле, более прямой путь применения алгоритма Карацубы в сравнении с (27.5): отпадает необходимость сохранять и повторно использовать eg и fh (но в (27.5) проще выглядят дополнительные операции).

192. Доказать соотношение (27.10).

Указание. Рассмотреть количество умножений чисел битовой длины 1 в процессе применения алгоритма Карацубы к входу размера m и показать, что $G(m)$ является для него нижней оценкой.

193. Можно ли подобрать целое положительное s такое, что для сложности умножения Тоома с разбиением чисел на s частей выполнено $T_{\text{TM}}^{(s)}(m) = O(m^{1,01})$?

194. а) Для любого $\varepsilon > 0$ можно указать $N > 0$ такое, что $3T_{\text{KM}}(m) < T_{\text{KM}}(2m) < (3 + \varepsilon)T_{\text{KM}}(m)$ при всех $m \geq N$.

б) Для любого $\varepsilon > 0$ можно указать $N > 0$ такое, что $7T_{\text{St}}(n) < T_{\text{St}}(2n) < (7 + \varepsilon)T_{\text{St}}(n)$ при всех $n \geq N$.

Указание. См. (27.10), (27.8) и (27.14).

195. Теоремы о рекуррентных неравенствах часто формулируются иначе. Например, в [5] и [8] теорема дается, с точностью до обозначений и используемых слов, в следующем виде. Пусть s — целое ≥ 2 , и при любом n вида s^k , $k = 1, 2, \dots$, вещественная функция $f(n)$ натурального аргумента удовлетворяет неравенству

$$f(n) \leq wf\left(\frac{n}{s}\right) + cn^d,$$

где w, c, d — константы, причем $w > 0, c > 0, d \geq 0$. Пусть при этом $f(n)$ не убывает на каждом отрезке $[s^{k-1} + 1, s^k]$. Тогда

$$f(n) = \begin{cases} O(n^d \log n), & \text{если } d = \log_s w, \\ O(n^d), & \text{если } d > \log_s w, \\ O(n^{\log_s w}), & \text{если } d < \log_s w. \end{cases}$$

Доказать эту теорему. (Заметим, что в условии теоремы 26.1 нет требования неубывания $f(n)$ на каждом отрезке $[s^{k-1} + 1, s^k]$, но зато требуется, чтобы, например, неравенство (26.1) выполнялось для всех n , а не только для n вида 2^k .)

196. Произведение двух данных комплексных чисел $(a+bi)(c+di) = (ac-bd) + (ad+bc)i$ можно вычислить с помощью нескольких сложений и вычитаний и лишь трех умножений вещественных чисел¹.

Указание. Можно начать вычисления с $v_1 = a(d - c)$, $v_2 = c(a + b)$, $v_3 = b(d + c)$.

197. Как показано в тексте, алгоритм Штрассена позволяет перемножать $(n \times n)$ -матрицы с элементами, принадлежащими некоторому кольцу R (не обязательно коммутативному), используя $O(n^\omega)$, $\omega = \log_2 7$, умножений элементов R . Можно уменьшить значение ω , если найти способ перемножения (3×3) -матриц за 21 умножение или способ перемножения (4×4) -матриц за 41 умножение.

¹ Невозможность алгоритма, использующего только два умножения вещественных чисел, показана в [87, разд. 4].

Глава 7

Сводимость

В этой главе мы рассматриваем только временную сложность алгоритмов и считаем, что размер входа — это неотрицательное целое число.

§ 28. Линейная сводимость

Ниже в сопоставляемых друг с другом вычислительных задачах подразумеваются преобразования каких-то математических объектов, представляемых одинаково для всех задач, участвующих в сопоставлении. Размер входа для алгоритмов решения рассматриваемых задач тоже должен определяться единообразно. Затраты на выполнение вычислений измеряются количеством операций из одного и того же набора (например, арифметических операций, битовых операций и т. д.).

Определение 28.1. Пусть P и Q — две вычислительные задачи. Если любому алгоритму A_Q решения задачи Q можно сопоставить такой алгоритм A_P решения задачи P , что

$$T_{A_P}(n) = O(T_{A_Q}(n)), \quad (28.1)$$

то говорят, что задача P *линейно сводится* к задаче Q , и пишут $P \leq Q$. В специально оговариваемых случаях утверждение $P \leq Q$ предполагает, что рассматриваются лишь такие алгоритмы A_Q , сложности которых удовлетворяют некоторым фиксированным условиям.

Слово «линейно» в этом определении означает лишь то, что сложность получаемого алгоритма A_P не является, скажем, квадратом, кубом и т. д. сложности алгоритма A_Q . Присутствие условий (ограничений) на сложности алгоритмов решения задачи Q часто облегчает или просто делает возможным доказательство равенства (28.1). В то же время, предполагается, что эти ограничения отсекают нерациональные алгоритмы; тогда смысл отношения $P \leq Q$ состоит в том,

что каждому приемлемому («хорошему») алгоритму решения задачи Q мы можем сопоставить алгоритм решения задачи P такой, что выполнено (28.1).

Пример 28.1. Будем рассматривать задачи умножения M и возведения в квадрат S неотрицательных целых чисел, заданных в двоичной системе, считая размером входа в первой задаче максимальную из битовых длин чисел n_1, n_2 , а во второй — битовую длину данного числа n (будем обозначать этот размер через m), в качестве затрат будем рассматривать битовые затраты.

Из равенства $n^2 = n \cdot n$ следует, что $S \leq M$, так как это равенство дает требуемый определением 28.1 алгоритм. Дает ли равенство

$$n_1 \cdot n_2 = \frac{(n_1 + n_2)^2 - n_1^2 - n_2^2}{2} \quad (28.2)$$

основание считать, что $M \leq S$? Битовая длина числа $n_1 + n_2$ может оказаться равной $m + 1$. Если бы для возведения в квадрат использовался некоторый чрезвычайно нерациональный алгоритм, имеющий, скажем, сложность $m!$, то соответствующий алгоритм умножения имел бы сложность порядка $(m + 1)!$. Равенство $(m + 1)! = O(m!)$ места не имеет. Можно попытаться определить умножение через возведение в квадрат как-то иначе, но можно и не отказываться от формулы (28.2): при установлении линейной сводимости обычно считается, что сложность любого приемлемого алгоритма выполнения какой-либо мультипликативной арифметической операции (умножения, возведения в квадрат, деления и т. д.) удовлетворяет, хотя бы для всех достаточно больших m , условиям

$$1) T(m) \geq m,$$

$$2) T(m) \text{ не убывает при возрастании } m,$$

$$3) T(2m) \leq 4T(m)$$

(условие 3 отсекает алгоритмы умножения, возведения в квадрат и т. д., сложность которых растет быстрее чем m^2). Приняв, что сложность алгоритма A_S удовлетворяет этим условиям, мы имеем

$$T_{A_M}(m) \leq T_{A_S}(m + 1) \leq T_{A_S}(2m) \leq 4T_{A_S}(m),$$

что дает $T_{A_M}(m) = O(T_{A_S}(m))$ для описанного с помощью (28.2) алгоритма A_M умножения. Мы используем здесь то, что операции вычитания и деления на 2 имеют сложность $O(m)$, и то, что $T_{A_S}(m) \geq m$ по условию 1.

Таким образом, если принимаются ограничения 1—3 для сложностей алгоритмов решения задачи S , то $M \leq S$.

Пример 28.2. Из формулы (23.2) нельзя сделать вывода, что задача построения транзитивно-рефлексивного замыкания графа с n вершинами (или, в другой терминологии, транзитивно-рефлексивного замыкания булевой матрицы порядка n) линейно сводится к задаче умножения двух булевых матриц порядка n , так как само число умножений матриц в этой формуле зависит от n . Но мы покажем, что соответствующая линейная сводимость имеет место, если принять, что для любого из рассматриваемых алгоритмов умножения булевых матриц порядка n его сложность $B(n)$ (примем это упрощенное обозначение) удовлетворяет нижеследующим условиям 1), 2), 3), причем для условий 2) и 3) будет достаточно, если они удовлетворяются только для всех достаточно больших значений n :

- 1) $B(1) = 1$,
- 2) $B(n)$ не убывает при возрастании n ,
- 3) $4B(n) \leq B(2n) \leq 8B(n)$.

Комментарий к условию 3. Неравенство $B(2n) \leq 8B(n)$ отсекает те алгоритмы умножения булевых матриц, сложность которых растет быстрее чем n^3 . Алгоритм должен построить n^2 элементов матрицы-произведения, поэтому его сложность не может расти медленнее чем n^2 , и в соответствии с этим в ограничения включается неравенство $4B(n) \leq B(2n)$.

Для булевой матрицы C порядка n будем, как и прежде, обозначать ее транзитивно-рефлексивное замыкание через C^* .

Пусть G — ориентированный граф с n вершинами и C — его матрица смежности. Предположим вначале, что n — четное число: $n = 2l$. Разобьем множество вершин $V = \{v_1, v_2, \dots, v_{2l}\}$ графа G на два подмножества $V_1 = \{v_1, v_2, \dots, v_l\}$ и $V_2 = \{v_{l+1}, v_{l+2}, \dots, v_{2l}\}$, а множество ребер графа G — на четыре подмножества $E_{11}, E_{12}, E_{21}, E_{22}$: начало каждого из ребер из множества E_{pq} принадлежит V_p , а конец — V_q , где $p, q \in \{1, 2\}$. Если исходная матрица C представлена в блочном виде

$$\begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix},$$

где каждый блок — это матрица порядка l , то матрица C_{pq} несет полную информацию о ребрах из множества E_{pq} . Рассмотрим пути в графе G , соединяющие вершины из множества V_1 . Назовем *ходом* путь из вершины в вершину этого множества, представляющий собой

- либо продвижение по ребру, принадлежащему E_{11} ,
- либо продвижение по ребру, принадлежащему E_{12} , за которым следует некоторый путь по ребрам из E_{22} и затем продвижение по ребру, принадлежащему E_{21} .

Непосредственно видно, что (i, j) -элемент матрицы $C_{11} \vee (C_{12} \wedge C_{22}^* \wedge C_{21})$ равен 1, если и только если существует ход, соединяющий вершины $v_i, v_j \in V_1$. При этом ясно, что из v_i можно добраться в v_j ($v_i, v_j \in V_1$) по ребрам графа G , если и только если существует последовательность ходов, приводящая из v_i в v_j , т. е. если и только если (i, j) -элемент матрицы

$$(C_{11} \vee (C_{12} \wedge C_{22}^* \wedge C_{21}))^*$$

равен 1. Обозначим эту матрицу через F_{11} . Для некоторых матриц F_{12}, F_{21}, F_{22} (их вид определим ниже) имеем блочную запись матрицы C^* :

$$C^* = \begin{pmatrix} F_{11} & F_{12} \\ F_{21} & F_{22} \end{pmatrix}$$

При этом матрица F_{pq} , $p, q \in \{1, 2\}$, несет информацию о существовании путей из вершин множества V_p в вершины множества V_q . Справедливы равенства

$$F_{12} = F_{11} \wedge C_{12} \wedge C_{22}^*,$$

$$F_{21} = C_{22}^* \wedge C_{21} \wedge F_{11},$$

$$F_{22} = C_{22}^* \vee (C_{22}^* \wedge C_{21} \wedge F_{11} \wedge C_{12} \wedge C_{22}^*);$$

эти выражения удобны для вычисления матрицы C^* : четыре матрицы $F_{p,q}$, $p, q \in \{1, 2\}$, можно найти, выполнив два построения транзитивно-рефлексивных замыканий, шесть умножений и два сложения матриц порядка l :

$$X = C_{22}^*,$$

$$Y = C_{12} \wedge X,$$

$$Z = X \wedge C_{21},$$

$$F_{11} = (C_{11} \vee (Y \wedge C_{21}))^*,$$

$$F_{12} = F_{11} \wedge Y,$$

$$F_{21} = Z \wedge F_{11},$$

$$F_{22} = X \vee (F_{21} \wedge Y).$$

Мы указали рекурсивный переход от матриц порядка $2l$ к матрицам порядка l . Учитывая, что $C^* = C$ для матрицы первого порядка, мы получаем алгоритм вычисления C^* для случаев, когда порядок n матрицы равен 2^k , $k \geq 0$. Пусть для умножения булевых матриц используется алгоритм со сложностью $B(n)$, которая удовлетворяет условиям 1—3, приведенным в начале этого примера. Для $n = 2^k$ мы имеем следующее соотношение, которому будет удовлетворять сложность

$T(n)$ полученного алгоритма построения транзитивно-рефлексивного замыкания:

$$T(2^k) \leq \begin{cases} 0, & \text{если } k = 0, \\ 2T(2^{k-1}) + 6B(2^{k-1}) + 2 \cdot 2^{2(k-1)}, & \text{если } k > 0. \end{cases} \quad (28.3)$$

В силу условия 3 имеем

$$B(2^{k-1}) \geq 4B(2^{k-2}) \geq \dots \geq 4^{k-1}B(2^0) = 2^{2(k-1)}B(1)$$

при $k \geq 2$. Дополнительное использование условия 1 (т.е. условия $B(1) = 1$) дает нам $B(2^{k-1}) \geq 2^{2(k-1)}$ при $k \geq 1$. Заменяя $2^{2(k-1)}$ во второй строке соотношения (28.3) на $B(2^{k-1})$, получаем

$$T(2^k) \leq \begin{cases} 0, & \text{если } k = 0, \\ 2T(2^{k-1}) + 8B(2^{k-1}), & \text{если } k > 0. \end{cases} \quad (28.4)$$

Докажем по индукции, что

$$T(2^k) \leq 4B(2^k), \quad k = 0, 1, \dots \quad (28.5)$$

Для $k = 0$ это неравенство очевидно, так как $T(1) = 0$, $B(1) = 1$. Пусть $k > 0$ и неравенство выполнено для $k - 1$. Тогда $2T(2^{k-1}) \leq 8B(2^{k-1})$, и, как следствие (28.4), мы получаем $T(2^k) \leq 16B(2^{k-1})$. В силу условия 3 мы имеем $B(2^{k-1}) \leq \frac{1}{4}B(2^k)$, откуда следует (28.5). Таким образом, для $n = 2^k$ получили $T(n) = O(B(n))$.

Если n не есть число вида 2^k , то от матрицы C переходим к матрице

$$C' = \begin{pmatrix} C & 0 \\ 0 & I_s \end{pmatrix},$$

порядка 2^k , взяв единичную матрицу I_s некоторого порядка, меньшего чем порядок матрицы C . Таким образом, порядок матрицы C' меньше, чем $2n$, т.е. $2^k < 2n$. Из (28.5) следует, что

$$T(n) = T(2^k) \leq 4B(2^k).$$

Так как $2^k < 2n$ и функция $B(n)$ — неубывающая, получаем $4B(2^k) \leq 4B(2n)$. Отсюда

$$T(n) \leq 4B(2n) \leq 4 \cdot 8 \cdot B(n) = 32B(n)$$

для всех n и

$$T(n) = O(B(n)), \quad (28.6)$$

что и требовалось.

Из сказанного следует, что если использовать для умножения квадратных булевых матриц алгоритм со сложностью, растущей медлен-

нее, чем n^3 , и удовлетворяющей условиям 1—3, то можно строить транзитивно-рефлексивное замыкание со сложностью, растущей медленнее, чем сложность алгоритма Уоршелла (пример 23.2). Напомним, что алгоритм Штрассена умножения матриц, модифицированный для булева случая (пример 27.2), имеет сложность $\tilde{O}(n^{\log_2 7})$. Но для небольших значений n условия 1—3 здесь выполняться не будут. Однако соотношение (28.6) можно доказать при более слабых предположениях, чем использованные выше, и обсуждавшееся доказательство не потребует больших изменений. А именно, достаточно предположить, что $B(n)$ удовлетворяет условиям 2—3 для $n > N$, где N — некоторое натуральное число (см. задачу 194б). Получится оценка $T(n) \leq cB(n)$, где c зависит от $B(N)$, но эта зависимость не влияет на (28.6).

Для нахождения транзитивно-рефлексивного замыкания ориентированного графа с n вершинами существует алгоритм, сложность которого допускает оценку $O(n^{2.82})$, или, более точно, оценку $\tilde{O}(n^{\log_2 7})$.

Как показывает последний пример, если $P \leq Q$ и эта линейная сводимость конструктивна, то прогресс в умении быстро решать задачу Q может в некоторых случаях обеспечить прогресс и в умении быстро решать задачу P . Но это не всегда так. Допустим, что n является нижней границей сложности для алгоритмов решения задачи Q и в то же время для задачи P имеется алгоритм решения со сложностью, меньшей n . Согласно определению 28.1 мы имеем $P \leq Q$, но наличие этой линейной сводимости ничего не дает для продвижения в умении быстро решать задачу P .

Может быть установлена и линейная сводимость задачи умножения булевых матриц к задаче построения транзитивно-рефлексивного замыкания (см. задачу 198).

Отношение линейной сводимости, очевидно, рефлексивно. Если не рассматривать упомянутых в определении 28.1 дополнительных условий, то это отношение будет транзитивным. В тех случаях, когда такие условия наложены, необходима осмотрительность: если для задач P , Q и R мы имеем $P \leq Q$ при некоторых условиях на сложность алгоритмов решения Q , а также $Q \leq R$ при соответствующих условиях на сложность алгоритмов решения R , то чтобы на основании этого утверждать, что $P \leq R$, надо дополнительно установить, что сложности тех алгоритмов решения задачи Q , которые сопоставляются алгоритмам решения задачи R , удовлетворяют условиям, налагаемым на сложности алгоритмов решения задачи Q при доказательстве отношения $P \leq Q$. Это обстоятельство иногда безосновательно игнорируется.

§ 29. Линейная сводимость и нижние границы сложности

Теорема 29.1. Пусть задачи P и Q таковы, что $P \leq Q$. Пусть $f(n)$ — асимптотическая нижняя граница сложности алгоритмов решения задачи P , и при этом соотношение $P \leq Q$ устанавливается без наложения ограничений на сложность алгоритмов решения задачи Q . Тогда $f(n)$ является асимптотической нижней границей сложности алгоритмов решения задачи Q .

Доказательство. Для каждого алгоритма A_Q решения задачи Q найдется такой алгоритм A_P решения задачи P , для которого $T_{A_P}(n) = O(T_{A_Q}(n))$, или, эквивалентно, $T_{A_Q}(n) = \Omega(T_{A_P}(n))$. Но $T_{A_P}(n) = \Omega(f(n))$, следовательно, $T_{A_Q}(n) = \Omega(f(n))$. $\square$

Если при соблюдении условия этого предложения для какого-то алгоритма A_Q решения задачи Q имеет место оценка $T_{A_Q}(n) = O(f(n))$, то этот алгоритм будет оптимальным по порядку сложности и $T_{A_Q}(n) = \Theta(f(n))$. (При наличии ограничений на сложность алгоритма A_Q , когда фактически предполагается, что A_Q принадлежит некоторому классу $\mathcal{Q}$, речь должна бы идти об оптимальности по порядку сложности в классе $\mathcal{Q}$; но если ограничения таковы, что в класс $\mathcal{Q}$ попадают наиболее рациональные алгоритмы, то упоминание класса $\mathcal{Q}$ не обязательно.)

Пример 29.1. Вновь рассмотрим задачу построения выпуклой оболочки конечного множества точек плоскости с помощью арифметических операций и сравнений. Многоугольник, являющийся выпуклой оболочкой, представляется массивом своих вершин в порядке их следования при обходе в некотором направлении, обычно против часовой стрелки, начиная с некоторой вершины. Мы покажем, что задача сортировки массивов вещественных чисел с помощью арифметических операций и сравнений линейно сводится к задаче построения выпуклой оболочки в этой постановке, считая, что в этих задачах затраты измеряются общим числом арифметических операций и сравнений. Но если порядок вершин выпуклой оболочки, которую надо построить, может быть произвольным, то задача меняется, и, в рамках этих рассмотрений, про нее мы ничего не утверждаем (см. замечание в конце этого параграфа).

Пусть $x_1, x_2, \dots, x_n$ — данный массив попарно различных вещественных чисел. Решив задачу построения выпуклой оболочки множества точек с координатами

$$(x_1, x_1^2), (x_2, x_2^2), \dots, (x_n, x_n^2)$$

(см. рис. 23), мы можем, двигаясь по вершинам построенного многоугольника, найти вершину с наименьшей абсциссой, а затем продвигаться по вершинам в порядке возрастания абсцисс. Сложность этой дополнительной части работы допускает оценку $O(n)$ и при этом легко видеть, что сложность любого алгоритма построения выпуклой оболочки не может быть меньше чем n . Таким образом, каждому алгоритму A построения выпуклой оболочки мы сопоставляем алгоритм сортировки со сложностью $O(T_A(n))$. Следовательно, задача сортировки вещественных чисел с помощью арифметических операций и сравнений линейно сводится к задаче построения выпуклой оболочки.

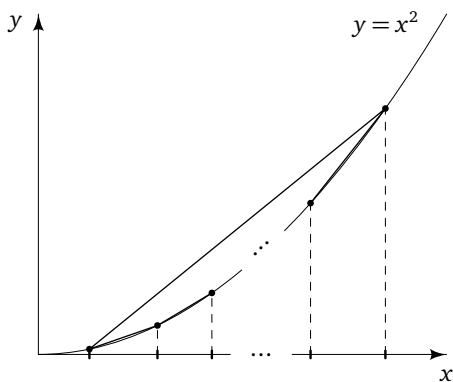


Рис. 23. Сведение сортировки чисел $x_1, x_2, \dots, x_n$, отмеченных на оси абсцисс, к построению выпуклой оболочки точек $(x_1, x_1^2), (x_2, x_2^2), \dots, (x_n, x_n^2)$.

Из результатов § 14 следует, что любая сортировка с помощью сравнений массивов длины n имеет сложность не менее $\log_2 n!$. Можно ли, привлекая помимо операций сравнения еще и арифметические операции, предложить алгоритм сортировки массивов вещественных чисел, сложность которого по числу сравнений была бы меньше, чем $\log_2 n!$, пусть даже при том, что потребовалось бы очень большое число арифметических операций? Ниже мы обосновываем отрицательный ответ на этот вопрос.

Дополнительное использование четырех арифметических операций означает, что сравниваться могут не только числа $x_1, x_2, \dots, x_n$, заданные изначально, но и значения рациональных функций от этих чисел.

Каждая рациональная функция $F(x_1, x_2, \dots, x_n)$ записывается как отношение двух полиномов

$$F(x_1, x_2, \dots, x_n) = \frac{p(x_1, x_2, \dots, x_n)}{q(x_1, x_2, \dots, x_n)} \quad (29.1)$$

(в этом параграфе мы рассматриваем рациональные функции и полиномы с вещественными коэффициентами). В частности, каждый полином $f(x_1, x_2, \dots, x_n)$ можно рассматривать как рациональную функцию

$$\frac{f(x_1, x_2, \dots, x_n)}{1}.$$

Предполагается, что если алгоритм предписывает сравнение, в котором участвует значение рациональной функции (29.1), то ее знаменатель $q(x_1, x_2, \dots, x_n)$ не обращается в 0 при рассматриваемых значениях $x_1, x_2, \dots, x_n$. Неравенство $F_1(x_1, x_2, \dots, x_n) < F_2(x_1, x_2, \dots, x_n)$ равносильно, очевидно, неравенству $F(x_1, x_2, \dots, x_n) < 0$, где

$$F(x_1, x_2, \dots, x_n) = F_1(x_1, x_2, \dots, x_n) - F_2(x_1, x_2, \dots, x_n).$$

То же самое для сравнений со знаками $>, =, \neq, \leq, \geq$.

Предложение 29.1. Функция $f(n) = \lceil \log_2 n! \rceil$ является нижней границей сложности по числу сравнений алгоритмов сортировки массивов длины n попарно различных вещественных чисел с помощью сравнений и четырех арифметических операций.

*Доказательство*¹. Точку $(x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, все компоненты которой попарно различны, будем называть *бесповторной*. Каждому массиву попарно различных вещественных чисел соответствует бесповторная точка в $\mathbb{R}^n$. Если высота дерева сортировки (см. § 14) оказалась меньше, чем $\log_2 n!$, то в этом дереве листьев меньше, чем $n!$. Следовательно, найдется лист, в который попадают бесповторные точки $(x_1^{(1)}, \dots, x_n^{(1)})$, $(x_1^{(2)}, \dots, x_n^{(2)})$, имеющие разные порядки расположения компонент. Алгоритм сортировки, рассчитывающий на «помощь» рациональных функций, в таком листе должен предложить набор рациональных функций

$$G_1(x_1, x_2, \dots, x_n), \dots, G_n(x_1, x_2, \dots, x_n), \quad (29.2)$$

который для любой бесповторной точки с числовыми компонентами $x'_1, x'_2, \dots, x'_n$, попадающей в результате предписываемых алгоритмом

¹ Идея элементарного доказательства обсуждалась автором с С. П. Поляковым. Наиболее раннее (довольно трудное для понимания) доказательство было опубликовано Н. Фридманом в [67].

сравнений в этот лист, позволяют получить компоненты этой точки в упорядоченном виде:

$$G_1(x'_1, x'_2, \dots, x'_n), \dots, G_n(x'_1, x'_2, \dots, x'_n),$$

т. е. значение $G_1(x'_1, x'_2, \dots, x'_n)$ должно быть наименьшим из $x'_1, x'_2, \dots, x'_n$, значение $G_2(x'_1, x'_2, \dots, x'_n)$ — вторым по величине и т. д. Однако рациональные функции не могут гарантировать такой результат по совокупности следующих причин:

1) Каждая неповторная точка $x' = (x'_1, x'_2, \dots, x'_n)$ обладает в $\mathbb{R}^n$ окрестностью, все точки которой имеют тот же самый порядок расположения компонент, что и x' (достаточно малое «шевеление» компонент не меняет их порядка расположения).

2) Любая рациональная функция $F(x_1, x_2, \dots, x_n)$, равная нулю на некотором непустом открытом подмножестве множества $\mathbb{R}^n$, равна нулю на всем множестве $\mathbb{R}^n$ (имеет нулевой полином в качестве числителя — см. задачу 203).

3) Если, например, значение $G_1(x'_1, x'_2, \dots, x'_n)$ равно наименьшему числу из $x'_1, x'_2, \dots, x'_n$, и если для какой-то точки наименьшее есть x'_i , то в силу двух предыдущих пунктов получаем, что рациональная функция

$$G_1(x_1, x_2, \dots, x_n) - x_i$$

будет нулевой рациональной функцией, т. е. всегда $G_1(x'_1, x'_2, \dots, x'_n) = x'_i$. Аналогичное рассуждение применимо к k -му по величине числу из $x'_1, x'_2, \dots, x'_n$ для $k > 1$. Это означает, что набор (29.2) рациональных функций — это просто $x_1, x_2, \dots, x_n$. Но такой набор не позволяет устанавливать различия порядков расположения компонент рассматриваемых неповторных точек. $\square$

Из доказанного предложения следует, что $f(n) = n \log n$ является асимптотической нижней границей сложности алгоритмов сортировки массивов длины n попарно различных вещественных чисел с помощью сравнений и четырех арифметических операций. По теореме 29.1 эта функция является также асимптотической нижней границей для алгоритмов построения выпуклой оболочки.

Любой алгоритм построения выпуклой оболочки с помощью арифметических операций и сравнений, который имеет сложность $O(n \log n)$, является оптимальным по порядку сложности по общему числу операций, и его сложность есть $\Theta(n \log n)$. Алгоритм Грэхема построения выпуклой оболочки (пример 3.1) является оптимальным по порядку сложности по общему числу операций, коль скоро используемая им сортировка имеет сложность $O(n \log n)$ по числу сравнений.

Добавим, что обсуждавшийся в примере 3.1 алгоритм Грэхема дополнительно использует сортировку углов по их величине. Но и сравнение величин углов можно производить, сравнивая длины отрезков или же квадраты этих длин, вычисляемых по теореме Пифагора.

Замечание. То, что нижняя граница сложности алгоритмов построения выпуклой оболочки совпадает с нижней границей сложности сортировки по числу сравнений, вызывает предположение, что причиной этого совпадения служит требование упорядоченности вершин выпуклой оболочки. Но если даже разрешить задавать выпуклую оболочку множеством ее вершин в произвольном порядке, то нижняя граница будет иметь вид $\Omega(n!)$ или $\Omega(n \log n)$, как прежде¹.

§ 30. Классы P и NP

Цель этого и следующих параграфов — дать общее, без деталей, представление о некоторых проблемах, касающихся алгоритмов полиномиально ограниченной сложности (см. § 5, определение 5.2) или, другими словами, полиномиальных алгоритмов.

Уже говорилось, иногда бывает очень трудно установить, существует ли для какой-то задачи полиномиальный алгоритм решения. В этой связи рассмотрим класс задач NP. В этот класс входят вычислительные задачи, где результатом может быть лишь 1 или 0, т.е. фактически «да» или «нет», что типично для задач распознавания, когда есть исходный объект x и требуется выяснить (проверить, распознать), существует ли объект y , удовлетворяющий вместе с x определенным условиям. Эти x и y в задачах класса NP представляются словами в некотором алфавите Λ . Длину слов x и y будем обозначать через $\|x\|$ и $\|y\|$. Размер входа x — это $\|x\|$, затраты измеряются числом операций над символами (по аналогии с битовыми операциями). Задаются полином $f(t)$ и двухместный предикат $U(x, y)$, определенный на словах в алфавите Λ . Предикат должен обладать тем свойством, что для слов x, y таких, что $\|y\| \leq f(\|x\|)$, значение $U(x, y)$ вычисляется за время, ограниченное некоторым полиномом от $\|x\|$. Соответствующая задача из класса NP состоит в том, чтобы выяснить, верно ли, что

$$\exists y, \|y\| \leq f(\|x\|) U(x, y) \quad (30.1)$$

для данного x . (Для многих задач класса NP предполагается, что $f(t) = t$, т.е. просто слово y не длиннее слова x .)

¹ См. [44, разд. 3.2].

Пусть, например, $\Lambda = \{0, 1\}$, и непустые слова рассматриваются как двоичные неотрицательные целые числа. Тогда, очевидно, задачу проверки, является ли данное число x составным, можно сформулировать как задачу класса NP: здесь предикат (условие) $U(x, y)$ — это

$$\llcorner 2 \leq y < x \text{ и } x \text{ делится на } y \llcorner,$$

$f(t) = t$. Проверка делимости x на y выполняется за время $O(m^2)$, где m — битовая длина числа x , т. е. длина числа x как слова в алфавите Λ .

Теперь введем в рассмотрение класс задач P. Каждая задача этого класса состоит в распознавании, обладает ли входное слово некоторым фиксированным свойством, и при этом свойство задано таким предикатом V , что значение $V(x)$ вычисляется полиномиальным алгоритмом. Легко устанавливается, что $P \subseteq NP$: мы можем, например, взять $U(x, y) = V(x)$, $f(t) = t$.

Обсуждавшаяся задача о составных числах принадлежит, как мы показали, NP. Благодаря существованию полиномиального алгоритма проверки простоты числа эта задача принадлежит также P. Но для некоторых задач из NP до сих пор неизвестно, принадлежат ли они P. Выражение (30.1) задает некоторый одноместный предикат, обозначим его через $W(x)$. Вопрос, на который ответ ясен не всегда, состоит в том, найдется ли полиномиальный алгоритм вычисления значения предиката $W(x)$. Пример задачи, для которой без труда устанавливается ее принадлежность NP, но не ясно, принадлежит ли она также и P, дает так называемая задача о выполнимости: определить для данной булевой формулы, содержащей переменные $x_1, x_2, \dots, x_n$, принимает ли она значение 1 (т. е. «истина») при каких-либо булевых значениях этих переменных. Здесь булева формула представляется словом x в алфавите $\{x, 0, 1, (,), \neg, \wedge, \vee\}$, для этого переменные $x_1, x_2, x_3, \dots$ записываются как $x1, x10, x11, \dots$: вслед за буквой x помещается номер переменной в двоичной системе. Набор значений переменных — это слово y из нулей и единиц, равное по длине числу переменных в формуле.

Нередко в обсуждениях принадлежности какой-то задачи распознавания классу NP сама эта задача трактуется как задача распознавания принадлежности заданного слова языку L — подмножеству множества Λ^* всех слов в рассматриваемом алфавите Λ , и при этом L состоит из всех слов x , для которых утверждение (30.1) выполнено (тогда слово y называют *сертификатом* слова x , хотя, возможно, логичнее было бы называть y сертификатом принадлежности слова x языку L). Мы обратимся к этому в заключительном абзаце этого параграфа и в § 32.

Если бы было доказано, что $P = NP$, то в широком спектре случаев мы бы легко устанавливали принадлежность задачи классу Р (мы устанавливали бы только *существование* полиномиального алгоритма; разработка такого алгоритма — это отдельный вопрос), так как принадлежность задачи классу NP нередко устанавливается достаточно легко. Неформально говоря, совпадение Р и NP означало бы следующее: если для любого слова y , соразмерного по длине с заданным словом x , мы умеем быстро проверять, удовлетворяет ли y вместе с x каким-то фиксированным условиям, то мы сумеем и быстро проверить, существует ли хотя бы одно слово y соответствующей длины, которое удовлетворяет вместе с x этим условиям.

Но равенство $P = NP$ по сей день не доказано, и специалисты склоняются к предположению, что оно неверно (проблема $P \stackrel{?}{=} NP$).

Разумеется, есть простой алгоритм проверки того, выполняется ли (30.1) для данного x — можно перебирать все слова y такие, что $\|y\| \leq f(\|x\|)$, проверяя каждый раз выполнение условия $U(x, y)$. Но при $\|x\| \rightarrow \infty$ сложность этого алгоритма растет экспоненциально.

Замечание об обозначениях классов. Обозначение Р происходит от английского слова *polynomial* — полином, полиномиальный. Странное же обозначение NP есть не что иное, как аббревиатура словосочетания *nondeterministic polynomial* — недетерминированный полиномиальный. Изначальное определение класса NP, появившееся в начале 70-х годов XX столетия, опиралось на понятие недетерминированного алгоритма (заметим, что если заданное слово x принадлежит языку L , то приводящая к получению значения 1 полиномиально ограниченная по длине последовательность переходов в недетерминированном алгоритме может быть представлена некоторым словом y ; это согласуется с рассмотренным нами определением класса NP на основе (30.1))¹.

§ 31. *Существование задач распознавания, не принадлежащих Р.* **Связь моделей МТ и РМ**

Возникает вопрос, существуют ли вообще задачи распознавания принадлежности слов некоторому языку, разрешимые алгоритмически, но не принадлежащие Р. Один из первых примеров задачи такого рода в 1973 г. был обнаружен М. Фишером и М. Рабином. Этот пример связан с так называемой арифметикой Пресбургера. Рассматриваются логические формулы, записанные с соблюдением обычных син-

¹ См., например, [20].

таксических правил с помощью некоторого числа переменных, знаков $+$, $=$, логических связок $\vee$, $\wedge$, $\neg$ и скобок; при этом каждая переменная должна быть связана одним из кванторов $\forall$, $\exists$, а множеством значений переменных является $\mathbb{N}$. Каждая такая формула, трактуемая как утверждение о неотрицательных целых числах, имеет значение «истина» или «ложь» (1 или 0). Арифметика Пресбургера — это совокупность всех истинных формул указанного вида. Так, формулы

$$\forall_x \exists_y (x + y = x), \quad \forall_x \forall_y (x + y = y + x) \wedge \neg (\forall_x \exists_y (x + y = y))$$

принадлежат арифметике Пресбургера, а формула $\forall_x \exists_y (x + y = y)$ — нет. Как и ранее, мы будем использовать переменные, имеющие вид буквы x , за которой следует некоторое двоичное число (номер переменной). В алфавите

$$\Lambda = \{x, 0, 1, +, =, \vee, \wedge, \neg, \forall, \exists, (,)\}$$

формула $\forall_x \exists_y (x + y = x)$ запишется в виде $\forall x1 \exists x10 (x1 + x10 = x1)$ и т. д. М. Пресбургером было в 1930 г. доказано существование алгоритма, который дает ответ на вопрос, является ли данное слово в алфавите Λ формулой арифметики Пресбургера (разумеется, основная задача, решаемая этим алгоритмом, состоит в различении синтаксически правильных формул, принимающих значение 1 и соответственно 0; слова, не являющиеся синтаксически правильными формулами, легко распознаются и отвергаются).

Теорема, доказанная в 1973 г. Фишером и Рабином, может быть сформулирована так (мы не приводим доказательства¹).

Теорема Фишера—Рабина. *Существует такая константа $c > 0$, что для любого алгоритма A проверки принадлежности слова из Λ^* множеству формул арифметики Пресбургера найдется целое $n_A \geq 0$ такое, что в Λ^* для каждого $n > n_A$ существует слово длины n , для которого алгоритму A требуется более $2^{2^{cn}}$ вычислительных шагов.*

Из теоремы Фишера—Рабина следует, что задача распознавания принадлежности формулы арифметике Пресбургера не попадает в класс P .

В доказательстве теоремы Фишера—Рабина вычислительные шаги соответствуют рассмотрению МТ в качестве модели вычислений.

С помощью модели МТ доказывается, например, и теорема о том, что если $t(n)$ — вычислимая с помощью некоторой машины Тьюринга функция натурального аргумента, принимающая натуральные значения, то существует такой язык в некотором алфавите, что сложность

¹ См. [65].

любого алгоритма (в виде машины Тьюринга) распознавания принадлежности слова этому языку будет больше $t(n)$ для всех достаточно больших n .¹ В определенном смысле эта теорема является более общей, чем теорема Фишера—Рабина, но она очень абстрактна. Теорема Фишера—Рабина примечательна тем, что в ней идет речь о некотором содержательном в математическом смысле языке (предикате).

Модель МТ, однако, выглядит избыточно затратной в сравнении с моделью РАМ в силу, например, того, что если некоторый символ хранится в ячейке a ленты и для определения хода дальнейших вычислений надо сравнить этот символ с символом, расположенным в ячейке b , которая находится далеко от ячейки a , то передвижение головки машины Тьюринга из a в b потребует большого числа тактов работы и, следовательно, больших затрат, а в случае модели РАМ сверх самой операции сравнения здесь нет затрат вообще.

Приведем соображение, которое можно оформить в виде теоремы², показывающее, что если некоторые вычисления имеют полиномиальную сложность при использовании РАМ, то они будут иметь полиномиальную сложность и при использовании МТ. Для простоты будем считать, что речь идет о преобразовании слов в алфавите $\{0, 1\}$ и соответственно о битовой сложности вычислений. Будем также считать, что для сложности $T(n)$ по числу битовых операций произвольного алгоритма и его пространственной сложности $S(n)$, измеряемой числом используемых дополнительных ячеек, в худшем случае выполняется соотношение $S(n) \leq C \cdot T(n)$ при некоторой положительной константе C .

Пусть $f: \Lambda^* \rightarrow \Lambda$ и $f_{\text{РАМ}}$ — некоторый алгоритм вычисления f с помощью РАМ, имеющий битовую сложность $T_{f_{\text{РАМ}}}(n)$, которая полиномиально ограничена по длине $n = \|x\|$ входа x (считаем, что в каждой ячейке РАМ размещается 0 или 1). Пусть $T_{f_{\text{РАМ}}}(n) < p(n)$, где p — некоторый полином. Тогда количество ячеек, используемых $f_{\text{РАМ}}$ при работе со словом x , сверх тех, в которых изначально хранится x , не превосходит $Cp(\|x\|)$. Если использовать МТ вместо РАМ, то возникающие дополнительные затраты на переходы от одной ячейки ленты к другой можно сделать меньшими, чем $\|x\| + Cp(\|x\|)$ при каждой битовой операции. Таким образом, общее число тактов работы МТ не превзойдет $p(\|x\|)(\|x\| + Cp(\|x\|))$, и сложность вычислений на МТ будет ограничена полиномом $p(n)(n + Cp(n))$.

¹ См., например, [52, разд. 2], [5, разд. 4.2]. Впервые эта теорема была доказана, вероятно, М. Блюмом.

² См. [8, разд. 1.7].

§ 32. Полиномиальная сводимость. NP-полные задачи

Мы уже упоминали, что довольно часто при обсуждении принадлежности какой-то задачи классу NP сама эта задача формулируется как задача распознавания принадлежности заданного слова языку L в рассматриваемом алфавите Λ , и при этом L состоит из всех слов x , для которых выполнено (30.1).

Пусть Λ_1, Λ_2 — некоторые алфавиты и $L_1 \subset \Lambda_1^*$, $L_2 \subset \Lambda_2^*$. Пусть задача F состоит в распознавании принадлежности произвольного слова из Λ_1^* языку L_1 , а задача G — в распознавании принадлежности произвольного слова из Λ_2^* языку L_2 .

Задача F полиномиально сводится к задаче G , если существует такая функция $\varphi: \Lambda_1^* \rightarrow \Lambda_2^*$, что временные затраты на вычисление $\varphi(x)$ ограничены некоторым полиномом от $\|x\|$ и при этом $x \in L_1 \Leftrightarrow \varphi(x) \in L_2$.

Нетрудно показать, что отношение полиномиальной сводимости транзитивно.

Неформально говоря, F полиномиально сводится к G , если любой алгоритм для G можно использовать и для F после быстрой (полиномиально ограниченной по времени) перекодировки входа.

С полиномиальной сводимостью связывается понятие NP-полной задачи. Все NP-полные задачи принадлежат NP, и каждая из этих задач замечательна тем, что к ней полиномиально сводится любая из задач класса NP. Таким образом, если бы хоть для одной из NP-полных задач удалось найти полиномиальный алгоритм решения, то это означало бы наличие полиномиальных алгоритмов решения всех задач, принадлежащих классу NP. Тем самым, это означало бы, что $P = NP$. Если хоть для одной из NP-полных задач не существует полиномиального алгоритма, то $P \neq NP$ (в силу того, что любая NP-полная задача принадлежит NP).

Сам факт существования NP-полных задач весьма нетривиален¹.

Теорема. NP-полные задачи существуют.

Существование NP-полных задач было открыто независимо С. А. Куком и Л. А. Левиным². Доказано, например, что в класс NP-пол-

¹ Мы не приводим доказательство теоремы о существовании NP-полных задач из-за того, что оно требует привлечения специфической техники доказательств утверждений о машинах Тьюринга; такого рода доказательства выглядят естественно в определенном контексте, который отличается от контекста этого курса. Отчасти это послужило и причиной того, что в § 31 мы оставили без доказательства теорему Фишера—Рабина.

² См. [33], [36]; в терминологии Л. А. Левина NP-полная задача — это универсальная задача перебора.

ных задач входит упоминавшаяся выше задача о выполнимости булевой формулы. Более того, это утверждение верно и в предположении, что формула задана в конъюнктивной нормальной форме (КНФ):

$$D_1 \wedge D_2 \wedge \dots \wedge D_m, \quad D_i = (l_{i1} \vee l_{i2} \vee \dots \vee l_{i,k_i}), \quad i = 1, 2, \dots, m, \quad (32.1)$$

при этом l_{ij} является *литералом*, т.е. некоторой переменной x_s или переменной с отрицанием $\neg x_s$. Задача выполнимости заданной в КНФ булевой формулы (в дальнейшем для краткости будем говорить о выполнимости КНФ) получила название Sat, от английского слова *satisfiability*, одно из значений которого — выполнимость¹.

Если какая-то NP-полная задача сводится к некоторой задаче $H \in \text{NP}$, то задача H тоже NP-полная, это следует из транзитивности отношения полиномиальной сводимости.

Установлением полиномиальной сводимости задачи Sat доказывался NP-полнота многих других задач из NP, среди которых, например, ряд задач на графах. При этом графы, как правило, задаются матрицами смежности. Алфавит может содержать цифры 0, 1 и какой-то знак для отделения друг от друга строк матрицы.

Пример 32.1. Покажем полиномиальную сводимость Sat к задаче о существовании в данном графе клики с m вершинами, т.е. существовании набора из m вершин, любые две из которых соединены ребром. Пусть КНФ F имеет вид (32.1). Построим граф G_F с $k_1 + k_2 + \dots + k_m$ вершинами, для которых используем обозначения

$$l_{ij}, \quad i = 1, 2, \dots, m, \quad j = 1, 2, \dots, k_i,$$

при этом вершины l_{ij} и l_{vw} соединяем ребром, если и только если выполнены два условия:

- $i \neq v$,
- конкретные литералы, скрывающиеся в (32.1) за обозначениями l_{ij} и l_{vw} , не являются отрицанием друг друга (скажем, если l_{ij} — это $\neg x_1$, а l_{vw} — это x_1 , то эти две вершины ребром не соединяются).

Построение матрицы смежности графа G_F может быть выполнено за полиномиально ограниченное время.

Из определения клики и способа построения графа G_F следует, что клика из m вершин в этом графе существует, если и только если формула (32.1) выполнима. В самом деле, при наличии такой клики полагаем $x_s = 0$, когда литерал $\neg x_s$ соответствует одной из вершин

¹ Прозрачное изложение идей доказательства принадлежности задачи Sat классу NP можно найти в [15]. См. также [20], [5], [31].

клики, а в остальных случаях $x_s = 1$. В результате каждое D_i в (32.1) равно 1.

Наоборот, если заданная формула (32.1) выполнима, то при соответствующих значениях всех переменных $x_1, x_2, \dots, x_n$ для каждого $i \leq m$ можно выбрать j такое, что l_{ij} — это литерал со значением 1. Сделав такой выбор, мы получаем набор вершин, образующий клику с m вершинами.

Если исходная КНФ имеет вид

$$(\neg x_1 \vee \neg x_2) \wedge (x_2) \wedge (x_1 \vee x_2),$$

то мы получаем граф G_F с пятью вершинами (рис. 24), в котором обнаруживается клика (l_{11}, l_{21}, l_{32}) с тремя вершинами. Это соответствует тому, что исходная формула принимает значение 1 при $x_1 = 0, x_2 = 1$.

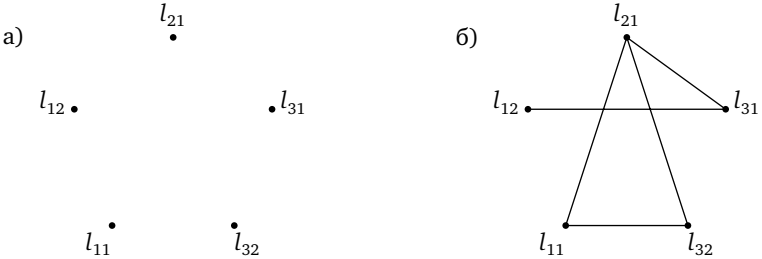


Рис. 24. Построение графа G_F для $F = (\neg x_1 \vee \neg x_2) \wedge (x_2) \wedge (x_1 \vee x_2)$: а) выбор вершин, б) проведение ребер.

Доказано также, что задача распознавания гамильтоновости графа является NP-полной¹. Упомянем еще одну очень известную NP-полную задачу, называемую задачей о рюкзаке. Задано конечное множество U , размер $s(u) \in \mathbb{N}^+$ и стоимость $v(u) \in \mathbb{Z}^+$ каждого элемента $u \in U$, а также $a, b \in \mathbb{Z}^+$. Существует ли такое подмножество $U' \subset U$, что $\sum_{u \in U'} s(u) \leq a$, $\sum_{u \in U'} v(u) \geq b$?

Для решения проблемы $P \stackrel{?}{=} NP$ достаточно со всей тщательностью рассмотреть какую-нибудь одну NP-полную задачу, например Sat, и ответить на вопрос о ее принадлежности классу P. Но этот заманчивый план до сих пор реализовать не удалось, усилия многих исследователей не привели к решению этой проблемы, хотя, как уже было сказано, и устоялось мнение, что, скорее всего, $P \neq NP$. Это предположение

¹ Огромное число примеров NP-полных задач собрано в [20].

влечет за собой рекомендацию: если доказано, что решаемая практическая задача (в надлежащей формализации в виде задачи распознавания принадлежности слова языку) является NP-полной, то не стоит рассчитывать на нахождение полиномиального алгоритма ее решения.

Заметим в заключение, что быстрый алгоритм проверки существования какого-то математического объекта может в некоторых случаях позволить быстро решать и задачу фактического построения этого объекта. Проиллюстрируем это примером.

Пример 32.2. Мы знаем, что задача проверки простоты натурального числа n принадлежит классу P, — алгоритм Агравала, Кайала и Саксены (пример 22.2) имеет битовую сложность $O(m^{11})$, где m — битовая длина n . Вопрос о существовании полиномиального алгоритма факторизации (разложения на простые множители) остается без ответа до сих пор, при том что алгоритмы факторизации имеют большую важность, например, для криптографии¹.

Легко показать, что следующая задача принадлежит NP: для заданных $n, k \in \mathbb{N}^+$, $k < n$, выяснить, имеется ли у числа n делитель l такой, что $1 < l \leq k$. Полиномиальный (по битовой длине m числа n) алгоритм решения этой задачи тоже неизвестен, но открытие такого алгоритма, назовем его A , автоматически дало бы полиномиальный алгоритм факторизации (кстати сказать, существование A автоматически следовало бы из равенства $P = NP$, если бы оно вдруг было доказано). Для этого можно воспользоваться бинарным поиском.

В самом деле, пусть уже установлено, что n не имеет делителей, меньших n_1 , где $n_1 < n$, и пусть n_2 таково, что $n_1 < n_2 < n$, и мы интересуемся наименьшим принадлежащим отрезку $[n_1, n_2]$ простым множителем числа n . Тогда, применяя A к n, n_3 , где $n_3 = \left\lfloor \frac{n_1 + n_2}{2} \right\rfloor$, мы сузим диапазон поиска примерно вдвое: в зависимости от результата применения A мы перейдем от отрезка $[n_1, n_2]$ либо к отрезку $[n_1, n_3]$, либо к отрезку $[n_3 + 1, n_2]$. Первоначально же полагаем $n_1 = 2, n_2 = n$. Применив алгоритм A не более $m = \lceil \log_2(n + 1) \rceil$ раз, мы найдем наименьший простой множитель t числа n . Повторяем те же вычисления для

$$n' = \frac{n}{t} \quad (32.2)$$

и т. д. Общее число простых множителей числа n с учетом их кратности ограничено сверху величиной $\log_2 n$ и, значит, величиной m . Бито-

¹ См. [18, разд. 5], [12, гл. 4].

вые затраты каждого деления (32.2) не превосходят Cm^2 , где C — некоторая константа. Если битовая сложность алгоритма A есть $O(m^d)$, то сложность описанного алгоритма факторизации будет допускать оценку $O(m^{d+2})$, т. е. этот алгоритм будет полиномиальным.

Задачи

198. Задача умножения квадратных булевых матриц линейно сводится к задаче построения транзитивно-рефлексивного замыкания (предполагается, что для сложностей рассматриваемых алгоритмов построения транзитивно-рефлексивного замыкания выполнено $T(3n) = O(T(n))$).

Указание. Пусть M_1 и M_2 — две булевы матрицы порядка n . Пусть X — булева матрица порядка $3n$:

$$\begin{pmatrix} 0 & M_1 & 0 \\ 0 & 0 & M_2 \\ 0 & 0 & 0 \end{pmatrix}.$$

Чему равны X^2, X^3 ? Воспользоваться формулой $X^* = I_{3n} \vee X \vee \dots \vee X^{3n-1}$ для транзитивно-рефлексивного замыкания.

199. Какие из следующих оценок допускает сложность по числу булевых операций любого из алгоритмов умножения булевых $(n \times n)$ -матриц:

- а) $\Theta(n^3)$, б) $O(n^3)$, в) $\Omega(n)$, г) $\Omega(n^{3/2})$, д) $O(n \log n)$,
 е) $\Omega(n^{\log_2 3})$, ж) $\Theta(n^{\log_2 7})$, з) $\Omega(n^2)$, и) $\Omega(n^3)$, к) $\Theta(n)$,
 л) $O(n^{\log_2 6\sqrt{2}})$, м) $\Omega(n^{5/3})$?

200. Существует ли алгоритм построения транзитивно-рефлексивного замыкания графа с n вершинами, основанный на булевых операциях и имеющий по числу этих операций сложность а) $O(n^{1.99})$, б) $O(n^{2.99})$?

201. Здесь речь идет о линейной сводимости $P \leq Q$ задач, связанных с мультипликативными операциями над квадратными числовыми матрицами порядка n . Рассматриваются лишь такие алгоритмы решения задачи Q , для сложности по числу арифметических операций каждого из которых выполняется соотношение $T(kn) = O(T(n))$, $k = 2, 3$.

Требуется показать, что задача умножения произвольных квадратных матриц линейно сводится к задаче

- а) умножения симметричных квадратных матриц;
 б) умножения верхних треугольных матриц;
 в) обращения невырожденных матриц.

Указание. Так же, как в предыдущей задаче, здесь можно прибегнуть к матрицам размера, большего n , используя исходные матрицы как блоки для построения новых матриц. В пункте в) полезно предварительно установить вид матрицы

$$\begin{pmatrix} I_n & M_1 & 0 \\ 0 & I_n & M_2 \\ 0 & 0 & I_n \end{pmatrix}^{-1},$$

где M_1, M_2 — исходные матрицы, I_n — единичная матрица порядка n .

202. Существует ли алгоритм построения выпуклой оболочки множества n точек на плоскости, основанный на арифметических операциях и сравнениях и имеющий сложность по числу этих операций а) $O(n \log \log n)$, б) $O(n^{1,001})$?

203. Доказать свойство 2 рациональных функций, сформулированное в § 29.

Указание. Достаточно доказать, что если полином $p(x_1, x_2, \dots, x_n)$ тождественно равен нулю на некотором непустом открытом множестве $U \subset \mathbb{R}^n$, то этот полином нулевой. При $n = 1$ утверждение очевидно. Пусть $n > 1$ и точка $v = (v_1, v_2, \dots, v_n) \in \mathbb{R}^n$ такова, что $p(v_1, v_2, \dots, v_n) \neq 0$. Пусть $u \in U$. Множество U — открытое, поэтому у точки u существует окрестность некоторого радиуса $r > 0$, целиком принадлежащая U . Пусть l — расстояние от u до v , а $c_1, c_2, \dots, c_n$ — координаты вектора единичной длины, направленного из u в v . Если t пробегает множество $\mathbb{R}$, то формулы

$$x_1 = u_1 + c_1 t, \quad x_2 = u_2 + c_2 t, \quad \dots, \quad x_n = u_n + c_n t \quad (32.3)$$

задают прямую в $\mathbb{R}^n$, причем при $t = 0$ получается точка u , а при $t = l$ — точка v . Остается рассмотреть для полинома $\bar{p}(t)$ одной переменной t , получающегося подстановкой (32.3) в $p(x_1, x_2, \dots, x_n)$, его значения в точке $t = l$ и на интервале $-r < t < r$.

204. Для каких целых $n \geq 1$ справедливо, что если некоторый полином с вещественными коэффициентами от $x_1, x_2, \dots, x_n$ обращается в нуль на бесконечном подмножестве множества $\mathbb{R}^n$, то этот полином является нулевым?

205. Функция $f(n) = \lceil \log_2(n+1) \rceil$ является нижней границей сложности по числу сравнений алгоритмов поиска места элемента в упорядоченном массиве длины n попарно различных вещественных чисел с помощью сравнений и четырех арифметических операций.

206. Пусть известен алгоритм, который по данным s, m , $s > 1$, $m \in \mathbb{N}^+$, строит m значащих двоичных цифр числа $\frac{1}{s}$ (построить m значащих цифр некоторого числа x , $0 < x < 1$, — это в данном

контексте означает отыскать первую ненулевую цифру после запятой в двоичной записи этого числа, а затем отбросить все цифры после m цифр, отсчитанных от найденной), и пусть сложность этого алгоритма есть $O(f(m))$, где $f(m)$ — некоторая функция такая, что дополнительно известен алгоритм умножения произвольных $a, b \in \mathbb{N}^+$, сложность которого тоже есть $O(f(m))$, где $m = \max\{\lambda(a), \lambda(b)\}$. На основе этих двух алгоритмов сконструировать алгоритм построения частного и остатка от деления положительных целых a и b , имеющий сложность $O(f(m))$, $m = \max\{\lambda(a), \lambda(b)\}$.

Указание. Нужно построить q и r такие, что $a = qb + r$, $0 \leq r < b$, или $a \frac{1}{b} = q + s$, $0 \leq s < 1$. Возникновение погрешности при вычислении $\frac{1}{b}$ может привести к тому, что найденное q будет отличаться на 1 от точного значения; несколько добавочных проб помогут найти точные q и r , не изменяя оценки $O(f(m))$ для сложности.

207. Пусть $c > 0$ и y_0 удовлетворяет неравенствам $\frac{1}{2c} \leq y_0 \leq \frac{1}{c}$; пусть последовательность $y_0, y_1, y_2, \dots$ получена по рекуррентной формуле

$$y_i = 2y_{i-1} - cy_{i-1}^2, \quad i = 1, 2, \dots$$

Тогда последовательность $y_0, y_1, \dots$ сходится к $\frac{1}{c}$.

208. (Продолжение предыдущей задачи.) Пусть $c \in \mathbb{N}^+$. Справедливо следующее утверждение¹. Пусть y_0 удовлетворяет неравенствам $\frac{1}{2c} \leq y_0 \leq \frac{1}{c}$ и последовательность $y_0, y_1, y_2, \dots$ получена по рекуррентной формуле

$$y_i = 2\tilde{y}_{i-1} - c_i\tilde{y}_{i-1}^2, \quad i = 1, 2, \dots, \quad (32.4)$$

где

- целое c_i таково, что $\lambda(c_i) = \lambda(c)$, и если $\lambda(c) > 2^i$, то первые 2^i цифр числа c_i совпадают с соответствующими цифрами числа c , а последующие цифры суть нули, если же $\lambda(c) \leq 2^i$, то $c_i = c$;
- $\tilde{y}_0$ получается из y_0 отбрасыванием всех цифр после первой значащей цифры;
- после вычисления значения y_i , $i > 0$, по рекуррентной формуле (32.4) в нем отбрасываются все цифры, идущие после первых 2^i значащих цифр, — это дает значение $\tilde{y}_i$.

Тогда первые $2^i - 3$ значащие цифры числа y_i совпадают с соответствующими цифрами числа $\frac{1}{c}$ при $i > 1$. Считая этот факт установ-

¹ См. [8, разд. 8.2].

ленным и используя решение задачи 206, доказать, что задача деления одного целого числа на другое с остатком линейно сводится к задаче умножения двух целых чисел. (Размером входа считается $m = \max\{\lambda(a), \lambda(b)\}$, где a и b — исходные числа, при этом считаем, что m есть число вида 2^k ; всюду подразумеваются битовые затраты; если это нужно, можно считать, что сложность $f(m)$ умножения удовлетворяет условиям $f(m) \leq f(2m) \leq 4f(m)$).

Указание. Соответствующий алгоритм построения частного и остатка уже описан в этой задаче и задаче 206. Достаточно доказать, что алгоритм приближенного обращения s , описанный в этой задаче, имеет сложность $R(2^k)$ такую, что $R(2^k) \leq \gamma f(2^k)$ для некоторой константы γ . Подобрать γ так, чтобы доказательство проводилось индукцией, и индуктивный переход был основан на неравенстве

$$R(2^k) \leq R(2^{k-1}) + 2f(2^{k-1}) + \delta 2^{k-1},$$

где константа δ определяется, в частности, тем, какой алгоритм сложения чисел используется.

209. Верно ли, что для доказательства того, что $P = NP$, достаточно показать, что хотя бы одна задача из NP принадлежит P ?

210. Существуют ли в NP задачи, не являющиеся NP -полными?

211. Если бы оказалось, что полиномиального алгоритма распознавания простоты натурального числа не существует (забудем об алгоритме Агравала, Кайала и Саксены), то из этого бы следовало, что $P \neq NP$.

212. Дизъюнктивная нормальная форма (ДНФ) определяется как

$$C_1 \vee C_2 \vee \dots \vee C_m, \quad C_i = (l_{i1} \wedge l_{i2} \wedge \dots \wedge l_{ik_i}), \quad i = 1, 2, \dots, m,$$

при этом каждое l_{ij} является литералом. Задача выполнимости ДНФ принадлежит P .

213. Найти ошибку или пробел в следующем доказательстве того, что $Sat \in P$. Очевидно, что любую КНФ можно преобразовать в эквивалентную ДНФ (см. задачу 212), поэтому задача выполнимости КНФ сводится к задаче выполнимости ДНФ, а эта задача принадлежит P .

214. Для выполнимой булевой формулы назовем соответствующий набор значений переменных *выполняющим*. Если $P = NP$, то существует полиномиальный алгоритм, который строит выполняющий набор для данной булевой формулы, если эта формула выполнима, и пустое слово, если формула невыполнима.

Приложение А

Некоторые алгоритмы сортировки и поиска

А1. Сортировка

Для простоты считаем, что требуется упорядочить по возрастанию числовой массив $x_1, x_2, \dots, x_n$ с попарно различными элементами. Размер входа — число n . Мы называем *сегментом* массива $x_1, x_2, \dots, x_n$ любую его часть $x_i, x_{i+1}, \dots, x_{k-1}, x_k$, $1 \leq i \leq k \leq n$, которая по условию или по построению является упорядоченной.

1. Пузырьковая сортировка. Последовательным просмотром всех $x_1, x_2, \dots, x_n$ определяется x_i такое, что $x_i > x_{i+1}$; затем x_i и x_{i+1} меняются местами, просмотр продолжается с элемента x_{i+1} и т.д. Тем самым в результате первого просмотра всего массива наибольший элемент передвинется на последнее место. Следующие просмотры начинаются опять сначала, после уменьшения на единицу количества просматриваемых элементов. Массив будет упорядочен после просмотра, который охватывал только первый и второй элементы, или же раньше, если при некотором просмотре не обнаружено x_i такого, что $x_i > x_{i+1}$.

2. Сортировка выбором. Выполняется $n - 1$ шаг. На i -м шаге ($i = 1, 2, \dots, n - 1$) среди элементов $x_i, x_{i+1}, \dots, x_n$ отыскивается наименьший и переставляется с x_i .

3. Сортировка простыми вставками (два варианта). Пусть после нескольких шагов сортировки элементы $x_1, x_2, \dots, x_i$ уже упорядочены (образуют сегмент): $x_1 < x_2 < \dots < x_i$. Тогда на следующем шаге элемент x_{i+1} вставляется в этот сегмент таким образом, что элементы $x_1, x_2, \dots, x_{i+1}$ оказываются упорядоченными (сегмент расширяется). В конечном счете получаем сегмент $x_1, x_2, \dots, x_n$. В первом варианте сортировки место вставки определяется последовательными сравнениями x_{i+1} с $x_i, x_{i-1}, \dots$, во втором — последовательными сравнениями x_{i+1} с $x_1, x_2, \dots$.

4. Сортировка бинарными вставками. Отличается от сортировки простыми вставками тем, что место x_i в сегменте $x_1, x_2, \dots, x_{i-1}$ определяется алгоритмом бинарного поиска (см. А2, п. 4).

5. Сортировка слияниями. Разнообразные виды этой сортировки используют слияние сегментов. Сначала мы рассмотрим процедуру слияния, а затем опишем два варианта сортировки, основанной на этой процедуре.

Слияние. Пусть для элементов массива $e_1, e_2, \dots, e_m$ выполнено $e_1 < e_2 < \dots < e_k$ и $e_{k+1} < e_{k+2} < \dots < e_m$, $k \leq m$. Массив $f_1, f_2, \dots, f_m$, который является результатом слияния массивов $e_1, e_2, \dots, e_k$ и $e_{k+1}, e_{k+2}, \dots, e_m$, можно получить за m шагов. После i -го шага элементы $f_1, f_2, \dots, f_i$ уже имеют нужные значения, целые p и q ($p + q = i$) показывают, сколько элементов из числа $e_1, e_2, \dots, e_k$ и $e_{k+1}, e_{k+2}, \dots, e_m$ уже использовано.

Рекурсивный вариант сортировки слияниями. При $n = 1$ массив упорядочен. Пусть $n > 1$, тогда массив $x_1, x_2, \dots, x_n$ разбивается на два примерно равных по длине подмассива $x_1, x_2, \dots, x_{\lfloor n/2 \rfloor}$ и $x_{\lfloor n/2 \rfloor + 1}, x_{\lfloor n/2 \rfloor + 2}, \dots, x_n$. Сортировка применяется рекурсивно к этим подмассивам, после чего выполняется слияние.

Сортировка фон Неймана. Первоначально элементы массива $x_1, x_2, \dots, x_n$ рассматриваются как упорядоченные одноэлементные сегменты. Затем в массиве $y_1, y_2, \dots, y_n$ образуются упорядоченные сегменты длины 2, получающиеся слиянием x_1 и x_2 , x_3 и x_4 , x_5 и x_6 , ... Последний сегмент будет иметь один или два элемента в зависимости от четности n . Полученные сегменты сливаются в упорядоченные сегменты длины 4 (кроме последнего, который тоже упорядочен, но, возможно, имеет длину 1, 2 или 3), они последовательно попадают в массив $x_1, x_2, \dots, x_n$. Процесс укрупнения сегментов продолжается дальше. В некий момент массив $x_1, x_2, \dots, x_n$ или $y_1, y_2, \dots, y_n$ содержит только один упорядоченный сегмент.

6. Быстрая сортировка. Эта сортировка основывается на процедуре разбиения массива. Перед описанием сортировки мы рассмотрим эту процедуру.

Разбиение. Берется первый элемент массива и сравнивается со всеми остальными. Меньшие его элементы помещаются в начальную часть массива, большие — в концевую. Сам первоначально взятый элемент помещается между этими двумя частями, это — то место, которое ему надлежит занимать в упорядоченном массиве. Дополнительный массив для этой процедуры не требуется, достаточно двух переменных p и q , показывающих, сколько элементов в начальной

и конечной частях уже занято. Элемент, взятый первым, расположен на $(p + 1)$ -м месте и сравнивается со следующим за ним. Равенство $p + q = n - 1$ означает, что разбиение завершено.

Сортировка. Выполняется разбиение; в результате элемент, ранее располагавшийся в массиве первым, занимает нужное место (с некоторым номером k , $1 \leq k \leq n$). Затем быстрая сортировка применяется рекурсивно к $x_1, x_2, \dots, x_{k-1}$ и $x_{k+1}, x_{k+2}, \dots, x_n$.

Рандомизированный вариант. На этапе разбиения для сравнения со всеми элементами массива или его части используется не первый, а случайно выбранный из рассматриваемых на этом этапе элементов.

А2. Поиск

Числовой массив $x_1, x_2, \dots, x_n$ имеет попарно различные элементы. В п. 4 элементы предполагаются упорядоченными по возрастанию.

1. Поиск наименьшего. Просматриваются последовательно $x_2, x_3, \dots, x_n$ и каждый новый элемент x_i сравнивается с уже найденным наименьшим среди $x_1, x_2, \dots, x_{i-1}$.

2. Поиск m -го наименьшего. Элементы $x_1, x_2, \dots, x_n$ переставляются в соответствии с процедурой разбиения (см. алгоритм быстрой сортировки). Пусть элемент, бывший в исходном массиве первым, после выполнения процедуры стал k -м, $1 \leq k \leq n$. Если $m = k$, то задача решена. Если $m < k$, то разыскивается m -е наименьшее среди $x_1, x_2, \dots, x_{k-1}$; если $m > k$, то разыскивается $(m - k)$ -е наименьшее среди $x_{k+1}, x_{k+2}, \dots, x_n$.

3. Одновременный поиск наименьшего и наибольшего. Элементы $x_1, x_2, \dots, x_n$ просматриваются последовательными парами: x_1, x_2 , затем x_3, x_4 и т.д. (последний элемент может остаться без пары). При рассмотрении k -й пары x_{2k-1}, x_{2k} в ней выбираются наименьший и наибольший элементы, которые сравниваются с уже найденными наименьшим и, соответственно, наибольшим среди $x_1, x_2, \dots, x_{2k-2}$. Если n нечетно, то на последнем шаге x_n сравнивается с уже найденными наименьшим и наибольшим среди $x_1, x_2, \dots, x_{n-1}$.

4. Бинарный поиск места элемента. Кроме упорядоченного массива $x_1 < x_2 < \dots < x_n$ дано число y , для которого априори может осуществляться любая из возможностей

$$y \leq x_1, \quad x_1 < y \leq x_2, \quad \dots, \quad x_{n-1} < y \leq x_n, \quad x_n < y.$$

Этим возможностям присваиваются номера $1, 2, \dots, n + 1$. Требуется найти номер фактически осуществившейся возможности. Первоначальный диапазон поиска — от 1 до $n + 1$. Каждый шаг бинарного поиска сужает диапазон примерно вдвое: если перед очередным шагом диапазон был от p до q , то y сравнивается с x_r , $r = \lfloor (p + q)/2 \rfloor$. При $x_r < y$ диапазон дальнейшего поиска — от $r + 1$ до q (в дальнейшем рассматривается сегмент $x_{r+1}, x_{r+2}, \dots, x_{q-1}$), в противном случае — от p до r (в дальнейшем рассматривается сегмент $x_p, x_{p+1}, \dots, x_r$). И так далее до совпадения границ диапазона.

Приложение В

Оценивание сумм значений монотонных функций

Предложение В.1. Пусть f — неубывающая или невозрастающая непрерывная на отрезке $[n_0, n_1]$ функция, $n_0, n_1 \in \mathbb{Z}$. Тогда

$$\int_{n_0}^{n_1} f(x) dx + f(n_0) \leq \sum_{k=n_0}^{n_1} f(k) \leq \int_{n_0}^{n_1} f(x) dx + f(n_1) \quad (\text{В.1})$$

в случае неубывающей f и

$$\int_{n_0}^{n_1} f(x) dx + f(n_1) \leq \sum_{k=n_0}^{n_1} f(k) \leq \int_{n_0}^{n_1} f(x) dx + f(n_0) \quad (\text{В.2})$$

в случае невозрастающей f .

Доказательство. Пусть f — неубывающая функция. Рис. 25 показывает, что $\sum_{k=n_0}^{n_1-1} f(k) \leq \int_{n_0}^{n_1} f(x) dx$ (значение $\sum_{k=n_0}^{n_1-1} f(k)$ равно сумме

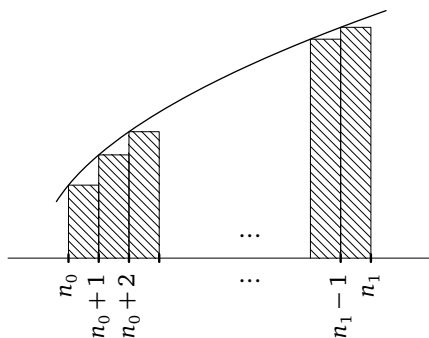


Рис. 25. $\sum_{k=n_0}^{n_1-1} f(k) \leq \int_{n_0}^{n_1} f(x) dx$.

площадей выделенных прямоугольников с вертикальными сторонами $f(n_0), f(n_0 + 1), \dots, f(n_1 - 1)$). Соответственно, рис. 26 показывает,

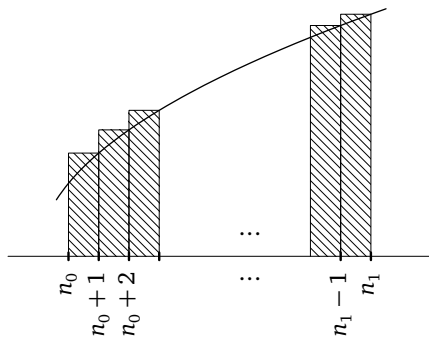


Рис. 26. $\sum_{k=n_0+1}^{n_1} f(k) \geq \int_{n_0}^{n_1} f(x) dx$.

что $\sum_{k=n_0+1}^{n_1} f(k) \geq \int_{n_0}^{n_1} f(x) dx$ (значение $\sum_{k=n_0+1}^{n_1} f(k)$ равно сумме площадей выделенных прямоугольников с вертикальными сторонами $f(n_0 + 1), f(n_0 + 2), \dots, f(n_1)$). Это дает нам (B.1). Неравенства (B.2) доказываются аналогично. $\square$

Рассмотрим некоторые примеры. Функция $\sqrt{x}$ не убывает на правой полуоси. Имеем

$$\int_1^n \sqrt{x} dx + 1 \leq \sum_{k=1}^n \sqrt{k} \leq \int_1^n \sqrt{x} dx + \sqrt{n},$$

т. е.

$$\frac{2}{3}(\sqrt{n})^3 + \frac{1}{3} \leq \sum_{k=1}^n \sqrt{k} \leq \frac{2}{3}(\sqrt{n})^3 + \sqrt{n} - \frac{2}{3},$$

и, как следствие,

$$\sum_{k=1}^n \sqrt{k} = \frac{2}{3}(\sqrt{n})^3 + O(\sqrt{n}). \quad (\text{B.3})$$

Аналогично выводится, что при любом вещественном $\alpha \geq 0$ и $n \rightarrow \infty$ выполняется $\sum_{k=0}^n k^\alpha \sim \frac{n^{\alpha+1}}{\alpha+1}$. (Справедлива ли эта формула при всех $\alpha \neq -1$?)

Функция $\frac{1}{x}$ не возрастает при $x \geq 1$. Имеем

$$\int_1^n \frac{dx}{x} + \frac{1}{n} \leq \sum_{k=1}^n \frac{1}{k} \leq \int_1^n \frac{dx}{x} + 1,$$

это дает нам

$$\ln n \leq \sum_{k=1}^n \frac{1}{k} \leq \ln n + 1,$$

и, как следствие,

$$\sum_{k=1}^n \frac{1}{k} = \ln n + O(1).$$

Приложение С

Проблема орбит

С1. Показательная функция и логарифмическая оценка

В этом разделе приложения рассказывается об одной вычислительной задаче и об эффективном решении ее некоторого частного случая; почти все факты приводятся без доказательств. В разделе С2 мы рассмотрим оставшийся случай с большими подробностями.

Речь идет об одном из вариантов известной «проблемы орбит». Даны две квадратные матрицы A и B одинакового порядка с элементами из поля $\mathbb{Q}$. Существуют ли натуральные m такие, что $A^m = B$? Если да, то надо указать все такие m . Рассмотрение собственных значений матриц A и B приводит к вопросу о распознавании существования и поиске натуральных m таких, что

$$a^m = b \tag{С.1}$$

для данных алгебраических чисел a и b .

Напомним, что число $a \in \mathbb{C}$ называется *алгебраическим*, если существует полином над полем $\mathbb{Q}$, для которого a является корнем. В качестве такого полинома удобно рассматривать *унитарный*¹ полином, т. е. полином с единичным старшим коэффициентом. Можно показать, что для всякого алгебраического числа существует единственный унитарный неприводимый над $\mathbb{Q}$ полином, для которого a является корнем. Алгебраическое число $a \in \mathbb{C}$ называется *целым алгебраическим*, если соответствующий неприводимый унитарный полином является полиномом над $\mathbb{Z}$, т. е. имеет целые коэффициенты². Нетрудно, например, убедиться, что число $(1 + \sqrt{5})/2$ — целое алгебраическое, а число $(3 + 4\sqrt{-1})/5$ — нецелое алгебраическое.

¹ Используются также термины *приведенный*, *нормированный*.

² Для избежания терминологической путаницы между целыми алгебраическими числами и обычными целыми, т. е. элементами $\mathbb{Z}$, последние в теории алгебраических чисел часто называют *целыми рациональными* числами.

Если $|a| \neq 1$, то с исследованием и решением уравнения (С.1) серьезных трудностей не возникает, так как модуль значения показательной функции a^m монотонно возрастает или монотонно убывает, и можно получить логарифмическое ограничение на m . Задача становится интересной при $|a| = 1$ и при дополнительном условии, что a не является корнем из 1. (Если a есть корень из единицы, то задача тривиальна.)

Как это следует из элементарной теории алгебраических чисел, вопрос можно переформулировать так: дан неприводимый над $\mathbb{Q}$ полином $f(x)$, $\deg f(x) = n$, и некоторый полином $q(x)$ над $\mathbb{Q}$ степени $\deg q(x) < n$; существуют ли такие натуральные m , что

$$a^m = q(a) \quad (\text{С.2})$$

при любых $a \in \mathbb{C}$, для которых $f(a) = 0$, и если да, то как найти такие m ? Мы сосредоточимся на последнем вопросе, опуская мелкие подробности перехода к нему от проблемы орбит.

Особенность постановки вопроса об уравнении (С.2) состоит в том, что речь идет не об одном фиксированном корне полинома $f(x)$, но обо всех его корнях. Легко показать, что если равенство (С.2) верно для одного какого-то корня a_0 полинома $f(x)$, то оно верно для всех корней этого полинома. В самом деле, если для некоторого фиксированного m полиномы $x^m - q(x)$ и $f(x)$ имеют общий корень a_0 , то эти полиномы имеют нетривиальный общий делитель. Наибольший общий делитель этих полиномов может быть найден алгоритмом Евклида и, следовательно, должен иметь рациональные коэффициенты. Из этого и из неприводимости $f(x)$ над $\mathbb{Q}$ следует, что наибольший общий делитель равен $f(x)$. Отсюда получаем, что $x^m - q(x)$ делится на $f(x)$. Это означает, что каждый корень $f(x)$ является также корнем $x^m - q(x)$.

Таким образом, задача о всей совокупности корней эквивалентна задаче об одном фиксированном корне, и задача об одном фиксированном корне эквивалентна задаче о любом другом корне. Это обстоятельство позволяет справиться со случаем целого алгебраического a . К решению подводит серия результатов о корнях унитарных полиномов над $\mathbb{Z}$, начавшаяся с работ Кронекера и завершившаяся в 60—70-е годы XX века исследованиями А. Шинцеля, Г. Цассенхауза, П. Бланксби и Х. Монтгомери¹. Выяснено, что если унитарный полином над $\mathbb{Z}$ степени n таков, что его корни не являются корнями из

¹ См. [84], [63]. Результаты Кронекера, Шинцеля и Цассенхауза изложены в [43, разд. 20.2].

единицы, то у него найдется по меньшей мере один корень a , для которого $|a| > 1$. Бланксби и Монтгомери показали, что по меньшей мере для одного корня выполняется неравенство

$$|a| \geq 1 + \frac{1}{30n^2 \ln(6n)}. \quad (C.3)$$

Из этого позднее было выведено, что если корни неприводимого унитарного полинома $f(x)$ степени n с целочисленными коэффициентами не являются корнями из единицы, то для любого натурального решения m уравнения (C.2) выполнено неравенство

$$m \leq n + 60n^2 \ln(6n)(\ln(n+1) + \ln|q(x)|), \quad (C.4)$$

где $|q(x)|$ обозначает длину вектора коэффициентов полинома $q(x)$: если $q(x) = c_k x^k + \dots + c_1 x + c_0$, то

$$|q(x)| = \sqrt{c_k^2 + \dots + c_1^2 + c_0^2}.$$

Решения уравнения (C.2) для различных корней a полинома $f(x)$ совпадают. Доказанное может быть сформулировано следующим образом.

Предложение C.1. Пусть $f(x)$ — неприводимый над $\mathbb{Q}$ полином степени n , корни которого не являются корнями из единицы. Пусть $q(x)$ — некоторый полином над $\mathbb{Q}$, степень которого меньше n . Тогда для любого корня a полинома $f(x)$ существует не более одного натурального решения уравнения (C.2), при этом решения m уравнения (C.2) для различных корней a полинома $f(x)$ совпадают, и имеет место оценка (C.4).

Несложно показать, что верны следующие утверждения:

1. Пусть $h(x) \in \mathbb{Q}$ и $r(x)$ — остаток от деления $h(x)$ на $f(x)$. Пусть $a \in \mathbb{C}$, $f(a) = 0$. Тогда $h(a) = r(a)$.

2. Пусть $q_1(x), q_2(x), q_3(x), \dots$ суть остатки от деления $x, x^2, x^3, \dots$ на $f(x)$. Тогда

- при $k > 1$ полином $q_k(x)$ равен остатку от деления $xq_{k-1}(x)$ на $f(x)$;
- равенство (C.2) имеет место, если и только если $q(x) = q_m(x)$.

Из этих утверждений следует, что после того, как установлена граница M для возможного значения m , распознавание существования и поиск натурального решения уравнения (C.2) можно выполнить, затратив $O(nM)$ арифметических операций над рациональными числами.

С2. Неудачно выбранный размер входа

Обратимся к уравнению (С.2), предполагая, что среди коэффициентов унитарного неприводимого над $\mathbb{Q}$ полинома $f(x)$ имеется по крайней мере один нецелый рациональный; без специальных оговорок считаем, что корни $f(x)$ не являются корнями из единицы. Возникает вопрос: можно ли и для этого случая получить подобную (С.4) верхнюю оценку величины m ? На протяжении ряда лет считалось, что ответ положительный¹ и что существует полином трех переменных $P(x_1, x_2, x_3)$ такой, что даже при наличии среди коэффициентов $f(x)$ нецелых рациональных чисел имеет место оценка

$$m < P(n, \log_2 |f|, \log_2 |q|), \quad (\text{С.5})$$

но затем выяснилось, что это неверно². Проблема заслуживает детального рассмотрения, поскольку ряд ее моментов является принципиальным. Вывод о существовании полинома $P(x_1, x_2, x_3)$ был сделан на основании утверждения о том, что если среди коэффициентов $f(x)$ есть нецелые числа, то имеет место неравенство

$$m \leq (n-1) \log_2 |f|. \quad (\text{С.6})$$

Неравенство (С.6) опровергается несложно (сразу заметно, что правая часть этого неравенства не зависит от вида полинома $q(x)$, что подозрительно). Мы, тем не менее, сосредоточим усилия на другом — покажем, что верхняя оценка (С.5) не может иметь места не только при полиномиальной, но и, например, при показательной функции того или иного вида (скажем, вида $2^{x_1^{x_2^{x_3}}}$) и вообще при любой непрерывной функции $P(x_1, x_2, x_3)$. Это, в частности, лишает возможности характеризовать вычислительную сложность алгоритма как полиномиальную, экспоненциальную и т.д.

Указанное обстоятельство в значительной мере является следствием плохо выбранных компонент n , $|f(x)|$ и $|q(x)|$ размера входа. Если мы зафиксируем n и $f(x)$, а затем будем слегка изменять значения коэффициентов полинома $q(x)$ (тем самым две компоненты размера остаются фиксированными, а третья изменяется очень мало), то при этом могут возникать уравнения вида (С.2), имеющие натуральные решения m , и эти m для разных уравнений могут, как будет показано, отличаться друг от друга сколь угодно сильно. Мы укажем натуральное n , неприводимый полином $f(x) \in \mathbb{Q}[x]$ степени n , последова-

¹ См. [76]; в этой же работе получена оценка (С.4) как следствие оценки (С.3).

² См. [59].

тельность $q_1(x), q_2(x), \dots \in \mathbb{Q}[x]$ полиномов степени меньшей, чем n , и вещественные A, B , $A < B$, такие, что $A < |q_k(x)| < B$, $k = 1, 2, \dots$, и при этом каждое из уравнений

$$a^m = q_k(a), \quad f(a) = 0, \quad (\text{C.7})$$

имеет решение $m = k$. Возьмем

$$a = \frac{3 + 4\sqrt{-1}}{5}, \quad (\text{C.8})$$

тогда $f(x) = x^2 - \frac{6}{5}x + 1$, $n = 2$ и $|f(x)| = \sqrt{1 + \frac{36}{25} + 1} = \frac{\sqrt{86}}{5}$. Пусть $b_1, b_2, \dots$ и $c_1, c_2, \dots$ — последовательности рациональных чисел таких, что $a^k = b_k a + c_k$, и пусть $q_k(x) = b_k x + c_k \in \mathbb{Q}[x]$ (таким образом, $q(x)$ равен остатку от деления x^k на $f(x) = x^2 - \frac{6}{5}x + 1$). Очевидно, что $b_1 = 1$, $c_1 = 0$.

Используя утверждения 1, 2, сформулированные в конце предыдущего раздела этого приложения, индукцией по k легко доказывается, что

$$b_{k+1} = \frac{6}{5}b_k + c_k, \quad c_{k+1} = -b_k$$

для $k > 0$. Последняя система двух рекуррентных уравнений решается легко, так как второе уравнение позволяет заменить в первом c_k на $-b_{k-1}$ и получить для b_k линейное рекуррентное уравнение второго порядка с постоянными коэффициентами. В итоге имеем (см. приложение F)

$$b_k = \frac{5}{8}\sqrt{-1} \left(\left(\frac{3 - 4\sqrt{-1}}{5} \right)^k - \left(\frac{3 + 4\sqrt{-1}}{5} \right)^k \right) = \frac{5}{4} \sin(k\theta)$$

и

$$c_k = -\frac{5}{8}\sqrt{-1} \left(\left(\frac{3 - 4\sqrt{-1}}{5} \right)^{k-1} - \left(\frac{3 + 4\sqrt{-1}}{5} \right)^{k-1} \right) = -\frac{5}{4} \sin((k-1)\theta),$$

где $\theta \in \mathbb{R}$ выбрано так, чтобы выполнялось $a = e^{\theta\sqrt{-1}}$. Получаем

$$|q_k| = \sqrt{b_k^2 + c_k^2} = \frac{5}{4} \sqrt{\sin^2 k\theta + \sin^2(k-1)\theta} \leq \frac{5}{4} \sqrt{2}.$$

Для функции $h(x) = \sqrt{\sin^2 x + \sin^2(x - \theta)}$ положим $\mu = \min_{0 \leq x < 2\pi} (h(x)) \geq 0$.

Так как $\sin(\theta) = \frac{4}{5}$, то θ не является целым кратным π , поэтому $\mu > 0$. Таким образом,

$$|q_k| = \frac{5}{4} h(k\theta) \geq \frac{5}{4} \mu > 0.$$

Положим $A = \frac{5}{4}\mu$, $B = \frac{5}{4}\sqrt{2}$. Для любой функции $F(x_1, x_2, x_3)$, непрерывной на множестве

$$V = \left\{ (x_1, x_2, x_3) : x_1 = 2, x_2 = \frac{\sqrt{86}}{5}, A < x_3 < B \right\},$$

мы можем выбрать целое k , большее максимума функции $F(x_1, x_2, x_3)$ на V , и это приведет к уравнению (С.7) с решением m , большим $F(n, |f(x)|, |q(x)|)$.

Таким образом, нами доказана следующая теорема.

Теорема С.1. Пусть алгоритм проверки существования и поиска решения m уравнения (С.2) состоит в получении верхней границы M для m и в последующей проверке всех значений $m = 1, 2, \dots, M$. Тогда сложность этого алгоритма по числу таких проверок не допускает оценки сверху вида $F(n, |f(x)|, |q(x)|)$, где F — какая-либо непрерывная функция трех переменных.

Отсутствие непрерывной функции, ограничивающей затраты, является, как уже упоминалось, следствием неудачно выбранных компонент размера входа. Можно показать¹, что если среди коэффициентов унитарного неприводимого полинома $f(x)$ имеется хотя бы один, не являющийся целым рациональным, и если $C, D \in \mathbb{Z}$ таковы, что Ca является целым алгебраическим для любого корня a полинома $f(x)$ и $Dq(x) \in \mathbb{Z}[x]$, то для решения m уравнения (С.2) выполнено

$$m \leq (n-1) \log_2 C + \log_2 D. \quad (\text{С.9})$$

Границы (С.4) и (С.9) приводят к алгоритму решения проблемы орбит.

В качестве C может быть взято наименьшее общее кратное знаменателей коэффициентов полинома $f(x)$, в качестве D — наименьшее общее кратное знаменателей коэффициентов полинома $q(x)$.

Неверно, что C всегда можно взять меньшим², чем $|f(x)|$. В этом можно убедиться, вернувшись к $f(x) = x^2 - \frac{6}{5}x + 1$.

¹ См. уже упоминавшуюся работу [59].

² В [76] авторы исходили из предположения, что всегда можно взять $C < |f(x)|$.

Приложение D

Об одном свойстве сумм неотрицательных случайных величин

Здесь доказывается теорема 17.1 из § 17.

Пусть $\xi_1, \xi_2, \dots$ — последовательность неотрицательных случайных величин на некотором вероятностном пространстве. Пусть числовая последовательность $h_1, h_2, \dots$ такова, что для каждого $k \geq 1$ выполнено $E(\xi_k | \xi_1, \xi_2, \dots, \xi_{k-1}) \leq h_k$ при всех значениях $\xi_1, \xi_2, \dots, \xi_{k-1}$. Зафиксируем неотрицательное число q и введем целочисленную случайную величину

$$\tau = \min \left\{ n : \sum_{k=1}^n \xi_k \geq q \right\}.$$

Пусть $\tau < \infty$ всюду на рассматриваемом вероятностном пространстве. Тогда $E\left(\sum_{k=1}^{\tau} h_k\right) \geq q$.

Рассмотрим случайные величины χ_k , $k = 1, 2, \dots$, где $\chi_1 = 1$, а при $k > 1$ выполняется $\chi_k = 1$, если $\xi_1 + \xi_2 + \dots + \xi_{k-1} < q$, и $\chi_k = 0$ в противном случае. Заметим, что

$$\chi_k = \begin{cases} 1, & \text{если } \tau \geq k, \\ 0, & \text{если } \tau < k, \end{cases}$$

и $1 = \chi_1 \geq \chi_2 \geq \dots$

Положим $p_k = P(\tau = k) = P(\chi_k = 1) - P(\chi_{k+1} = 1)$, $k = 1, 2, \dots$. Ясно, что $P(\chi_k = 1) = 1 - p_1 - p_2 - \dots - p_{k-1}$. Имеем

$$\begin{aligned} E\left(\sum_{k=1}^{\tau} h_k\right) &= \sum_{k=1}^{\infty} \sum_{j=1}^k h_j P(\tau = k) = \\ &= \sum_{k=1}^{\infty} (P(\chi_k = 1) - P(\chi_{k+1} = 1)) \sum_{j=1}^k h_j = \end{aligned}$$

$$\begin{aligned}
&= \sum_{k=1}^{\infty} P(\chi_k = 1) \sum_{j=1}^k h_j - \sum_{k=1}^{\infty} P(\chi_{k+1} = 1) \sum_{j=1}^k h_j = \\
&= \sum_{k=1}^{\infty} P(\chi_k = 1) \sum_{j=1}^k h_j - \sum_{k=2}^{\infty} P(\chi_k = 1) \sum_{j=1}^{k-1} h_j = \\
&= \sum_{k=1}^{\infty} P(\chi_k = 1) \sum_{j=1}^k h_j - \sum_{k=2}^{\infty} P(\chi_k = 1) \left(\sum_{j=1}^k h_j - h_k \right) = \\
&= \sum_{k=1}^{\infty} P(\chi_k = 1) \sum_{j=1}^k h_j - \sum_{k=2}^{\infty} P(\chi_k = 1) \sum_{j=1}^k h_j + \sum_{k=2}^{\infty} P(\chi_k = 1) h_k = \\
&= h_1 P(\chi_1 = 1) + \sum_{k=2}^{\infty} h_k P(\chi_k = 1) = \\
&= \sum_{k=1}^{\infty} h_k P(\chi_k = 1).
\end{aligned}$$

Таким образом,

$$E\left(\sum_{k=1}^{\tau} h_k\right) = \sum_{k=1}^{\infty} h_k P(\chi_k = 1). \quad (D.1)$$

Введем случайные величины $\eta_k = \chi_k \xi_k$, $k = 1, 2, \dots$; из определения случайных величин χ_k и условия $\tau < \infty$ следует, что

$$\sum_{k=1}^{\infty} \eta_k \geq q. \quad (D.2)$$

Докажем, что для $k = 1, 2, \dots$

$$E\eta_k \leq h_k P(\chi_k = 1). \quad (D.3)$$

Рассмотрим полную группу событий W_1, W_2 : в W_1 попадают те элементы вероятностного пространства, для которых $\xi_1 + \xi_2 + \dots + \xi_{k-1} < q$, в W_2 — для которых $\xi_1 + \xi_2 + \dots + \xi_{k-1} \geq q$. С учетом определения случайной величины χ_k , равной 1 на множестве W_1 и 0 на множестве W_2 , получаем по формуле полного математического ожидания:

$$\begin{aligned}
E\eta_k &= E(\chi_k \xi_k) = \\
&= E(\chi_k \xi_k | W_1) P(W_1) + E(\chi_k \xi_k | W_2) P(W_2) = \\
&= E(\xi_k | W_1) P(W_1) = \\
&= E(\xi_k | W_1) P(\chi_k = 1),
\end{aligned}$$

т. е.

$$E\eta_k = E(\xi_k|W_1)P(\chi_k = 1). \quad (D.4)$$

Так как по условию неравенство $E(\xi_k|\xi_1, \xi_2, \dots, \xi_{k-1}) \leq h_k$ выполняется при всех значениях $\xi_1, \xi_2, \dots, \xi_{k-1}$, то оно выполняется на W_1 . С учетом (D.4) имеем $E(\xi_k|W_1)P(\chi_k = 1) \leq h_k$, откуда следует (D.3).

Из (D.1), (D.2), (D.3) получаем

$$q = Eq \leq E\left(\sum_{k=1}^{\infty} \eta_k\right) = \sum_{k=1}^{\infty} E\eta_k \leq \sum_{k=1}^{\infty} h_k P(\chi_k = 1) = E\left(\sum_{k=1}^{\tau} h_k\right),$$

что и требовалось¹.

¹ Сходное, но менее подробное доказательство имеется в [2, гл. III, § 5, теорема 1], но там в условии теоремы пропущено требование конечности τ (см. задачу 136). Добавим, что доказанное утверждение следует из более общих теорем о случайных суммах — см. [30, § 4.1]

Приложение Е

О сортировке, оптимальной по числу сравнений в худшем случае

Долгое время считалось правдоподобным предположение, что сортировкой, оптимальной по числу сравнений в худшем случае, является сортировка бинарными вставками, о сложности $T_B(n)$ которой, вместе с этим, было известно, что $T_B(5) = 8$, при том, что наибольшая из обоснованных нижних границ для числа сравнений, необходимых для сортировки пяти элементов, есть $\lceil \log_2 5! \rceil = 7$. Это порождало гипотезу, что сложность $T_{\text{opt}}(n)$ оптимальной сортировки для некоторых значений n больше, чем $\lceil \log_2 n! \rceil$, и первое такое значение n равно пяти. Но в 1956 г. Г. Б. Демутом был найден алгоритм сортировки пяти элементов, который требует всего семь сравнений.

Этот алгоритм можно легко изобразить с помощью рисунков, на которых те точки, которые соответствуют сравнивавшимся элементам, соединяются стрелкой, ведущей от большего элемента к меньшему.

Сравниваем первый элемент со вторым и третий с четвертым, а потом сравниваем меньшие найденные элементы; на это уйдет три сравнения (рис. 27). Этим, в частности, для трех элементов из пяти мы устанавливаем их относительный порядок. Затем для пятого элемента, который еще не сравнивался, мы находим место среди трех уже упорядоченных элементов бинарным поиском; на это уйдет два сравнения. Тем самым мы приходим к одному из двух случаев, изображенных на рис. 28. И в том, и в другом случае для завершения сортировки с помощью бинарного поиска места элемента достаточно двух сравнений: в случае, изображенном на рис. 28а, вставка производится в массив из двух элементов, в случае, изображенном на рис. 28б, — из трех элементов. В итоге семи сравнений оказывается достаточно.

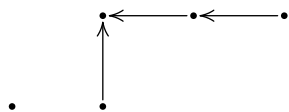


Рис. 27. Сортировка пяти элементов: ситуация после трех сравнений.

Рис. 28. Сортировка пяти элементов: ситуация после семи сравнений. (а) Вставка в массив из двух элементов. (б) Вставка в массив из трех элементов.



Рис. 28. Сортировка пяти элементов, ситуация после нахождения места пятого элемента среди трех упорядоченных элементов: а) пятый элемент меньше всех трех упорядоченных элементов; б) пятый элемент больше наименьшего из трех упорядоченных элементов. (В обоих случаях еще двух сравнений достаточно для завершения сортировки.)

Указанный алгоритм сортировки пяти элементов был позднее обобщен на произвольное число элементов, эта сортировка получила название сортировки слияниями и вставками. Через $T_F(n)$ обычно обозначается сложность этой сортировки по числу сравнений. Обнаружено, что $T_F(n) = \lceil \log_2 n! \rceil$ для $n = 1, 2, \dots, 11$. Но при этом $T_F(12) = 30$, хотя $\lceil \log_2 12! \rceil = 29$, и возникает вопрос, всегда ли возможна сортировка двенадцати элементов не более чем двадцатью девятью сравнениями? Аналогичные вопросы могут возникнуть и для больших значений n .

Как говорилось в § 14, имеется алгоритм, который, исходя из $n > 0$, строит оптимальный по числу сравнений алгоритм сортировки массивов из n элементов (алгоритм строит алгоритм). Этот алгоритм построения оптимального алгоритма сортировки требует огромной работы, даже если применить все средства экономии перебора. Но, тем не менее, на этом пути с помощью компьютера показано, что $T_{\text{opt}}(12) = 30$, что больше, чем $\lceil \log_2 12! \rceil$. К моменту выхода первого издания знаменитой монографии Д. Кнута «Искусство программирования» — ее тома с названием «Сортировка и поиск» [27], значение $T_{\text{opt}}(13)$ известно не было. К настоящему же времени по крайней мере для всех $n \leq 1000$ значения $T_{\text{opt}}(n)$ известны¹. Положение с $n \leq 13$ таково:

n :	1	2	3	4	5	6	7	8	9	10	11	12	13
$\lceil \log_2 n! \rceil$:	0	1	3	5	7	10	13	16	19	22	26	29	33
$T_B(n)$:	0	1	3	5	8	10	14	17	21	25	29	33	37
$T_F(n)$:	0	1	3	5	7	10	13	16	19	22	26	30	34
$T_{\text{opt}}(n)$:	0	1	3	5	7	10	13	16	19	22	26	30	34

¹ См. <https://oeis.org/A036604/b036604.txt>

Также известно, что $T_{\text{opt}}(14) = 38$, $T_{\text{opt}}(15) = 42$, ..., $T_{\text{opt}}(1000) = 8641$. Исследования, не связанные с компьютерным вычислением $T_{\text{opt}}(n)$, показали, что имеется бесконечно много значений n , для которых $T_F(n) > T_{\text{opt}}(n)$. Значение $n = 47$ — наименьшее из известных¹.

¹ См. [27, разд. 5.3.1].

Приложение F

Метод построения общего решения линейного рекуррентного уравнения с постоянными коэффициентами

Напомним метод построения общего решения линейного рекуррентного уравнения с постоянными коэффициентами¹. Начнем со случая однородного уравнения

$$a_d y(n) + a_{d-1} y(n-1) + \dots + a_0 y(n-d) = 0. \quad (\text{F.1})$$

Пусть $\lambda_1, \lambda_2, \dots, \lambda_s$ — все различные корни характеристического уравнения

$$a_d \lambda^d + a_{d-1} \lambda^{d-1} + \dots + a_0 = 0 \quad (\text{F.2})$$

и $m_1, m_2, \dots, m_s$ — кратности этих корней, $m_1 + m_2 + \dots + m_s = d$. Общим решением уравнения (F.1) является

$$\sum_{k=1}^s (C_{k,m_k-1} n^{m_k-1} + \dots + C_{k1} n + C_{k0}) \lambda_k^n, \quad (\text{F.3})$$

где все C_{ij} — произвольные постоянные.

Общее решение неоднородного линейного рекуррентного уравнения с постоянными коэффициентами

$$a_d y(n) + a_{d-1} y(n-1) + \dots + a_0 y(n-d) = f(n), \quad (\text{F.4})$$

где $f(n)$ — известная функция, является суммой общего решения соответствующего однородного уравнения (F.1) и какого-нибудь частного решения неоднородного уравнения (F.4).

Если правая часть $f(n)$ уравнения (F.4) представлена в виде $f(n) = g(n) + \dots + h(n)$ и если известны частные решения $v(n), \dots, w(n)$ для уравнений, левые части которых совпадают с левой частью уравнения (F.4), а правые части равны соответственно $g(n), \dots, h(n)$, то $v(n) + \dots + w(n)$ будет частным решением уравнения (F.4).

¹ См., например, [17, гл. V, § 4], [47, гл. 3].

Если правая часть $f(n)$ неоднородного уравнения (F.4) представляет собой *квазиполином*, т. е. равна $p(n)\mu^n$, где $p(n)$ — полином некоторой степени $l \geq 0$, а μ — некоторое (комплексное) число, то (F.4) имеет частное решение в виде квазиполинома $q(n)\mu^n$, и при этом полином $q(n)$ имеет степень $l + t$, где t — кратность μ как корня характеристического уравнения (F.2) (если μ не является корнем этого уравнения, то считаем кратность нулевой: $t = 0$). Коэффициенты полинома $q(n)$ находятся методом неопределенных коэффициентов.

Частные решения с заданными начальными значениями находятся подстановкой в общее решение начальных значений и определением постоянных из возникшей системы линейных алгебраических уравнений. (С помощью этого метода выводится и формула Бине (10.2).)

Приложение G

Об одном семействе алгебраических уравнений

В примере 24.2 мы столкнулись с рекуррентным уравнением

$$y(n) - y(n-1) - \dots - y(n-k) = 1, \quad (\text{G.1})$$

$y(0) = y(1) = \dots = y(k-1) = 0$. В качестве частного решения этого неоднородного уравнения при $k > 1$ можно взять $\frac{-1}{k-1}$. Нам предстоит теперь заняться общим решением соответствующего однородного уравнения, характеристическим уравнением которого служит

$$\lambda^k - \lambda^{k-1} - \dots - \lambda - 1 = 0. \quad (\text{G.2})$$

Свойства корней уравнения (G.2) описываются следующим предложением.

Предложение G.1. При $k > 1$ уравнение (G.2) имеет k различных корней. При этом в точности один из них — мы будем называть его главным корнем уравнения (G.2) и обозначать через α_k — по модулю превосходит единицу. Главный корень — вещественное число, удовлетворяющее условию¹ $2 - \frac{1}{k} \leq \alpha_k < 2$.

Доказательство. Поскольку предстоит использовать некоторую технику теории аналитических функций, мы перейдем к привычному обозначению z для комплексной переменной и будем рассматривать уравнение $z^k - z^{k-1} - \dots - z - 1 = 0$, левую часть которого обозначим через $v_k(z)$. Положим $V_k(z) = (z-1)v_k(z) = z^{k+1} - 2z^k + 1$.

Уравнение $V_k(z) = 0$ в сравнении с $v_k(z) = 0$ имеет дополнительный корень $z = 1$; мы докажем утверждение предложения для $V_k(z) = 0$,

¹ См. [42, гл. 13, п. 7—12], [80], [27, разд. 5.4.2, упр. 7]. В [42] доказано также, что все корни, отличные от главного корня α_k , по модулю не превосходят $\alpha_k - 1 < 1$.

$k > 1$, откуда будет следовать требуемое. Доказательство мы проведем в три этапа, установив справедливость следующих утверждений:

(i) для любого $k > 1$ все корни уравнения $V_k(x) = 0$ простые (т. е. имеют кратность 1);

(ii) круг любого радиуса r , $1 < r < 2 - \frac{1}{k}$, с центром в $z = 0$ содержит ровно k корней уравнения $V_k(z) = 0$;

(iii) уравнение $V_k(z) = 0$ имеет по меньшей мере один вещественный корень на полуинтервале $\left[2 - \frac{1}{k}, 2\right)$.

Для доказательства утверждения (i) заметим, прежде всего, что

$$\text{нод}(V_k(z), V'_k(z)) = \text{нод}(V_k(z), z^{k-1}((k+1)z - 2k)).$$

Но $V_k(z)$ не обращается в 0 при $z = 0$ и при $z = \frac{2k}{k+1}$. Отсюда следует, что $\text{нод}(V_k(z), V'_k(z)) = 1$. Это говорит о том, что корни $V_k(z)$ простые.

Для доказательства утверждения (ii) положим $W_k(z) = z^{k+1} - 2z^k$. Если на некоторой окружности выполняется неравенство $|W_k(z)| > 1$, то внутри этой окружности полиномы $V_k(z)$ и $W_k(z)$ по теореме Руше¹ имеют одинаковое число корней. Рассмотрим окружность S_σ радиуса $1 + \sigma$, $0 < \sigma < 1$, с центром в $z = 0$. Так как

$$|W_k(z)| = |z^{k+1} - 2z^k| \geq |2z^k| - |z^{k+1}|,$$

то на окружности S_σ выполняется $|W_k(z)| \geq (1 + \sigma)^k(1 - \sigma)$. Далее, очевидно, $(1 + \sigma)^k \geq 1 + k\sigma$, и мы получаем, что если σ удовлетворяет неравенству

$$(1 + k\sigma)(1 - \sigma) > 1, \tag{G.3}$$

то условия теоремы Руше выполнены, и полином $V_k(z)$ имеет внутри S_σ столько же корней, сколько их имеет $W_k(x)$, т. е. k . Неравенство (G.3) выполнено для всех значений σ , принадлежащих интервалу с концами, являющимися корнями квадратного уравнения $k\sigma^2 - (k-1)\sigma = 0$, — эти корни суть 0 и $1 - \frac{1}{k}$. Это доказывает утверждение (ii).

¹ См., например, [49, гл. 5, § 3] или [43, п. 1.1]. Применительно к полиномиальному случаю эта теорема допускает следующую формулировку. Пусть полином $V(z)$ представлен в виде суммы $W(z) + \tilde{W}(z)$ двух полиномов, и на контуре некоторой области значение $|\tilde{W}(z)|$ меньше значения $|W(z)|$. Тогда внутри этой области число корней полиномов $V(z)$ и $W(z)$ одинаково.

Наконец, утверждение (iii) следует из того, что $v_k(1) < 0$, $v_k(2) > 0$ и при этом уравнение $v_k(z) = 0$ не имеет корней на интервале $(1, 2 - \frac{1}{k})$. $\square$

Рекуррентное уравнение (G.1) имеет, таким образом, общее решение

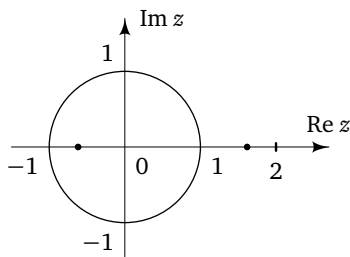
$$y(n) = C_k \alpha_k^n + C_{k-1} \alpha_{k-1}^n + \dots + C_1 \alpha_1^n - \frac{1}{k-1}$$

(α_k — главный корень характеристического уравнения (G.2), все остальные корни $\alpha_1, \alpha_2, \dots, \alpha_{k-1}$ по модулю не превосходят единицу). Нас интересует частное решение, удовлетворяющее условию

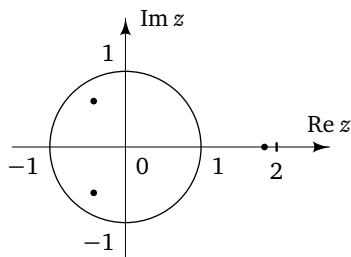
$$y(0) = y(1) = \dots = y(k-1) = 0.$$

И без нахождения всех C_i ясно, что $C_k \neq 0$, — иначе последовательность $y(k), y(k+1), \dots$ была бы ограниченной.

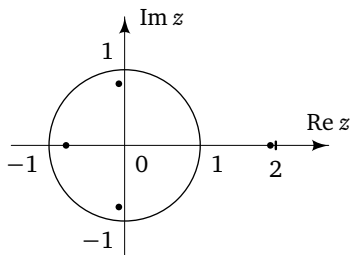
На рис. 29 показано расположение корней уравнений $v_k(z) = 0$, $k = 2, 3, 4$.



а) $z^2 - z - 1 = 0$



б) $z^3 - z^2 - z - 1 = 0$



в) $z^4 - z^3 - z^2 - z - 1 = 0$

Итак, нам удалось обойтись без фактического нахождения корней характеристического уравнения. Более того, речь шла не об одном конкретном уравнении, а о семействе, в которое входят уравнения сколь угодно высоких степеней.

Мы получили доказательство предложения 24.1 из § 24.

Приложение Н

Об алгоритмах типа Монте-Карло, предназначенных для распознавания свойств математических объектов

Подкласс рандомизированных алгоритмов, так называемых алгоритмов типа Монте-Карло или вероятностных алгоритмов, находит применения в численных методах; широко известен, например, такого типа метод приближенного вычисления определенных интегралов. Эти численные алгоритмы получают обоснование и оценку точности¹. Ниже мы коснемся иной области их применения, а именно — возможности проверки наличия того или иного свойства у заданного объекта (числа, матрицы и т. д.).

Рассматривается некоторое множество M . Фиксировано свойство P , которым может обладать или не обладать каждый из элементов множества M . Задача состоит в проверке для данного конкретного $a \in M$, обладает ли этот элемент свойством P . То, что a обладает свойством P , или же, то же самое, что a удовлетворяет условию P , будем записывать как $P(a)$. Пусть ситуация такова, что известно некоторое сравнительно легко проверяемое необходимое условие Q того, что для a выполнено P , иначе говоря, $P(a) \Rightarrow Q(a)$. Но при этом условие Q не является достаточным, — вообще говоря, не верно, что $Q(a) \Rightarrow P(a)$, т.е. не верно, что если выполнено $Q(a)$, то обязательно должно быть выполнено $P(a)$. Довольно часто условие Q является параметризованным, содержа кванторы всеобщности, т.е. формулировка этого условия включает обороты типа «для любого целого положительного $m...$ », «для любых $x_1, \dots, x_n \in \{0, 1\}...$ » и т.д. Вновь, как и в случае алгоритмов типа Лас-Вегас, рассматриваемая задача (в данном случае — проверка обладания свойством P) решается так, что каждому входу $a \in M$ алгоритма сопоставляется вероятностное пространство сценариев. При этом, следование любому из сценариев ведет к получению конкретного, не содержащему кванторов

¹ Уже упоминалась книга И. М. Соболя [53].

всеобщности случайного варианта (теста), $\tilde{Q}$ условия Q . Для этого, например, каждому связанному одним из кванторов всеобщности параметров присваивается случайное значение соответствующего типа, это даст желаемый вариант, назовем его $\tilde{Q}$. Условие $\tilde{Q}$ дает нам некоторый *тест* того, имеет ли место $P(a)$. Так как $\tilde{Q}(a)$ является необходимым условием для $P(a)$, то невозможен случай, когда рассматриваемый вход a не удовлетворяет условию $\tilde{Q}$, но обладает свойством P : если для a не имеет места $\tilde{Q}(a)$, то это гарантирует, что не имеет места и $P(a)$. Представляется важным вычисление или оценивание вероятности события «рассматриваемое $a \in M$ удовлетворяет сгенерированному тесту $\tilde{Q}$, но не обладает при этом свойством P ». Можно считать, что условие Q выбрано удачно, если для $\tilde{Q}$, случайно (как это объяснено выше) сгенерированного исходя из Q , эта вероятность не превосходит $\frac{1}{2}$.

Пусть эта вероятность равна p . Если условие $\tilde{Q}$ выполнено для a (иначе говоря, a выдерживает тест $\tilde{Q}$), то считаем, что с вероятностью $1 - p$ рассматриваемое a обладает свойством P . Если условия $\tilde{Q}$ генерируются независимо k раз и каждый раз такое условие оказывается выполненным, то эта вероятность равна соответственно $1 - p^k$.

Как иллюстрацию рассмотрим пример рандомизированного алгоритма типа Монте-Карло, появившегося одним из самых первых в литературе. Его автор — Р. Фрейвалдс¹.

Пример Н.1. Даны три $(n \times n)$ -матрицы A , B и C , элементы которых можно считать, скажем, рациональными числами. Требуется проверить, верно ли, что $AB = C$. Пользуясь каким-либо алгоритмом умножения матриц, имеющим сложность $\Theta(n^\omega)$, $\omega > 2$, по числу арифметических операций, мы можем выполнить эту проверку, с такой же сложностью $\Theta(n^\omega)$. Для этой же проверки можно использовать и рандомизированный алгоритм типа Монте-Карло. (В роли a выступит тройка матриц размера $n \times n$.) Если x — некоторый вектор-столбец с n компонентами, то вычисление двух векторов $A(Bx)$ и Cx выполняется с суммарным числом операций $O(n^2)$. Пусть вектор-столбец $x \in \{0, 1\}^n$ выбран случайно, и все возможности выбора равновероятны. Пусть D — ненулевая матрица размера $n \times n$. Тогда вероятность того, что для выбранного x выполняется $Dx = 0$, не превосходит $\frac{1}{2}$. В самом деле, пусть $Dx = 0$ и элемент d_{ij} матрицы D не равен нулю. Имеем для i -го

¹ См. оригинальную публикацию [66] и последующее изложение в [81, Sect. 7.1], [32, разд. 4.1].

элемента вектора Dx :

$$\sum_{k=1}^n d_{ik}x_k = 0. \quad (\text{H.1})$$

Для каждого удовлетворяющего равенству (H.1) вектора x найдется в $\{0, 1\}^n$ такой вектор, для которого (H.1) не выполняется — в силу того, что $d_{ij} \neq 0$, нам будет достаточно изменить компоненту x_j вектора x . Все векторы из $\{0, 1\}^n$ разбиваются на такого рода пары. Вероятность события $Dx = 0$ не может превосходить $\frac{1}{2}$.

Рассматривая в качестве D матрицу $AB - C$, получаем, что вероятность неравенства $AB - C \neq 0$ не превосходит $\frac{1}{2}$. Следовательно, при случайном выборе вектора x мы имеем следующее:

если $A(Bx) \neq Cx$, то $AB \neq C$;

если $A(Bx) = Cx$, то $AB = C$ с вероятностью не меньшей $\frac{1}{2}$.

Если, скажем, предъявлено семь независимо выбранных векторов $x \in \{0, 1\}^n$, для каждого из которых $A(Bx) \neq Cx$, то это будет означать, что $AB \neq C$ с вероятностью, не превосходящей $\frac{1}{2^7} = \frac{1}{128} < 0,01$, соответственно $AB = C$ выполняется с вероятностью, не меньшей, чем $1 - \frac{1}{128} > 0,99$.

Замечание. Те события в вероятностных пространствах, которые рассматриваются в обсуждениях алгоритмов типа Монте-Карло, проверяющих путем тестирования наличие тех или иных свойств математических объектов, связаны не с входами алгоритмов, а с используемыми тестами. Предъявление вероятности или ее оценки в итоге такого обсуждения есть, фактически, способ заявить о степени возможного доверия результатам тестирования.

Литература

- [1] С. А. Абрамов. Исследование алгоритмов одновременного нахождения наибольшего и наименьшего элементов массива // ЖВМ и МФ. 1982. Т. 22, № 2. С. 424—428.
- [2] С. А. Абрамов. Элементы анализа программ. Частичные функции на множестве состояний. М.: Наука, 1986.
- [3] С. А. Абрамов, Е. А. Бордаченкова, Д. Е. Хмельнов. Расширяемое эссе как гипертекстовая схема информационного и учебного материала // ЖВМ и МФ. 2013. Т. 53, № 3. С. 495—501.
- [4] С. А. Абрамов, Г. Г. Гнездилова. Алгоритм управления вопросником в автоматизированной обучающей системе // Вестн. Моск. ун-та. Сер. 15. Вычисл. матем. и киб. 1988. № 2. С. 47—52.
- [5] В. Б. Алексеев. Введение в теорию сложности алгоритмов: Учебное пособие для студентов. М.: Издательский отдел ф-та ВМК МГУ, 2002.
- [6] М. В. Алехнович, А. Я. Белов. Сложность алгоритмов при построениях циркулем и линейкой. // Фундамент. и прикл. матем. 2001. Т. 7, № 2. С. 597—614.
- [7] В. И. Арнольд. Динамика, статистика и проективная геометрия полей Гауля. М.: МЦНМО, 2005.
- [8] А. Ахо, Дж. Хопкрофт, Дж. Ульман. Построение и анализ вычислительных алгоритмов. М.: Мир, 1979.
- [9] М. Балазар. Асимптотический закон распределения простых чисел. М.: МЦНМО, 2013.
- [10] Н. Г. де Брейн. Асимптотические методы в анализе. М.: ИЛ, 1961.
- [11] А. А. Васин, В. В. Морозов. Теория игр и модели математической экономики: Учебное пособие. М.: МАКС Пресс, 2005.
- [12] Введение в криптографию / Под общей редакцией В. В. Ященко. М.: МЦНМО: ЧеРо, 2000.
- [13] Н. К. Верещагин, А. Шень. Начала теории множеств. М.: МЦНМО, 2008.
- [14] Н. Вирт. Алгоритмы + структуры данных = программы. М.: Мир, 1985.
- [15] М. Н. Вялый. Сложность вычислительных задач // Математическое просвещение. Третья серия. Вып. 4. М.: МЦНМО, 2000. С. 81—114.

- [16] С. Б. Гашков, В. Н. Чубариков. Арифметика. Алгоритмы. Сложность вычислений. М.: Высшая школа, 2000.
- [17] А. О. Гельфонд. Исчисление конечных разностей. М.: Наука, 1967.
- [18] О. Н. Герман, Ю. В. Нестеренко. Теоретико-числовые методы в криптографии. М.: Издательский центр «Академия», 2012.
- [19] Р. Грэхем. Начала теории Рамсея. М.: Мир, 1984.
- [20] М. Гэри, Д. Джонсон. Вычислительные машины и труднорешаемые задачи. М.: Мир, 1982.
- [21] С. Дасгупта, Х. Пападимитриу, У. Вазирани. Алгоритмы. М.: МЦНМО, 2014.
- [22] В. А. Ильин, Г. Д. Ким. Линейная алгебра и аналитическая геометрия. М.: Изд-во МГУ, 1998.
- [23] А. А. Карацуба. Сложность вычислений // Труды Математического института РАН. 1995. Т. 211. С. 186—203.
- [24] А. А. Карацуба, Ю. П. Офман. Умножение многозначных чисел на автоматах // Докл. АН СССР. 1962. Т. 145, № 2. С. 293—294.
- [25] Д. Кнут. Искусство программирования для ЭВМ. Т. 1. Основные алгоритмы. М.; СПб.; Киев: ИД «Вильямс», 1999.
- [26] Д. Кнут. Искусство программирования для ЭВМ. Т. 2. Получисленные алгоритмы. М.; СПб.; Киев: ИД «Вильямс», 2000.
- [27] Д. Кнут. Искусство программирования для ЭВМ. Т. 3. Сортировка и поиск. М.; СПб.; Киев: ИД «Вильямс», 2001.
- [28] Т. Кормен, Ч. Лейзерсон, Р. Ривест. Алгоритмы: построение и анализ. М.: МЦНМО, 2000.
- [29] А. И. Кострикин. Введение в алгебру. М.: Наука, 1977.
- [30] В. М. Круглов, В. Ю. Королев. Предельные теоремы для случайных сумм. М.: Изд-во МГУ, 1990.
- [31] В. Н. Крупский. Введение в сложность вычислений. М.: Факториал-Пресс, 2006.
- [32] Н. Н. Кузюрин, С. А. Фомин. Эффективные алгоритмы и сложность вычислений: Учебное пособие. М.: МФТИ, 2007.
- [33] С. А. Кук. Сложность процедур вывода теорем // Кибернетический сборник, нов. серия. 1975. Вып. 12. С. 5—15.
- [34] С. С. Лавров. Программирование. Математические основы, средства, теория. СПб.: БХВ-Петербург, 2001.
- [35] В. Н. Латышев. Комбинаторная теория колец: сложность алгебраических алгоритмов. М.: Изд-во Моск. ун-та, 1987.

- [36] Л. А. Левин. Универсальные задачи перебора // Проблемы передачи информации. 1973. Т. 9, № 3. С. 115—116.
- [37] Дж. Макконелл. Анализ алгоритмов. Вводный курс. М.: Техносфера, 2002.
- [38] А. С. Малистов. О поиске медианы массива за линейное время // Математическое просвещение. Третья серия, вып. 21. М.: МЦНМО, 2017. С. 265—270.
- [39] А. А. Марков, Н. М. Нагорный. Теория алгорифмов. М.: Фазис, 1996.
- [40] Ф. Мостеллер. 50 занимательных вероятностных задач с решениями. М.: URSS, 2005.
- [41] Ю. В. Нестеренко. Теория чисел. М: ИЦ «Академия», 2008.
- [42] А. М. Островский. Решение уравнений и систем уравнений. М.: ИЛ, 1963.
- [43] В. В. Прасолов. Многочлены. М.: МЦНМО, 2014.
- [44] Ф. Препарата, М. Шеймос. Вычислительная геометрия: Введение. М.: Мир, 1989.
- [45] А. В. Прохоров, В. Г. Ушаков, Н. Г. Ушаков. Задачи по теории вероятностей. Основные понятия. Предельные теоремы. Случайные процессы. М.: МЦНМО, 2023.
- [46] Э. Рейнгольд, Ю. Нивергельт, Н. Део. Комбинаторные алгоритмы. Теория и практика. М.: Мир, 1980.
- [47] В. К. Романко. Разностные уравнения. М.: БИНОМ. Лаборатория знаний, 2006.
- [48] А. А. Сапоженко. Некоторые вопросы сложности алгоритмов: Учебное пособие. М.: Издательский отдел ф-та ВМК МГУ, 2001.
- [49] А. Г. Свешников, А. Н. Тихонов. Теория функций комплексной переменной. М.: Физматлит, 1999.
- [50] В. Серпинский. 250 задач по теории чисел. Москва; Ижевск: Регулярная и хаотическая динамика, 2004.
- [51] А. Л. Тоом. О сложности схемы из функциональных элементов, реализующей умножение целых чисел // Докл. АН СССР. 1963. Т. 150, № 2. С. 496—498.
- [52] Дж. Хартманис, Дж. Э. Хопкрофт. Обзор теории сложности // Кибернетический сборник. 1974. Вып. 11. С. 131—176.
- [53] И. М. Соболев. Метод Монте-Карло. М.: Наука, 1989.
- [54] М. Хейле, М. Кауэрс, М. Зайдль. Получение многих методов умножения матриц // Материалы III международной конференции «Компьютерная алгебра» (Федеральный исследовательский центр «Информатика и управление», Российский университет дружбы народов). М.: 2019. С. 23—24.
- [55] П. Л. Чебышев. Полное собрание сочинений. Т. 1. М.; Л.: АН СССР, 1944.

- [56] И. Р. Шафаревич. Избранные главы алгебры. М.: Журнал «Математическое образование», 2000.
- [57] А. Шень. Программирование: теоремы и задачи. М.: МЦНМО, 2017.
- [58] Математическая энциклопедия. Т. 1. М.: Издательство «Советская энциклопедия», 1977.
- [59] S. A. Abramov, M. Bronstein. Hypergeometric dispersion and the orbit problem // Proceedings of the 2000 International Symposium on Symbolic and Algebraic Computation. New York: ACM, 2000. P. 8—13.
- [60] M. Agrawal, N. Kayal, N. Saxena. PRIMES is in P // Ann. of Math. (2). 2004. V. 160, № 2. P. 781—793.
- [61] S. Baase, A. van Gelder. Computer algorithms: Introduction to design and analysis. Boston, MA: Addison-Wesley, 2000.
- [62] E. Bach, J. Shallit. Algorithmic number theory. V. 1. Efficient algorithms. Cambridge, MA: MIT Press, 1996.
- [63] P. E. Blanksby, H. L. Montgomery. Algebraic integers near the unit circle // Acta Arith. 1971. V. 18. P. 355—369.
- [64] G. Brassard, P. Bratley. Fundamentals of algorithmics. Englewood Cliffs, NJ: Prentice Hall, 1996.
- [65] M. J. Fischer, M. O. Rabin. Super-exponential complexity of Presburger arithmetic // Complexity of computation (Proc. SIAM-AMS Sympos., New York, 1973). Providence, RI: AMS, 1974. (SIAM-AMS Proc.; Vol. VII). P. 27—41.
- [66] R. Freivalds. Probabilistic machines can use less running time // Proceedings of IFIP Congress (1977). Amsterdam: North-Holland, 1977. P. 839—842.
- [67] N. Friedman. Some results on the effect of arithmetics on comparison problems // Proc. 13th IEEE Ann. Symp. on Switching and Automata Theory. Los Alamitos, CA: IEEE Computer Society, 1972. P. 139—143.
- [68] M. Fürer. Faster Integer Multiplication // Proceedings of the 39th Annual ACM Symposium on Theory of Computing. New York: ACM, 2007. P. 57—66.
- [69] M. Fürer. Faster Integer Multiplication Algorithm // SIAM J. Comp. 2009. V. 39, № 3. P. 979—1005.
- [70] F. Le Gall. Powers of Tensors and Fast Matrix Multiplication // ISSAC 2014 — Proceedings of the 39th International Symposium on Symbolic and Algebraic Computation. New York: ACM, 2014. P. 296—303.
- [71] J. von zur Gathen, J. Gerhard. Modern computer algebra. New York: Cambridge University Press, 1999.
- [72] D. Harvey, J. van der Hoeven. Integer multiplication in time $O(n \log n)$. 2018. <http://www.texmacs.org/joris/nlogn/nlogn.pdf>
- [73] P. Hoffman. The man who loved only numbers. New York: Hyperion, 1998.

- [74] *J. E. Hopcroft, L. R. Kerr.* Some techniques for proving certain simple programs optimal // Proc. 10th Annual Symposium on Switching and Automata Theory (SWAT 1969). P. 36—45.
- [75] *S. Jukna.* Extremal Combinatorics. Berlin; Heidelberg: Springer Verlag, 2022.
- [76] *R. Kannan, R. J. Lipton.* Polynomial-time algorithm for the orbit problem // J. Assoc. Comput. Mach. 1986. V. 33, № 4. P. 808—821.
- [77] *D. E. Knuth.* Big omicron and big omega and big theta // ACM SIGACT News. 1976. V. 8, iss. 2. P. 18—23.
- [78] *J. B. Laderman.* A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications // Bull. Amer. Math. Soc. 1976. V. 82, № 1. P. 126—128.
- [79] *J. C. Lagarias.* The $3x + 1$ problem and its generalizations // The American Mathematical Monthly. 1989. V. 92, № 1. P. 3—23.
- [80] *E. P. Miles, Jr.* Generalized Fibonacci numbers and associated matrices // The American Mathematical Monthly. 1960. V. 67, № 8. P. 745—752.
- [81] *R. Motwani, P. Raghavan.* Randomized algorithms. Cambridge: Cambridge University Press, 1995.
- [82] *I. Pohl.* A sorting problem and its complexity // Comm. ACM. 1972. V. 15, № 6. P. 462—466.
- [83] *E. M. Reingold, A. I. Stocks.* Simple proofs of lower bounds for polynomial evaluation // Complexity of computer computations (Proc. Sympos., IBM Thomas J. Watson Res. Center, Yorktown Heights, N. Y., 1972). New York: Plenum, 1972. P. 21—30.
- [84] *A. Schinzel, H. Zassenhaus.* A refinement of two theorems of Kronecker // Michigan Math. J. 1965. V. 12. P. 81—85.
- [85] *R. Sedgewick, P. Flajolet.* An introduction to the analysis of algorithms. Boston, MA: Addison-Wesley, 1996.
- [86] *A. Turing.* On computable numbers, with an application to the Entscheidungsproblem // Proc. London Math. Soc. (2). 1936. V. 42, № 3. P. 230—265.
- [87] *S. Winograd.* On multiplication of 2×2 matrices // Linear Algebra and Appl. 1971. V. 4. P. 381—388.
- [88] *A. C. Yao.* Probabilistic computation: towards a unified measure of complexity // 18th Annual Symposium on Foundations of Computer Science (Providence, RI, 1977). Long Beach, CA: IEEE, 1977. P. 222—227

Предметный указатель

Алгоритм

- Агравала, Кайала и Саксены (AKS) 157
- бинарный возведения в степень (RS) 34, 114
- Грэхема (G) 23, 204
- Евклида (E) 78, 151, 152
- — расширенный (EE) 81, 153, 155
- Карацубы (KM) 180
- китайский 164
- кратных карт 96
- Ле Галля 187
- наивного
- — деления с остатком (ND) 149
- — умножения (NM) 146
- оптимальный 115
- — по порядку сложности 120
- пробных делений (TD) 10, 31
- рандомизированный 58, 65, 135
- сложения (Add) 144
- Тоома (TM) 185
- Уоршелла 161, 200
- Флойда 166
- Фюрера 151, 187
- Харли и ван дер Ховена 187
- Шеймоса—Хоя (SH) 39
- Шёнхаге—Штрассена 187
- Штрассена умножения матриц (St) 183

Асимптотики символ

- — o 20, 22
- — O 18, 22
- — $\tilde{O}$ 158
- — Θ 18, 22

— — Ω 18, 22

— — $\sim$ 21

Задача NP-полная 210

Затраты алгоритма (функции затрат) 9

Класс

— NP 205

— P 206

Медиана массива 65

Модель вычислений 17, 142, 209

Нижняя граница сложности 111

— асимптотическая 120

Оценка точная 19, 22

Поиск 220

— бинарный места элемента (BS) 82, 220

— наименьшего элемента 111, 115, 220

— наименьшего и наибольшего элементов 117, 128, 220

— m -го наименьшего элемента 220

Полиномиальная ограниченность 48

Принцип Яо 135

Проблема $P \stackrel{?}{=} NP$ 207

Размер входа 10

Сводимость

— линейная 195

Сводимость полиномиальная 210
 Сегмент массива 82
 Сертификат 206
 Сложность 11
 — аддитивная 15
 — битовая 143
 — в среднем 44
 — в худшем случае 11
 — временная 11
 — мультипликативная 14
 — по числу операций 13
 — пространственная 11
 — словесная 146
 Сортировка 218
 — быстрая (QS) 38, 219
 — — рандомизированная 60
 — вставками
 — — бинарными (B) 86, 219
 — — простыми (I_1, I_2) 10, 218
 — выбором 16, 218

— оптимальная (opt) 234
 — пузырьковая 17, 54, 218
 — слияниями и вставками (F) 235
 — слияниями рекурсивная (MS) 171, 174, 219
 — фон Неймана (vN) 84, 219
 Стратегия «разделяй и властвуй» 170
 Схемы Горнера оптимальность 117

 Теорема
 — о рекуррентном неравенстве 175
 — о существовании NP-полных задач 210
 — Ферма малая 156
 — Фишера—Рабина 208
 Тропические вычисления 166

 Формула полного математического ожидания 51

Оглавление

Предисловие	3
Введение	5
Часто используемые обозначения	8
Глава 1. Сложности алгоритмов как функции числовых аргументов. Сложность в худшем случае	
§ 1. Затраты алгоритма для данного входа. Сложность по числу операций	9
§ 2. Асимптотические оценки: формализм	17
§ 3. Асимптотические оценки: два примера	22
§ 4. Длина числа как возможный размер входа	31
Задачи	35
Глава 2. Сложность в среднем	
§ 5. Понятие сложности в среднем	44
§ 6. Сортировка и конечные вероятностные пространства. Применение формулы полного математического ожидания	48
§ 7. Пример медленного роста сложности в среднем в сравнении со сложностью в худшем случае	54
§ 8. Функция затрат рандомизированного алгоритма	58
Задачи	66
Глава 3. Оценивание числа шагов (итераций) алгоритма	
§ 9. Функции, убывающие по ходу выполнения алгоритма	78
§ 10. Качество оценок	87
§ 11. Завершимость работы алгоритма	93
§ 12. Вложенные циклы: дополнительные примеры	99
§ 13. Нецелые размеры входа и непрерывные оценочные функции	101
Задачи	104
Глава 4. Нижняя граница сложности алгоритмов некоторого класса. Оптимальные алгоритмы	
§ 14. Понятие нижней границы сложности	111
§ 15. Оптимальные алгоритмы	114

§ 16. Асимптотические нижние границы. Алгоритм, оптимальный по порядку сложности	120
§ 17. Нижняя граница сложности в среднем	124
§ 18. Нижние границы сложности рандомизированных алгоритмов. Принцип Яо	133
Задачи	136

Глава 5. Битовая сложность

§ 19. Битовые операции	142
§ 20. Наивная арифметика: умножение	146
§ 21. Наивная арифметика: деление с остатком	149
§ 22. Модулярная арифметика	153
§ 23. Булева арифметика	159
Задачи	163

Глава 6. Рекуррентные соотношения как средство анализа сложности алгоритмов

§ 24. Простейшие рекуррентные уравнения	167
§ 25. Об одном классе нелинейных рекуррентных соотношений	170
§ 26. Асимптотические оценки решений рекуррентных неравенств	174
§ 27. Упрощающее добавление нулей	179
Задачи	188

Глава 7. Сводимость

§ 28. Линейная сводимость	195
§ 29. Линейная сводимость и нижние границы сложности	201
§ 30. Классы P и NP	205
§ 31. Существование задач распознавания, не принадлежащих P. Связь моделей МТ и РАМ	207
§ 32. Полиномиальная сводимость. NP-полные задачи	210
Задачи	214

Приложение А. Некоторые алгоритмы сортировки и поиска	218
---	-----

Приложение В. Оценивание сумм значений монотонных функций	222
---	-----

Приложение С. Проблема орбит	225
--	-----

Приложение Д. Об одном свойстве сумм неотрицательных случайных величин	231
--	-----

Приложение Е. О сортировке, оптимальной по числу сравнений в худшем случае	234
--	-----

Приложение Ф. Метод построения общего решения линейного рекуррентного уравнения с постоянными коэффициентами	237
--	-----

Приложение Г. Об одном семействе алгебраических уравнений	239
Приложение Н. Об алгоритмах типа Монте-Карло, предназначенных для распознавания свойств математических объектов . .	243
Литература	246
Предметный указатель	251